
OP-TEE Documentation

TrustedFirmware.org

Jul 12, 2024

CONTENTS

1	Getting started	3
1.1	About OP-TEE	3
1.2	Coding standards	4
1.3	Contribute	5
1.4	Contact	8
1.5	License headers	10
1.6	Platforms supported	10
1.7	Presentations	12
1.8	Releases	14
2	Architecture	19
2.1	Core	19
2.2	Cryptographic implementation	50
2.3	Device Tree	54
2.4	Device tree bindings	57
2.5	File structure	59
2.6	GlobalPlatform API	61
2.7	Libraries	72
2.8	Porting guidelines	73
2.9	Secure boot	79
2.10	Secure storage	80
2.11	Subkeys	89
2.12	Trusted Applications	92
2.13	Virtualization	102
2.14	SPMC	104
2.15	Arm Security Extensions	115
2.16	Platform documentation	116
3	Build and run	119
3.1	Prerequisites	119
3.2	Device specific information	122
3.3	AOSP	149
3.4	Linux kernel TEE framework	152
3.5	OP-TEE gits	152
3.6	Toolchains	185
3.7	Trusted Applications	186
3.8	StandAloneMM	195
3.9	OP-TEE with Rust	196
3.10	Linux userland integration	198

4	Debugging techniques	205
4.1	Abort dumps / call stack	205
4.2	Ftrace (function tracing)	207
4.3	Gprof	209
5	Frequently Asked Questions	211
5.1	Abbreviations	213
5.2	Architecture	213
5.3	Board support	215
5.4	Building	215
5.5	Certification and security reviews	218
5.6	Contribution	219
5.7	Interfaces	219
5.8	Hardware and peripherals	220
5.9	License	221
5.10	Promotion	221
5.11	Security vulnerabilities	221
5.12	Source code	222
5.13	Testing	222
5.14	Trusted Applications	222

This is the official location for OP-TEE documentation.

GETTING STARTED

This contains general information about OP-TEE, how to get in contact, how to contribute, how to report security issues etc. It is intended for people who are new to OP-TEE.

1.1 About OP-TEE

OP-TEE is a Trusted Execution Environment (TEE) designed as companion to a non-secure Linux kernel running on Arm; Cortex-A cores using the TrustZone technology. OP-TEE implements *TEE Internal Core API* v1.3.1 which is the API exposed to Trusted Applications and the *TEE Client API* v1.0, which is the API describing how to communicate with a TEE. Those APIs are defined in the *GlobalPlatform API* specifications.

The non-secure OS is referred to as the Rich Execution Environment (REE) in TEE specifications. It is typically a Linux OS flavor as a GNU/Linux distribution or the AOSP.

OP-TEE is designed primarily to rely on the Arm TrustZone technology as the underlying hardware isolation mechanism. However, it has been structured to be compatible with any isolation technology suitable for the TEE concept and goals, such as running as a virtual machine or on a dedicated CPU.

The main design goals for OP-TEE are:

- **Isolation** - the TEE provides isolation from the non-secure OS and protects the loaded Trusted Applications (TAs) from each other using underlying hardware support,
- **Small footprint** - the TEE should remain small enough to reside in a reasonable amount of on-chip memory as found on Arm based systems,
- **Portability** - the TEE aims at being easily pluggable to different architectures and available HW and has to support various setups such as multiple client OSes or multiple TEEs.

1.1.1 OP-TEE components

OP-TEE is divided in various components:

- A secure privileged layer, executing at Arm secure PL-1 (v7-A) or EL-1 (v8-A) level.
- A set of secure user space libraries designed for Trusted Applications needs.
- A Linux kernel TEE framework and driver (merged to the official tree in v4.12).
- A Linux user space library designed upon the GlobalPlatform *TEE Client API* specifications.
- A Linux user space supplicant daemon (tee-supplciant) responsible for remote services expected by the TEE OS.
- A test suite (xtest), for doing regression testing and testing the consistency of the API implementations.
- An example git containing a couple of simple host- and TA-examples.

- And some build scripts, debugging tools to ease its integration and the development of Trusted Applications and secure services.

These components are available from several git repositories. The main ones are [build](#), [optee_os](#), [optee_client](#), [optee_test](#), [optee_examples](#) and the [Linux kernel TEE framework](#).

1.1.2 History

OP-TEE was initially developed by ST-Ericsson (and later on by STMicroelectronics), but this was before OP-TEE got the name “OP-TEE” and was turned into an open source project. Back then it was a closed source and a proprietary TEE project. In 2013, ST-Ericsson obtained GlobalPlatform’s compliance qualification with this implementation, proving that the APIs were behaving as expected according to the GlobalPlatform specifications.

Later on the same year (2013) Linaro was about to form Security Working Group (SWG) and one of the initial key tasks for SWG was to work on an open source TEE project. After talking to various TEE vendors Linaro ended up working with STMicroelectronics TEE project. But before being able to open source it there was a need to replace some proprietary components with open source components. For a couple of months Linaro/SWG together with engineers from STMicroelectronics re-wrote major parts (crypto library, secure monitor, build system etc), cleaned up the project by enforcing [Coding standards](#), running [checkpatch](#) etc.

June 12 2014 was the day when OP-TEE was “born” as an open source project. At that day the OP-TEE team pushed the [first commit](#) to GitHub. A bit after this Linaro also made a [press release](#) about this. That press release contains a bit more information. At the first year as an open source project it was owned by STMicroelectronics but maintained by Linaro and STMicroelectronics. In 2015 there was an ownership transfer of OP-TEE from STMicroelectronics to Linaro. In September 2019, ownership was transferred from Linaro to the TrustedFirmware.org project (see [_blogpost](#) for more information). Maintenance is a shared responsibility between the members for TrustedFirmware.org and some community maintainers representing other companies who are using OP-TEE.

1.2 Coding standards

In this project we are trying to adhere to the same coding convention as used in the Linux kernel (see [CodingStyle](#)). We achieve this by running [checkpatch](#) from Linux kernel. However there are a few exceptions that we had to make since the code also follows GlobalPlatform standards. The exceptions are as follows:

1. **CamelCase** for GlobalPlatform types is allowed.
2. We **do not** run checkpatch on third party code that we might use in this project, such as LibTomCrypt, MPA, newlib etc. The reason for that and not doing checkpatch fixes for third party code is because we would probably deviate too much from upstream and therefore it would be hard to rebase against those projects later on and we don’t expect that it is easy to convince other software projects to change coding style.
3. **All** variables **shall be** initialized to a well known value in one or another way. The reason for that is that we have had potential security issues in the past that originated from not having variables initialized with a well defined value. We have also investigate various toolchain flags that are supposed to help out finding uninitialized variables. Unfortunately our conclusion is that you cannot trust the compilers here, since there are corner cases where compilers cannot reliably give a warning.

Variables are initialized according to these general guidelines:

- Scalars (and types like `time_t` which are standardized as scalars) are initialized with `0`, unless another value makes more sense.
- For [optee_client](#) we need maximum portability. So use `{ 0 }` for struct types where the first element is known to be a scalar and `memset()` otherwise unless there is a good reason not to do so.

- For the rest of the code we assume that a recent version of GCC or Clang is used so we initialize structs with `{ }` in order to avoid the more clumsy `memset()` procedure. Types like `pthread_t` which can be a scalar or a composite type are initialized with `memset()` in order to minimize the amount of future headache. Arrays are initialized with `{ }`, too.

Unsigned integer constants are defined using the `U()`, `UL()` or `ULL()` macros, depending on the required width. `U()` is a good choice for 32-bit values. Any of the minimum width cousins `UINT{8, 16, 32, 64}_C()` are also accepted for compatibility. This makes the sign and size of the integer well defined instead of depend on how large the value is or which compiler is used. For example:

```
#define MY_UNSIGNED_CONSTANT U(123)
```

1.2.1 Running checkpatch

Regarding the checkpatch tool, it is not included directly into this project. Please use `checkpatch.pl` from the Linux kernel git in combination with the local [checkpatch script](#). Environment variable `CHECKPATCH` is expected to provide the path to the Linux checkpatch script path, i.e.:

```
export CHECKPATCH=/path/to/linux/scripts/checkpatch.pl
./scripts/checkpatch.sh HEAD
./scripts/checkpatch.sh --diff github/master HEAD
```

There are also targets for common use cases in the Makefiles:

```
export CHECKPATCH=/path/to/linux/scripts/checkpatch.pl
make checkpatch          #check staging and working area
make checkpatch-staging  #check staging area (added, but not committed files)
make checkpatch-working  #check working area (modified, but not added files)
```

1.3 Contribute

Contributions to OP-TEE are managed by the OP-TEE *Core Team* and anyone can contribute to OP-TEE as long as it is understood that it will require a *Signed-off-by* tag from the one submitting the patch(es). The Signed-off-by tag is a simple line at the end of the explanation for the patch, which certifies that you wrote it or otherwise have the right to pass it on as an open source patch (see below). You thereby assure that you have read and are following the rules stated in the *Developer Certificate of Origin* as stated below.

1.3.1 Developer Certificate of Origin

Developer Certificate of Origin
Version 1.1

Copyright (C) 2004, 2006 The Linux Foundation and its contributors.
660 York Street, Suite 102,
San Francisco, CA 94110 USA

Everyone is permitted to copy and distribute verbatim copies of this
license document, but changing it is not allowed.

(continues on next page)

(continued from previous page)

Developer's Certificate of Origin 1.1

By making a contribution to this project, I certify that:

- (a) The contribution was created in whole or in part by me and I have the right to submit it under the open source license indicated in the file; or
- (b) The contribution is based upon previous work that, to the best of my knowledge, is covered under an appropriate open source license and I have the right under that license to submit that work with modifications, whether created in whole or in part by me, under the same open source license (unless I am permitted to submit under a different license), as indicated in the file; or
- (c) The contribution was provided directly to me by some other person who certified (a), (b) or (c) and I have not modified it.
- (d) I understand and agree that this project and the contribution are public and that a record of the contribution (including all personal information I submit with it, including my sign-off) is maintained indefinitely and may be redistributed consistent with this project or the open source license(s) involved.

We have borrowed this procedure from the Linux kernel project to improve tracking of who did what, and for legal reasons.

To sign-off a patch, just add a line in the commit message saying:

```
Signed-off-by: Random J Developer <random@developer.example.org>
```

Use your real name or on some rare cases a company email address, but we disallow pseudonyms or anonymous contributions.

1.3.2 GitHub

This section describes how to use GitHub for OP-TEE development and contributions.

Setting up an account

You do not need to own a GitHub account in order to clone a repository. But if you want to contribute, you need to create an account at [GitHub](#). Note that a free plan is sufficient to collaborate.

SSH is recommended to access your GitHub repositories securely and without supplying your username and password each time you pull or push something. To configure SSH for GitHub, please refer to [Connecting to GitHub with SSH](#).

Forking

Only owners of the OP-TEE projects have write permissions to the git repositories of those projects. Contributors **should fork** OP-TEE/*.[git](#) and/or linaro-swg/*.[git](#) into their own account, then work on this forked repository. The complete documentation about **forking** can be found at [fork a repo](#).

Creating pull requests

When you want to submit a patch to the OP-TEE project, you are supposed to create a [pull request](#) to the git where you forked your git from. How that is done using GitHub is explained at the GitHub [pull request](#) page.

Commit messages

- The **subject line** should explain **what** the patch does as precisely as possible. It is usually prefixed with keywords indicating which part of the code is affected, but not always. Avoid lines longer than 80 characters.
- The **commit description** should give more details on **what** is changed, and explain **why** it is done. Indication on how to enable and use some particular feature can be useful, too. Try to limit line length to 72 characters, except when pasting some error message (compiler diagnostic etc.). Long lines are allowed to accommodate URLs, too (preferably use URLs in a Fixes: or Link: tag).
- The commit message **must** end with a blank line followed by some tags, including your Signed-off-by: tag. By applying such a tag to your commit, you are effectively declaring that your contribution follows the terms stated by *Developer Certificate of Origin* (in the DCO section there is also a complete example).
- Other tags may be used, such as:
 - Tested-by: Test R <test@r.com>
 - Acked-by: Acke R <acke@r.com>
 - Suggested-by: Suggeste R <suggeste@r.com>
 - Reported-by: Reporte R <reporte@r.com>
- When citing a previous commit, whether it is in the text body or in a Fixes: tag, always use the format shown above (12 hexadecimal digits prefix of the commit SHA1, followed by the commit subject in double quotes and parentheses).

Review feedback

It is very likely that you will get review comments from other OP-TEE users asking you to fix certain things etc. When fixing review comments, do:

- **Add fixup** patches on **top** of your existing branch. **Do not** squash and force push while fixing review comments.
- When all comments have been addressed, just write a simple messages in the comments field saying something like “All comments have been addressed”. By doing so you will notify the maintainers that the fix might be ready for review again.

Finalizing your contribution

Once you and reviewers have agreed on the patch set, which is when all the people who have commented on the pull request have given their Acked-by: or Reviewed-by:, you need to consolidate your commits:

Use `git rebase -i` to squash the fixup commits (if any) into the initial ones. For instance, suppose the `git log --oneline` for your contribution looks as follows when the review process ends:

```
<sha1-commit4> [Review] Do something
<sha1-commit3> [Review] Do something
<sha1-commit2> Do something else
<sha1-commit1> Do something
```

Then you would do:

```
$ git rebase -i <sha1-commit1>^
```

Edit the commit script so it looks like so:

```
pick <sha1-commit1> Do something
squash <sha1-commit3> [Review] Do something
squash <sha1-commit4> [Review] Do something
pick <sha1-commit2> Do something else
```

Add the proper tags (Acked-by: ..., Reviewed-by: ..., Tested-by: ...) to the commit message(s) for each and every commit as provided by the people who reviewed and/or tested the patches.

Hint: `git commit --fixup=<sha1-of-commit-to-fix>` and later on `git rebase -i --autosquash <sha1-of-first-commit-in-patch-serie>^1` is pretty convenient to use when adding review patches and doing the final squash operation.

Once `rebase -i` is done, you need to force-push (`-f`) to your GitHub branch in order to update the pull request page.

```
$ git push -f <my-remote> <my-branch>
```

After completing this it is the project maintainers job to apply your patches to the master branch.

1.4 Contact

1.4.1 GitHub

Our preference is to use GitHub for communication. The reason for that is that it is an open source project, so there should be no real reason to hide discussions from other people. GitHub also makes it possible for anyone to chime in into discussion etc. So besides sending patches as pull requests on GitHub we also encourage people to use the “issues” to report bugs, give suggestions, ask questions etc.

Please try to use the “issues” in the relevant git. I.e., if you want to discuss something related to `optee_client`, then use “issues” at [optee_client](#) and so on. If you have a general question etc about OP-TEE that doesn’t really belong to a specific git, then please use [issues at optee_os](#) in that case.

1.4.2 Email

You can reach the [Core Team](#) by sending an email to `op-tee[at]lists.trustedfirmware.org`. However note that it's a public mailinglist and **not** just TrustedFirmware engineers behind that email address.

For pure Linux kernel patches, please use the appropriate Linux kernel mailinglist, basically run the `get_maintainer.pl` script in the Linux kernel tree to figure out where to send your patches.

```
$ cd <linux-kernel>
$ ./scripts/get_maintainer.pl drivers/tee/
```

1.4.3 IRC

Some of the OP-TEE developers can be reached at Freenode (`chat.freenode.net`) at channel `#linaro-security`. Having that said, the activity there is a bit limited, so it is probably **not** the best place to discuss OP-TEE.

1.4.4 Vulnerability reporting

As part of the TrustedFirmware.org organization, the OP-TEE project uses the security incident procedure outlined on the [TrustedFirmware.org security incident](#) page. We offer two methods for reporting security issues. The first is the traditional method of sending email to the addresses listed on the [Mailing Aliases](#) page. The alternative method is through GitHub's [GitHub Security Advisories](#) page for OP-TEE.

We prefer the [GitHub Security Advisories](#) page because they simplify the sharing and communication of reports. However, this also requires a GitHub account and we recognize that not everyone can or has the ability to report security issues via GitHub; therefore, we also accept reports via email.

Note that OP-TEE is a reference implementation for developers and device manufacturers and by being a reference implementation it is not always running a secure device configuration by default (see [Platform ports](#) for more information). Therefore we ask people to think twice whether the security incident report should go to:

- a) The OP-TEE project? Is it an issue in the generic code?
- b) The chipmaker? Does it only affect a certain platform? Is it a configuration described only under NDA?
- c) The ones making the end product? Is the issue only present on a certain device?

In some cases, the OP-TEE team works directly with chipmakers. However, it is not uncommon for products to be manufactured using OP-TEE without the OP-TEE project's knowledge. In such instances, we recommend sending the security issue report to the manufacturer of the final product, who should then, if necessary, contact the OP-TEE project and/or the chipmaker.

1.4.5 Core Team

The core team consists of TrustedFirmware.org engineers. See also "THE REST" in the [OP-TEE MAINTAINERS](#) file, which oversees the essential activities, such as performing releases, merging patches, and being the first to respond to security incidents.

1.5 License headers

This document defines the format of the copyright and license headers in OP-TEE source files. Such headers shall comply with the rules described here, which are compatible with the rules adopted by the Linux kernel community.

1.5.1 New source files

- **Rule 1.1** Shall contain exactly one SPDX license identifier, which can express a single or multiple licenses (refer to [SPDX](#) for syntax details).
- **Rule 1.2** The SPDX license identifier shall be added as a comment line. It shall be the first possible line in the file which can contain a comment. The comment style shall depend on the file type:
 - **Rule 1.2.1** C source: `// SPDX-License-Identifier: <expression>`
 - **Rule 1.2.2** C header: `/* SPDX-License-Identifier: <expression> */`
 - **Rule 1.2.3** Assembly: `/* SPDX-License-Identifier: <expression> */`
 - **Rule 1.2.4** Python, shell: `# SPDX-License-Identifier: <expression>`
- **Rule 1.3** Shall contain at least one copyright line
- **Rule 1.4** Shall not contain the mention ‘All rights reserved’
- **Rule 1.5** Shall not contain any license notice other than the SPDX license identifier

Note that files imported from external projects are not new files. The rules for pre-existing files (below) apply.

1.5.2 Pre-existing or imported files

- **Rule 2.1** SPDX license identifiers shall be added according to the license notice(s) in the file and the rules above (1.1 and 1.2*)
- **Rule 2.2** It is recommended that license notices be removed once the corresponding identifier has been added. Note however that this may only be done by the copyright holder(s) of the file.
- **Rule 2.3** Similar to 2.2, and subject to the same conditions, the text: “All rights reserved” shall be removed also.

1.6 Platforms supported

Several platforms are supported. In order to manage slight differences between platforms, a `PLATFORM_FLAVOR` flag has been introduced. The `PLATFORM` and `PLATFORM_FLAVOR` flags define the whole configuration for a chip the where the Trusted OS runs. Note that there is also a composite form which makes it possible to append `PLATFORM_FLAVOR` directly, by adding a dash in-between the names. The composite form is shown below for the different boards. For more specific details about build flags etc, please read [Configuration and flags](#). Some platforms have different sub-maintainers, please refer to the file [MAINTAINERS](#) for contact details for various platforms.

Table 1: Platforms officially supported in OP-TEE

Platform	Composite PLATFORM flag	Publicly available?	Ma
ARM Juno Board	<code>PLATFORM=vexpress-juno</code>	Yes	Ye
Atmel ATSAMA5D2-XULT Board	<code>PLATFORM=sam</code>	Yes	Ye
Broadcom ns3	<code>PLATFORM=bcm-ns3</code>	No	Ye

continues on

Table 1 – continued from previous page

Platform	Composite PLATFORM flag	Publicly available?	Ma
DeveloperBox (Socionext Synquacer SC2A11)	PLATFORM=synquacer	Yes	Ye
FSL ls1021a	PLATFORM=ls-ls1021atwr	Yes	Ye
NXP ls1043ardb	PLATFORM=ls-ls1043ardb	Yes	Ye
NXP ls1046ardb	PLATFORM=ls-ls1046ardb	Yes	Ye
NXP ls1012ardb	PLATFORM=ls-ls1012ardb	Yes	Ye
NXP ls1028ardb	PLATFORM=ls-ls1028ardb	Yes	Ye
NXP ls1088ardb	PLATFORM=ls-ls1088ardb	Yes	Ye
NXP ls2088ardb	PLATFORM=ls-ls2088ardb	Yes	Ye
NXP ls1012afwry	PLATFORM=ls-ls1012afwry	Yes	Ye
FSL i.MX6 Quad SABRE Lite Board	PLATFORM=imx-mx6qsabreelite	Yes	Ye
FSL i.MX6 Quad SABRE SD Board	PLATFORM=imx-mx6qsabresd	Yes	Ye
SolidRun i.MX6 Quad Hummingboard Edge	PLATFORM=imx-mx6qhmbedge	Yes	Ye
SolidRun i.MX6 Dual Hummingboard Edge	PLATFORM=imx-mx6dhmbedge	Yes	Ye
SolidRun i.MX6 Dual Lite Hummingboard Edge	PLATFORM=imx-mx6dlhmbedge	Yes	Ye
SolidRun i.MX6 Solo Hummingboard Edge	PLATFORM=imx-mx6shmbedge	Yes	Ye
FSL i.MX6 UltraLite EVK Board	PLATFORM=imx-mx6ulevk	Yes	Ye
NXP i.MX7Dual SabreSD Board	PLATFORM=imx-mx7dsabresd	Yes	Ye
NXP i.MX7Solo WaRP7 Board	PLATFORM=imx-mx7swarp7	Yes	Ye
NXP i.MX8MQEVK Board	PLATFORM=imx-imx8mqevk	Yes	Ye
NXP i.MX8MMEVK Board	PLATFORM=imx-imx8mmevk	Yes	Ye
ARM Foundation FVP	PLATFORM=vexpress-fvp	Yes	Ye
HiSilicon D02	PLATFORM=d02	No	Ye
HiSilicon Hi3519AV100 Demo Board	PLATFORM=hisilicon-hi3519av100_demo	No	Ye
HiKey Board (HiSilicon Kirin 620)	PLATFORM=hikey` or `PLATFORM=hikey-hikey	Yes	Ye
HiKey960 Board (HiSilicon Kirin 960)	PLATFORM=hikey-hikey960	Yes	Ye
Marvell ARMADA 7K Family	PLATFORM=marvell-armada7k8k	Yes	Ye
Marvell ARMADA 8K Family	PLATFORM=marvell-armada7k8k	Yes	Ye
Marvell ARMADA 3700 Family	PLATFORM=marvell-armada3700	Yes	Ye
MediaTek MT8173 EVB Board	PLATFORM=mediatek-mt8173	No	Ye
Poplar Board (HiSilicon Hi3798C V200)	PLATFORM=poplar	Yes	Ye
QEMU	PLATFORM=vexpress-qemu_virt	Yes	Ye
QEMUv8	PLATFORM=vexpress-qemu_armv8a	Yes	Ye
Raspberry Pi 3	PLATFORM=rpi3	Yes	Ye
Renesas RCAR	PLATFORM=rcar	No	Ye
Renesas RZ/G	PLATFORM=rzg	Yes	Ye
Rockchip PX30	PLATFORM=rockchip-px30	No	Ye
Rockchip RK322X	PLATFORM=rockchip-rk322x	No	Ye
Rockchip RK3399	PLATFORM=rockchip-rk3399	Yes	Ye
STMicroelectronics b2260 - h410 (96boards fmt)	PLATFORM=stm-b2260	No	Ye
STMicroelectronics b2120 - h310 / h410	PLATFORM=stm-cannes	No	Ye
STMicroelectronics STM32MP1 series	PLATFORM=stm32mp1	Yes	Ye
Allwinner A64 Pine64 Board	PLATFORM=sunxi-sun50i_a64	Yes	Ye
Texas Instruments AM65x	PLATFORM=k3-am65x	Yes	Ye
Texas Instruments DRA7xx	PLATFORM=ti-dra7xx	Yes	Ye
Texas Instruments AM57xx	PLATFORM=ti-am57xx	Yes	Ye
Texas Instruments AM43xx	PLATFORM=ti-am43xx	Yes	Ye
AMD/Xilinx Versal ACAP	PLATFORM=versal	Yes	Ye
Xilinx Zynq 7000 ZC702	PLATFORM=zynq7k-zc702	Yes	No
Xilinx Zynq UltraScale+ MPSOC	PLATFORM=zynqmp-zcu102	Yes	No
Spreadtrum SC9860	PLATFORM=sprd-sc9860	No	No

1.7 Presentations

Below are presentations coming from engineers working with OP-TEE in one or another way. Note that the older they are, the less relevant is the information in them. So do not trust blindly what was said back in the days, cross check with latest version to understand whether things have changed or not.

The links are sorted in chronological order, newest first and oldest at the end.

- **LVC21F**
 - Demo - PKCS#11 in OP-TEE ([video](#))
- **LVC21**
 - LVC21-118 - ASLR in OP-TEE ([slides](#), [video](#))
 - LVC21-201 - Security Working Group (SWG) Lightning Talk ([slides](#))
 - LVC21-215 - PKCS#11 in OP-TEE ([slides](#), [video](#))
 - LVC21-305 - OP-TEE as a Secure Partition running on SPM using ARMv8.4-A SEL2 feature ([slides](#), [video](#))
 - LVC21-311 - Virtualization for OP-TEE ([slides](#), [video](#))
- **LVC20**
 - LVC20-112 - PSA Secure Partitions in OP-TEE ([slides](#), [video](#))
 - LVC20-118 - SCMI server in TEE ([slides](#), [video](#))
 - LVC20-204 - Encrypted firmwares and how to bake them right ([slides](#), [video](#))
- **SAN19**
 - SAN19-107 - Secure Data Path on Linux and NXP i.MX 8M ([slides](#), [video](#))
 - SAN19-207 - SCMI server in secure world ([slides](#), [video](#))
 - SAN19-225 - Fuzzing embedded (trusted) operating systems using AFL ([slides](#), [video](#))
 - SAN19-226 - Enabling AOSP FBE for OP-TEE Keymaster ([slides](#), [video](#))
 - SAN19-411 - Runtime Secure Keys in OP-TEE ([slides](#), [video](#))
 - SAN19-413 - TEE based Trusted Keys in Linux ([slides](#), [video](#))
 - SAN19-507 - HDCP and OP-TEE ([slides](#), [video](#))
- **BKK19**
 - BKK19-419 - Debugging with OP-TEE ([slides](#), [video](#))
 - BKK19-415 - OP-TEE: Shared memory between TAs ([slides](#), [video](#))
 - BKK19-403 - Using DTB overlays in OP-TEE ([slides](#), [video](#))
 - BKK19-215 - TPM in TEE ([slides](#), [video](#))
 - BKK19-117 - Security WG Lightning talks ([slides](#), [video](#))
- **YVR18**
 - YVR18-414 - Keymaster and Gatekeeper ([slides](#), [video](#))
 - YVR18-117 - SWG updates since HKG18 ([slides](#), [video](#))
- **HKG18**

- HKG18-402 - Build secure key management services in OP-TEE ([slides](#), [video](#))
- **SFO17**
 - SFO17-309 - Secure storage updates ([slides](#), [video](#))
- **Webinar**
 - TEE Linux kernel support and open source security ([slides](#), [video](#))
- **BUD17**
 - BUD17-416 - Benchmark and profiling in OP TEE ([slides](#), [video](#))
 - BUD17-400 - Secure Data Path with OPTTEE ([slides](#), [video](#))
- **LAS16**
 - LAS16-504 - Secure Storage updates in OP-TEE ([slides](#), [video](#))
 - LAS16-406 - Android Widevine on OP-TEE ([slides](#), [video](#))
 - LAS16-111 - Easing Access to ARM TrustZone OP TEE and Raspberry Pi 3 ([slides](#), [video](#))
- **BKK16**
 - BKK16-201 - PlayReady OP-TEE Integration with Secure Video Path ([slides](#), [video](#))
 - BKK16-110 - A Gentle Introduction to Trusted Execution and OP-TEE ([slides](#))
- **SFO15**
 - SFO15-503 - Secure storage in OP-TEE ([slides](#), [video](#))
 - SFO15-205 - OP-TEE Content Decryption with Microsoft PlayReady on ARM TrustZone ([slides](#), [video](#))
 - SFO15-200 - TEE kernel driver ([slides](#), [video](#))
- **HKG15**
 - HKG15-311 - OP-TEE for Beginners and Porting Review ([slides](#), [video](#))
 - HKG15-307 - OP-TEE pager ([slides](#), [video](#))
- **LCU14**
 - LCU14-306 - OP-TEE Future Enhancements ([slides](#))
 - LCU14-302 - How to port OP-TEE to another platform ([slides](#), [video](#))
 - LCU14-107 - OP-TEE on ARMv8-A ([slides](#), [video](#))
 - LCU14-103 - How to create and run Trusted Applications on OP-TEE ([slides](#), [video](#))
- **LCA14**
 - LCA14-502 - The way to a generic TrustZone solution ([slides](#))
 - LCA14-418 - Testing a secure framework ([slides](#))

1.8 Releases

1.8.1 Cadence

New versions of OP-TEE are released four times a year, i.e., quarterly releases.

Release dates

Starting from version 3.10.0 we track the old and also show the releases being planned for the future in the table below. The dates will tell whether it is an old, upcoming or future release.

Version	Release date
OP-TEE 4.4.0	18/Oct/24
OP-TEE 4.3.0	12/Jul/24
OP-TEE 4.2.0	12/Apr/24
OP-TEE 4.1.0	19/Jan/24
OP-TEE 4.0.0	20/Oct/23
OP-TEE 3.22.0	14/Jul/23
OP-TEE 3.21.0	14/Apr/23
OP-TEE 3.20.0	20/Jan/23
OP-TEE 3.19.0	14/Oct/22
OP-TEE 3.18.0	15/July/22
OP-TEE 3.17.0	15/Apr/22
OP-TEE 3.16.0	28/Jan/22
OP-TEE 3.15.0	18/Oct/21
OP-TEE 3.14.0	16/July/21
OP-TEE 3.13.0	30/Apr/21
OP-TEE 3.12.0	20/Jan/21
OP-TEE 3.11.0	16/Oct/20
OP-TEE 3.10.0	21/Aug/20

1.8.2 Changelog

The changelog is stored in the [optee_os](#) git ([CHANGELOG.md](#)). There you can see what has been done between the different releases in terms of commits as well as pull requests.

1.8.3 Versioning schema

OP-TEE follows [Semantic Versioning 2.0.0](#). What that means in practice is well described at the link just shown.

1.8.4 Release procedure

There are certain steps that needs to be done when making a release. This checklist here serves as guidance to the one in charge of making a new release. Roughly start with this 2-3 weeks before the targeted release date.

tl;dr

Table 2: Short version of the OP-TEE release procedure

When (nus)	(Tmi- nus)	Action	Ex- am- ple
3w		Create release pull request	PR#3099
3w		Inform maintainers about upcoming release	
1w		Increment the revision number in mk/config.mk	CFG_OPTEE_REVIS ? = 3 CFG_OPTEE_REVIS ? = x
1w		Create release candidate tag in optee_* + build.git	git tag - a 3.x.y- rc1 - m “3.x.y- rc1”
1w		Let maintainers know about the release candidate tag	
1w		Test platform builds / devices	
Release day		Update CHANGELOG.md	changelog ex- am- ple
Release day		Collect/merge Tested-By tags	com- mit ex- am- ple
Release day		Create release tag in optee_* + build.git + linux.git	git tag - a 3.x.y - m “3.x.y” git tag - a optee- 3.x.y

Long version

1. Create a “release pull request” at GitHub ought to collect Tested-By tags from various maintainers. As an example, see [PR#3099](#).
2. Send email to all maintainers to let them know about the upcoming release. The addresses to the maintainers can be found in the [MAINTAINERS](#) file.

Hint: With this command you will get all email addresses

```
$ scripts/get_maintainer.py --release-to
```

3. Increment the revision number in *mk/config.mk*: `CFG_OPTEE_REVISION_MAJOR` and `CFG_OPTEE_REVISION_MINOR`. These values are made available to TAs and to the Normal World driver at boot time.
4. Create a release candidate (RC) tag (annotated tag, i.e., `git tag -a 3.x.y-rc1 -m "3.x.y-rc1"`) in the following gits `optee_*` and `build.git`. One way to do it is like this

```
$ export VER=3.x.y-rc1
$ for d in optee* build; do ( cd $d; git tag -a $VER -m $VER ); done
$ for d in optee* build; do ( cd $d; git push origin $VER ); done
```

5. Send a follow up email to all maintainers to let them know that there is a release tag ready to be tested on their devices for the platforms that they are maintaining.
6. In case major regressions are found, then fix those and create a another release candidate tag (i.e., repeat step 3 and 4 until there are no remaining issues left).
7. On release day: Update [CHANGELOG.md](#) see this [changelog example](#) to see how that should look like.
8. Collect all tags (Tested-By etc) from maintainers and use those in the commit message, for an example see this [commit example](#).
9. Create a release tag (annotated tag, i.e., `git tag -a 3.x.y -m "3.x.y"`) in the following gits `optee_*` and `build.git`. Tag the tip of the `optee` branch in `linux.git`, the name of the tag has to be prefixed with `optee-` to avoid confusions. For instance: `git tag -a optee-3.x.y -m "optee-3.x.y"`.

Hint: You can use the same steps as in step 4, when creating the tags.

10. Create a new branch in [manifest](#) from `master` where the name corresponds to the release you are preparing. I.e., `git checkout -b 3.x.y origin/master`.
11. Update all [manifest](#) XML-files in the [manifest](#) git, so they refer to the tag in the release we are working with (see [3.6.0 stable](#) commit as an example). This can be done with the [make_stable.sh](#) script. Now it is also time to push the new branch and tag it. Example:

```
$ export VER=3.x.y
$ cd manifest
$ ./make_stable.sh -o -r $VER
$ git diff # make sure everything looks good
$ git commit -a -m "OP-TEE $VER stable"
$ git remote add upstream git@github.com:OP-TEE/manifest
```

(continues on next page)

(continued from previous page)

```
$ git push upstream  
$ git tag -a $VER -m $VER  
$ git push upstream tag $VER
```

12. Send a last email to maintainers and other stakeholders telling that the release has been completed.

ARCHITECTURE

2.1 Core

2.1.1 Interrupt handling

This section describes how *optee_os* handles switches of world execution context based on *SMC* exceptions and interrupt notifications. Interrupt notifications are *IRQ/FIQ* exceptions which may also imply switching of world execution context: normal world to secure world, or secure world to normal world.

Use cases of world context switch

This section lists all the cases where OP-TEE OS is involved in world context switches. *Optee_os* executes in the secure world. World switch is done by the core's secure monitor level/mode, referred below as the Monitor.

When the normal world invokes the secure world, the normal world executes a *SMC* instruction. The *SMC* exception is always trapped by the Monitor. If the related service targets the trusted OS, the Monitor will switch to OP-TEE OS world execution. When the secure world returns to the normal world, OP-TEE OS executes a *SMC* that is caught by the Monitor which switches back to the normal world.

When a secure interrupt is signaled by the Arm GIC, it shall reach the OP-TEE OS interrupt exception vector. If the secure world is executing, OP-TEE OS will handle interrupt straight from its exception vector. If the normal world is executing when the secure interrupt raises, the Monitor vector must handle the exception and invoke OP-TEE OS to serve the interrupt.

When a non-secure interrupt is signaled by the Arm GIC, it shall reach the normal world interrupt exception vector. If the normal world is executing, it will handle straight the exception from its exception vector. If the secure world is executing when the non-secure interrupt raises, OP-TEE OS will temporarily return back to normal world via the Monitor to let normal world serve the interrupt.

Core exception vectors

Monitor vector is *VBAR_EL3* in AArch64 and *MVBAR* in Armv7-A/AArch32. Monitor can be reached while normal world or secure world is executing. The executing secure state is known to the Monitor through the *SCR_NS*.

Monitor can be reached from a *SMC* exception, an *IRQ* or *FIQ* exception (so-called interrupts) and from asynchronous aborts. Obviously monitor aborts (data, prefetch, undef) are local to the Monitor execution.

The Monitor can be external to OP-TEE OS (case *CFG_WITH_ARM_TRUSTED_FW=y*). If not, provides a local secure monitor *core/arch/arm/sm*. Armv7-A platforms should use the OP-TEE OS secure monitor. Armv8-A platforms are likely to rely on an [Trusted Firmware A](#).

When executing outside the Monitor, the system is executing either in the normal world (SCR_NS=1) or in the secure world (SCR_NS=0). Each world owns its own exception vector table (state vector):

- VBAR_EL2 or VBAR_EL1 non-secure or VBAR_EL1 secure for AArch64.
- HVBAR or VBAR non-secure or VBAR secure for Armv7-A and AArch32.

All SMC exceptions are trapped in the Monitor vector. IRQ/FIQ exceptions can be trapped either in the Monitor vector or in the state vector of the executing world.

When the normal world is executing, the system is configured to route:

- secure interrupts to the Monitor that will forward to OP-TEE OS
- non-secure interrupts to the executing world exception vector.

When the secure world is executing, the system is configured to route:

- secure and non-secure interrupts to the executing OP-TEE OS exception vector. OP-TEE OS shall forward the non-secure interrupts to the normal world.

Optee_os non-secure interrupts are always trapped in the state vector of the executing world. This is reflected by a static value of SCR_(IRQ|FIQ).

Native and foreign interrupts

Two types of interrupt are defined from OP-TEE OS point of view.

- **Native interrupt** - The interrupt handled by OP-TEE OS, secure interrupts targetting S-EL1 or secure privileged mode
- **Foreign interrupt** - The interrupt not handled by OP-TEE OS, non-secure interrupts targetting normal world or secure interrupts targetting EL3.

For Arm **GICv2** mode, a native interrupt is signalled with a FIQ and a foreign interrupt is signalled with an IRQ. For Arm **GICv3** mode, a foreign interrupts is signalled as a FIQ which could be handled by either secure world (aarch32 Monitor mode or aarch64 EL3) or normal world.

Arm GICv3 mode can be enabled by setting CFG_ARM_GICV3=y. Native interrupts must be securely routed to OP-TEE OS. Foreign interrupts, when trapped during secure world execution might need to be efficiently routed to the normal world.

IRQ and FIQ keeps their meaning in normal world so for clarity we will keep using those names in the normal world context.

Normal World invokes OP-TEE OS using SMC

Entering the Secure Monitor

The monitor manages all entries and exits of secure world. To enter secure world from normal world the monitor saves the state of normal world (general purpose registers and system registers which are not banked) and restores the previous state of secure world. Then a return from exception is performed and the restored secure state is resumed. Exit from secure world to normal world is the reverse.

Some general purpose registers are not saved and restored on entry and exit, those are used to pass parameters between secure and normal world (see [ARM_DEN0028A_SMC_Calling_Convention](#) for details).

Entry and exit of Trusted OS

On entry and exit of Trusted OS each CPU is uses a separate entry stack and runs with IRQ and FIQ masked. SMCs are categorised in two flavors: **fast** and **yielding**.

- For **fast** SMCs, OP-TEE OS will execute on the entry stack with IRQ/FIQ masked until the execution returns to normal world.
- For **yielding** SMCs, OP-TEE OS will at some point execute the requested service with interrupts unmasked. In order to handle interrupts, mainly forwarding of foreign interrupts, OP-TEE OS assigns a trusted thread ([core/arch/arm/kernel/thread.c](#)) to the SMC request. The trusted thread stores the execution context of the requested service. This context can be suspended and resumed as the requested service executes and is interrupted. The trusted thread is released only once the service execution returns with a completion status.

For **yielding** SMCs, OP-TEE OS allocates or resumes a trusted thread then unmask the IRQ and FIQ lines. When the OP-TEE OS needs to invoke the normal world from a foreign interrupt or a remote service call, OP-TEE OS masks IRQ and FIQ and suspends the trusted thread. When suspending, OP-TEE OS gets back to the entry stack.
- **Both** fast and yielding SMCs end on the entry stack with IRQ and FIQ masked and OP-TEE OS invokes the Monitor through a SMC to return to the normal world.

Deliver non-secure interrupts to Normal World

Forward a Foreign Interrupt from Secure World to Normal World

When a foreign interrupt is received in secure world as an IRQ or FIQ exception then secure world:

1. Saves trusted thread context (entire state of all processor modes for Armv7-A)
2. Masks all interrupts (IRQ and FIQ)
3. Switches to entry stack
4. Issues an SMC with a value to indicate to normal world that an IRQ has been detected and last SMC call should be continued

The monitor restores normal world context with a return code indicating that an IRQ is about to be delivered. Normal world issues a new SMC indicating that it should continue last SMC.

The monitor restores secure world context which locates the previously saved context and checks that it is a return from a foreign interrupt that is requested before restoring the context and lets the secure world foreign interrupt handler return from exception where the execution would be resumed.

Note that the monitor itself does not know or care that it has just forwarded a foreign interrupt to normal world. The bookkeeping is done in the trusted thread handling in OP-TEE OS. Normal world is responsible to decide when the secure world thread should resume execution (for details, see [Thread handling](#)).

Deliver a foreign interrupt to normal world when ``SCR\_NS`` is set

Since SCR_IRQ is cleared, an IRQ will be delivered using the exception vector (VBAR) in the normal world. The IRQ is received as any other exception by normal world, the monitor and the OP-TEE OS are not involved at all.

Deliver secure interrupts to Secure World

A secure (foreign) interrupt can be received during two different states, either in normal world (SCR_NS is set) or in secure world (SCR_NS is cleared). When the secure monitor is active (Armv8-A EL3 or Armv7-A Monitor mode) FIQ and IRQ are masked. FIQ reception in the two different states is described below.

Deliver secure interrupt to secure world when SCR_NS is set

When the monitor traps a secure interrupt it:

1. Saves normal world context and restores secure world context from last secure world exit (which will have IRQ and FIQ blocked)

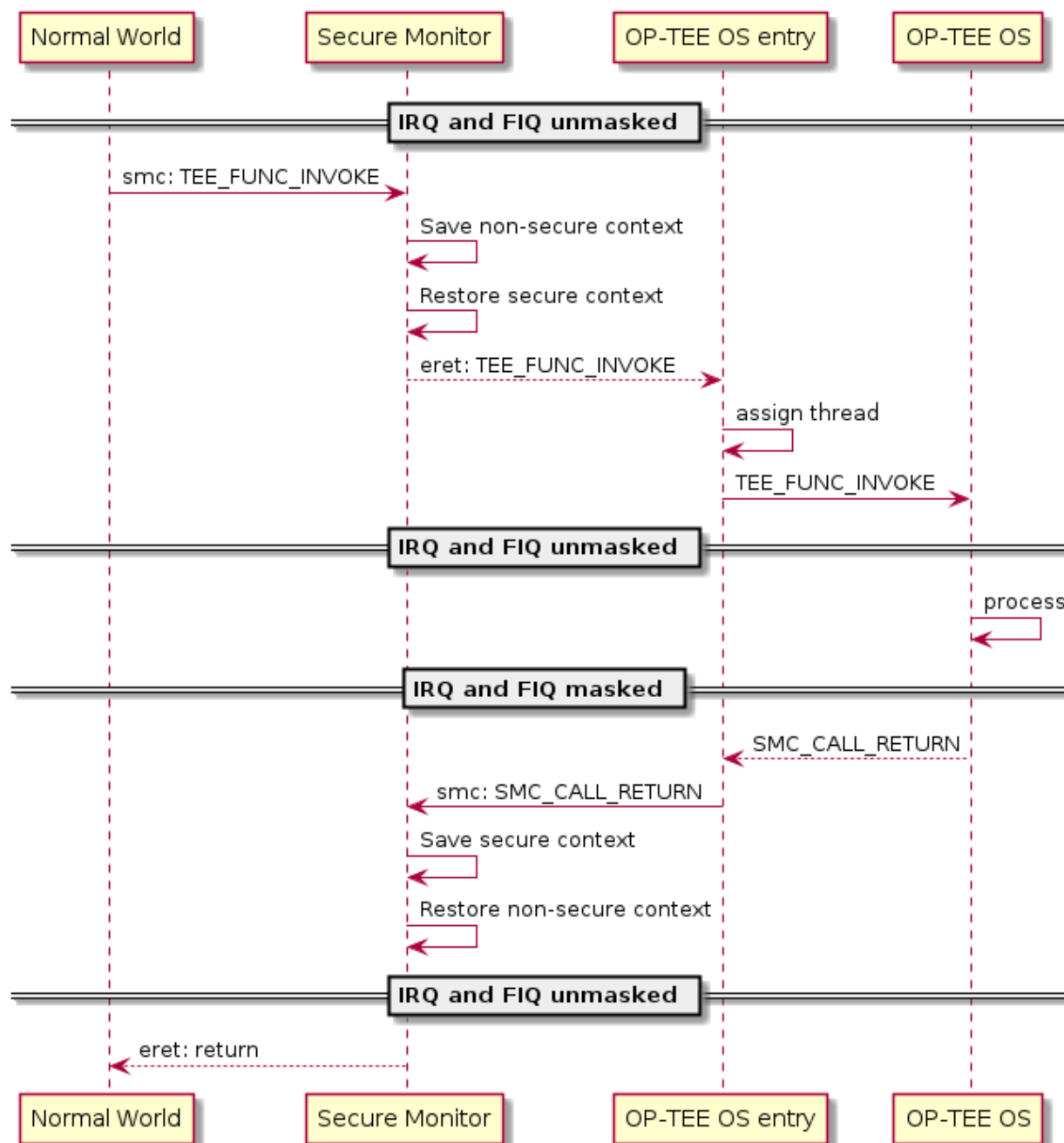


Fig. 1: SMC entry to secure world

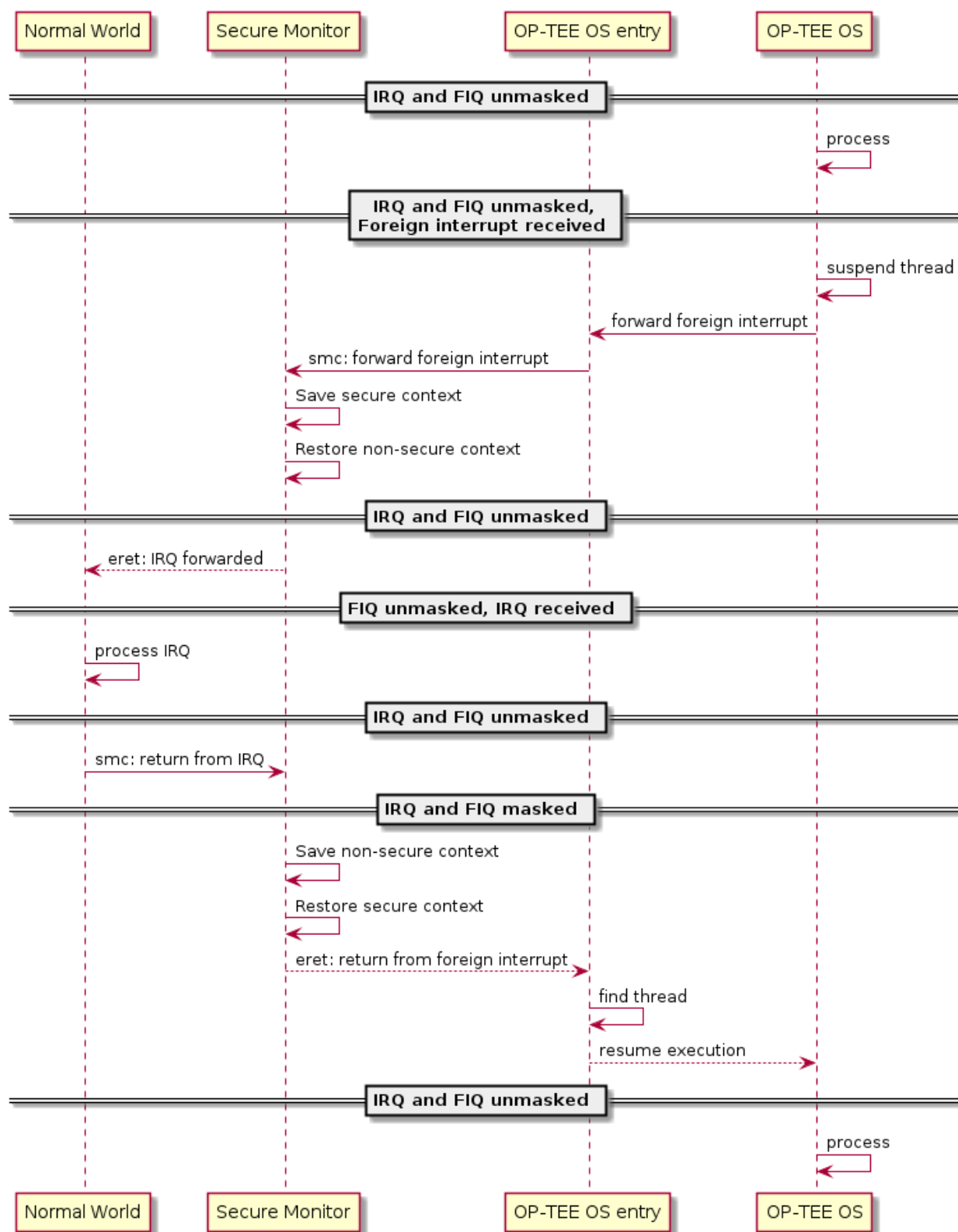


Fig. 2: Foreign interrupt received in secure world and forwarded to normal world

2. Clears SCR_FIQ when clearing SCR_NS
3. Does a return from exception into OP-TEE OS via the secure interrupt entry point
4. OP-TEE OS handles the native interrupt directly in the entry point
5. OP-TEE OS issues an SMC to return to normal world
6. The monitor saves the secure world context and restores the normal world context
7. Does a return from exception into the restored context

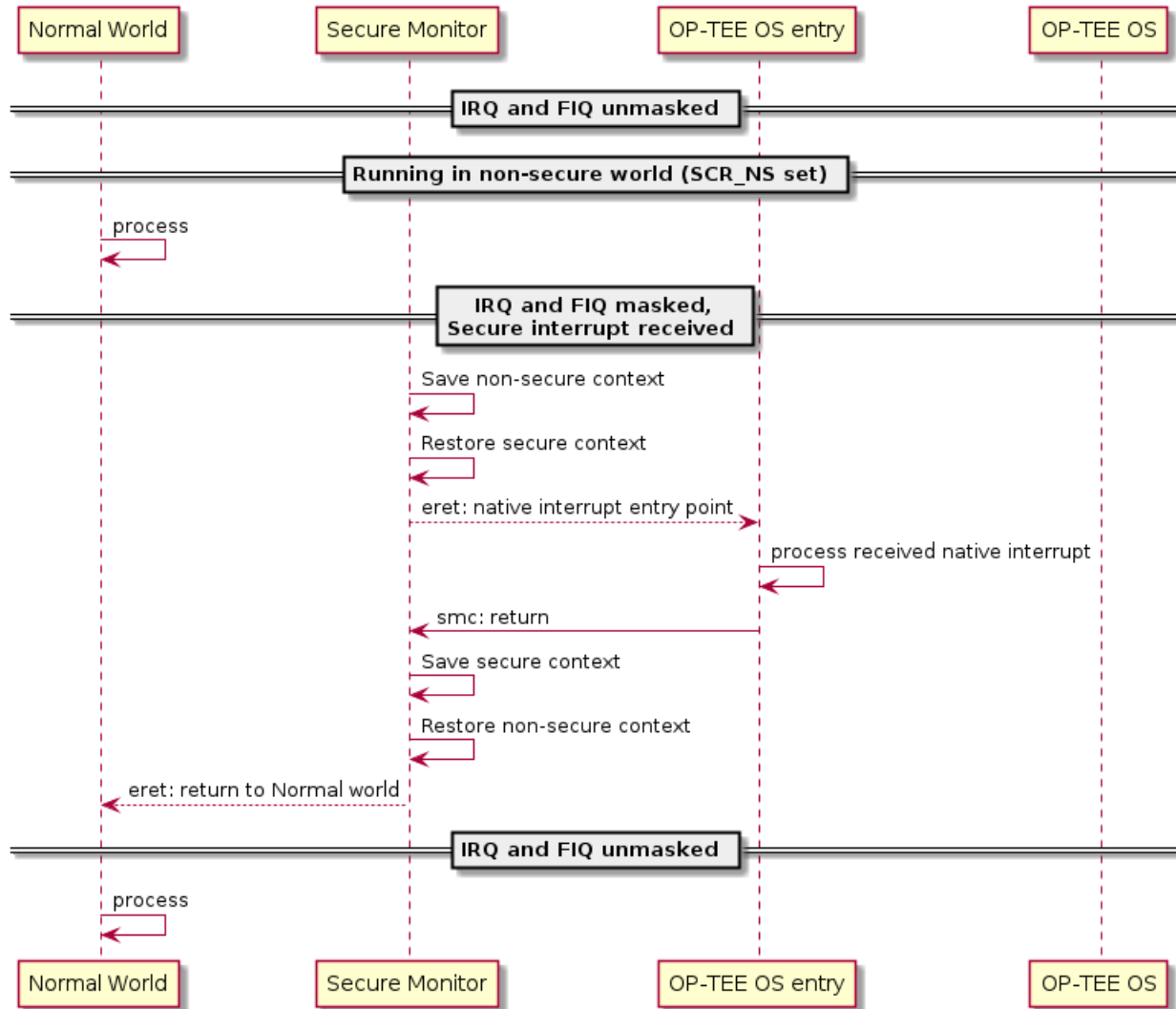


Fig. 3: Secure interrupt received when SCR_NS is set

Deliver FIQ to secure world when SCR_NS is cleared

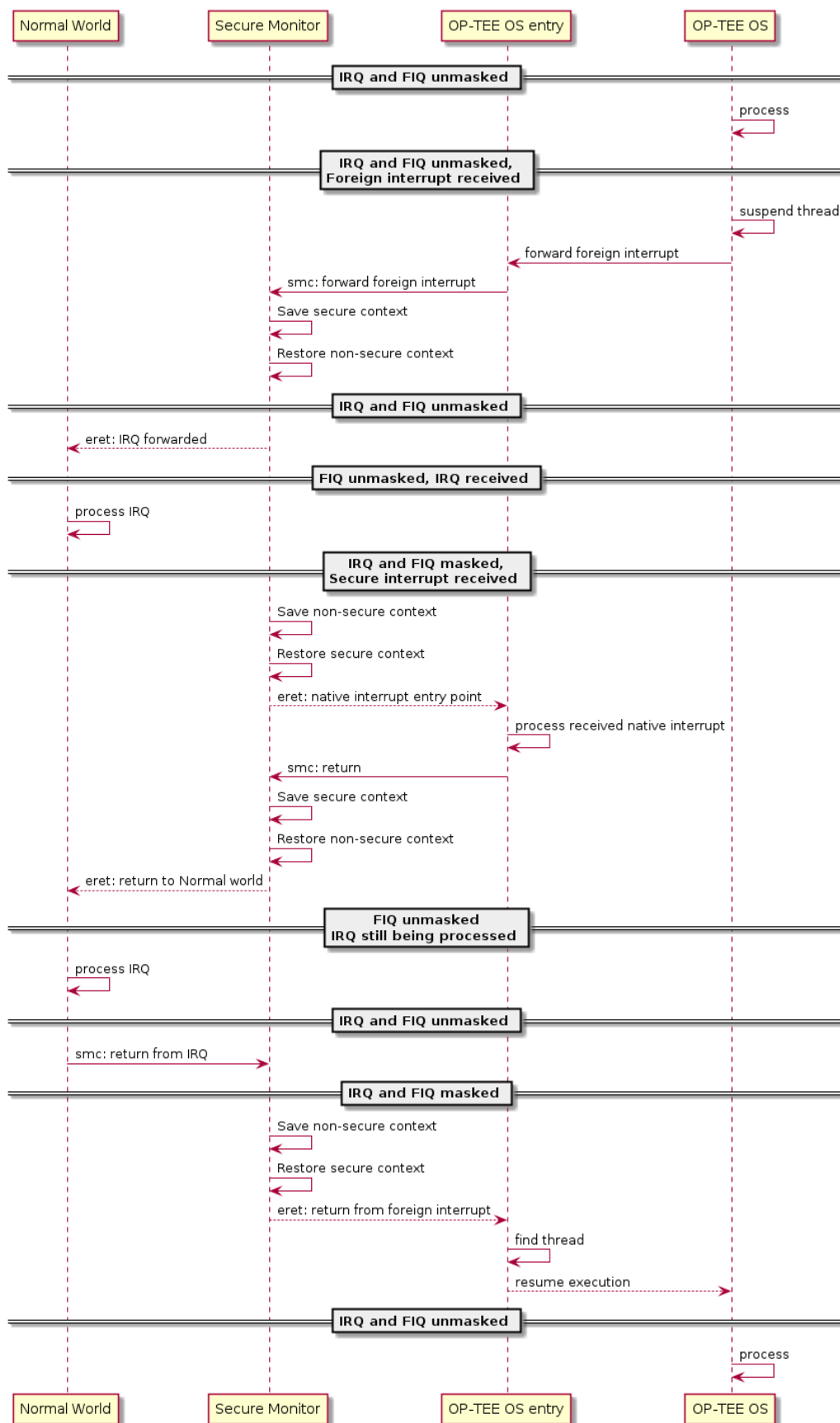


Fig. 4: FIQ received while processing an IRQ forwarded from secure world

Trusted thread scheduling

Trusted thread for standard services

OP-TEE yielding services are carried through standard SMC. Execution of these services can be interrupted by foreign interrupts. To suspend and restore the service execution, optee_os assigns a trusted thread at yielding SMC entry.

The trusted thread terminates when optee_os returns to the normal world with a service completion status.

A trusted thread execution can be interrupted by a native interrupt. In this case the native interrupt is handled by the interrupt exception handlers and once served, optee_os returns to the execution trusted thread.

A trusted thread execution can be interrupted by a foreign interrupt. In this case, optee_os suspends the trusted thread and invokes the normal world through the Monitor (optee_os so-called RPC services). The trusted threads will resume only once normal world invokes the optee_os with the RPC service status.

A trusted thread execution can lead optee_os to invoke a service in normal world: access a file, get the REE current time, etc. The trusted thread is first suspended then resumed during remote service execution.

Scheduling considerations

When a trusted thread is interrupted by a foreign interrupt and when optee_os invokes a normal world service, the normal world gets the opportunity to reschedule the running applications. The trusted thread will resume only once the client application is scheduled back. Thus, a trusted thread execution follows the scheduling of the normal world caller context.

Optee_os does not implement any thread scheduling. Each trusted thread is expected to track a service that is invoked from the normal world and should return to it with an execution status.

The OP-TEE Linux driver (as implemented in `drivers/tee/optee` since Linux kernel 4.12) is designed so that the Linux thread invoking OP-TEE gets assigned a trusted thread on TEE side. The execution of the trusted thread is tied to the execution of the caller Linux thread which is under the Linux kernel scheduling decision. This means trusted threads are scheduled by the Linux kernel.

Trusted thread constraints

TEE core handles a static number of trusted threads, see `CFG_NUM_THREADS`.

Trusted threads are expensive on memory constrained system, mainly because of the execution stack size.

On SMP systems, optee_os can execute several trusted threads in parallel if the normal world supports scheduling of processes. Even on UP systems, supporting several trusted threads in optee_os helps normal world scheduler to be efficient.

Core handlers for native interrupts

OP-TEE core provides methods for device drivers to setup and register handler functions for native interrupt controller drivers (see: `ref: native_foreign_irqs`). Interrupt handlers can be nested as when an interrupt controller exposes interrupts which signaling is multiplexed on an interrupt controlled by a parent interrupt controller.

Interrupt controllers are represented by an instance of `struct itr_chip`. An interrupt controller exposes a given number of interrupts, each identified by an index from 0 to N-1 where N is the total number of interrupts exposed by that controller. In the literature, an interrupt index identifier is called interrupt number.

Interrupt management API functions

Interrupt management resources are declared in header file `interrupt.h`. Interrupt consumers main API functions are:

- `interrupt_enable()` and `interrupt_disable()` to respectively enable or disable an interrupt.

- `interrupt_mask()` and `interrupt_unmask()` to respectively mask or unmask an interrupt. Masking of an enabled interrupt temporarily disables the interrupt while unmasking enables a previously masked interrupt. `interrupt_mask()` and `interrupt_unmask()` are allowed to be called from an interrupt context, but `interrupt_enable()` and `interrupt_disable()` not so.
- `interrupt_configure()` to configure an interrupt detection mode and priority.
- `interrupt_add_handler()` to configure an interrupt and register an interrupt handler function, see below.
- `interrupt_remove_handler()` to unregister an interrupt handler function from an interrupt.

Interrupt controller drivers

An interrupt controller instance, named chip (`struct itr_chip`) defines operation function handlers for management of the interrupt(s) it controls. An interrupt chip driver must provide operation handler functions `.add`, `.mask`, `.unmask`, `.enable` and `.disable`. There are other operation handler functions that are optional, as for example `.raise_pi`, `.raise_sgi` and `.set_priority`.

An interrupt chip driver registers the controller instance with API function `itr_chip_init()`. The driver calls the registered interrupt consumer(s) handler(s) with API function `interrupt_call_handlers()`.

CPU main interrupt controller driver

The CPU interrupt controller (e.g. a GIC instance on Arm architecture CPUs) is called the main interrupt controller. Its driver must register as main controller using API function `interrupt_main_init()`. The function is in charge of calling `itr_chip_init()` for that chip instance.

Interrupt consumer drivers can get a reference to the main interrupt controller with the API function `interrupt_get_main_chip()`.

Interrupt handlers

Interrupt handler functions are callback functions registered by interrupt consumer drivers that core shall call when the related interrupt occurs. Structure `struct itr_handler` references a handler. It contains the handler function entry point, the interrupt number, the interrupt controller device and a few more parameters.

An interrupt handler function return value is of type `enum itr_return`. It shall return `ITRR_HANDLED` when the interrupt is served and `ITRR_NONE` when the interrupt cannot be served.

The interrupt handler runs in an interrupt context rather than a thread context. When this occurs, all other interrupts are masked, necessitating fast execution of the interrupt handler to avoid delaying or missing out on other interrupts. When an interrupt occurs that requires the completion of long-running operations, the interrupt handler should request the OP-TEE bottom half thread (see [Notifications](#)) to execute those operations.

API function `interrupt_add_handler()`, `interrupt_add_handler_with_chip()` and `interrupt_alloc_add_handler()` configure and register a handler function to a given interrupt.

API function `interrupt_remove_handler()` and `interrupt_remove_free_handler()` unregister a registered handler.

Interrupt consumer driver

A typical implementation of a driver consuming an interrupt includes retrieving of the interrupt resource (interrupt controller and interrupt number in that controller), configuring the interrupt, registering a handler for the interrupt and enabling/disabling the interrupt.

For example, the dummy driver below prints a debug trace when the related interrupt occurs:

```
static struct itr_handler *foo_int1_handler;

static struct foo_int1_data = {
    /* field with some interrupt handler private data */
}
```

(continues on next page)

(continued from previous page)

```

};

static enum itr_return foo_it_handler_fn(struct itr_handler *h)
{
    foo_acknowledge_interrupt(h->it);
    DMSG("Interrupt FOO%u served", h->it);

    return ITRR_HANDLED;
}

static TEE_Result foo_initialization(void)
{
    TEE_Result res = TEE_ERROR_GENERIC;

    res = interrupt_alloc_add_handler(itr_core_get(),
                                     GIC_INT_F00,
                                     foo_it_handler_fn,
                                     ITRF_TRIGGER_LEVEL,
                                     &foo_int1_data,
                                     &foo_int1_handler);

    if (res)
        return res;

    interrupt_enable(itr_chip, it_num);

    return TEE_SUCCESS;
}

static void foo_release(void)
{
    if (foo_int1_handler) {
        interrupt_disable(foo_int1_handler->chip,
                          foo_int1_handler->it);

        interrupt_remove_free_handler(&foo_int1_handler);
    }
}

```

2.1.2 Notifications

There are two kinds of notifications that secure world can use to make normal world aware of some event.

1. Synchronous notifications delivered with `OPTEE_RPC_CMD_NOTIFICATION` using the `OPTEE_RPC_NOTIFICATION_SEND` parameter.
2. Asynchronous notifications delivered with a combination of a non-secure interrupt and a fast call from the non-secure interrupt handler.

Secure world can wait in normal for a notification to arrive. This allows the calling thread to sleep instead of spinning when waiting for something. This happens for instance when a thread waits for a mutex to become available.

Synchronous notifications are limited by depending on RPC for delivery, this is only usable from a normal thread context. Secure interrupt handler or other atomic context cannot use synchronous notifications due to this.

Asynchronous notifications uses a platform specific way of triggering a non-secure interrupt. This is done with `itr_raise_pi()` in a way suitable for a secure interrupt handler or another atomic context. This is useful when using a top half and bottom half kind of design in a device driver. The top half is done in the secure interrupt handler which then triggers normal world to make a yielding call into secure world to do the bottom half processing.

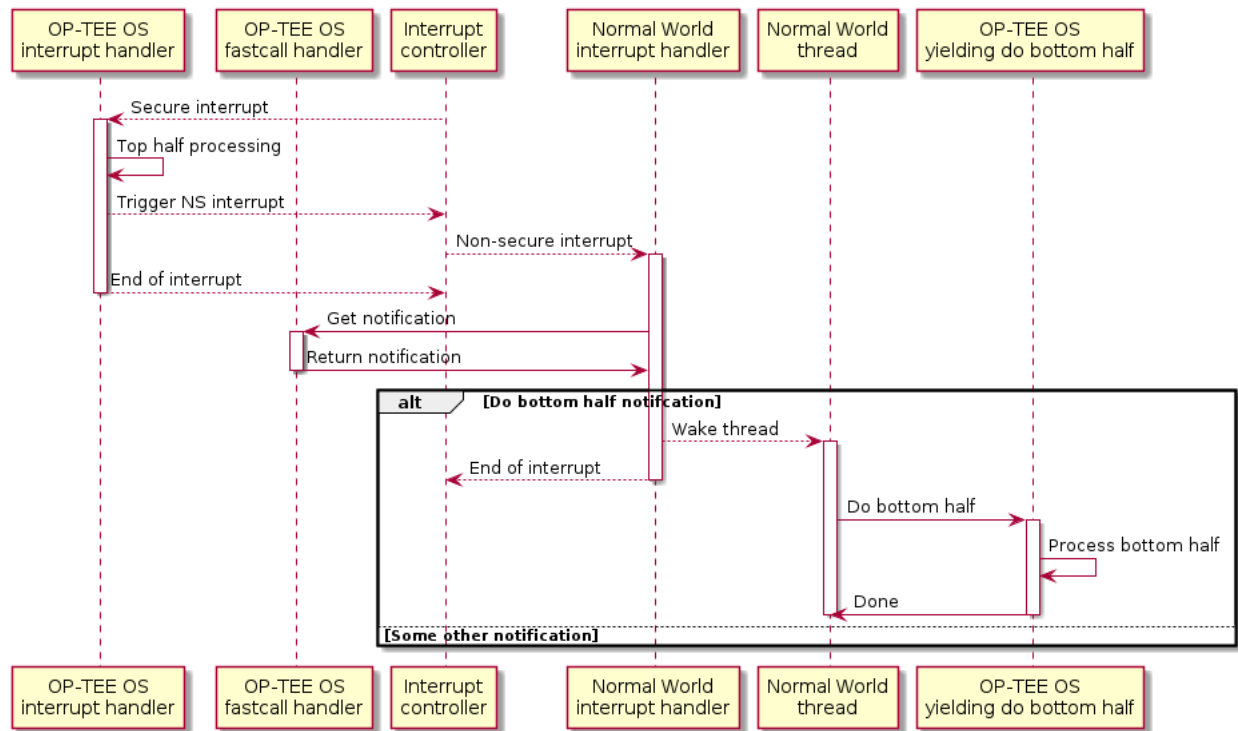


Fig. 5: Top half, bottom half example

Notifications are identified with a value, allocated as:

0 - 63

Mixed asynchronous and synchronous range

64 - Max

Synchronous only range

If the **Max** value is smaller than 63, then there's only the mixed range.

If asynchronous notifications are enabled then is the value 0 reserved for signalling the a driver need a bottom half call, that is the yielding call `OPTEE_MSG_CMD_DO_BOTTOM_HALF`.

The rest of the asynchronous notification values are managed with two functions `notif_alloc_async_value()` and `notif_free_async_value()`.

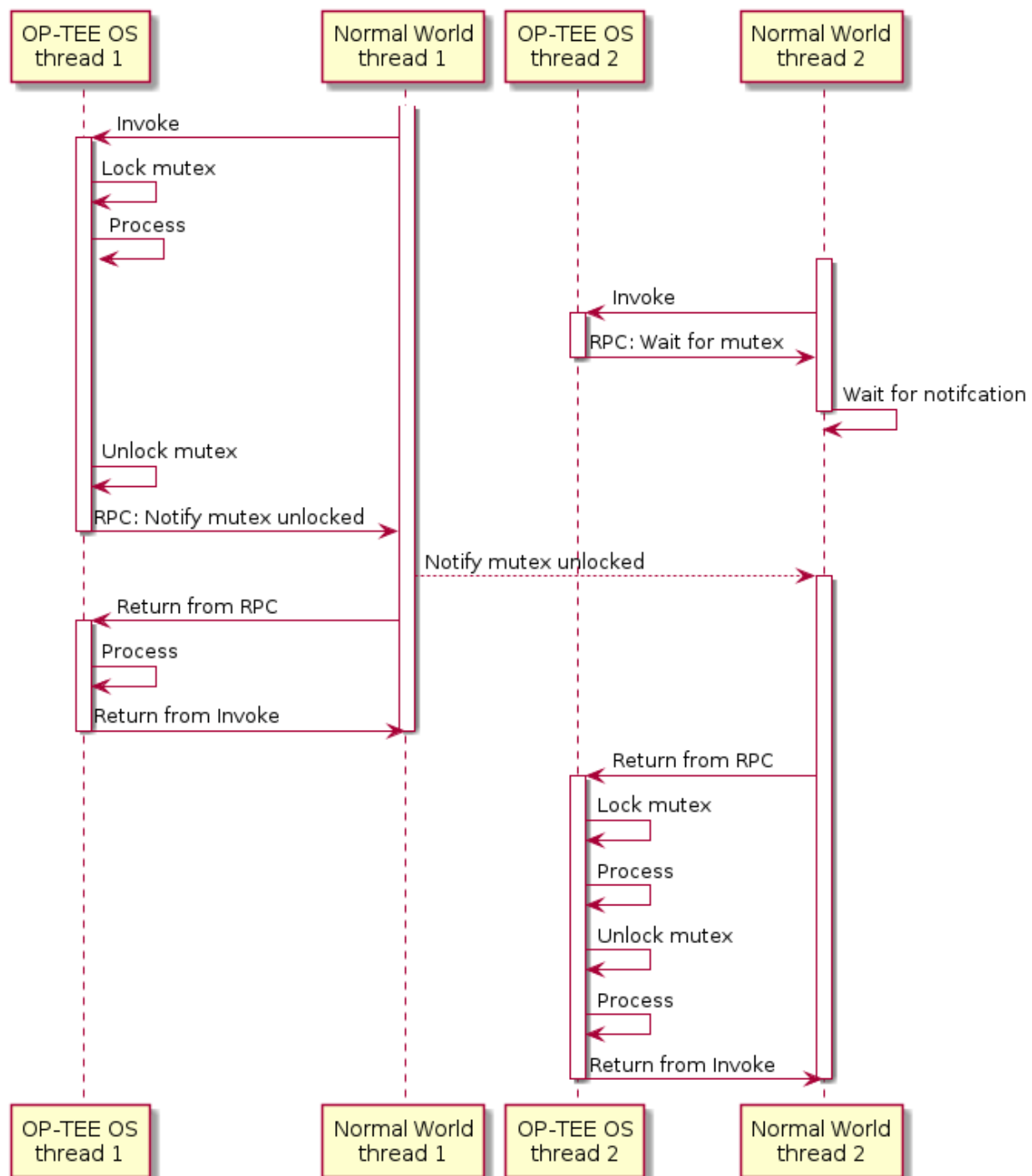


Fig. 6: Synchronous example

2.1.3 Memory objects

A memory object, **MOBJ**, describes a piece of memory. The interface provided is mostly abstract when it comes to using the MOBJ to populate translation tables etc. There are different kinds of MOBJs describing:

- **Physically contiguous memory**
 - created with `mobj_phys_alloc(...)`.
 - **Virtual memory**
 - one instance with the name `mobj_virt` available.
 - spans the entire virtual address space.
 - **Physically contiguous memory allocated from a `tee_mm_pool_t` ***
 - created with `mobj_mm_alloc(...)`.
 - **Paged memory**
 - created with `mobj_paged_alloc(...)`.
 - only contains the supplied size and makes `mobj_is_paged(...)` return true if supplied as argument.
 - **Secure copy paged shared memory**
 - created with `mobj_seccpy_shm_alloc(...)`.
 - makes `mobj_is_paged(...)` and `mobj_is_secure(...)` return true if supplied as argument.
-

2.1.4 MMU

Translation tables

OP-TEE supports two translation table formats:

1. Short-descriptor translation table format, available on ARMv7-A and ARMv8-A AArch32
2. Long-descriptor translation format, available on ARMv7-A with LPAE and ARMv8-A

ARMv7-A without LPAE (Large Physical Address Extension) must use the short-descriptor translation table format only. ARMv8-A AArch64 must use the long-descriptor translation format only.

Translation table format is a static build time configuration option, `CFG_WITH_LPAE`. The design around the translation table handling has been centered around these factors:

1. Share translation tables between CPUs when possible to save memory and simplify paging
2. Support non-global CPU specific mappings to allow executing different TAs in parallel.

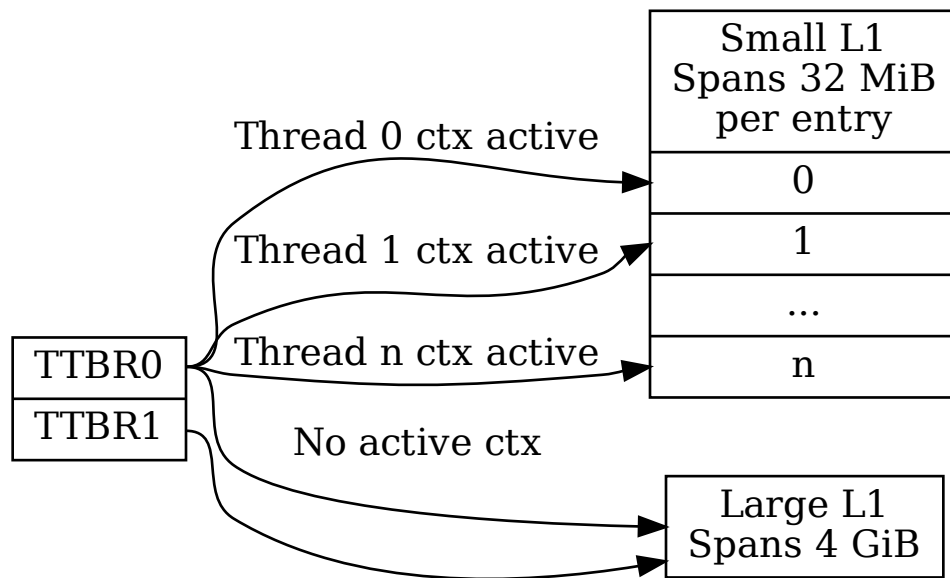
Short-descriptor translation table format

Several L1 translation tables are used, one large spanning 4 GiB and two or more small tables spanning 32 MiB. The large translation table handles kernel mode mapping and matches all addresses not covered by the small translation tables. The small translation tables are assigned per thread and covers the mapping of the virtual memory space for one TA context.

Memory space between small and large translation table is configured by TTBCR. TTBR1 always points to the large translation table. TTBR0 points to the a small translation table when user mapping is active and to the large translation table when no user mapping is currently active. For details about registers etc, please refer to a Technical Reference Manual for your architecture, for example [Cortex-A53 TRM](#).

The translation tables has certain alignment constraints, the alignment (of the physical address) has to be the same as the size of the translation table. The translation tables are statically allocated to avoid fragmentation of memory due to the alignment constraints.

Each thread has one small L1 translation table of its own. Each TA context has a compact representation of its L1 translation table. The compact representation is used to initialize the thread specific L1 translation table when the TA context is activated.



Long-descriptor translation table format

Each CPU is assigned a L1 translation table which is programmed into Translation Table Base Register 0 (TTBR0 or TTBR0_EL1 as appropriate).

L1 and L2 translation tables are statically allocated and initialized at boot. Normally there is only one shared L2 table, but with ASLR enabled the virtual address space used for the shared mapping may need to use two tables. An unused entry in the L1 table is selected to point to the per thread L2 table. With ASLR configured this means that different per thread entry may be selected each time the system boots. Note that this entry will only point to a table when the per thread mapping is activated.

The L2 translation tables in their turn point to L3 tables which use the small page granularity of 4 KiB. The shared mappings has the L3 tables initialized too at boot, but the per thread L3 tables are dynamic and are only assigned when the mapping is activated.

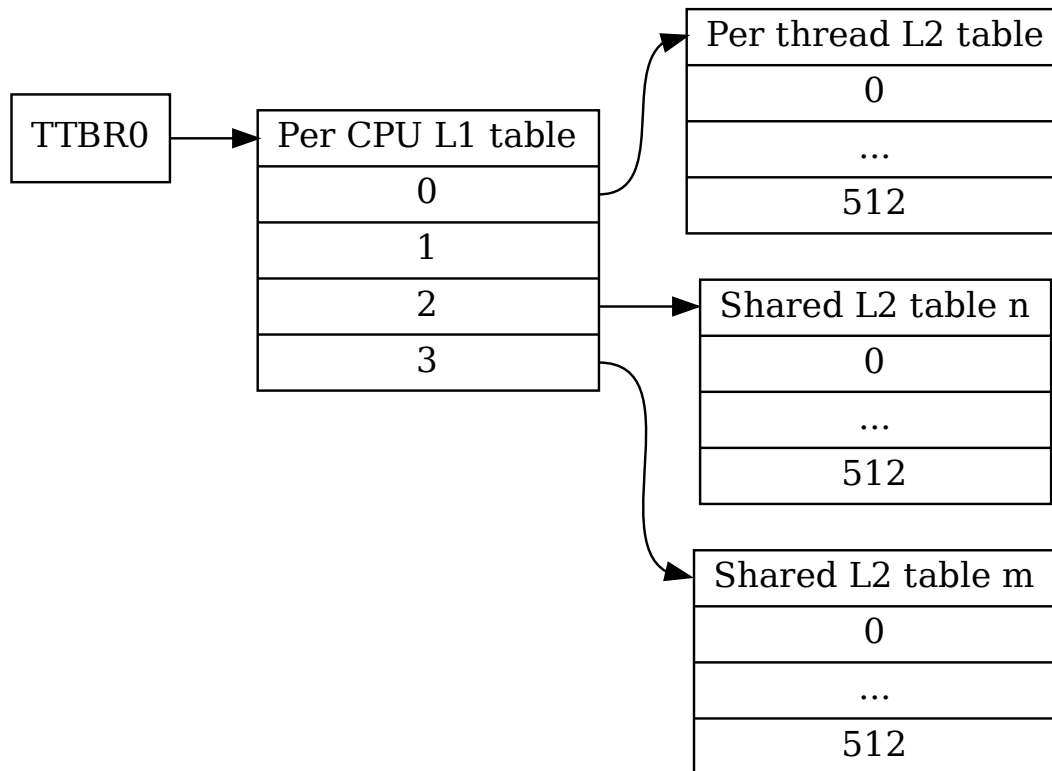


Fig. 7: Example translation table setup with 4GiB virtual address space with L3 tables excluded

Page table cache

Page tables used to map TAs are managed with the page table cache. When the context of a TA is unmapped, all its page tables are released with a call to `pgt_free()`. All page tables needed when mapping a TA are allocated using `pgt_alloc()`.

A fixed maximum number of translation tables are available in a pool. One thread may execute a TA which needs all or almost all tables. This can block TAs from being executed by other threads. To ensure that all TAs eventually will be permitted to execute `pgt_alloc()` temporarily frees eventual tables allocated before waiting for tables to become available.

The page table cache behaves differently depending on configuration options.

Without paging (`CFG_WITH_PAGER=n`)

This is the easiest configuration. All page tables are statically allocated in the `.nozi.pgt_cache` section. `pgt_alloc()` allocates tables from the free-list and `pgt_free()` returns the tables directly to the free-list.

With paging enabled (`CFG_WITH_PAGER=y`)

Page tables are allocated as zero initialized locked pages during boot using `tee_pager_alloc()`. Locked pages are populated with physical pages on demand from the pager. The physical page can be released when not needed any longer with `tee_pager_release_phys()`.

With `CFG_WITH_LPAE=y` each translation table has the same size as a physical page which makes it easy to release the physical page when the translation table isn't needed any longer. With the short-descriptor table format (`CFG_WITH_LPAE=n`) it becomes more complicated as four translation tables are stored in each page. Additional book-keeping is used to tell when the page for used by four separate translation tables can be released.

With paging of user TA enabled (`CFG_PAGED_USER_TA=y`)

With paging of user TAs enabled a cache of recently used translation tables is used. This can save us from a storm of page faults when restoring the mappings of a recently unmapped TA. Which translation tables should be cached is indicated with reference counting by the pager on used tables. When a table needs to be forcefully freed `tee_pager_pgt_save_and_release_entries()` is called to let the pager know that the table can't be used any longer.

When a mapping in a TA is removed it also needs to be purged from cached tables with `pgt_flush_ctx_range()` to prevent old mappings from being accidentally reused.

Switching to user mode

This section only applies with following configuration flags:

- `CFG_WITH_LPAE=n`
- `CFG_CORE_UNMAP_CORE_AT_EL0=y`

When switching to user mode only a minimal kernel mode mapping is kept. This is achieved by selecting a zeroed out big L1 translation in TTBR1 when transitioning to user mode. When returning back to kernel mode the original L1 translation table is restored in TTBR1.

Switching to normal world

When switching to normal world either via a foreign interrupt (see *Native and foreign interrupts* or RPC there is a chance that secure world will resume execution on a different CPU. This means that the new CPU need to be configured with the context of the currently active TA. This is solved by always setting the TA context in the CPU when resuming execution.

2.1.5 Pager

OP-TEE currently requires >256 KiB RAM for OP-TEE kernel memory. This is not a problem if OP-TEE uses TrustZone protected DDR, but for security reasons OP-TEE may need to use TrustZone protected SRAM instead. The amount of available SRAM varies between platforms, from just a few KiB up to over 512 KiB. Platforms with just a few KiB of SRAM cannot be expected to be able to run a complete TEE solution in SRAM. But those with 128 to 256 KiB of SRAM can be expected to have a capable TEE solution in SRAM. The pager provides a solution to this by demand paging parts of OP-TEE using virtual memory.

Secure memory

TrustZone protected SRAM is generally considered more secure than TrustZone protected DRAM as there is usually more attack vectors on DRAM. The attack vectors are hardware dependent and can be different for different platforms.

Backing store

TrustZone protected DRAM or in some cases non-secure DRAM is used as backing store. The data in the backing store is integrity protected with one hash (SHA-256) per page (4KiB). Readonly pages are not encrypted since the OP-TEE binary itself is not encrypted.

Partitioning of memory

The code that handles demand paging must always be available as it would otherwise lead to deadlock. The virtual memory is partitioned as:

Type	Sections
unpaged	text rodata data bss heap1 nozi heap2
init / paged	text_init rodata_init
paged	text_pageable rodata_pageable
demand alloc	

Where `nozi` stands for “not zero initialized”, this section contains entry stacks (thread stack when TEE pager is not enabled) and translation tables (TEE pager cached translation table when the pager is enabled and LPAE MMU is used).

The `init` area is available when OP-TEE is initializing and contains everything that is needed to initialize the pager. After the pager has been initialized this area will be used for demand paged instead.

The `demand alloc` area is a special area where the pages are allocated and removed from the pager on demand. Those pages are returned when OP-TEE does not need them any longer. The thread stacks currently belongs this area. This means that when a stack is not used the physical pages can be used by the pager for better performance.

The technique to gather code in the different area is based on compiling all functions and data into separate sections. The unpaged text and rodata is then gathered by linking all object files with `--gc-sections` to eliminate sections that are outside the dependency graph of the entry functions for unpaged functions. A script analyzes this ELF file and generates the bits of the final link script. The process is repeated for init text and rodata. What is not “unpaged” or “init” becomes “paged”.

Partitioning of the binary

Note: The struct definitions provided in this section are explicitly covered by the following dual license:

SPDX-License-Identifier: (BSD-2-Clause OR GPL-2.0)

The binary is partitioned into four parts as:

Binary
Header
Init
Hashes
Pageable

The header is defined as:

```
#define OPTEE_MAGIC          0x4554504f
#define OPTEE_VERSION        1
#define OPTEE_ARCH_ARM32     0
#define OPTEE_ARCH_ARM64     1

struct optee_header {
    uint32_t magic;
    uint8_t version;
    uint8_t arch;
    uint16_t flags;
    uint32_t init_size;
    uint32_t init_load_addr_hi;
    uint32_t init_load_addr_lo;
    uint32_t init_mem_usage;
    uint32_t paged_size;
};
```

The header is only used by the loader of OP-TEE, not OP-TEE itself. To initialize OP-TEE the loader loads the complete binary into memory and copies what follows the header and the following `init_size` bytes to `(init_load_addr_hi << 32 | init_load_addr_lo)`. `init_mem_usage` is used by the loader to be able to check that there is enough physical memory available for OP-TEE to be able to initialize at all. The loader supplies in `r0/x0` the address of the first byte following what was not copied and jumps to the load address to start OP-TEE.

In addition to overall binary with partitions inside described as above, three extra binaries are generated simultaneously during build process for loaders who support loading separate binaries:

v2 binary
Header

v2 binary
Init
Hashes

v2 binary

Pageable

In this case, loaders load header binary first to get image list and information of each image; and then load each of them into specific load address assigned in structure. These binaries are named with v2 suffix to distinguish from the existing binaries. Header format is updated to help loaders loading binaries efficiently:

```
#define OPTEE_IMAGE_ID_PAGER    0
#define OPTEE_IMAGE_ID_PAGED   1

struct optee_image {
    uint32_t load_addr_hi;
    uint32_t load_addr_lo;
    uint32_t image_id;
    uint32_t size;
};

struct optee_header_v2 {
    uint32_t magic;
    uint8_t version;
    uint8_t arch;
    uint16_t flags;
    uint32_t nb_images;
    struct optee_image optee_image[];
};
```

Magic number and architecture are identical as original. Version is increased to two. `load_addr_hi` and `load_addr_lo` may be `0xFFFFFFFF` for pageable binary since pageable part may get loaded by loader into dynamic available position. `image_id` indicates how loader handles current binary. Loaders who don't support separate loading just ignore all v2 binaries.

Initializing the pager

The pager is initialized as early as possible during boot in order to minimize the “init” area. The global variable `tee_mm_vcore` describes the virtual memory range that is covered by the level 2 translation table supplied to `tee_pager_init(...)`.

Assign pageable areas

A virtual memory range to be handled by the pager is registered with a call to `tee_pager_add_core_area()`.

```
bool tee_pager_add_area(tee_mm_entry_t *mm,
                       uint32_t flags,
                       const void *store,
                       const void *hashes);
```

which takes a pointer to `tee_mm_entry_t` to tell the range, flags to tell how memory should be mapped (readonly, execute etc), and pointers to backing store and hashes of the pages.

Assign physical pages

Physical SRAM pages are supplied by calling `tee_pager_add_pages(...)`

```
void tee_pager_add_pages(tee_vaddr_t vaddr,
                        size_t npages,
                        bool unmap);
```

`tee_pager_add_pages(...)` takes the physical address stored in the entry mapping the virtual address `vaddr` and `npages` entries after that and uses it to map new pages when needed. The `unmap` parameter tells whether the pages should be unmapped immediately since they does not contain initialized data or be kept mapped until they need to be recycled. The pages in the “init” area are supplied with `unmap == false` since those page have valid content and are in use.

Invocation

The pager is invoked as part of the abort handler. A pool of physical pages are used to map different virtual addresses. When a new virtual address needs to be mapped a free physical page is mapped at the new address, if a free physical page cannot be found the oldest physical page is selected instead. When the page is mapped new data is copied from backing store and the hash of the page is verified. If it is OK the pager returns from the exception to resume the execution.

Data structures

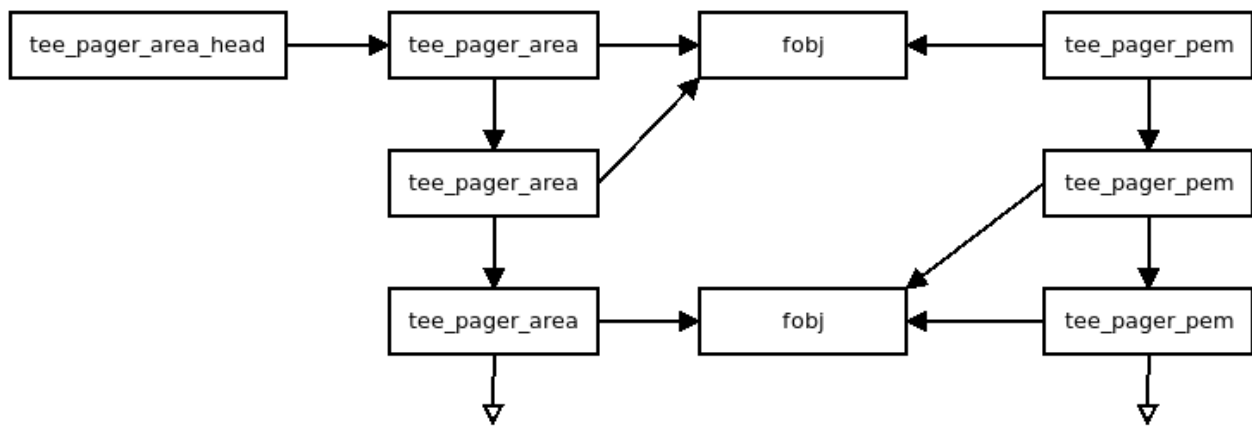


Fig. 8: How the main pager data structures relates to each other

struct tee_pager_area

This is a central data structure when handling paged memory ranges. It's defined as:

```
struct tee_pager_area {
    struct fobj *fobj;
    size_t fobj_pgoffs;
    enum tee_pager_area_type type;
    uint32_t flags;
    vaddr_t base;
```

(continues on next page)

(continued from previous page)

```

size_t size;
struct pgt *pgt;
TAILQ_ENTRY(tee_pager_area) link;
TAILQ_ENTRY(tee_pager_area) fobj_link;
};

```

Where base and size tells the memory range and fobj and fobj_pgoffs holds the content. A struct tee_pager_area can only use struct fobj and one struct pgt (translation table) so memory ranges spanning multiple fobjs or pgts are split into multiple areas.

struct fobj

This is a polymorph object, using different implementations depending on how it's initialized. It's defines as:

```

struct fobj_ops {
    void (*free)(struct fobj *fobj);
    TEE_Result (*load_page)(struct fobj *fobj, unsigned int page_idx,
                           void *va);
    TEE_Result (*save_page)(struct fobj *fobj, unsigned int page_idx,
                           const void *va);
};

struct fobj {
    const struct fobj_ops *ops;
    unsigned int num_pages;
    struct refcount refc;
    struct tee_pager_area_head areas;
};

```

num_pages

Tells how many pages this fobj covers.

refc

A reference counter, everyone referring to a fobj need to increase and decrease this as needed.

areas

A list of areas using this fobj, traversed when making a virtual page unavailable.

struct tee_pager_pmem

This structure represents a physical page. It's defined as:

```

struct tee_pager_pmem {
    unsigned int flags;
    unsigned int fobj_pgidx;
    struct fobj *fobj;
    void *va_alias;
    TAILQ_ENTRY(tee_pager_pmem) link;
};

```

PMEM_FLAG_DIRTY

Bit is set in flags when the page is mapped read/write at at least one location.

PMEM_FLAG_HIDDEN

Bit is set in `flags` when the page is hidden, that is, not accessible anywhere.

fobj_pgidx

The page at this index in the `fobj` is used in this physical page.

fobj

The `fobj` backing this page.

va_alias

Virtual address where this physical page is updated when loading it from backing store or when writing it back.

All `struct tee_pager_pmem` are stored either in the global list `tee_pager_pmem_head` or in `tee_pager_lock_pmem_head`. The latter is used by pages which are mapped and then locked in memory on demand. The pages are returned back to `tee_pager_pmem_head` when the pages are explicitly released with a call to `tee_pager_release_phys()`.

A physical page can be used by more than one `tee_pager_area` simultaneously. This is also known as shared secure memory and will appear as such for both read-only and read-write mappings.

When a page is hidden it's unmapped from all translation tables and the `PMEM_FLAG_HIDDEN` bit is set, but kept in memory. When a physical page is released it's also unmapped from all translation tables and its content is written back to storage, then the `fobj` field is set to `NULL` to note the physical page as unused.

Note that when `struct tee_pager_pmem` references a `fobj` it doesn't update the reference counter since it's already guaranteed to be available due the `struct tee_pager_area` which must reference the `fobj` too.

Paging of user TA

Paging of user TAs can optionally be enabled with `CFG_PAGED_USER_TA=y`. Paging of user TAs is analogous to paging of OP-TEE kernel parts but with a few differences:

- Read/write pages are paged in addition to read-only pages
- Page tables are managed dynamically

`tee_pager_add_uta_area(...)` is used to setup initial read/write mapping needed when populating the TA. When the TA is fully populated and relocated `tee_pager_set_uta_area_attr(...)` changes the mapping of the area to strict permissions used when the TA is running.

Paging shared secure memory

Shared secure memory is achieved by letting several `tee_pager_area` using the same backing `fobj`. When a `tee_pager_area` is allocated and assigned a `fobj` it's also added to a list for `tee_pager_areas` using this `fobj`. This helps when a physical page is released.

When a fault occurs first a matching `tee_pager_area` is located. Then `tee_pager_pmem_head` is searched to see if a physical page already holds the page of the `fobj` needed. If so the `pgt` is updated to map the physical page at the appropriate location. If no physical page was holding the page a new physical page is allocated, initialized and finally mapped.

In order to make as few updates to mappings as possible changes to less restricted, no access -> read-only or read-only to read-write, is done only for the virtual address was used when the page fault occurred. Changes in the other direction has to be done in all translation tables used to map the physical page.

2.1.6 Stacks

Different stacks are used during different stages. The stacks are:

- **Secure monitor stack** (128 bytes), bound to the CPU. Only available if OP-TEE is compiled with a secure monitor always the case if the target is Armv7-A but never for Armv8-A.
- **Temp stack** (small ~1KB), bound to the CPU. Used when transitioning from one state to another. Interrupts are always disabled when using this stack, aborts are fatal when using the temp stack.
- **Abort stack** (medium ~2KB), bound to the CPU. Used when trapping a data or pre-fetch abort. Aborts from user space are never fatal the TA is only killed. Aborts from kernel mode are used by the pager to do the demand paging, if pager is disabled all kernel mode aborts are fatal.
- **Thread stack** (large ~8KB), not bound to the CPU instead used by the current thread/task. Interrupts are usually enabled when using this stack.

Notes for Armv7-A/AArch32

Stack	Comment
Temp	Assigned to SP_SVC during entry/exit, always assigned to SP_IRQ and SP_FIQ
Abort	Always assigned to SP_ABT
Thread	Assigned to SP_SVC while a thread is active

Notes for AArch64

There are only two stack pointers, SP_EL1 and SP_EL0, available for OP-TEE in AArch64. When an exception is received stack pointer is always SP_EL1 which is used temporarily while assigning an appropriate stack pointer for SP_EL0. SP_EL1 is always assigned the value of `thread_core_local[cpu_id]`. This structure has some spare space for temporary storage of registers and also keeps the relevant stack pointers. In general when we talk about assigning a stack pointer to the CPU below we mean SP_EL0.

Boot

During early boot the CPU is configured with the temp stack which is used until OP-TEE exits to normal world the first time.

Notes for AArch64

SPSEL is always 0 on entry/exit to have SP_EL0 acting as stack pointer.

Normal entry

Each time OP-TEE is entered from normal world the temp stack is used as the initial stack. For fast calls, this is the only stack used. For normal calls an empty thread slot is selected and the CPU switches to that stack.

Normal exit

Normal exit occurs when a thread has finished its task and the thread is freed. When the main thread function, `tee_entry_std(...)`, returns interrupts are disabled and the CPU switches to the temp stack instead. The thread is freed and OP-TEE exits to normal world.

RPC exit

RPC exit occurs when OP-TEE need some service from normal world. RPC can currently only be performed with a thread is in running state. RPC is initiated with a call to `thread_rpc(...)` which saves the state in a way that when the thread is restored it will continue at the next instruction as if this function did a normal return. CPU switches to use the temp stack before returning to normal world.

Foreign interrupt exit

Foreign interrupt exit occurs when OP-TEE receives a foreign interrupt. For Arm GICv2 mode, foreign interrupt is sent as IRQ which is always handled in normal world. Foreign interrupt exit is similar to RPC exit but it is `thread_irq_handler(...)` and `elx_irq(...)` (respectively for Armv7-A/Aarch32 and for Aarch64) that saves the thread state instead. The thread is resumed in the same way though. For Arm GICv3 mode, foreign interrupt is sent as FIQ which could be handled by either secure world (EL3 in AArch64) or normal world. This mode is not supported yet.

Notes for Armv7-A/AArch32

SP_IRQ is initialized to temp stack instead of a separate stack. Prior to exiting to normal world CPU state is changed to SVC and temp stack is selected.

Notes for AArch64

SP_EL0 is assigned temp stack and is selected during IRQ processing. The original SP_EL0 is saved in the thread context to be restored when resuming.

Resume entry

OP-TEE is entered using the temp stack in the same way as for normal entry. The thread to resume is looked up and the state is restored to resume execution. The procedure to resume from an RPC exit or an foreign interrupt exit is exactly the same.

Syscall

Syscall's are executed using the thread stack.

Notes for Armv7-A/AArch32

Nothing special SP_SVC is already set with thread stack.

Notes for syscall AArch64

Early in the exception processing the original SP_EL0 is saved in `struct thread_svc_regs` in case the TA is executed in AArch64. Current thread stack is assigned to SP_EL0 which is then selected. When returning SP_EL0 is assigned what is in `struct thread_svc_regs`. This allows `tee_svc_sys_return_helper(...)` having the syscall exception handler return directly to `thread_unwind_user_mode(...)`.

2.1.7 Shared Memory

Shared Memory is a block of memory that is shared between the non-secure and the secure world. It is used to transfer data between both worlds.

The shared memory is allocated and managed by the non-secure world, i.e. the Linux OP-TEE driver. Secure world only considers the individual shared buffers, not their pool. Each shared memory is referenced with associated attributes:

- Buffer start address and byte size,
- Cache attributes of the shared memory buffer,
- List of chunks if mapped from noncontiguous pages.

Shared memory buffer references manipulated must fit inside one of the shared memory areas known from the OP-TEE core. OP-TEE supports two kinds of shared memory areas: an area for contiguous buffers and an area for noncontiguous buffers. At least one has to be enabled.

Contiguous shared memory is the historical OP-TEE legacy shared memory scheme where a specific physical memory area is shared. Nowadays, platforms tend to describe the physical memory layout and enable noncontiguous dynamic shared memory, allowing the non-secure OS to use its native system memory as legitimate shared memory references.

Contiguous shared buffers

Configuration directives `CFG_SHMEM_START` and `CFG_SHMEM_SIZE` define a share memory area where shared memory buffers are contiguous. Generic memory layout registers it as the `MEM_AREA_NSEC_SHM` memory area.

The non-secure world issues `OPTEE_SMC_GET_SHM_CONFIG` to retrieve contiguous shared memory area configuration:

- Physical address of the start of the pool
- Size of the pool
- Whether or not the memory is cached

Contiguous shared memory (also known as static or reserved shared memory) is enabled with the configuration flag `CFG_CORE_RESERVED_SHM=y`.

Noncontiguous shared buffers

To benefit from noncontiguous shared memory buffers, platform shall enable dynamic shared memory (`CFG_CORE_DYN_SHM=y`). When enabled, OP-TEE core is given the main memory address range(s) seen from non-secure OS. Non-secure client application can simply register a memory buffer as (e.g. with `TEEC_RegisterSharedMemory()`) so that it can be used in OP-TEE communication.

This feature requires Linux OP-TEE driver to properly handle the memory references of its various clients memories: userland applications, kernel drivers, remote services.

This feature also requires OP-TEE core to know the legitimate addresses ranges where non-secure can claim to use a shared memory page. The OP-TEE core generic boot sequence discovers dynamic shared areas from the device tree (memory nodes) and/or areas explicitly registered by the platform (`register_ddr()`).

Non-secure side needs to register buffers as 4kByte chunks lists into OP-TEE core using the `OPTEE_MSG_CMD_REGISTER_SHM` API prior referencing to them using the OP-TEE invocation API.

For performance reasons, the TEE Client Library (`libtee`) uses noncontiguous shared memory when available since it avoids copies in some situations.

Shared Memory Chunk Allocation

It is the Linux kernel driver for OP-TEE that is responsible for allocating chunks of shared memory. OP-TEE linux kernel driver relies on linux kernel generic allocation support (`CONFIG_GENERIC_ALLOCATION`) to allocation/release of shared memory physical chunks. OP-TEE linux kernel driver relies on linux kernel dma-buf support (`CONFIG_DMA_SHARED_BUFFER`) to track shared memory buffers references.

Registering shared memory

Only dynamic or physically non-contiguous shared memory needs to be registered. Static or physically contiguous shared memory is already known to OP-TEE OS.

SMC based OP-TEE MSG ABI

With the SMC based OP-TEE MSG ABI there are a few exceptions where memory doesn't need to be shared before it can be accessed from OP-TEE OS. These are:

1. When issuing the SMC `OPTEE_SMC_CALL_WITH_ARG` where the physical address of the supplied `struct optee_msg_arg` is passed in one of the registers.
2. When issuing the SMC `OPTEE_SMC_CALL_RETURN_FROM_RPC` as a return from the request `OPTEE_SMC_RETURN_RPC_ALLOC` to allocate memory. This RPC return is combined with an implicit registration of shared memory. The registration is ended with a `OPTEE_SMC_RETURN_RPC_FREE` request.

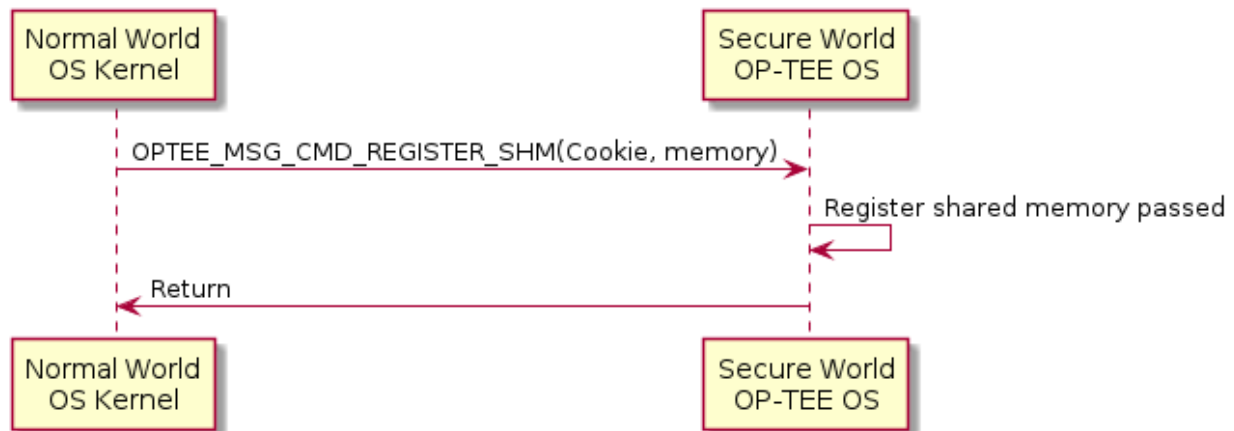


Fig. 9: Register shared memory example

FF-A based OP-TEE MSG ABI

With the FF-A based OP-TEE MSG ABI memory must always be registered before it can be used by OP-TEE OS. This case can potentially also involve another component in secure world, SPMC at S-EL2 a secure hypervisor which controls which memory OP-TEE OS can see or use.

In the case where there are no SPMC at S-EL2 OP-TEE OS will take care of that part of the communication with normal world. This means that for normal world communication with OP-TEE OS is the same regardless of the presence of a secure hypervisor.

Registration of shared memory is a two step procedure. It's first registered with a call to the SPMC which returns a cookie or global memory handle. This cookie is later used when calling OP-TEE OS, if the cookie isn't already known

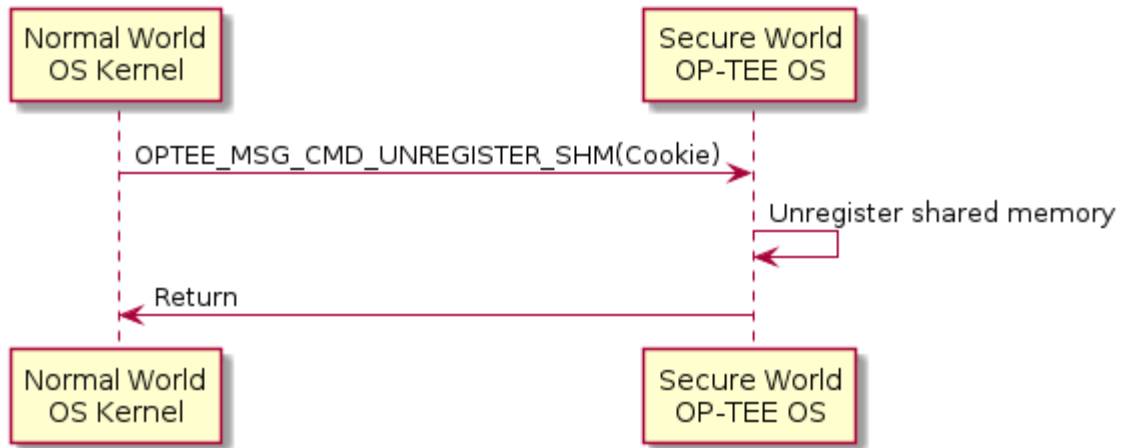


Fig. 10: Unregister shared memory example

to OP-TEE OS it will ask the SPMC to make the memory available. This lazy second step is a way of saving an extra round trip to secure world.

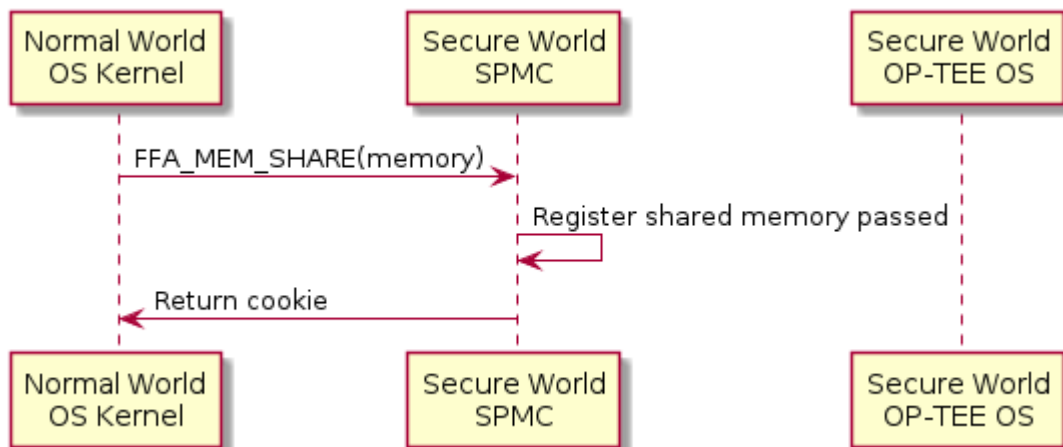


Fig. 11: Register shared memory example

Unregistration of shared memory is also done in two steps. First with a call to OP-TEE and then with a call to the SPMC. If the lazy second step of shared memory has not been done, then OP-TEE OS doesn't need to interact with the SPMC.

Using shared memory

From the Client Application

The client application can ask for shared memory allocation using the GlobalPlatform Client API function `TEEC_AllocateSharedMemory(...)`. The client application can also register a memory through the GlobalPlatform Client API function `TEEC_RegisterSharedMemory(...)`. The shared memory reference can then be used as parameter when invoking a trusted application.

From the Linux Driver

Occasionally the Linux kernel driver needs to allocate shared memory for the communication with secure world, for example when using buffers of type `TEEC_TempMemoryReference`.

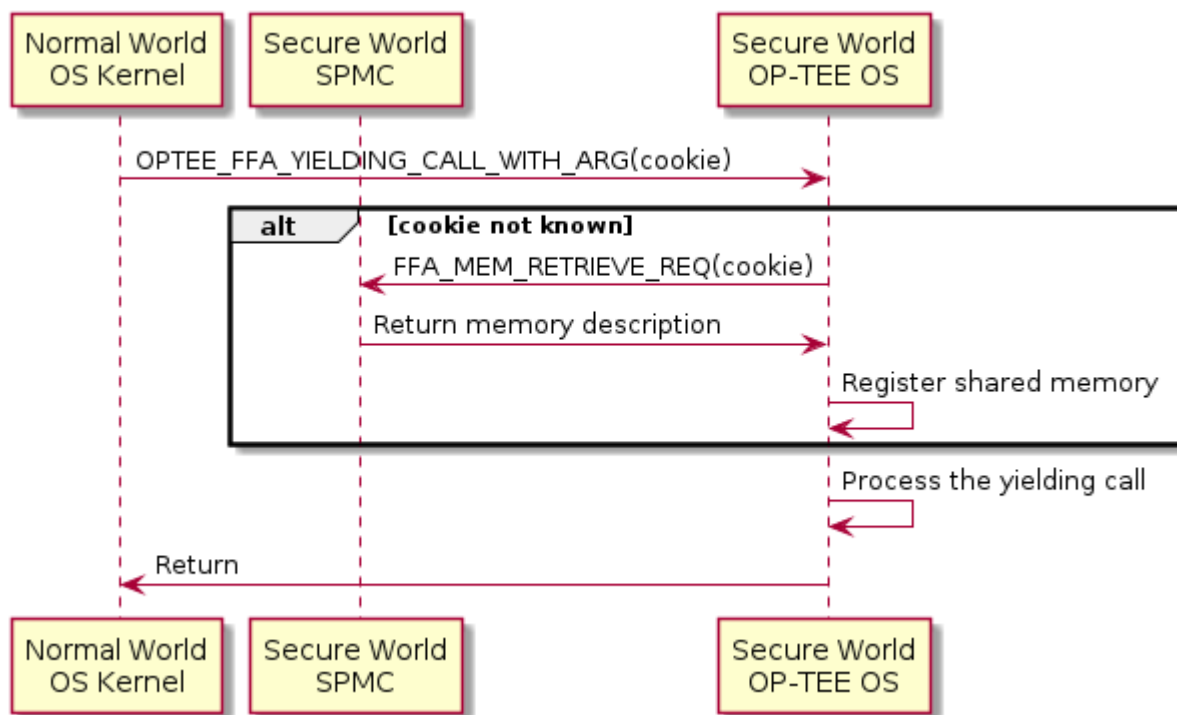


Fig. 12: Calling OP-TEE OS with shared memory

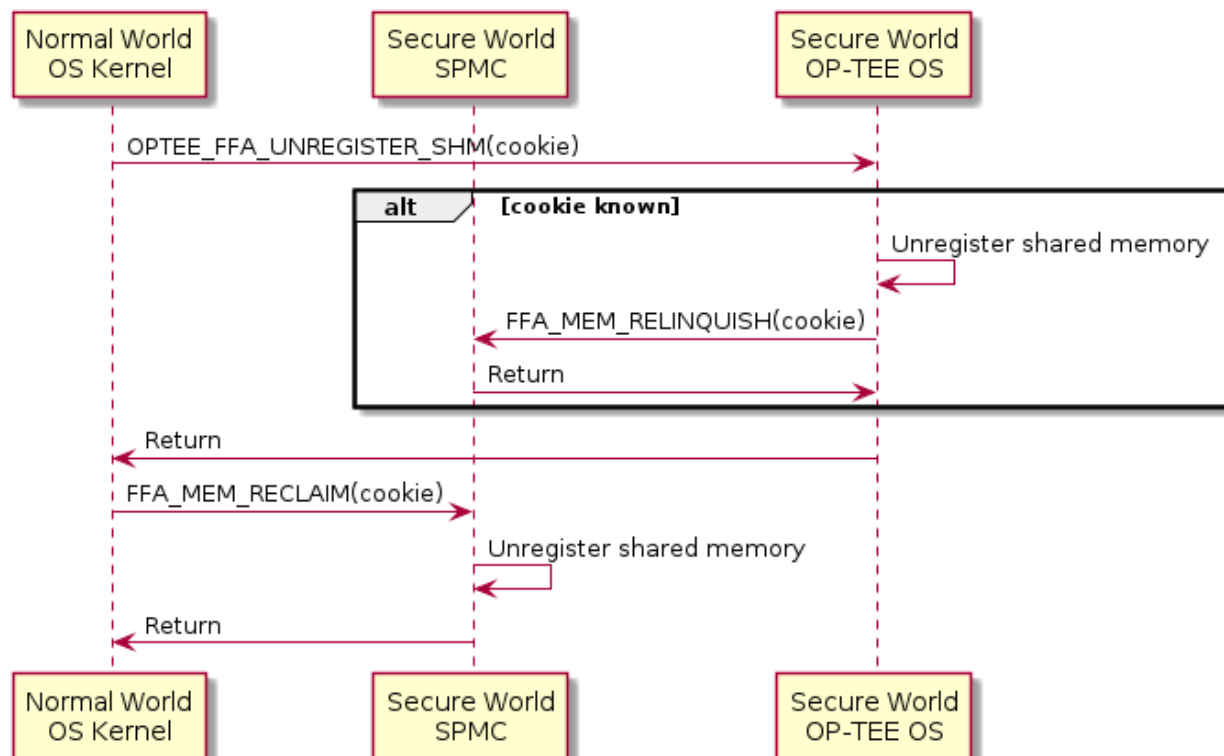


Fig. 13: Unregister shared memory

From OP-TEE core

In case OP-TEE core needs information from TEE supplicant (dynamic TA loading, REE time request,...), shared memory must be allocated. Allocation depends on the use case. OP-TEE core asks for the following shared memory allocation:

- `optee_msg_arg` structure, used to pass the arguments to the non-secure world, where the allocation will be done by sending a `OPTEE_SMC_RPC_FUNC_ALLOC` message.
- In some cases, a payload might be needed for storing the result from TEE supplicant, for example when loading a Trusted Application. This type of allocation will be done by sending the message `OPTEE_MSG_RPC_CMD_SHM_ALLOC(OPTEE_MSG_RPC_SHM_TYPE_APPL, ...)`, which then will return:
 - the physical address of the shared memory
 - a handle to the memory, that later on will be used later on when freeing this memory.

From TEE Supplicant

TEE supplicant is also working with shared memory, used to exchange data between normal and secure worlds. TEE supplicant receives a memory address from the OP-TEE core, used to store the data. This is for example the case when a Trusted Application is loaded. In this case, TEE supplicant must register the provided shared memory in the same way a client application would do, involving the Linux driver.

2.1.8 SMC

SMC Interface

OP-TEE's SMC interface is defined in two levels using `optee_smc.h` and `optee_msg.h`. The former file defines SMC identifiers and what is passed in the registers for each SMC. The latter file defines the OP-TEE Message protocol which is not restricted to only SMC even if that currently is the only option available.

SMC communication

The main structure used for the SMC communication is defined in `struct optee_msg_arg` (in `optee_msg.h`). If we are looking into the source code, we could see that communication mainly is achieved using `optee_msg_arg` and `thread_smc_args` (in `thread.h`), where `optee_msg_arg` could be seen as the main structure. What will happen is that the *Linux kernel TEE framework* driver will get the parameters either from `optee_client` or directly from an internal service in Linux kernel. The TEE driver will populate the `struct optee_msg_arg` with the parameters plus some additional bookkeeping information. Parameters for the SMC are passed in registers 1 to 7, register 0 holds the SMC id which among other things tells whether it is a standard or a fast call.

2.1.9 Thread handling

OP-TEE core uses a couple of threads to be able to support running jobs in parallel (not fully enabled!). There are handlers for different purposes. In `thread.c` you will find a function called `thread_init_primary(...)` which assigns `init_handlers` (functions) that should be called when OP-TEE core receives standard or fast calls, FIQ and PSCI calls. There are default handlers for these services, but the platform can decide if they want to implement their own platform specific handlers instead.

Synchronization primitives

OP-TEE has three primitives for synchronization of threads and CPUs: *spin-lock*, *mutex*, and *condvar*.

Spin-lock

A spin-lock is represented as an `unsigned int`. This is the most primitive lock. Interrupts should be disabled before attempting to take a spin-lock and should remain disabled until the lock is released. A spin-lock is initialized with `SPINLOCK_UNLOCK`.

Table 1: Spin lock functions

Function	Purpose
<code>cpu_spin_lock</code> <code>..)</code>	Locks a spin-lock
<code>cpu_spin_try1</code> <code>..)</code>	Locks a spin-lock if unlocked and returns <code>0</code> else the spin-lock is unchanged and the function returns <code>!0</code>
<code>cpu_spin_unlo</code> <code>..)</code>	Unlocks a spin-lock

Mutex

A mutex is represented by `struct mutex`. A mutex can be locked and unlocked with interrupts enabled or disabled, but only from a normal thread. A mutex cannot be used in an interrupt handler, abort handler or before a thread has been selected for the CPU. A mutex is initialized with either `MUTEX_INITIALIZER` or `mutex_init(. ..)`.

Table 2: Mutex functions

Function	Purpose
<code>mutex_lock(. ..)</code>	Locks a mutex. If the mutex is unlocked this is a fast operation, else the function issues an RPC to wait in normal world.
<code>mutex_unlock(. ..)</code>	Unlocks a mutex. If there is no waiters this is a fast operation, else the function issues an RPC to wake up a waiter in normal world.
<code>mutex_trylock(. ..)</code>	Locks a mutex if unlocked and returns <code>true</code> else the mutex is unchanged and the function returns <code>false</code> .
<code>mutex_destroy(. ..)</code>	Asserts that the mutex is unlocked and there is no waiters, after this the memory used by the mutex can be freed.

When a mutex is locked it is owned by the thread calling `mutex_lock(...)` or `mutex_trylock(...)`, the mutex may only be unlocked by the thread owning the mutex. A thread should not exit to TA user space when holding a mutex.

Condvar

A condvar is represented by `struct condvar`. A condvar is similar to a `pthread_condvar_t` in the pthreads standard, only less advanced. Condition variables are used to wait for some condition to be fulfilled and are always used together a mutex. Once a condition variable has been used together with a certain mutex, it must only be used with that mutex until destroyed. A condvar is initialized with `CONDVAR_INITIALIZER` or `condvar_init(...)`.

Table 3: Condvar functions

Function	Purpose
<code>condvar_wait(...)</code>	Atomically unlocks the supplied mutex and waits in normal world via an RPC for the condition variable to be signaled, when the function returns the mutex is locked again.
<code>condvar_signal(...)</code>	Wakes up one waiter of the condition variable (waiting in <code>condvar_wait(...)</code>).
<code>condvar_broadcast(...)</code>	Wake up all waiters of the condition variable.

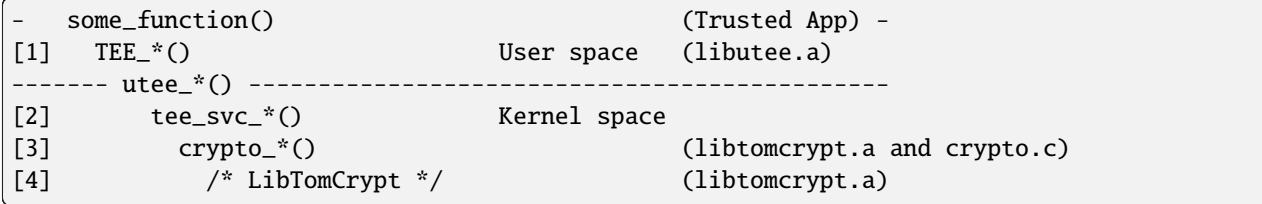
The caller of `condvar_signal(...)` or `condvar_broadcast(...)` should hold the mutex associated with the condition variable to guarantee that a waiter does not miss the signal.

2.2 Cryptographic implementation

This document describes how the TEE Cryptographic Operations API is implemented, how the default crypto provider may be configured at compile time, and how it may be replaced by another implementation.

2.2.1 Overview

There are several layers from the Trusted Application to the actual crypto algorithms. Most of the crypto code runs in kernel mode inside the TEE core. Here is a schematic view of a typical call to the crypto API. The numbers in square brackets ([1], [2]...) refer to the sections below.



2.2.2 [1] The TEE Cryptographic Operations API

OP-TEE implements the Cryptographic Operations API defined by the GlobalPlatform association in the *TEE Internal Core API*. This includes cryptographic functions that span various cryptographic needs: message digests, symmetric ciphers, message authentication codes (MAC), authenticated encryption, asymmetric operations (encryption/decryption or signing/verifying), key derivation, and random data generation. These functions make up the TEE Cryptographic Operations API.

The Internal API is implemented in `tee_api_operations.c`, which is compiled into a static library: `${O}/ta_arm{32, 64}-lib/libutee/libutee.a`.

Most API functions perform some parameter checking and manipulations, then invoke some `utee_*` function to switch to kernel mode and perform the low-level work.

The `utee_*` functions are declared in `utee_syscalls.h` and implemented in `utee_syscalls_asm.S`. They are simple system call wrappers which use the `SVC` instruction to switch to the appropriate system service in the OP-TEE kernel.

2.2.3 [2] The crypto services

All cryptography-related system calls are declared in `tee_svc_cryp.h` and implemented in `tee_svc_cryp.c`. In addition to dealing with the usual work required at the user/kernel interface (checking parameters and copying memory buffers between user and kernel space), the system calls invoke a private abstraction layer: the **Crypto API**, which is declared in `crypto.h`. It serves two main purposes:

1. Allow for alternative implementations, such as hardware-accelerated versions.
2. Provide an easy way to disable some families of algorithms at compile-time to save space. See *LibTomCrypt* below.

2.2.4 [3] `crypto_*`()

The `crypto_*`() functions implement the actual algorithms and helper functions. TEE Core has one global active implementation of this interface. The default implementation, mostly based on *LibTomCrypt*, is as follows:

Listing 1: File: `core/crypto/crypto.c`

```
/*
 * Default implementation for all functions in crypto.h
 */

#if !defined(_CFG_CRYPTOWITH_HASH)
TEE_Result crypto_hash_get_ctx_size(uint32_t algo __unused,
                                     size_t *size __unused)
{
    return TEE_ERROR_NOT_IMPLEMENTED;
}
...
#endif /*_CFG_CRYPTOWITH_HASH*/
```

Listing 2: File: core/lib/libtomcrypt/tee_ltc_provider.c

```
#if defined(_CFG_CRYPTOWITH_HASH)
TEE_Result crypto_hash_get_ctx_size(uint32_t algo, size_t *size)
{
    /* ... */
    return TEE_SUCCESS;
}

#endif /* _CFG_CRYPTOWITH_HASH */
```

As shown above, families of algorithms can be disabled and `crypto.c` will provide default null implementations that will return `TEE_ERROR_NOT_IMPLEMENTED`.

2.2.5 Public/private key format

`crypto.h` uses implementation-specific types to hold key data for asymmetric algorithms. For instance, here is how a public RSA key is represented:

Listing 3: File: core/include/crypto/crypto.h

```
struct rsa_public_key {
    struct bignum *e;    /* Public exponent */
    struct bignum *n;    /* Modulus */
};
```

This is also how such keys are stored inside the TEE object attributes (`TEE_ATTR_RSA_PUBLIC_KEY` in this case). `struct bignum` is an opaque type, known to the underlying implementation only. `struct bignum_ops` provides functions so that the system services can manipulate data of this type. This includes allocation/deallocation, copy, and conversion to or from the big endian binary format.

Listing 4: File: core/include/crypto/crypto.h

```
struct bignum *crypto_bignum_allocate(size_t size_bits);

TEE_Result crypto_bignum_bin2bn(const uint8_t *from, size_t fromsize,
                                struct bignum *to);

void crypto_bignum_bn2bin(const struct bignum *from, uint8_t *to);
/* ... */
```

2.2.6 [4] LibTomCrypt

Some algorithms may be disabled at compile time if they are not needed, in order to reduce the size of the OP-TEE image and reduces its memory usage. This is done by setting the appropriate configuration variable. For example:

```
$ make CFG_CRYPTOAES=n          # disable AES only
$ make CFG_CRYPTOAES,DES=n      # disable symmetric ciphers
$ make CFG_CRYPTOAES,DSA,RSA,DH,ECC=n # disable public key algorithms
$ make CFG_CRYPTOAES=n          # disable all algorithms
```

Please refer to `core/lib/libtomcrypt/sub.mk` for the list of all supported variables.

Note that the application interface is **not** modified when algorithms are disabled. This means, for instance, that the functions `TEE_CipherInit()`, `TEE_CipherUpdate()` and `TEE_CipherFinal()` would remain present in `libutee`. a even if all symmetric ciphers are disabled (they would simply return `TEE_ERROR_NOT_IMPLEMENTED`).

2.2.7 Add a new software based crypto implementation

To add a new software based implementation, the default one in `core/lib/libtomcrypt` in combination with what is in `core/crypto` should be used as a reference. Here are the main things to consider when adding a new crypto provider:

- Put all the new code in its own directory under `core/lib` unless it is code that will be used regardless of which crypto provider is in use. How we are dealing with AES-GCM in `core/crypto` could serve as an example.
- Avoid modifying `tee_svc_cryp.c`. It should not be needed.
- Although not all crypto families need to be defined, all are required for compliance to the GlobalPlatform specification.
- If you intend to make some algorithms optional, please try to re-use the same names for configuration variables as the default implementation.

2.2.8 [5] Support for crypto IC

Some cryptographic co-processors and secure elements are supported under a Generic Cryptographic Driver interface, connecting the TEE Crypto generic APIs to the HW driver interface. This interface is in `core/drivers/crypto/crypto_api` and should be followed when adding support for new devices.

At the time of writing, OP-TEE does not support the [GP TEE Secure Element API](#) and therefore the access to the secure element - the NXP EdgeLock® SE05x - follows the Cryptographic Operations API presenting a single session to the device. This session is shared with the normal world through the PKCS#11 interface but also through a more generic interface (`libseetec`) which allows clients to send Application Protocol Data Units (APDUs) directly to the device.

Notice that cryptographic co-processors do not necessarily comply with all the GP requirements tested and covered by the OP-TEE sanity test suite (`optee_test`). In those cases where the cryptographic operations are not supported - i.e: the SE05x does not implement all RSA key sizes - we opted for disabling those particular tests at build time rather than letting them fail.

Some cryptographic co-processors may have limitations regarding the range of key sizes and supported ciphers. For instance, the AMD/Xilinx Versal ACAP Cryptographic driver may have constraints on key sizes, while NXP SE5X HSM modules may lack support for RSA or ECC. In such cases, especially when dealing with unsupported key sizes, it may be necessary to resort to a software implementation of the cipher, typically utilizing LibTomCrypt.

Note: While the Hardware Security Modules or Cryptographic hardware processors supported by OP-TEE may achieve FIPS 140-2 certification at level 3, the software implementations of certain algorithms that OP-TEE may fall-back to cannot attain certification beyond level 2.

2.2.9 NXP SE05X Family of Secure Elements

This family of I2C bus devices are supported through the se050 cryptographic driver located at [core/drivers/crypto/se050](#). Before the REE boots, the session with the device is established using one of the OP-TEE supported I2C platform device drivers. Once the REE is up, the cryptographic driver can be configured to use the I2C driver in the REE (via RPC service) or continue using the one in OP-TEE.

Unless the Secure Element owns the I2C bus (no other elements on the bus, no runtime-PM and so forth), it is recommended to route all traffic via the Normal World. Initial communication with the device is not data intensive and therefore slow I2C drivers - perhaps those not using DMA channels - do not represent much of a performance drag; the situation changes once clients start hammering the device.

If using the REE for I2C transfers, it is also **imperative** to configure the driver so that the [GP Secure Channel Protocol 03](#) is enabled prior to exiting the Secure World; this way all communication between the processor and the secure element is encrypted and MAC authenticated. Please check the usage of the `CFG_CORE_SE05X_SCP03_EARLY` configuration option.

Aside of the secure element integration as an OP-TEE cryptographic driver, OP-TEE also presents an Application Protocol Data Units (APDU) interface to users via its OP-TEE client.

Using this interface, privileged applications can control the Secure Element to inject or delete keys or certificates, encrypt, decrypt, sign and verify data and so forth. An application implementing a subset of those functions can be seen in this Foundries.io repository: [fio-se05x-cli](#)

This reference code is not fully functional in mainline as it's not yet possible to import keys and certificates from the Secure Element into OP-TEE's PKCS#11 implementation. However, a user could still clear the Secure Element NVM memory and read certificates stored in it.

2.3 Device Tree

OP-TEE core can use the device tree format to inject platform configuration information during platform initialization and possibly some run time contexts.

Device Tree technology allows to describe platforms from ASCII source files so-called DTS files. These can be used to generate a platform description binary image, so-called DTB, embedded in the platform boot media for applying expected configuration settings during the platform initializations.

This scheme relaxes design constraints on the OP-TEE core implementation as most of the platform specific hardware can be tuned without modifying C source files or adding configuration directives in the build environments.

2.3.1 Secure and Non-Secure Device Trees

There can be several device trees embedded in the target system and some can be shared across the boot stages.

- Boot loader stages may load a device tree structure in memory for all boot stage to get platform configuration from. If such device tree data are to be accessed by the non-secure world, they shall be located in non-secure memory. Secure world may use its content during OP-TEE core initialization.
- Boot loader stages may load a device tree structure in secure memory for the benefit of the secure world only. Such device tree blob shall be located in secure memory. Secure world could use its content but this is currently not implemented in the latest OP-TEE release.
- OP-TEE core can also embedded a device tree structure to describe the platform.
- Non-secure world can embed its own device tree structure(s) and/or rely on a device tree structure loaded by the secure world during its initialization which happen before non-secure world is booted.

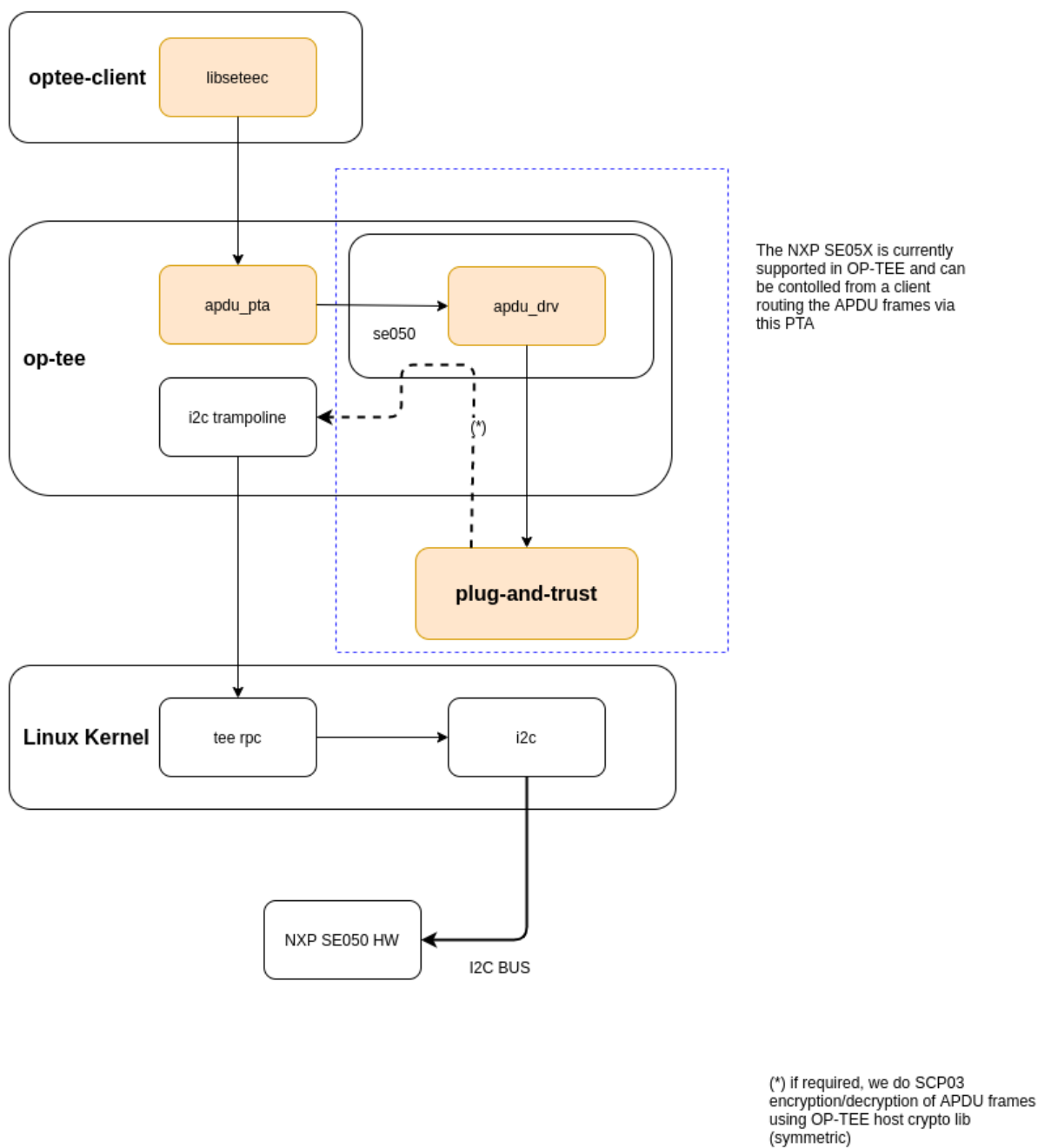


Fig. 14: Access to the Secure Element from libseetec and the APDU PTA.

Obviously the non-secure world will not be able to access a device tree image located in a secure memory which non-secure world has no access to.

When OP-TEE core is built with `CFG_DT=y`, non-secure and secure device trees can be accessed by OP-TEE core to get some platform configuration information.

2.3.2 Generic boot and DTBs

Generic boot sequence gets discovers main memory address ranges from preferably embedded DTB (section *Embedded Secure Device Tree*), defaulting to early boot external DTB (section *Early boot external device tree*).

Generic boot uses early boot external DTB (section *Early boot external device tree*) to share platform configuration information with the non-secure world.

Platform and drivers can call OP-TEE DT API (`core/include/kernel/dt.h`) to access embedded and/or external DTBs.

2.3.3 Early boot external device tree

The bootloader provides arguments to OP-TEE core when it boots it. Among those, the physical memory base address of a non-secure device tree image accessible to OP-TEE core, or a null address value in absence of such DTB.

Platform configuration may statically define such DTB location using the build configuration directive `CFG_DT_ADDR`.

When an external DTB is referred, OP-TEE core gets the console configuration if the platform has registered a compatible driver by adding attribute `__dt_driver` to a defined `const struct dt_driver` instance.

When an external DTB is referred, OP-TEE core adds into this DTB the description of some OP-TEE resources. These information can be used by the non-secure world to properly communicate with OP-TEE. This scheme assumes the image is located in non-secure memory.

Modifications made by OP-TEE core on the non-secure device tree image provided by early boot and passed to non-secure world are the following:

- Add an OP-TEE node if none found with the related invocation parameters.
- Add a reserved memory node for the few memory areas that shall be reserved to the secure world and non accessed by the non-secure world.
- Add a PSCI description node if none found.

Early boot DTB can be accessed by OP-TEE core only during its initialization, before non-secure world boots as it is expected the DTB memory location has likely been replaced with runtime contexts content.

Assuming there is no embedded DTB (section *Embedded Secure Device Tree*) OP-TEE core discovers the main memory address ranges from the non-secure DTB.

2.3.4 Early boot device tree overlay

There are two possibilities for OP-TEE core to provide a device tree overlay to the non-secure world.

- Append OP-TEE nodes to an existing DTB overlay located in early boot DTB. (`CFG_DT_ADDR` or boot argument register `R2/X2`).
- Generate a new DTB overlay image at location defined by `CFG_DT_ADDR`.

In the later case, memory referred by configuration directive `CFG_DT_ADDR` shall not contain a valid DTB image when OP-TEE core is booted. A subsequent non-secure boot stage should merge the OP-TEE DTB overlay image into another DTB.

A typical bootflow for this would be Trusted Firmware-A -> OP-TEE -> U-Boot with U-Boot in charge of merging OP-TEE DTB overlay located at CFG_DT_ADDR into a DTB U-Boot has loaded from elsewhere.

This functionality is enabled when CFG_EXTERNAL_DTB_OVERLAY=y.

2.3.5 Embedded Secure Device Tree

When OP-TEE core is built with configuration directive CFG_EMBED_DTB=y, directive CFG_EMBED_DTB_SOURCE_FILE shall provide the relative path of the DTS file inside directory core/arch/\$(ARCH)/dts from which a DTB is generated and embedded in a read-only section of OP-TEE core.

Refer to core/include/kernel/dt.h for API to access embedded DTB.

Section *Generic boot and DTBs* documents the generic boot sequence against embedded DTB.

2.3.6 OP-TEE Specific Bindings

Google Widevine device-tree bindings

2.4 Device tree bindings

2.4.1 Google Widevine device-tree bindings

```
%YAML 1.2
---
$id: http://devicetree.org/schemas/options/op-tee/google,widevine.yaml#
$schema: http://devicetree.org/meta-schemas/core.yaml#

title: Google Widevine initialization parameters

maintainers:
- Jeffrey Kardatzke <jkardatzke@chromium.org>
- Yi Chou <yich@chromium.org>

description:
  Widevine is Google's content protection system for DRM (digital rights
  management) contents.
  The necessary fields to initialize the Widevine related functions in
  OP-TEE. This node does not represent a real device, but serves as a
  place for passing data between firmware and OP-TEE.
  The content of this node should not be shared with the Linux kernel.

properties:
  op-tee,hardware-unique-key:
    $ref: /schemas/types.yaml#/definitions/uint8-array
    maxItems: 32
    description: |
      The hardware-unique key of the OP-TEE. It will be used to derive
      the secure storage key.
      For more information, please reference:
      https://optee.readthedocs.io/en/latest/architecture/porting_guidelines.html
```

(continues on next page)

(continued from previous page)

```

↪ #hardware-unique-key

tcg,tpm-auth-public-key:
  $ref: /schemas/types.yaml#/definitions/uint8-array
  maxItems: 1024
  description: |
    The TPM auth public key. Used to communicate the TPM from OP-TEE.
    The format of data should be TPM2B_PUBLIC.
    For more information, please reference the 12.2.5 section:
    https://trustedcomputinggroup.org/wp-content/uploads/TCG_TPM2_r1p59_Part2_
↪ Structures_pub.pdf

google,widevine-root-of-trust-ecc-p256:
  $ref: /schemas/types.yaml#/definitions/uint8-array
  maxItems: 32
  description: |
    The Widevine root of trust secret. Used to sign the Widevine
    request in OP-TEE. The value is an ECC NIST P-256 scalar.
    For more information, please reference the G.1.2 section:
    https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-186.pdf

required:
- op-tee,hardware-unique-key
- tcg,tpm-auth-public-key
- google,widevine-root-of-trust-ecc-p256

additionalProperties: false

examples:
- |
  options {
    google,widevine {
      op-tee,hardware-unique-key = [
        12 f7 98 d2 0e d2 85 92 a5 82 bf 98 b8 99 2b c0
        c6 6f 19 85 79 86 65 18 55 eb ff 9b 6c c0 ac 27
      ];
      tcg,tpm-auth-public-key = [
        00 76 00 23 00 0b 00 02 04 b2 00 20 e1 47 bf 27
        e1 74 30 c8 16 ab 72 4d 5c 77 e1 5c 61 2d 56 81
        b3 35 cd 9d eb 67 41 37 69 f0 32 41 00 10 00 10
        00 03 00 10 00 20 70 9a df 50 f9 0f d5 f4 40 e0
        ea 2c e8 f2 26 9f 0e 5c 02 70 16 c3 6c c1 83 03
        2d 04 10 bd 85 7a 00 20 83 03 c2 66 6e 01 32 34
        5c 5e 80 22 c7 48 24 3c 70 6b b8 e4 24 42 74 a9
        cf fc ab f8 30 e9 de 51
      ];
      google,widevine-root-of-trust-ecc-p256 = [
        ac 0d 86 c3 d7 b5 b7 a2 6f c3 d9 93 f7 de bc bb
        d5 c4 25 9b 21 5f 36 af b5 dd 6d 29 9d 08 c0 10
      ];
    };
  };

```

2.5 File structure

This page describes organization of the tree structure in *optee_os*.

The description is divided into different tables. First the flat top directory followed by the *core/* directory tree with the *core/arch/arm/* tree in separate table. There are two more tables covering the *lib/* and *ta/* trees.

2.5.1 Top level directories

Directory	Description
<i>core/</i>	Files that are only used building OP-TEE core, the privileged mode part
<i>keys/</i>	Secure keys or not so secure example keys
<i>ldelf/</i>	Ldelf the user mode ELF loader, for instance used to load TAs
<i>lib/</i>	Libraries that are used both when building more than one component, for instance, OP-TEE core, ldelf, or TAs
<i>mk/</i>	Makefiles supporting the build system
<i>scripts/</i>	Helper scripts for miscellaneous tasks
<i>ta/</i>	Files that are only used when building TAs
<i>out/</i>	Created when building unless a different out directory is specified with <i>O=...</i> on the command line

2.5.2 *core/*

Directory	Description
<i>arch/</i>	Architecture and platform specific files
<i>arch/arm/</i>	Arm specific architecture and platform files
<i>crypto/</i>	Crypto infrastructure including software implementations of certain algorithms.
<i>drivers/</i>	Various device drivers
<i>include/</i>	Header files of resources exported to the rest of the core
<i>include/crypto/</i>	Include files related to files in <i>/core/crypto</i>
<i>include/drivers/</i>	Include files related to device drivers
<i>include/dt-bindings/</i>	Include files for the device tree bindings
<i>include/kernel/</i>	Include files related to files in <i>/core/kernel</i>
<i>include/mm/</i>	Include files related to memory management and files in <i>/core/mm</i>
<i>include/tee/</i>	Include files related to files in <i>/core/tee</i>
<i>kernel/</i>	Miscellaneous architecture neutral files
<i>lib/</i>	Libraries that are used by core only
<i>lib/libfdt/</i>	Flat Device Trees manipulation library
<i>lib/libfdt/include/</i>	Include files related to libfdt
<i>lib/libtomcrypt/</i>	Libtomcrypt crypto library
<i>lib/libtomcrypt/in</i>	Include files related to libtomcrypt
<i>lib/libtomcrypt/src</i>	Source files of libtomcrypt
<i>lib/zlib/</i>	Zlib compression library
<i>mm/</i>	Architecture neutral memory management
<i>pta/</i>	Various pseudo TAs
<i>tee/</i>	Architecture neutral TEE files

2.5.3 core/arch/arm/

Directory	Description
cpu/	CPU specific settings
crypto/	Architecture specific software implementations of crypto algorithms
dts/	Device tree source files
include/	Header files of resources exported to the rest of the core
include/crypto/	Architecture specific include files related to /core/crypto or /core/arch/arm/crypto files
include/kernel/	Architecture specific include files related to /core/kernel or /core/arch/arm/kernel files
include/mm/	Architecture specific include files related to /core/mm or /core/arch/arm/mm files
include/sm/	Include files related to the secure monitor
include/tee/	Architecture specific include files related to /core/tee or /core/arch/arm/tee files
kernel/	Miscellaneous low level architecture specific files
plat-*/	Specific files for the different supported platform
mm/	Memory management
tee/	TEE files
sm/	Secure Monitor, ARMv7-A only

2.5.4 lib/

Directory	Description
libdl/	Implementation of dlopen(), dlsym() and dlclose() used by TAs and ldelf
libdl/include/	Include files for libdl
libmbedtls/	Mbed TLS crypto library
libmbedtls/core/	Glue code only compiled with core to connect with the core internal <crypto/crypto.h> API.
libmbedtls/include/	Include files with configuration of Mbed TLS
libmbedtls/mbedtls	Top directory of the imported Mbed TLS source tree
libmbedtls/mbedtls	Mbed TLS include files
libmbedtls/mbedtls	Mbed TLS implementation
libunw/	Unwind library
libunw/include/	Include files for libunw
libutee/	Libutee which provide the implementation of TEE Internal Core API.
libutee/arch/	Architecture specific implementation
libutee/include/	Include files related to libutee and the header files for TEE Internal Core API
libutils/	The reduced “libc” of OP-TEE
libutils/ext/	Extensions to a standard libc
libutils/ext/arch/	Architecture specific implementation of the extensions
libutils/ext/include/	Include files related to the extensions
libutils/isoc/	A subset of ISOC
libutils/isoc/arch/	Architecture specific
libutils/isoc/include/	Header files related to the provided subset of ISOC
libutils/isoc/newlib/	Routines imported from newlib

2.5.5 ta/

Directory	Description
trusted_keys	Trusted key TA
trusted_keys/inclu	Header file of the ABI provided by the trusted key TA
arch	Architecture specific files needed to compile a TA
mk	Makefile includes needed to build TAs and the TA dev kit
avb	TA to support AVB (Android Verified Boot)
avb/include	Header file of the ABI provided by the AVB TA
pkcs11	TA to support PKCS#11
pkcs11/src	Source code for the PKCS#11 TA
pkcs11/include	Header file for the ABI provided by the PKCS#11 TA

2.6 GlobalPlatform API

2.6.1 Introduction

GlobalPlatform works across industries to identify, develop and publish specifications which facilitate the secure and interoperable deployment and management of multiple embedded applications on secure chip technology. OP-TEE has support for GlobalPlatform TEE Client API [Specification](#) v1.0 (GPD_SPE_007) plus Errata and Precisions 2.0 (GPD_EPR_028) and TEE Internal Core API Specification v1.3.1 (GPD_SPE_010).

2.6.2 TEE Client API

The TEE Client API describes and defines how a client running in a rich operating environment (REE) should communicate with the TEE. To identify a Trusted Application (TA) to be used, the client provides an **UUID**. All TA's exposes one or several functions. Those functions corresponds to a so called **commandID** which also is sent by the client.

TEE Contexts

The TEE Context is used for creating a logical connection between the client and the TEE. The context must be initialized before the TEE Session can be created. When the client has completed a job running in secure world, it should finalize the context and thereby also release resources.

TEE Sessions

Sessions are used to create logical connections between a client and a specific Trusted Application. When the session has been established the client has opened up the communication channel towards the specified Trusted Application identified by the UUID. At this stage the client and the Trusted Application can start to exchange data.

TEE Shared memory

The TEE Client API describes many ways of sharing memory between the client and the TEE. Some ways are more efficient than others due to how they are implemented, but they have all their advantages too. For example, using a temporary memory reference (TEEC_TempMemoryReference) is often convenient, but depending on the situation often not the most efficient. A temporary memory reference is established internally in the TEE Client library before it is used, and when the call to secure world has returned it is torn down again. That results in a few extra re-entries into the TEE.

For more efficient communication a shared memory block (TEEC_SharedMemory) should be used since it can be reused between calls and also tuned in more ways. A shared memory block can be initialized either with TEEC_RegisterSharedMemory() or TEEC_AllocateSharedMemory().

TEEC_RegisterSharedMemory() sometimes fails to establish zero-copy shared memory and must in those cases fall back to a temporary “shadow buffer”. The TEE framework will for instance refuse to register a memory block that is mapped read-only in the client. Another reason can be if FF-A is used and a part of the memory range has been registered previously.

TEEC_AllocateSharedMemory() is the best choice to establish zero-copy shared memory. If TEEC_RegisterSharedMemory() must be used instead because the buffer is allocated in advance or externally there are still a few things that helps avoid a fallback to a “shadow buffer”. Make sure that the memory range is normal read/write memory and if possible use page-aligned memory buffers.

TEE Client API example / usage

Below you will find the main functions as defined by GlobalPlatform and are used in the communication between the client and the TEE.

```
TEEC_Result TEEC_InitializeContext(  
    const char* name,  
    TEEC_Context* context)  
  
void TEEC_FinalizeContext(  
    TEEC_Context* context)  
  
TEEC_Result TEEC_OpenSession (  
    TEEC_Context* context,  
    TEEC_Session* session,  
    const TEEC_UUID* destination,  
    uint32_t connectionMethod,  
    const void* connectionData,  
    TEEC_Operation* operation,  
    uint32_t* returnOrigin)  
  
void TEEC_CloseSession (  
    TEEC_Session* session)  
  
TEEC_Result TEEC_InvokeCommand(  
    TEEC_Session* session,  
    uint32_t commandID,  
    TEEC_Operation* operation,  
    uint32_t* returnOrigin)
```

In principle the commands are called in this order:

```
TEEC_InitializeContext(...)
TEEC_OpenSession(...)
TEEC_InvokeCommand(...)
TEEC_CloseSession(...)
TEEC_FinalizeContext(...)
```

It is not uncommon that `TEEC_InvokeCommand(...)` is called several times in a row when the session has been established.

For a complete example, please see chapter **5.2 Example 1: Using the TEE Client API** in the GlobalPlatform TEE Client API [Specification](#) v1.0.

2.6.3 TEE Internal Core API

The Internal Core API is the API that is exposed to the Trusted Applications running in the secure world. The TEE Internal API consists of four major parts:

1. Trusted Storage API for Data and Keys
2. Cryptographic Operations API
3. Time API
4. Arithmetical API

Examples / usage

Calling the Internal Core API is done in the same way as described above using Client API. The best place to find information how this should be done is in the TEE Internal Core API [Specification](#) which contains many examples of how to call the various APIs. One can also have a look at the examples in the [optee_examples](#) git.

2.6.4 Extensions

In addition to what is stated in *TEE Internal Core API*, there are some non-official extensions in OP-TEE.

Trusted Applications should include header file `tee_internal_api_extensions.h` to import the definitions of the extensions. For each extension, a configuration directive prefixed `CFG_` allows one to disable support for the extension when building the OP-TEE packages.

Cache Maintenance Support

Following functions have been introduced in order to allow Trusted Applications to operate with the data cache:

```
TEE_Result TEE_CacheClean(char *buf, size_t len);
TEE_Result TEE_CacheFlush(char *buf, size_t len);
TEE_Result TEE_CacheInvalidate(char *buf, size_t len);
```

These functions are available to any Trusted Application defined with the flag `TA_FLAG_CACHE_MAINTENANCE` set on, see *Cache maintenance Flag*. When not set, each function returns the error code `TEE_ERROR_NOT_SUPPORTED`. Within these extensions, a Trusted Application is able to operate on the data cache, with the following specification:

Function	Description
TEE_CacheCle	Write back to memory any dirty data cache lines. The line is marked as not dirty. The valid bit is unchanged.
TEE_CacheFlt	Purges any valid data cache lines. Any dirty cache lines are first written back to memory, then the cache line is invalidated.
TEE_CacheInv	Invalidate any valid data cache lines. Any dirty line are not written back to memory.

In the following two cases, the error code `TEE_ERROR_ACCESS_DENIED` is returned:

- The memory range has not the write access, that is `TEE_MEMORY_ACCESS_WRITE` is not set.
- The memory is **not** user space memory.

You may disable this extension by setting the following configuration variable in `conf.mk`:

```
CFG_CACHE_API := n
```

PKCS#1 v1.5 RSASSA without hash OID

This extension adds identifier `TEE_ALG_RSASSA_PKCS1_V1_5` to allow signing and verifying messages with RSASSA-PKCS1-v1_5, in [RFC 3447](#), without including the OID of the hash in the signature. You may disable this extension by setting the following configuration variable in `conf.mk`:

```
CFG_CRYPTO_RSASSA_NA1 := n
```

The TEE Internal Core API was extended with a new algorithm descriptor.

Algorithm	Possible Modes
TEE_ALG_RS	TEE_MODE_SIGN / TEE_MODE_VERIFY

Algorithm	Identifier
TEE_ALG_RS	0xF0000830

Concat KDF

Support for the Concatenation Key Derivation Function (Concat KDF) according to [SP 800-56A \(Recommendation for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography\)](#) can be found in OP-TEE. You may disable this extension by setting the following configuration variable in `conf.mk`:

```
CFG_CRYPTO_CONCAT_KDF := n
```

Implementation notes

All key and parameter sizes **must** be multiples of 8 bits. That is:

- Input parameters: the shared secret (`Z`) and `OtherInfo`.
- Output parameter: the derived key (`DerivedKeyingMaterial`).

In addition, the maximum size of the derived key is limited by the size of an object of type `TEE_TYPE_GENERIC_SECRET` (512 bytes). This implementation does **not** enforce any requirement on the content of the `OtherInfo` parameter. It is the application's responsibility to make sure this parameter is constructed as specified by the NIST specification if compliance is desired.

API extension

To support Concat KDF, the *TEE Internal Core API* v1.3.1 was extended with new algorithm descriptors, new object types, and new object attributes as described below.

p.95 Add new object type to `TEE_PopulateTransientObject`

The following entry shall be added to **Table 5-8**:

Object type	Parts
<code>TEE_TYPE_C</code>	The <code>TEE_ATTR_CONCAT_KDF_Z</code> part (input shared secret) must be provided.

p.121 Add new algorithms for `TEE_AllocateOperation`

The following entry shall be added to **Table 6-3**:

Algorithm	Possible Modes
<code>TEE_ALG_CC</code>	<code>TEE_MODE_DERIVE</code>
<code>TEE_ALG_CC</code>	
<code>TEE_ALG_CC</code>	
<code>TEE_ALG_CC</code>	
<code>TEE_ALG_CC</code>	
<code>TEE_ALG_CC</code>	

p.126 Explain usage of HKDF algorithms in `TEE_SetOperationKey`

In the bullet list about operation mode, the following shall be added:

- For the Concat KDF algorithms, the only supported mode is `TEE_MODE_DERIVE`.

p.150 Define `TEE_DeriveKey` input attributes for new algorithms

The following sentence shall be deleted:

The `TEE_DeriveKey` function can only be used with the algorithm `TEE_ALG_DH_DERIVE_SHARED_SECRET`.

The following entry shall be added to **Table 6-7**:

Algorithm	Possible operation parameters
<code>TEE_ALG_CONCAT_KDF_SHA1_DERIVE_KEY</code>	<code>TEE_ATTR_CONCAT_KDF_DKM_LENGTH</code> : up to 512 bytes. This parameter is mandatory: <code>TEE_ATTR_CONCAT_KDF_OTHER_INFO</code>
<code>TEE_ALG_CONCAT_KDF_SHA224_DERIVE_KEY</code>	
<code>TEE_ALG_CONCAT_KDF_SHA256_DERIVE_KEY</code>	
<code>TEE_ALG_CONCAT_KDF_SHA384_DERIVE_KEY</code>	
<code>TEE_ALG_CONCAT_KDF_SHA512_DERIVE_KEY</code>	
<code>TEE_ALG_CONCAT_KDF_SHA512_DERIVE_KEY</code>	

p.152 Add new algorithm identifiers

The following entries shall be added to **Table 6-8**:

Algorithm	Identifier
TEE_ALG_CONCAT_KDF_SHA1_DERIVE_KEY	0x800020C1
TEE_ALG_CONCAT_KDF_SHA224_DERIVE_KEY	0x800030C1
TEE_ALG_CONCAT_KDF_SHA256_DERIVE_KEY	0x800040C1
TEE_ALG_CONCAT_KDF_SHA384_DERIVE_KEY	0x800050C1
TEE_ALG_CONCAT_KDF_SHA512_DERIVE_KEY	0x800060C1

p.154 Define new main algorithm

In **Table 6-9** in section 6.10.1, a new value shall be added to the value column for row bits [7:0]:

Bits	Function	Value
Bits [7:0]	Identify the main underlying algorithm itself	... 0xC1: Concat KDF

The function column for bits[15:12] shall also be modified to read:

Bits	Function	Value
Bits [15:12]	Define the message digest for asymmetric signature algorithms or Concat KDF	

p.155 Add new object type for Concat KDF input shared secret

The following entry shall be added to **Table 6-10**:

Name	Identifier	Possible sizes
TEE_TYPE_CONCAT_KDF_Z	0xA10000C1	8 to 4096 bits (multiple of 8)

p.156 Add new operation attributes for Concat KDF

The following entries shall be added to **Table 6-11**:

Name	Value	Pro-tection	Type	Comment
TEE_ATTR_CONCAT_KI	0xC000C	Pro-protected	Ref	The shared secret (Z)
TEE_ATTR_CONCAT_KI	0xD000C	Public	Ref	OtherInfo
TEE_ATTR_CONCAT_KI	0xF0000	Public	Value	The length (in bytes) of the derived keying material to be generated, maximum 512. This is KeyDataLen / 8.

HKDF

OP-TEE implements the *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)* as specified in [RFC 5869](#). This file documents the extensions to the *TEE Internal Core API* v1.3.1 that were implemented to support this algorithm.

Note that the implementation follows the recommendations of version 1.3.1 of the specification for adding new algorithms. It should make it compatible with future changes to the official specification. You can disable this extension by setting the following in `conf.mk`:

```
CFG_CRYPTO_HKDF := n
```

p.95 Add new object type to TEE_PopulateTransientObject

The following entry shall be added to **Table 5-8**:

Object type	Parts
TEE_TYPE_HKDF_IKM	The TEE_ATTR_HKDF_IKM (Input Keying Material) part must be provided.

p.121 Add new algorithms for TEE_AllocateOperation

The following entry shall be added to **Table 6-3**:

Algorithm	Possible Modes
TEE_ALG_HKDF_MD5_DERIVE_KEY TEE_ALG_HKDF_SHA224_DERIVE_KEY TEE_ALG_HKDF_SHA384_DERIVE_KEY TEE_ALG_HKDF_SHA512_DERIVE_KEY	TEE_ALG_HKDF_SHA1_DERIVE_KEY TEE_ALG_HKDF_SHA256_DERIVE_KEY TEE_ALG_HKDF_SHA512_DERIVE_KEY TEE_MODE_DERIVE

p.126 Explain usage of HKDF algorithms in TEE_SetOperationKey

In the bullet list about operation mode, the following shall be added:

- For the HKDF algorithms, the only supported mode is TEE_MODE_DERIVE.

p.150 Define TEE_DeriveKey input attributes for new algorithms

The following sentence shall be deleted:

```
The TEE_DeriveKey function can only be used with the algorithm
TEE_ALG_DH_DERIVE_SHARED_SECRET
```

The following entry shall be added to **Table 6-7**:

Algorithm	Possible operation parameters
TEE_ALG_HKDF_MD5_DERIVE_KEY TEE_ALG_HKDF_SHA1_DERIVE_KEY TEE_ALG_HKDF_SHA224_DERIVE_KEY TEE_ALG_HKDF_SHA256_DERIVE_KEY TEE_ALG_HKDF_SHA384_DERIVE_KEY TEE_ALG_HKDF_SHA512_DERIVE_KEY TEE_ALG_HKDF_SHA512_DERIVE_KEY	TEE_ATTR_HKDF_OKM_LENGTH: Number of bytes in the Output Keying Material TEE_ATTR_HKDF_SALT (optional) Salt to be used during the extract step TEE_ATTR_HKDF_INFO (optional) Info to be used during the expand step

p.152 Add new algorithm identifiers

The following entries shall be added to **Table 6-8**:

Algorithm	Identifier
TEE_ALG_HKDF_MD5_DERIVE_KEY	0x800010C0
TEE_ALG_HKDF_SHA1_DERIVE_KEY	0x800020C0
TEE_ALG_HKDF_SHA224_DERIVE_KEY	0x800030C0
TEE_ALG_HKDF_SHA256_DERIVE_KEY	0x800040C0
TEE_ALG_HKDF_SHA384_DERIVE_KEY	0x800050C0
TEE_ALG_HKDF_SHA512_DERIVE_KEY	0x800060C0

p.154 Define new main algorithm

In **Table 6-9** in section 6.10.1, a new value shall be added to the value column for row `bits [7:0]`:

Bits	Function	Value
Bits [7:0]	Identify the main underlying algorithm itself	... 0xC0: HKDF

The function column for `bits[15:12]` shall also be modified to read:

Bits	Function	Value
Bits [15:12]	Define the message digest for asymmetric signature algorithms or HKDF	

p.155 Add new object type for HKDF input keying material

The following entry shall be added to **Table 6-10**:

Name	Identifier	Possible sizes
TEE_TYPE_HKDF_IKM	0xA10000C0	8 to 4096 bits (multiple of 8)

p.156 Add new operation attributes for HKDF salt and info

The following entries shall be added to **Table 6-11**:

Name	Value	Pro-tection	Type	Comment
TEE_ATTR_HKDF_IKM	0xC0000	Pro- tected	Ref	
TEE_ATTR_HKDF_SALT	0xD0000	Public	Ref	
TEE_ATTR_HKDF_INFO	0xD0000	Public	Ref	
TEE_ATTR_HKDF_OKM_LENGTH	0xF0000	Public	Value	

PBKDF2

This document describes the OP-TEE implementation of the key derivation function, *PBKDF2* as specified in [RFC 2898](#) section 5.2. This RFC is a republication of PKCS #5 v2.0 from RSA Laboratories' Public-Key Cryptography Standards (PKCS) series. You may disable this extension by setting the following configuration variable in `conf.mk`:

```
CFG_CRYPTOPBKDF2 := n
```

API extension

To support PBKDF2, the *TEE Internal Core API* v1.3.1 was extended with a new algorithm descriptor, new object types, and new object attributes as described below.

p.95 Add new object type to TEE_PopulateTransientObject

The following entry shall be added to **Table 5-8**:

Object type	Parts
TEE_TYPE_PBKDF2_PASSWORD	The TEE_ATTR_PBKDF2_PASSWORD part must be provided.

p.121 Add new algorithms for TEE_AllocateOperation

The following entry shall be added to **Table 6-3**:

Algorithm	Possible Modes
TEE_ALG_PBKDF2_HMAC_SHA1_DERIVE_KEY	TEE_MODE_DERIVE

p.126 Explain usage of PBKDF2 algorithm in TEE_SetOperationKey

In the bullet list about operation mode, the following shall be added:

- For the PBKDF2 algorithm, the only supported mode is TEE_MODE_DERIVE.

p.150 Define TEE_DeriveKey input attributes for new algorithms

The following sentence shall be deleted:

```
The TEE_DeriveKey function can only be used with the algorithm
TEE_ALG_DH_DERIVE_SHARED_SECRET
```

The following entry shall be added to **Table 6-7**:

Algorithm	Possible operation parameters
TEE_ALG_PBKDF2_HM	TEE_ATTR_PBKDF2_DKM_LENGTH: up to 512 bytes. This parameter is mandatory. TEE_ATTR_PBKDF2_SALT TEE_ATTR_PBKDF2_ITERATION_COUNT: This parameter is mandatory.

p.152 Add new algorithm identifiers

The following entries shall be added to **Table 6-8**:

Algorithm	Identifier
TEE_ALG_PBKDF2_HMAC_SHA1_DERIVE_KEY	0x800020C2

p.154 Define new main algorithm

In **Table 6-9** in section 6.10.1, a new value shall be added to the value column for row bits [7:0]:

Bits	Function	Value
Bits [7:0]	Identify the main underlying algorithm itself	... 0xC2: PBKDF2

The function column for bits [15:12] shall also be modified to read:

Bits	Function	Value
Bits [15:12]	Define the message digest for asymmetric signature algorithms or PBKDF2	

p.155 Add new object type for PBKDF2 password

The following entry shall be added to **Table 6-10**:

Name	Identifier	Possible sizes
TEE_TYPE_PBKDF2_PASSWORD	0xA1000C2	8 to 4096 bits (multiple of 8)

p.156 Add new operation attributes for Concat KDF

The following entries shall be added to **Table 6-11**:

Name	Value	Pro-tection	Type	Comment
TEE_ATTR_PBKDF2_PASSWORD	0xC0000	Pro- tected	Ref	
TEE_ATTR_PBKDF2_SALT	0xD0000	Public	Ref	
TEE_ATTR_PBKDF2_ITERATION_CC	0xF0000	Public	Value	
TEE_ATTR_PBKDF2_DKM_LENGTH	0xF0000	Public	Value	The length (in bytes) of the derived key- ing material to be generated, maximum 512.

Loadable plugins framework

This framework makes the supplicant a bit more flexible in terms of providing services. It is possible to design any REE service for the TEE as a tee-suppliant plugin. It makes it easy to:

- add new features to the supplicant that aren't needed in upstream, e.g. Rich OS-specific services
- sync an own fork of the supplicant with the upstream version

To create a plugin, developers have to implement the following structure from the `public/tee_plugin_method.h` file from the `optee_client` git.:

```
struct plugin_method {
    const char *name; /* short friendly name of the plugin */
    TEEC_UUID uuid;
    TEEC_Result (*init)(void);
```

(continues on next page)

(continued from previous page)

```
TEEC_Result (*invoke)(unsigned int cmd, unsigned int sub_cmd,
                      void *data, size_t in_len, size_t *out_len);
};
```

The plugin framework is based on the RPC - OPTEE_MSG_RPC_CMD_PLUGIN. This is a unified interface between TEE and plugins. TEE can only access the plugins by its UUID.

After implementing this structure, a plugin has to be compiled as a shared object. The objects have to be placed into the directory defined by CFG_TEE_PLUGIN_LOAD_PATH. This path can be set in the `config.mk` file in the `optee_client` git. By default it is set to `/usr/lib/tee-supplciant/plugins/`.

The supplicant loads all of the plugins from the directory during the startup process using `libdl`. After this, any requests to plugins from TEE will be processed in the common RPC handler.

On TEE side users can use any plugin by its UUID from TAs code and from the OP-TEE kernel code. The following function has been introduced like an extension of the TEE API to allow Trusted Applications to operate with plugins:

```
/*
 * tee_invoke_supp_plugin() - invoke a tee-supplciant's plugin
 * @uuid:      uuid of the plugin
 * @cmd:       command for the plugin
 * @sub_cmd:   subcommand for the plugin
 * @buf:       data [to/from] the plugin [in/out]
 * @len:       length of the input buf
 * @outlen:    pointer to length of the output data (if they will be used)
 *
 * Return TEE_SUCCESS on success or TEE_ERROR_* on failure.
 */
TEEC_Result tee_invoke_supp_plugin(const TEE_UUID *uuid, uint32_t cmd,
                                  uint32_t sub_cmd, void *buf, size_t len,
                                  size_t *outlen);
```

This API calls the `system-pta`, which uses the RPC to call a plugin. See `OPTEE_RPC_CMD_SUPP_PLUGIN` in the `core/include/optee_rpc_cmd.h` file from `optee_os` git. If there is a need to use plugins from the OP-TEE kernel, then the following function can be called directly:

```
TEEC_Result tee_invoke_supp_plugin_rpc(const TEE_UUID *uuid, uint32_t cmd,
                                       uint32_t sub_cmd, void *buf, size_t len,
                                       size_t *outlen);
```

Note: One buffer is used for input data to a plugin and for output data from a plugin. See an example of using this feature in the `optee_examples` git.

2.7 Libraries

2.7.1 libutils

OP-TEE core and OP-TEE development kit for Trusted Application provide a standard C library that is named **libutils**. It implements many standard functions like `snprintf()`, `strncmp()`, `memcpy()`, `malloc()`, `qsort()`, and many more but not all standard C library functions.

Note however that Trusted Applications implemented in C should use GP TEE Internal Core API functions rather than their standard C library function equivalent (e.g. `TEE_MemMove()` instead of `memcpy()` and `memmove()`, or `TEE_Malloc()` instead of `malloc()` and friends). This makes those TAs implementation more portable to other GP TEE compliant environments.

When `CFG_ULIBS_SHARED` is enabled, **libutils** is assigned UUID **71855bba-6055-4293-a63f-b0963a737360**.

2.7.2 libutee

The *TEE Internal Core API* describes services that are provided to Trusted Applications. **libutee** is a library that implements this API.

libutee is designed as a userland library specifically dedicated to OP-TEE Trusted Applications and aims at being executed in the non-privileged secure userspace.

Some services for this API are fully statically implemented inside the libutee library while some services for the API are implemented inside the OP-TEE core (privileged level) and libutee calls such services through system calls.

When `CFG_ULIBS_SHARED` is enabled, **libutee** is assigned UUID **4b3d937e-d57e-418b-8673-1c04f2420226**.

2.7.3 libmbdttls

OP-TEE OS source tree provides support of the Mbed TLS library, named **libmbdttls**.

A specific build sequence can compile an instance of **libmbdttls** and link it to OP-TEE core. Another build sequence compiles an instance of **libmbdttls** that can be linked with Trusted Applications.

When Mbed TLS is embedded in OP-TEE core, it is used as the default software implementation for most cryptography operations. When so, **libtomcrypt** is still used as default software implementation for few crypto operations. Embedding Mbed TLS in OP-TEE core requires `CFG_CRYPTOLIB_NAME=mbdttls` and `CFG_CRYPTOLIB_DIR=core/lib/libmbdttls`.

When `CFG_ULIBS_SHARED` is enabled, **libmbdttls** userland library is assigned UUID **87bb6ae8-4b1d-49fe-9986-2b966132c309**.

2.7.4 libunw

OP-TEE OS source tree implements execution stack back trace debug facilities available to both OP-TEE core and Trusted Applications. The feature relies on a library named **libunw**.

libunw, when linked to a Trusted Application, is always linked as a static library.

2.7.5 libdl

libdl library implement API function `dlopen()`, `dlsym()` and `dlclose()` used by Trusted Applications to support dynamic shared libraries.

When `CFG_ULIBS_SHARED` is enabled, **libdl** is assigned UUID **be807bbd-81e1-4dc4-bd99-3d363f240ece**.

2.7.6 Static vs Shared libraries

OP-TEE core supports only static libraries that are linked at build time to produce the monolithic OP-TEE core image.

OP-TEE Trusted Applications can support both static and shared libraries. In the latter case, each shared library is identified by a UUID and OP-TEE OS is in charge of dynamically loading the required shared libraries in the address space of the Trusted Application when this one uses a resource of the related library.

In order to support shared library, OP-TEE OS shall be built with `CFG_ULIBS_SHARED=y`. Shared library binary images are generated as **.elf** and **.ta** files, like Trusted Applications are, and shall be installed the same way as Trusted Applications are, see [ref:ta_locations](#).

2.8 Porting guidelines

This document serves a dual purpose:

- Serve as a base for getting OP-TEE up and running on a new device with initial xtest validation passing. This is the first part of this document (section 2).
- Highlight the missing pieces if you intend to make a real secure product, that is what the second part of this document is about.

We are trying our best to implement full end to end security in OP-TEE in a generic way, but due to the nature of devices being different, NDA etc, it is not always possible for us to do so and in those cases, we most often try to write a generic API, but we will just stub the code. This porting guideline highlights the missing pieces that must be addressed in a real secure consumer device. Hopefully we will sooner or later get access to devices where we at least can make reference implementations publicly available to everyone for the missing pieces we are talking about here.

2.8.1 Add a new platform

The first thing you need to do after you have decided to port OP-TEE to another device is to add a new platform device. That can either be adding a new platform variant (`PLATFORM_FLAVOR`) if it is a device from a family already supported, or it can be a brand new platform family (`PLATFORM`). Typically this initial setup involve configuring UART, memory addresses etc. For simplicity let us call our fictive platform for “gendev” just so we have something to refer to when writing examples further down.

core/arch/arm

In `core/arch/arm` you will find all the currently supported devices. That is where you are supposed to add a new platform or modify an existing one. Typically you will find this set of files in a specific platform folder:

```
$ ls
conf.mk  main.c  platform_config.h  sub.mk
```

So for the gendev platform it means that the files should be placed in this folder:

```
core/arch/arm/plat-gendev
```

conf.mk

This is the device specific makefile where you define configurations unique to your platform. This mainly comprises two things:

- OP-TEE configuration variables (CFG_), which may be assigned values in two ways. `CFG_F00 ?= bar` should be used to provide a default value that may be modified at compile time. On the other hand, variables that must be set to some value and cannot be modified should be set by: `$(call force,CFG_F00,bar)`.
- Compiler flags for the TEE core, the user mode libraries and the Trusted Applications, which may be added to macros used by the build system. Please see [Configuration and flags](#) and similar sections on that page.

It is recommended to use a existing platform configuration file as a starting point. For instance, [core/arch/arm/plat-hikey/conf.mk](#).

The platform `conf.mk` file should at least define the default platform flavor for the platform, the core configurations (architecture and number of cores), the main configuration directives (generic boot, arm trusted firmware support, generic time source, console driver, etc...) and some platform default configuration settings.

```
PLATFORM_FLAVOR ?= hikey

include core/arch/arm/cpu/cortex-armv8-0.mk

$(call force,CFG_TEE_CORE_NB_CORE,8)
$(call force,CFG_GENERIC_BOOT,y)
$(call force,CFG_PL011,y)
$(call force,CFG_PM_STUBS,y)
$(call force,CFG_SECURE_TIME_SOURCE_CNTPCT,y)
$(call force,CFG_WITH_ARM_TRUSTED_FW,y)
$(call force,CFG_WITH_LPAE,y)

ta-targets = ta_arm32
ta-targets += ta_arm64

CFG_NUM_THREADS ?= 8
CFG_CRYPT0_WITH_CE ?= y
CFG_WITH_STACK_CANARIES ?= y
CFG_CONSOLE_UART ?= 3
CFG_DRAM_SIZE_GB ?= 2
```

main.c

This platform specific file will contain power management handlers and code related to the UART. We will talk more about the information related to the handlers further down in this document. For our gendev device it could look like this (here we are excluding the necessary license header to save some space):

```
#include <console.h>
#include <drivers/serial8250_uart.h>
#include <kernel/generic_boot.h>
#include <kernel/panic.h>
#include <kernel/pm_stubs.h>
#include <mm/core_mmu.h>
#include <platform_config.h>
#include <stdint.h>
```

(continues on next page)

(continued from previous page)

```

#include <tee/entry_fast.h>
#include <tee/entry_std.h>

static void main_fiq(void)
{
    panic();
}

static const struct thread_handlers handlers = {
    .std_smc = tee_entry_std,
    .fast_smc = tee_entry_fast,
    .nintr = main_fiq,
    .cpu_on = cpu_on_handler,
    .cpu_off = pm_do_nothing,
    .cpu_suspend = pm_do_nothing,
    .cpu_resume = pm_do_nothing,
    .system_off = pm_do_nothing,
    .system_reset = pm_do_nothing,
};

const struct thread_handlers *generic_boot_get_handlers(void)
{
    return &handlers;
}

/*
 * Register the physical memory area for peripherals etc. Here we are
 * registering the UART console.
 */
register_phys_mem(MEM_AREA_IO_NSEC, CONSOLE_UART_BASE, SERIAL8250_UART_REG_SIZE);

static struct serial8250_uart_data console_data;

void console_init(void)
{
    serial8250_uart_init(&console_data, CONSOLE_UART_BASE,
                        CONSOLE_UART_CLK_IN_HZ, CONSOLE_BAUDRATE);
    register_serial_console(&console_data.chip);
}

```

platform_config.h

This is a mandatory header file for every platform, since there are several files relaying upon the existence of this particular file. This file is where you will find the major differences between different platforms, since this is where you do the memory configuration, define base addresses etc. we are going to list a few here, but it probably makes more sense to have a look at the already existing `platform_config.h` files for the other platforms. Our fictive gendev could look like this:

```

#ifndef PLATFORM_CONFIG_H
#define PLATFORM_CONFIG_H

/* Make stacks aligned to data cache line length */
#define STACK_ALIGNMENT 64

```

(continues on next page)

(continued from previous page)

```

/* 8250 UART */
#define CONSOLE_UART_BASE    0xcafebabe /* UART0 */
#define CONSOLE_BAUDRATE    115200
#define CONSOLE_UART_CLK_IN_HZ    19200000

/* Optional: when used with CFG_WITH_PAGER, defines the device SRAM */
#define TZSRAM_BASE        0x3F000000
#define TZSRAM_SIZE        (200 * 1024)

/* Mandatory main secure RAM usually DDR */
#define TZDRAM_BASE        0x60000000
#define TZDRAM_SIZE        (32 * 1024 * 1024)

/* Mandatory TEE RAM location and core load address */
#define TEE_RAM_START      TZDRAM_BASE
#define TEE_RAM_PH_SIZE    TEE_RAM_VA_SIZE
#define TEE_RAM_VA_SIZE    (4 * 1024 * 1024)
#define TEE_LOAD_ADDR      (TZDRAM_BASE + 0x200000)

/* Mandatory TA RAM (external less secure RAM) */
#define TA_RAM_START      (TZDRAM_BASE + TEE_RAM_VA_SIZE)
#define TA_RAM_SIZE        (TZDRAM_SIZE - TEE_RAM_VA_SIZE)

/* Mandatory: for static SHM, need a hardcoded physical address */
#define TEE_SHMEM_START    0x08000000
#define TEE_SHMEM_SIZE    (4 * 1024 * 1024)

#endif /* PLATFORM_CONFIG_H */

```

This is minimal amount of information in the `platform_config.h` file. I.e., the memory layout for on-chip and external RAM. Note that parts of the DDR typically will need to be shared with normal world, so there is need for some kind of memory firewall for this (more about that further down). As you can see we have also added the UART configuration here, i.e., the `DEVICE0_xyz` part.

Official board support in OP-TEE?

We do encourage everyone to submit their board support to the OP-TEE project itself, so it becomes part of the official releases and will be maintained by the OP-TEE community itself. If you intend to do so, then there are a few more things that you are supposed to do.

Update platforms supported

There is a section at the [Platforms supported](#) page that lists all devices officially supported in OP-TEE, that is where you also shall list your device. It should contain the name of the platform, then composite `PLATFORM` flag and whether the device is publicly available or not. If there is a product page on the internet for the device, please also create a link when writing the device name.

Update `.shippable.yml`

Since we are using Shippable to test pull requests etc, we would like that you also add your device to the `.shippable.yml` file, so that it will at least be built when someone is doing a pull request. Add a line at the end of file:

```
- _make PLATFORM=<platform-name>_
```

Maintainer

If you are submitting the board support upstream we are going to ask you to become the maintainer for the device you have added. This means that you should also update the [MAINTAINERS.md](#) file accordingly. By being a maintainer for a device you are responsible to keep it up to date and you will be asked every quarter as part of the OP-TEE release schedule to test your device running the latest OP-TEE software.

Update build.git and manifest.git

This isn't strictly necessary, but we are trying to create and maintain OP-TEE developer builds that should make it easy to setup, build and deploy OP-TEE on various devices. We encourage all maintainers to do the same for the boards they are in charge of. Therefore please consider creating a new *manifest* (and a new **.mk* in *build*) for the device you have added to OP-TEE.

2.8.2 Hardware Unique Key

Most devices have some kind of Hardware Unique Key (HUK) that is mainly used to derive other keys. The HUK could for example be used when deriving keys used in secure storage etc. The important thing with the HUK is that it needs to be well protected and in the best case the HUK should never ever be readable directly from software, not even from the secure side. There are different solutions to this, crypto accelerator might have support for it or, it could involve another secure co-processor.

In OP-TEE the HUK is just **stubbed** and you will see that in the function called `tee_otp_get_hw_unique_key(...)` in `core/include/kernel/tee_common_otp.h`. In a real secure product you **must** replace this with something else. If your device lacks the hardware support for a HUK, then you must at least change this to something else than just zeroes. But, remember it is not good secure practice to store a key in software, especially not the key that is the root for everything else, so this is not something we recommend that you should do.

2.8.3 Secure Clock

The Time API in GlobalPlatform Internal Core API specification defines three sources of time; system time, TA persistent time and REE time. The REE time is by nature considered as an unsecure source of time, but the other two should in a fully trustable hardware make use of trustable source of time, i.e., a secure clock. Note that from GlobalPlatform point of view it is not required to make use of a secure clock, i.e., it is OK to use time from REE, but the level of trust should be reflected by the `gpd.tee.systemTime.protectionLevel` property and the `gpd.tee.TAPersistentTime.protectionLevel` property (100=REE controlled clock, 1000=TEE controlled clock). So the functions that one needs to pay attention to are `tee_time_get_sys_time(...)` and `tee_time_get_ta_time(...)`. If your hardware has a secure clock, then you probably want to change the implementation there to instead use the secure clock (and then you would also need to update the property accordingly, i.e., `tee_time_get_sys_time_protection_level()` and the variable `ta_time_prot_lvl` in `tee_svc.c`).

2.8.4 Root and Chain of Trust

To be able to assure that your devices are running the (untampered) binaries you intended to run you will need to establish some kind of trust anchor on the devices.

The most common way of doing that is to put the root public key in some read only memory on the device. Quite often SoC's/OEM's stores public key(s) directly or the hash(es) of the public key(s) in [OTP](#). When the boot ROM (which indeed needs to be ROM) is about to load the first stage bootloader it typically reads the public key from the software binary itself, hash the key and compare it to the key in [OTP](#). If they are matching, then the boot ROM can be sure that the first stage bootloader was indeed signed with the corresponding private key.

In OP-TEE you will not find any code at all related to this and this is a good example when it is hard for us to do this in a generic way since device manufacturers all tend to do this in their own unique way and they are not very keen on sharing their low level boot details and security implementation with the rest of the world. This is especially true on ARMv7-A. For ARMv8-A it looks bit better, since Arm in Trusted Firmware A have implemented and defined how to abstract the chain of trust (see [auth-framework.rst](#)). We have successfully verified OP-TEE by using the authentication framework from Trusted Firmware A (see [Secure boot](#) for the details).

2.8.5 Hardware Crypto IP

By default OP-TEE uses a software crypto library (currently mbed TLS and LibTomCrypt) and you have the ability to enable Crypto Extensions that were introduced with ARMv8-A (if the device is capable of that). Some of the devices supported in OP-TEE OS repository have hardware crypto capabilities. A framework, named `drvccrypt` has been designed to integrate them. The `drvccrypt_register_*`() API functions allow drivers to register support for given cryptographic operations in OP-TEE core crypto API. Our [Cryptographic implementation](#) page describes in detail how the Crypto API is integrated.

2.8.6 Random Number Generator

By default OP-TEE is configured with a software PRNG. The entropy is added to software PRNG at various places, but unfortunately it is still quite easy to predict the data added as entropy. As a consequence, unless the RNG is based on hardware the generated random will be quite weak.

If your platform has a hardware entropy source, you should set `CFG_WITH_SOFTWARE_PRNG` to `n`, and provide an implementation for `hw_get_random_bytes()`, which returns multiple bytes of entropy.

When `CFG_WITH_SOFTWARE_PRNG=n`, the platform can enable a PTA service for normal world to retrieve good quality random bytes. See configuration switches `CFG_HWRNG_PTA` and `CFG_HWRNG_QUALITY`, from 0 to 1024.

When `CFG_WITH_SOFTWARE_PRNG=n`, the random number generator is made available to OP-TEE drivers and frameworks, including Trusted Applications (thoguh GP TEE Internal Core API) and normal world (when `CFG_HWRNG_PTA=y`).

2.8.7 Power Management / PSCI

In the [Add a new platform](#) section where we talked about the file `main.c`, we added a couple of handlers related to power management, we are talking about the following lines:

```
.cpu_on = cpu_on_handler,  
.cpu_off = pm_do_nothing,  
.cpu_suspend = pm_do_nothing,  
.cpu_resume = pm_do_nothing,  
.system_off = pm_do_nothing,  
.system_reset = pm_do_nothing,
```

The only function that actually does something there is the `cpu_on` function, the rest of them are stubbed. The main reason for that is because we think that how to suspend and resume is a device dependent thing. The code in OP-TEE is prepared so that callbacks etc from Trusted Firmware A will be routed to OP-TEE, but since the function(s) are just stubbed we will not do anything and just return. In a real production device, you would probably want to save and restore CPU states, secure hardware IPs' registers and TZASC and other memory firewall related setting when these callbacks are being called.

2.8.8 Memory firewalls / TZASC

Arm have defined a system IP / SoC peripheral called TrustZone Address Space Controller (TZASC, see [TZC-380](#) and [TZC-400](#)). TZASC can be used to configure DDR memory into separate regions in the physical address space, where each region can have an individual security level setting. After enabling TZASC, it will perform security checks on transactions to memory or peripherals. It is not always the case that TZASC is on a device, in some cases the SoC has developed something equivalent. In OP-TEE this is very well reflected, i.e., different platforms have different ways of protecting their memory. On ARMv8-A platforms we are in most of the cases using Trusted Firmware A as the boot firmware and there the secure bootloader is the one that configures secure vs non-secure memory using TZASC (see [plat_arm_security_setup](#) in TF-A). The takeaway here is that you must make sure that you have configured whatever memory firewall your device has such that it has a secure and a non-secure memory area.

2.8.9 Trusted Application private/public keypair

By default all Trusted Applications (TA's) are signed with the pre-generated 2048-bit RSA development key (private key). This key is located in the `keys` folder (in the root of `optee_os.git`) and is named `default_ta.pem`. This key **must** be replaced with your own key and you should **never ever** check-in this private key in the source code tree when in use in a real product. The recommended way to store private keys is to use some kind of [HSM](#) (Hardware Security Module), but an alternative would be temporary put the private key on a computer considered as secure when you are about to sign TA's intended to be used in real products. Typically it is only a few number of people having access to this type of key in company. The key handling in OP-TEE is currently a bit limited since we only support a single key which is used for all TA's. We have plans on extending this to make it a bit more flexible. Exactly when that will happen has not been decided yet.

2.8.10 Platform ports

OP-TEE is a reference implementation for developers and device manufacturers. This also implies that there are certain configurations and settings that cannot be done in OP-TEE reference code. In short, there are cases when the default configuration hasn't enabled all necessary security features for the end product. There are a couple of reasons for that.

- Chipmakers and Semiconductors might only share specifications telling how to securely configure their devices with partners who have signed an NDA with them.
- In some cases a setting might be perfectly fine when OP-TEE is used in one particular environment, but the same setting might be insecure in another environment.

Because of this we always urge companies and device manufacturers making the end product to follow the security guidelines from the chipmaker they are basing their products on. Refer also to [Platform documentation](#)

2.9 Secure boot

2.9.1 Armv8-A - Using the authentication framework in TF-A

This section gives a brief description on how to enable the verification of OP-TEE using the authentication framework in Trusted Firmware A (TF-A), i.e., something that could be used in an Armv8-A environment.

According to [user-guide.rst](#), there is no additional specific build options for the verification of OP-TEE. If we have enabled the authentication framework and specified the BL32 build option when building TF-A, the BL32 related certificates will be created automatically by the `cert_create` tool, and then these certificates will be verified during booting up.

To enable the authentication framework, the following steps should be followed according to [user-guide.rst](#). For more details about the authentication framework, please see [auth-framework.rst](#) and [trusted-board-boot.rst](#).

- Check out a recent version of the [mbed TLS](#) repository and then switch to tag mbedtls-2.2.0
- Besides the normal build options, add the following build options for TF-A

```
MBEDTLS_DIR=<path of the directory containing mbed TLS sources>
TRUSTED_BOARD_BOOT=1
GENERATE_COT=1
ARM_ROTPK_LOCATION=devel_rsa
ROT_KEY=<TF-A-PATH/plat/arm/board/common/rotpk/arm_rotprivk_rsa.pem>
```

Above steps have been tested on FVP platform, all verification steps are OK and xtest runs successfully without regression.

2.9.2 Armv7-A systems

Unlike for Armv8-A systems where one can use a more standardized way of doing secure boot by leverage the authentication framework as described above, most device manufacturers have their own way of doing secure boot. Please reach out directly to the manufacturer for the device you are working with to be able to understand how to do secure boot on their devices.

Note however that TF-A supports Armv7-A with Trustzone extension and we strongly encourage one to look at TF-A and use its BL2 as secure boot loader.

2.10 Secure storage

2.10.1 Background

Secure Storage in OP-TEE is implemented according to what has been defined in GlobalPlatform's *TEE Internal Core API* (here called Trusted Storage). This specification mandates that it should be possible to store general-purpose data and key material that guarantees confidentiality and integrity of the data stored and the atomicity of the operations that modifies the storage (atomicity here means that either the entire operation completes successfully or no write is done).

There are currently two secure storage implementations in OP-TEE:

- The first one relies on the normal world (REE) file system. It is described in this document and is the default implementation. It is enabled at compile time by `CFG_REE_FS=y`.
- The second one makes use of the Replay Protected Memory Block (RPMB) partition of an eMMC device, and is enabled by setting `CFG_RPMB_FS=y`. It is described in *RPMB Secure Storage*.

It is possible to use the normal world file systems and the RPMB implementations simultaneously. For this, two OP-TEE specific storage identifiers have been defined: `TEE_STORAGE_PRIVATE_REE` and `TEE_STORAGE_PRIVATE_RPMB`. Depending on the compile-time configuration, one or several values may be used. The value `TEE_STORAGE_PRIVATE` selects the REE FS when available, otherwise the RPMB FS (in this order).

2.10.2 REE FS Secure Storage

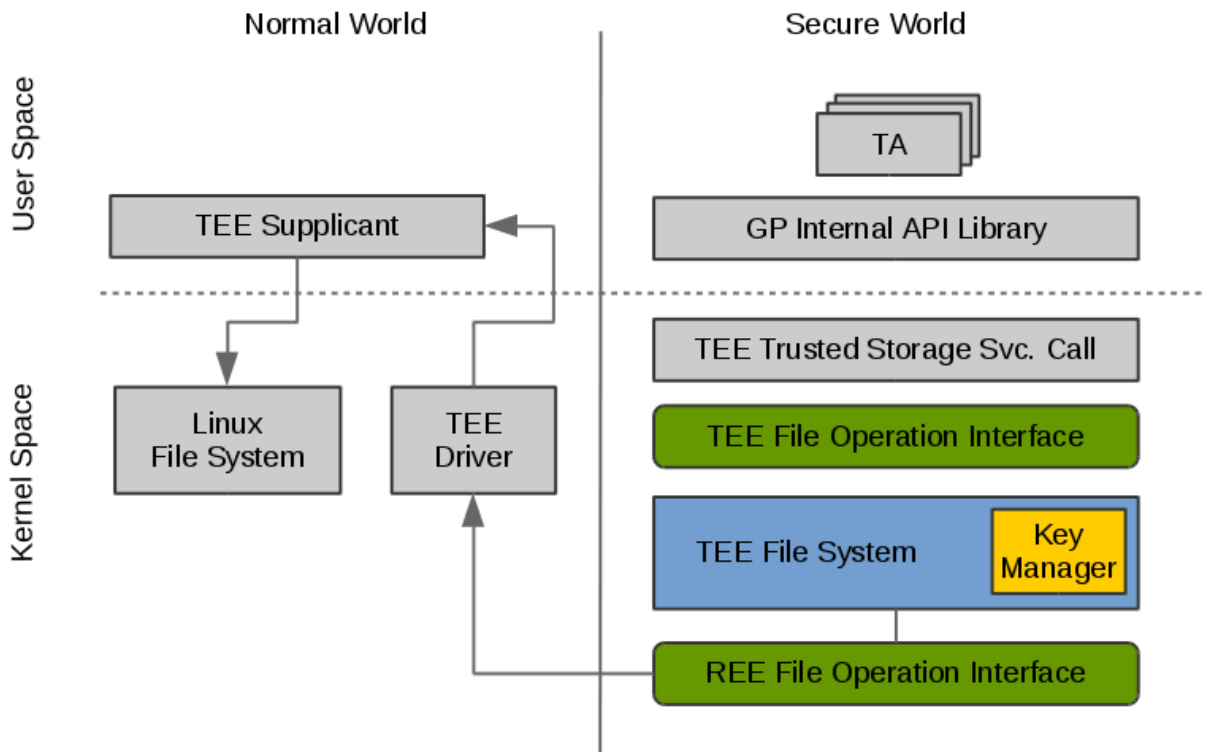


Fig. 15: Secure Storage System Architecture

Source Files in OP-TEE OS

Table 4: Secure storage files

Source file	Purpose
<code>core/tee/tee_svc</code>	TEE trusted storage service calls
<code>core/tee/tee_ree</code>	TEE file system & REE file operation interface
<code>core/tee/fs_htre</code>	Hash tree
<code>core/tee/tee_fs</code>	Key manager
<code>lib/libutee/</code>	GlobalPlatform Internal API library

Basic File Operation Flow

When a TA is calling the write function provided by GP Trusted Storage API to write data to a persistent object, a corresponding syscall implemented in TEE Trusted Storage Service will be called, which in turn will invoke a series of TEE file operations to store the data. TEE file system will then encrypt the data and send REE file operation commands and the encrypted data to TEE supplicant by a series of RPC messages. TEE supplicant will receive the messages and store the encrypted data accordingly to the Linux file system. Reading files are handled in a similar manner.

GlobalPlatform Trusted Storage Requirement

Below is an excerpt from the specification, listing the most vital requirements:

1. The Trusted Storage may be backed by non-secure resources as long as suitable cryptographic protection is applied, which **MUST** be as strong as the means used to protect the TEE code and data itself.
2. The Trusted Storage **MUST** be bound to a particular device, which means that it **MUST** be accessible or modifiable only by authorized TAs running in the same TEE and on the same device as when the data was created.
3. Ability to hide sensitive key material from the TA itself.
4. Each TA has access to its own storage space that is shared among all the instances of that TA but separated from the other TAs.
5. The Trusted Storage must provide a minimum level of protection against rollback attacks. It is accepted that the actually physical storage may be in an insecure area and so is vulnerable to actions from outside of the TEE. Typically, an implementation may rely on the REE for that purpose (protection level 100) or on hardware assets controlled by the TEE (protection level 1000).

(see GP TEE Internal Core API section 2.5 and 5.2)

If configured with `CFG_RPMB_FS=y` the protection against rollback is controlled by the TEE and is set to 1000. If `CFG_RPMB_FS=n`, there's no protection against rollback, and the protection level is set to 0.

TEE File Structure in Linux File System

OP-TEE by default uses `/data/tee/` as the secure storage space in the Linux file system. Each persistent object is assigned an internal identifier. It is an integer which is visible in the Linux file system as `/data/tee/<file number>`.

A directory file, `/data/tee/dirf.db`, lists all the objects that are in the secure storage. All normal world files are integrity protected and encrypted, as described below.

2.10.3 Key Manager

Key manager is a component in TEE file system, and is responsible for handling data encryption and decryption and also management of the sensitive key materials. There are three types of keys used by the key manager: the Secure Storage Key (SSK), the TA Storage Key (TSK) and the File Encryption Key (FEK).

Secure Storage Key (SSK)

SSK is a per-device key and is generated and stored in secure memory when OP-TEE is booting. SSK is used to derive the TA Storage Key (TSK).

SSK is derived by

$$\text{SSK} = \text{HMAC}_{\text{SHA256}}(\text{HUK}, \text{Chip ID} \parallel \text{"static string"})$$

The functions to get *Hardware Unique Key* (HUK) and chip ID depends on the platform implementation. Currently, in OP-TEE OS we only have a per-device key, SSK, which is used for secure storage subsystem, but, for the future we might need to create different per-device keys for different subsystems using the same algorithm as we generate the SSK; An easy way to generate different per-device keys for different subsystems is using different static strings to generate the keys.

Trusted Application Storage Key (TSK)

The TSK is a per-Trusted Application key, which is generated from the SSK and the TA's identifier (UUID). It is used to protect the FEK, in other words, to encrypt/decrypt the FEK.

TSK is derived by:

$$\text{TSK} = \text{HMAC}_{\text{SHA256}}(\text{SSK}, \text{TA_UUID})$$

File Encryption Key (FEK)

When a new TEE file is created, key manager will generate a new FEK by PRNG (pesudo random number generator) for the TEE file and store the encrypted FEK in meta file. FEK is used for encrypting/decrypting the TEE file information stored in meta file or the data stored in block file.

2.10.4 Hash Tree

The hash tree is responsible for handling data encryption and decryption of a secure storage file. The hash tree is implemented as a binary tree where each node (`struct tee_fs_htree_node_image` below) in the tree protects its two child nodes and a data block. The meta data is stored in a header (`struct tee_fs_htree_image` below) which also protects the top node.

All fields (header, nodes, and blocks) are duplicated with two versions, 0 and 1, to ensure atomic updates. See [core/tee/fs_htree.c](#) for details.

Meta Data Encryption Flow

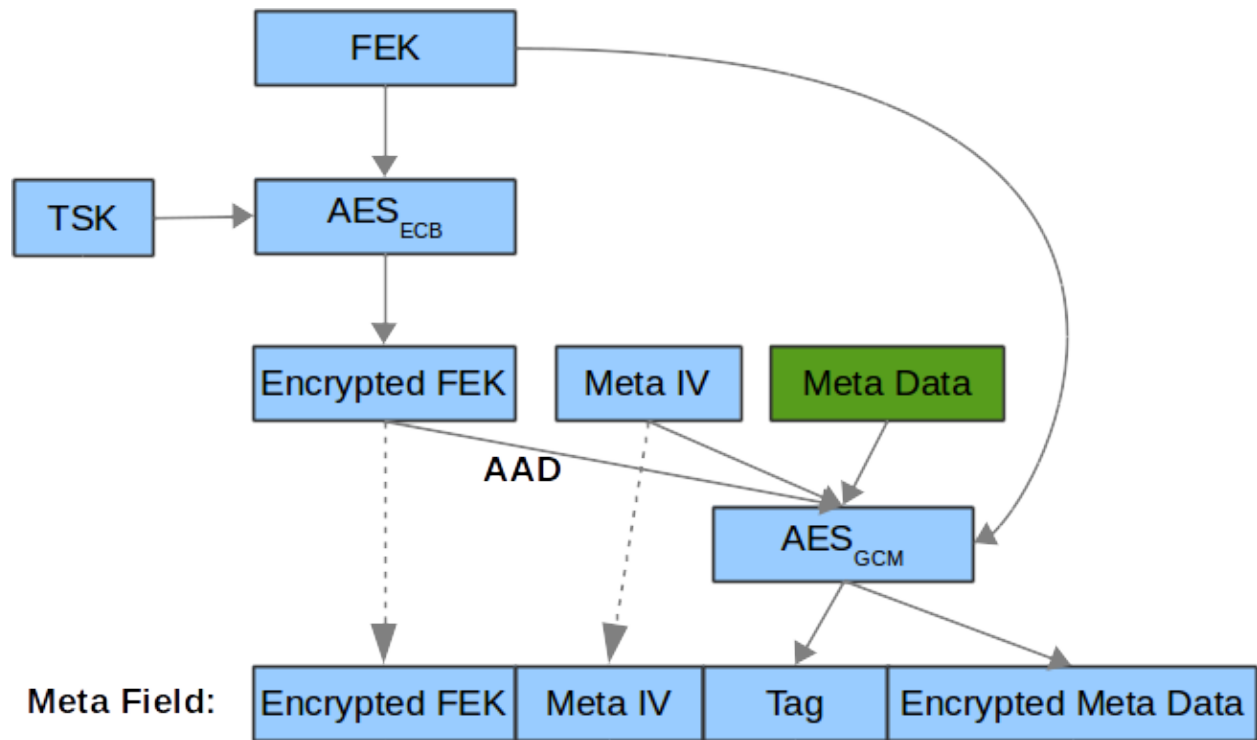


Fig. 16: Meta data encryption

A new meta IV will be generated by PRNG when a meta data needs to be updated. The size of meta IV is defined in `core/include/tee/fs_htree.h`, likewise are the data structures of meta data and node data are defined in `fs_htree.h` as follows:

```

struct tee_fs_htree_node_image {
    uint8_t hash[TEE_FS_HTREE_HASH_SIZE];
    uint8_t iv[TEE_FS_HTREE_IV_SIZE];
    uint8_t tag[TEE_FS_HTREE_TAG_SIZE];
    uint16_t flags;
};

struct tee_fs_htree_meta {
    uint64_t length;
};

struct tee_fs_htree_imeta {
    struct tee_fs_htree_meta meta;
    uint32_t max_node_id;
};

struct tee_fs_htree_image {
    uint8_t iv[TEE_FS_HTREE_IV_SIZE];
    uint8_t tag[TEE_FS_HTREE_TAG_SIZE];
    uint8_t enc_fek[TEE_FS_HTREE_FEK_SIZE];
    uint8_t imeta[sizeof(struct tee_fs_htree_imeta)];
};

```

(continues on next page)

(continued from previous page)

```
uint32_t counter;
};
```

Block Data Encryption Flow

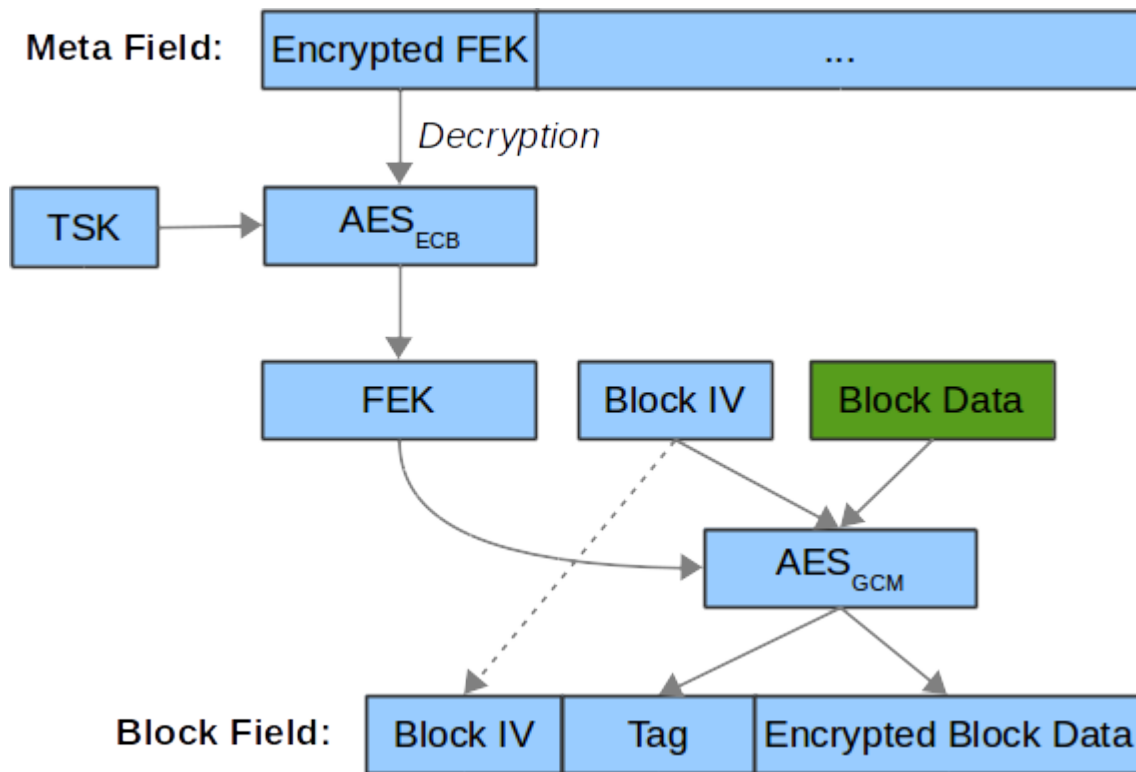


Fig. 17: Block data encryption

A new block IV will be generated by PRNG when a block data needs to be updated. The size of block IV is defined in `core/include/tee/fs_htree.h`.

2.10.5 Atomic Operation

According to GlobalPlatform Trusted Storage requirement of the atomicity, the following operations should support atomic update:

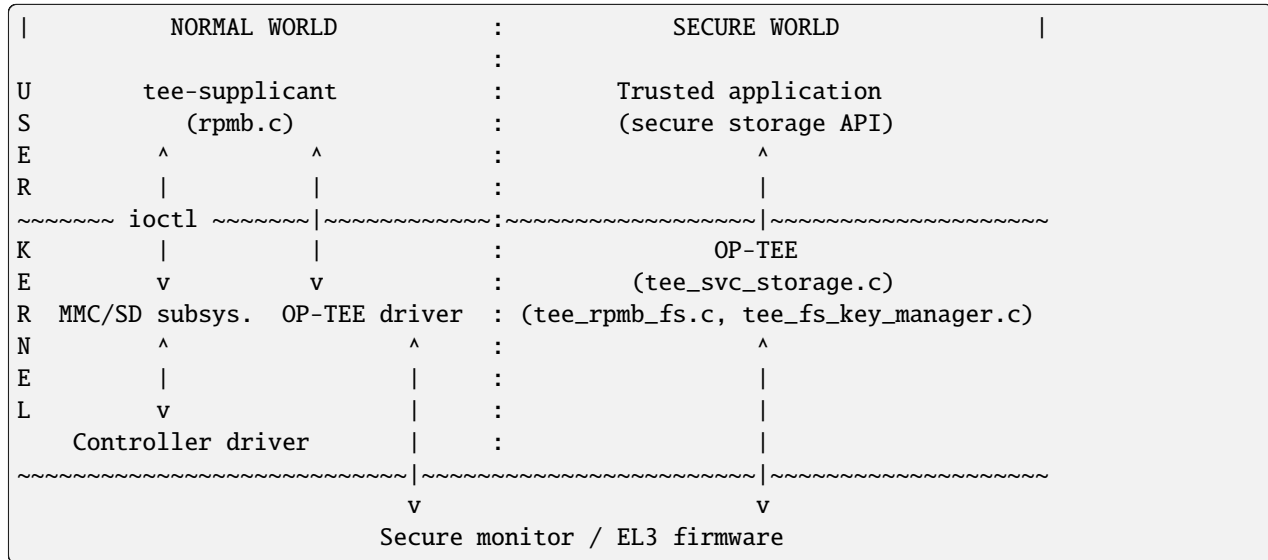
Write, Truncate, Rename, Create **and** Delete

The strategy used in OP-TEE secure storage to guarantee the atomicity is out-of-place update.

2.10.6 RPMB Secure Storage

This document describes the RPMB secure storage implementation in OP-TEE, which is enabled by setting `CFG_RPMB_FS=y`. Trusted Applications may use this implementation by passing a storage ID equal to `TEE_STORAGE_PRIVATE_RPMB`, or `TEE_STORAGE_PRIVATE` if `CFG_REE_FS` is disabled. For details about RPMB, please refer to the JEDEC eMMC specification (JESD84-B51).

The architecture is depicted below.



For information about the `ioctl()` interface to the MMC/SD subsystem in the Linux kernel, see the Linux core MMC header file [linux/mmc/core.h](#) and the [mmc-utils](#) repository.

The Secure Storage API

This part is common with the REE-based filesystem. The interface between the system calls in [core/tee/tee_svc_storage.c](#) and the RPMB filesystem is the `tee_file_operations`, namely `struct tee_file_ops`.

The RPMB filesystem

The FS implementation is entirely in [core/tee/tee_rpmb_fs.c](#) and the RPMB partition is divided in three parts:

- The first 128 bytes are reserved for partition data (`struct rpmb_fs_partition`).
- At offset 512 is the File Allocation Table (FAT). It is an array of `struct rpmb_fat_entry` elements, one per file. The FAT grows dynamically as files are added to the filesystem. Among other things, each entry has the start address for the file data, its size, and the filename.
- Starting from the end of the RPMB partition and extending downwards is the file data area.

Space in the partition is allocated by the general-purpose allocator functions, `tee_mm_alloc(...)` and `tee_mm_alloc2(...)`.

All file operations are atomic. This is achieved thanks to the following properties:

- Writing one single block of data to the RPMB partition is guaranteed to be atomic by the eMMC specification.
- The FAT block for the modified file is always updated last, after data have been written successfully.

- Updates to file content is done in-place only if the data do not span more than the “reliable write block count” blocks. Otherwise, or if the file needs to be extended, a new file is created.

Device access

There is no eMMC controller driver in OP-TEE. The device operations all have to go through the normal world. They are handled by the `tee-suppllicant` process which further relies on the kernel’s `ioctl()` interface to access the device. `tee-suppllicant` also has an emulation mode which implements a virtual RPMB device for test purposes.

RPMB operations are the following:

- Reading device information (partition size, reliable write block count).
- Programming the security key. This key is used for authentication purposes. Note that it is different from the Secure Storage Key (SSK) defined below, which is used for encryption. Like the SSK however, the security key is also derived from a hardware unique key or identifier. Currently, the function `tee_otp_get_hw_unique_key()` is used to generate the RPMB security key.
- Reading the write counter value. The write counter is used in the HMAC computation during read and write requests. The value is read at initialization time, and stored in `struct tee_rpmb_ctx`, i.e., `rpmb_ctx->wr_cnt`.
- Reading or writing blocks of data.

RPMB operations are initiated on request from the FS layer. Memory buffers for requests and responses are allocated in shared memory using `thread_rpc_alloc_payload(...)`. Buffers are passed to the normal world in a `TEE_RPC_RPMB_CMD` message, thanks to the `thread_rpc_cmd()` function. Most RPMB requests and responses use the data frame format defined by the JEDEC eMMC specification. HMAC authentication is implemented here also.

Security considerations

The RPMB partition in eMMC can not be accessed until a key has been programmed on the device: this is a one time action for the lifetime of the device. Once the key has been written on the eMMC controller, the controller uses it to authenticate requests.

The RPMB key can be programmed in a number of ways. The safest and most secure way is to program it from normal world during production in a secure environment. `MMC-tools` can for instance be used to program the RPMB key.

If you want OP-TEE to program the key automatically, OP-TEE must be configured with `CFG_RPMB_WRITE_KEY=y`.

Warning: Be aware that this configuration will send the RPMB key in clear to the non-secure side that relays the RPMB key programming request to the eMMC hardware device.

This configuration should only be enabled in a test or development environment.

OP-TEE can either embed a built-in RPMB key (`CFG_RPMB_TESTKEY=y`) or derive it from platform specific secrets (`CFG_RPMB_TESTKEY=n`). The former case might be useful during development while the later is recommended for production devices.

Deriving the key from secrets avoids OP-TEE from having to store it in memory therefore reducing the attack surface; OP-TEE derives the RPMB key from an internal set that includes the eMMC serial number and more importantly the Hardware Unique Key (HUK).

For this configuration to be effective, the **Hardware Unique Key** - a unique identifier for the particular instantiation of the SoC - must not be publicly accessible (please notice that not all platforms might be enforcing this requirement).

The need to keep the HUK secret is the reason why on security aware systems, the hardware will generate different HUK values depending on the security state of the platform: said differently, the SoC will generate different HUK depending if the BOOT ROM it is configured to boot signed images or not.

However notice that, since the RPMB key can only be written once on the controller, it follows that accessing RPMB before securing the board will cause future RPMB accesses to be denied once the board has been secured. To prevent this situation from happening, OP-TEE provides a software hook which platforms shall use to implement its security logic `plat_rpmb_key_is_ready()`.

Warning: For OP-TEE to be able to write the RPMB key, `CFG_RPMB_WRITE_KEY=y` must be configured and `plat_rpmb_key_is_ready()` must allow it at runtime.

When programming the RPMB key from normal world the RPMB key must be made available to that tool. The RPMB key must for security reasons normally not be known outside OP-TEE, but an exception might be made in the factory during the manufacturing process.

If the HUK is known, the script `scripts/derive_rpmb_key.py` can be used to derive the RPMB key.

Pros and cons with OP-TEE automatically writing the RPMB key:

1. Automatic writing can be triggered after each boot controlled entirely by the normal world, essentially tricking the secure world to reveal the secret RPMB key. Having a separate OP-TEE binary in the factory is not fully secure since it is enough to restore it from one device to achieve class breakage.
2. Automatic writing may target the wrong device if there is more than one RPMB since the device name it determined by the probe order. The probe order may differ between boot loader and the kernel or when rebooting. This problem can be addressed by supplying `-rpmb-cid` and the CID of the MMC device to use as argument.
3. Automatic writing must be used at the moment if tee-supplciant is configured to emulate RPMB since it starts from scratch at each boot.

Encryption

The FS encryption routines are in `core/tee/tee_fs_key_manager.c`. Block encryption protects file data. The algorithm is 128-bit AES in Cipher Block Chaining (CBC) mode with Encrypted Salt-Sector Initialization Vector (ESSIV), see `CBC-ESSIV` for details.

- During OP-TEE initialization, a 128-bit AES Secure Storage Key (SSK) is derived from a *Hardware Unique Key* (HUK). It is kept in secure memory and never written to disk. A Trusted Application Storage Key is derived from the SSK and the TA UUID.
- For each file, a 128-bit encrypted File Encryption Key (FEK) is randomly generated when the file is created, encrypted with the TSK and stored in the FAT entry for the file.
- Each 256-byte block of data is then encrypted in CBC mode. The initialization vector is obtained by the ESSIV algorithm, that is, by encrypting the block number with a hash of the FEK. This allows direct access to any block in the file, as follows:

```
FEK = AES-Decrypt(TSK, encrypted FEK);
k = SHA256(FEK);
IV = AES-Encrypt(128 bits of k, block index padded to 16 bytes)
Encrypted block = AES-CBC-Encrypt(FEK, IV, block data);
Decrypted block = AES-CBC-Decrypt(FEK, IV, encrypted block data);
```

SSK, TSK and FEK handling is common with the REE-based secure storage, while the AES CBC block encryption is used only for RPMB (the REE implementation uses GCM). The FAT is not encrypted.

REE FS hash state

If configured with both `CFG_REE_FS=y` and `CFG_RPMB_FS=y` the REE FS will create a special file, `dirfile.db.hash` in RPMB which hold a hash representing the state of REE FS.

2.10.7 Important caveats

Warning: Currently some OP-TEE platform are not able to support retrieval of the Hardware Unique Key or Chip ID required for secure operation. For those platforms, a constant key is used, resulting in no protection against decryption, or Secure Storage duplication to other devices. This is because information about how to retrieve key data from the SoC is considered sensitive by some vendors and it is not publicly available.

To allow Secure Storage to operate securely on your platform, you must define implementations in your platform code for:

```
void tee_opt_get_hw_unique_key(struct tee_hw_unique_key *hwkey);

int tee_opt_get_die_id(uint8_t *buffer, size_t len);
```

These implementations should fetch the key data from your SoC-specific e-fuses, or crypto unit according to the method defined by your SoC vendor.

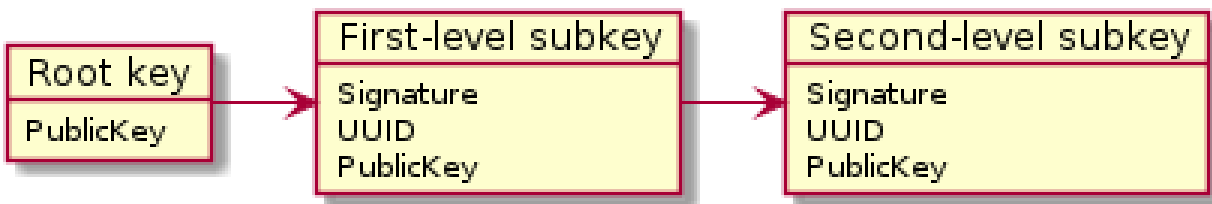
2.10.8 References

For more information about secure storage, please see SFO15-503, LAS16-504, SFO17-309 at [Presentations](#) and the *TEE Internal Core API* specification.

2.11 Subkeys

Subkeys is an OP-TEE-specific implementation to provide a public key hierarchy. Subkeys can be delegated to allow different actors to sign different TAs without sharing a private key.

The first key in the chain is verified using a root key. Example Subkey hierarchy:



Each subkey defines a UUIDv5 namespace¹ to which another signed subkey or TA must belong. This avoids that one subkey can be used to sign TAs which should be signed with a different key. Since the UUIDs are restricted by this there is also a special kind of subkey, called identity subkey, which uses the same UUID as the TA it is supposed to sign.

¹ UUIDv5 and namespaces are described in [RFC4122](#). Note that OP-TEE uses a truncated SHA-512 instead of the weak SHA-1 hash when deriving a new UUID from a namespace and name.

A subkey consists of two files, a private key pair in a .pem file and the signed public key in a .bin file. Subkeys reuse the signed header (SHDR) format used for signed TAs, followed by a payload holding a public key and UUID among other fields. A subkey is formatted with all fields in little endian as:

Size in bytes	Field Name	Protected by field	
Signed header (struct shdr)			
4	magic	hash	0x4f545348
4	img_type	hash	3, SHDR_SUBKEY
4	img_size	hash	
4	algo	hash	Signature algorithm, GP TEE_ALG_*
4	hash_size	hash	
4	sig_size	hash	
hash_size	hash	sig	
sig_size	sig	previous pub key	Root key or a subkey higher up in the chain
Subkey header (struct shdr_subkey)			
16	UUID	hash	UUIDv5 namespace for next subkey or TA
4	name_size	hash	Size of the UUIDv5 name for the next subkey or TA, 0 for identiy subkeys
4	subkey_version	hash	
4	max_depth	hash	
4	algo	hash	Signature algorithm for the next subkey or TA
4	attr_count	hash	
	Subkey attrs * attr_count		Attributes of the public key in this subkey
4	id	hash	GP TEE_ATTR_*
4	offs	hash	
4	size	hash	
opaque data padding up this binary to img_size		hash	The subkey attributes above point into this area using offset and size

All subkeys included in the subkey hierarchy are added in front when a TA is signed using a subkey. For example, if a TA is signed using the second-level subkey above it would look like this:

Size in bytes	Binary
first.img_s	First-level subkey Signed by Root key
first.name_	UUIDv5 name Not signed, used to prove that the next UUID is in the namespace of the First-level subkey. This size is from “name_size” of the previous subkey (First-level).
sec-ond.img_s	Second-level subkey Signed by First-level subkey
sec-ond.name_	UUIDv5 name Not signed used to prove that the next UUID is in the namespace of the Second-level subkey. This size is from “name_size” of the previous subkey (Second-level).
sec-ond.img_s	TA Signed by Second-level subkey

The signed TA binary is self-contained with all the public keys needed for verification included, except the public root key which is embedded in the TEE core binary.

The UUIDv5 name string is a separate field between subkeys and the next subkey or TA to allow a subkey to be used to sign more than one other subkey or TA.

A signed TA or subkey can be inspected using the `sign_encrypt.py` script, for example:

```
$ scripts/sign_encrypt.py display --in 5c206987-16a3-59cc-ab0f-64b9cfc9e758.ta
Subkey
struct shdr
  magic:      0x4f545348
  img_type:   3 (SHDR_SUBKEY)
  img_size:   320 bytes
  algo:       0x70414930 (TEE_ALG_RSASSA_PKCS1_PSS_MGF1_SHA256)
  hash_size:  32 bytes
  sig_size:   256 bytes
  hash:       f573f329fe77be686ce71647909c4ea35b5e1cd7de86369bd7d9fca31f6a4d65
struct shdr_subkey
  uuid:       f04fa996-148a-453c-b037-1dcfbad120a6
  name_size:  64
  subkey_version: 1
  max_depth:  4
  algo:       0x70414930 (TEE_ALG_RSASSA_PKCS1_PSS_MGF1_SHA256)
  attr_count: 2
  next name:  "mid_level_subkey"
Next header at offset: 692 (0x2b4)
Subkey
struct shdr
  magic:      0x4f545348
  img_type:   3 (SHDR_SUBKEY)
  img_size:   320 bytes
  algo:       0x70414930 (TEE_ALG_RSASSA_PKCS1_PSS_MGF1_SHA256)
  hash_size:  32 bytes
  sig_size:   256 bytes
  hash:       233a6dcf1a2cf69e50cde8e20c4129157da707c76fa86ce12ee31037edef02d7
struct shdr_subkey
  uuid:       1a5948c5-1aa0-518c-86f4-be6f6a057b16
  name_size:  64
  subkey_version: 1
  max_depth:  3
  algo:       0x70414930 (TEE_ALG_RSASSA_PKCS1_PSS_MGF1_SHA256)
  attr_count: 2
  next name:  "subkey1_ta"
Next header at offset: 1384 (0x568)
Bootstrap TA
struct shdr
  magic:      0x4f545348
  img_type:   1 (SHDR_BOOTSTRAP_TA)
  img_size:   84576 bytes
  algo:       0x70414930 (TEE_ALG_RSASSA_PKCS1_PSS_MGF1_SHA256)
  hash_size:  32 bytes
  sig_size:   256 bytes
  hash:       ea31ac7dc2cc06a9dc2853cd791dd00f784b5edc062ecfa274deeb66589b4ca5
```

(continues on next page)

(continued from previous page)

```
struct shdr_bootstrap_ta
  uuid:      5c206987-16a3-59cc-ab0f-64b9cfc9e758
  ta_version: 0
  TA offset: 1712 (0x6b0) bytes
  TA size:   84576 (0x14a60) bytes
```

2.12 Trusted Applications

There are two ways to implement Trusted Applications (TAs), Pseudo TAs and user mode TAs. User mode TAs are full featured Trusted Applications as specified by the *GlobalPlatform API* TEE specifications, these are simply the ones people are referring to when they are saying “Trusted Applications” and in most cases this is the preferred type of TA to write and use.

2.12.1 Pseudo Trusted Applications

A Pseudo Trusted Application is not a Trusted Application. A Pseudo TA is not a specific entity. A Pseudo TA is an interface. It is an interface exposed by the OP-TEE Core to its outer world: to secure client Trusted Applications and to non-secure client entities.

These are implemented directly to the OP-TEE core tree in, e.g., `core/pta` and are built along with and statically built into the OP-TEE core blob.

The Pseudo Trusted Applications included in OP-TEE already are OP-TEE secure privileged level services hidden behind a “GlobalPlatform TA Client” API. These Pseudo TAs are used for various purposes such as specific secure services or embedded tests services.

Pseudo TAs **do not** benefit from the GlobalPlatform Core Internal API support specified by the GlobalPlatform TEE specs. These APIs are provided to TAs as a static library each TA shall link against (the “*libutee*”) and that calls OP-TEE core service through system calls. As OP-TEE core does not link with *libutee*, Pseudo TAs can **only** use the OP-TEE core internal APIs and routines.

As Pseudo TAs runs at the same privileged execution level as the OP-TEE core code itself and that might or might not be desirable depending on the use case.

In most cases an unprivileged (user mode) TA is the best choice instead of adding your code directly to the OP-TEE core. However if you decide your application is best handled directly in OP-TEE core like this, you can look at `core/pta/stats.c` as a template and just add your Pseudo TA based on that to the `sub.mk` in the same directory.

2.12.2 User Mode Trusted Applications

User Mode Trusted Applications are loaded (mapped into memory) by OP-TEE core in the Secure World when something in Rich Execution Environment (REE) wants to talk to that particular application UUID. They run at a lower CPU privilege level than OP-TEE core code. In that respect, they are quite similar to regular applications running in the REE, except that they execute in Secure World.

Trusted Application benefit from the GlobalPlatform *TEE Internal Core API* as specified by the GlobalPlatform TEE specifications. There are several types of user mode TAs, which differ by the way they are stored.

2.12.3 TA locations

Plain TAs (user mode) can reside and be loaded from various places. There are three ways currently supported in OP-TEE.

Early TA

The so-called early TAs are virtually identical to the REE FS TAs, but instead of being loaded from the Normal World file system, they are linked into a special data section in the TEE core blob. Therefore, they are available even before `tee-suppllicant` and the REE's filesystems have come up. Please find more details in the [early TA commit](#).

REE filesystem TA

They consist of a [ELF](#) file, signed and optionally encrypted, named from the UUID of the TA and the suffix `.ta`. They are built separately from the OP-TEE core boot-time blob, although when they are built they use the same build system, and are signed with the key from the build of the original OP-TEE core blob.

Because the TAs are signed and optionally encrypted with `scripts/sign_encrypt.py`, they are able to be stored in the untrusted REE filesystem, and `tee-suppllicant` will take care of passing them to be checked and loaded by the Secure World OP-TEE core.

REE-FS TA rollback protection

OP-TEE core maintains a `ta_ver.db` file in secure storage to check for version of REE TAs as they are loaded from REE-FS in order to prevent against any TA version downgrades. TA version can be configured via TA build option: `CFG_TA_VERSION=<unsigned integer>`.

Note: Here rollback protection is effective only when `CFG_RPMB_FS=y`.

REE-FS TA formats

REE filesystem TAs come in three formats:

1. Legacy TAs signed, not encrypted, cannot be created anymore by the build scripts since version 3.7.0.
2. Bootstrap TAs, signed with the key from the build of the original OP-TEE core blob, not encrypted.
3. Encrypted TAs, sign-then-encrypt-then-MAC, encrypted with `TA_ENC_KEY` when `CFG_ENCRYPT_TA=y`. During OP-TEE runtime, the symmetric key used to decrypt TA has to be provided in a platform specific manner via overriding API:

```
TEE_Result tee_opt_get_ta_enc_key(uint32_t key_type, uint8_t *buffer,
                                size_t len);
```

REE-FS TA header structure

All REE filesystems TAs has common header, struct `shdr`, defined as:

```
enum shdr_img_type {
    SHDR_TA = 0,
    SHDR_BOOTSTRAP_TA = 1,
    SHDR_ENCRYPTED_TA = 2,
};

#define SHDR_MAGIC    0x4f545348

/**
 * struct shdr - signed header
 * @magic:      magic number must match SHDR_MAGIC
 * @img_type:   image type, values defined by enum shdr_img_type
 * @img_size:   image size in bytes
 * @algo:      algorithm, defined by public key algorithms TEE_ALG_*
 *             from TEE Internal API specification
 * @hash_size:  size of the signed hash
 * @sig_size:   size of the signature
 * @hash:      hash of an image
 * @sig:      signature of @hash
 */
struct shdr {
    uint32_t magic;
    uint32_t img_type;
    uint32_t img_size;
    uint32_t algo;
    uint16_t hash_size;
    uint16_t sig_size;
    /*
     * Commented out element used to visualize the layout dynamic part
     * of the struct.
     *
     * hash is accessed through the macro SHDR_GET_HASH and
     * signature is accessed through the macro SHDR_GET_SIG
     *
     * uint8_t hash[hash_size];
     * uint8_t sig[sig_size];
     */
};

#define SHDR_GET_SIZE(x)      (sizeof(struct shdr) + (x)->hash_size + \
                               (x)->sig_size)
#define SHDR_GET_HASH(x)     (uint8_t *)(((struct shdr *) (x)) + 1)
#define SHDR_GET_SIG(x)      (SHDR_GET_HASH(x) + (x)->hash_size)
```

The field `img_type` tells the type of TA, if it's `SHDR_TA` (0), it's a legacy TA. If it's `SHDR_BOOTSTRAP_TA` (1) it's a bootstrap TA.

The field `algo` tells the algorithm used. The script used to sign TAs currently uses `TEE_ALG_RSASSA_PKCS1_V1_5_SHA256` (0x70004830). This means RSA with PKCS#1v1.5 padding and SHA-256 hash function. OP-TEE accepts any of the `TEE_ALG_RSASSA_PKCS1_*` algorithms.

For bootstrap TAs struct `shdr` is followed by a subheader, struct `shdr_bootstrap_ta` which is defined as:

```
/**
 * struct shdr_bootstrap_ta - bootstrap TA subheader
 * @uuid:      UUID of the TA
 * @ta_version: Version of the TA
 */
struct shdr_bootstrap_ta {
    uint8_t uuid[sizeof(TEE_UUID)];
    uint32_t ta_version;
};
```

The fields `uuid` and `ta_version` allows extra checks to be performed when loading the TA. Currently only the `uuid` field is checked.

For encrypted TAs struct `shdr` is followed by a subheader, struct `shdr_bootstrap_ta` which is followed by another subheader, struct `shdr_encrypted_ta` defined as:

```
/**
 * struct shdr_encrypted_ta - encrypted TA header
 * @enc_algo:   authenticated encryption algorithm, defined by symmetric key
 *              algorithms TEE_ALG_* from TEE Internal API
 *              specification
 * @flags:      authenticated encryption flags
 * @iv_size:    size of the initialization vector
 * @tag_size:   size of the authentication tag
 * @iv:         initialization vector
 * @tag:        authentication tag
 */
struct shdr_encrypted_ta {
    uint32_t enc_algo;
    uint32_t flags;
    uint16_t iv_size;
    uint16_t tag_size;
    /*
     * Commented out element used to visualize the layout dynamic part
     * of the struct.
     *
     * iv is accessed through the macro SHDR_ENC_GET_IV and
     * tag is accessed through the macro SHDR_ENC_GET_TAG
     *
     * uint8_t iv[iv_size];
     * uint8_t tag[tag_size];
     */
};
```

The field `enc_algo` tells the algorithm used. The script used to encrypt TAs currently uses `TEE_ALG_AES_GCM` (0x40000810). OP-TEE core also accepts `TEE_ALG_AES_CCM` algorithm.

The field `flags` supports a single flag to tell encryption key type which is defined as:

```
#define SHDR_ENC_KEY_TYPE_MASK 0x1

enum shdr_enc_key_type {
    SHDR_ENC_KEY_DEV_SPECIFIC = 0,
```

(continues on next page)

(continued from previous page)

```
SHDR_ENC_KEY_CLASS_WIDE = 1,
};
```

REE-FS TA binary formats

TA binary follows the ELF file which normally is stripped as additional symbols etc will be ignored when loading the TA.

Legacy TA binary is formatted as:

```
hash = H(<struct shdr> || <stripped ELF>)
signature = RSA-Sign(hash)
legacy_binary = <struct shdr> || <hash> || <signature> || <stripped ELF>
```

Bootstrap TA binary is formatted as:

```
hash = H(<struct shdr> || <struct shdr_bootstrap_ta> || <stripped ELF>)
signature = RSA-Sign(<hash>)
bootstrap_binary = <struct shdr> || <hash> || <signature> ||
                   <struct shdr_bootstrap_ta> || <stripped ELF>
```

Encrypted TA binary is formatted as:

```
nonce = <unique random value>
ciphertext, tag = AES_GCM(<stripped ELF>)
hash = H(<struct shdr> || <struct shdr_bootstrap_ta> ||
        <struct shdr_encrypted_ta> || <nonce> || <tag> || <stripped ELF>)
signature = RSA-Sign(<hash>)
encrypted_binary = <struct shdr> || <hash> || <signature> ||
                  <struct shdr_bootstrap_ta> ||
                  <struct shdr_encrypted_ta> || <nonce> || <tag> ||
                  <ciphertext>
```

Verifying with Subkeys

A TA can be verified using a subkey or a chain of subkeys. This allows delegation of TA signing without distributing the root key. TAs signed with a subkey are confined to the UUID-V5 namespace of the subkey to avoid TA UUID clashes with different subkeys.

SHDR_SUBKEY is a type of header which enables chains of public keys. The public root key is used to verify the first public subkey, which then is used to verify the next public subkey and so on.

The TA is finally verified using the last subkey. All these headers are added in front of the TA binary so everything needed to verify the TA is available when it's loaded into memory.

For details on subkeys see also [Subkeys](#)

Loading REE-FS TA

A REE TA is loaded into shared memory using a series of RPC in *Loading a REE TA into nonsecure shared memory*. The payload memory is allocated via TEE-supplciant and later freed when the TA has been loaded into secure memory in *Freeing previously allocated nonsecure shared memory*.

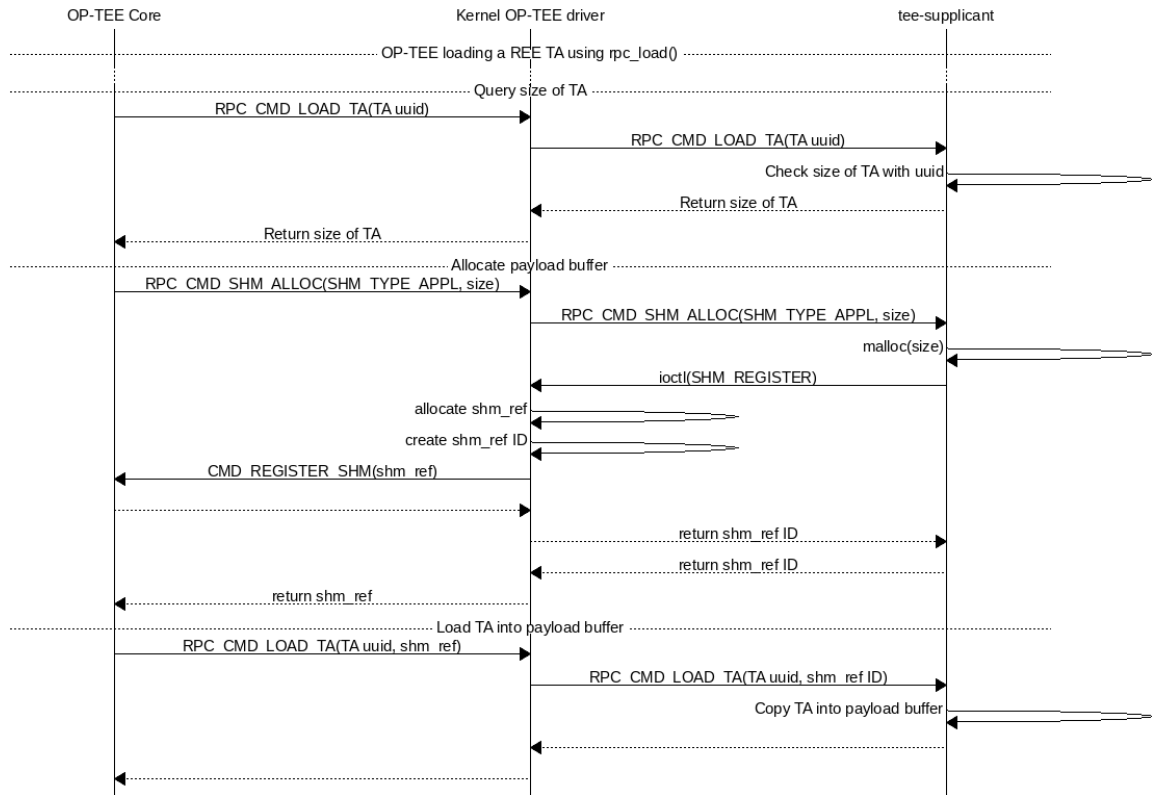


Fig. 18: Loading a REE TA into nonsecure shared memory

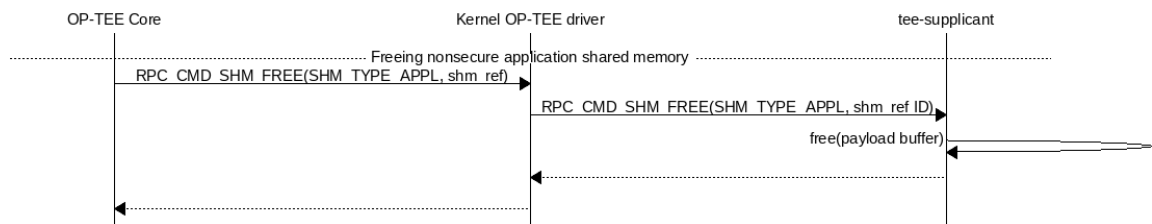


Fig. 19: Freeing previously allocated nonsecure shared memory

Secure Storage TA

These are stored in secure storage. The meta data is stored in a database of all installed TAs and the actual binary is stored encrypted and integrity protected as a separate file in the untrusted REE filesystem (flash). Before these TAs can be loaded they have to be installed first, this is something that can be done during initial deployment or at a later stage.

For test purposes the test program `xtest` can install a TA into secure storage with the command:

```
$ xtest --install-ta
```

TAs stored in secure storage are kept in a TA database. The TA database consists of a single file with the name `dirf.db` which is stored either in the REE filesystem based secure storage or in RPMB. The file is encrypted and integrity protected as any other object in secure storage. The TAs themselves are not stored in `dirf.db`, they are instead stored in the REE filesystem encrypted and integrity protected. One reason for this is that TAs can potentially be quite large, several megabytes, while secure storage is designed to hold only small objects counted in kilobytes.

`dirf.db` consists of an array of `struct tadb_entry`, defined as:

```
/*
 * struct tee_tadb_property
 * @uuid:      UUID of Trusted Application (TA) or Security Domain (SD)
 * @version:   Version of TA or SD
 * @custom_size: Size of customized properties, prepended to the encrypted
 *             TA binary
 * @bin_size:  Size of the binary TA
 */
struct tee_tadb_property {
    TEE_UUID uuid;
    uint32_t version;
    uint32_t custom_size;
    uint32_t bin_size;
};

#define TADB_IV_SIZE      TEE_AES_BLOCK_SIZE
#define TADB_TAG_SIZE     TEE_AES_BLOCK_SIZE
#define TADB_KEY_SIZE     TEE_AES_MAX_KEY_SIZE

/*
 * struct tadb_entry - TA database entry
 * @prop:            properties of TA
 * @file_number:     encrypted TA is stored in <file_number>.ta
 * @iv:              Initialization vector of the authentication crypto
 * @tag:             Tag used to validate the authentication encrypted TA
 * @key:             Key used to decrypt the TA
 */
struct tadb_entry {
    struct tee_tadb_property prop;
    uint32_t file_number;
    uint8_t iv[TADB_IV_SIZE];
    uint8_t tag[TADB_TAG_SIZE];
    uint8_t key[TADB_KEY_SIZE];
};
```

Entries where the UUID consists of zeros only are not valid and are ignored. The `file_number` field represents that name of the file stored in the REE filesystem. The filename is made from the decimal string representation of

`file_number` with `.ta` appended, or if it was to be printed: `printf("%u.ta", file_number)`.

The TA is decrypted using the authentication encryption algorithm AES-GCM initialized with the `iv` and `key` fields, the `tag` field is used when finalizing the decryption

A TA is looked up in the TA database by opening `dirf.db` and scanning through the elements which are of type `struct tadb_entry` until a matching UUID is found.

2.12.4 Loading and preparing TA for execution

User mode TAs are loaded into final memory in the same way using the user mode ELF loader `ldelf`. The different TA locations has a common interface towards `ldelf` which makes the user mode operations identical regardless of how the TA is stored.

The TA is loaded into secure memory in *Preparing TA for execution*.

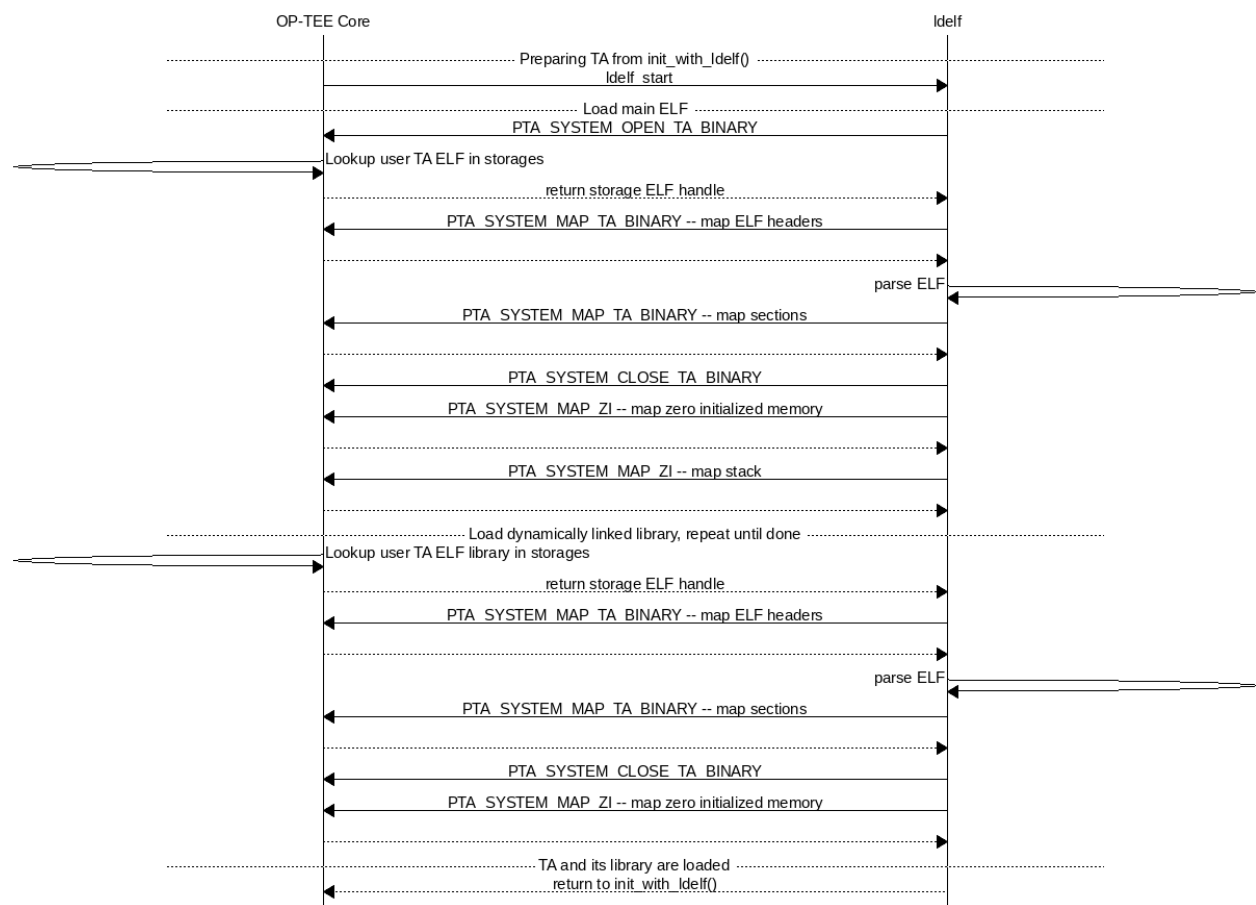


Fig. 20: Preparing TA for execution

After `ldelf` has returned with a TA prepared for execution it still remains in memory to serve the TA if `dlopen()` and friends are used. `ldelf` is also used to dump stack trace and detailed memory mappings if a TA is terminated via an abort.

A high level view of the entire flow from the client application in Linux user space where a session is opened to a TA is given in *Open session to a TA*.

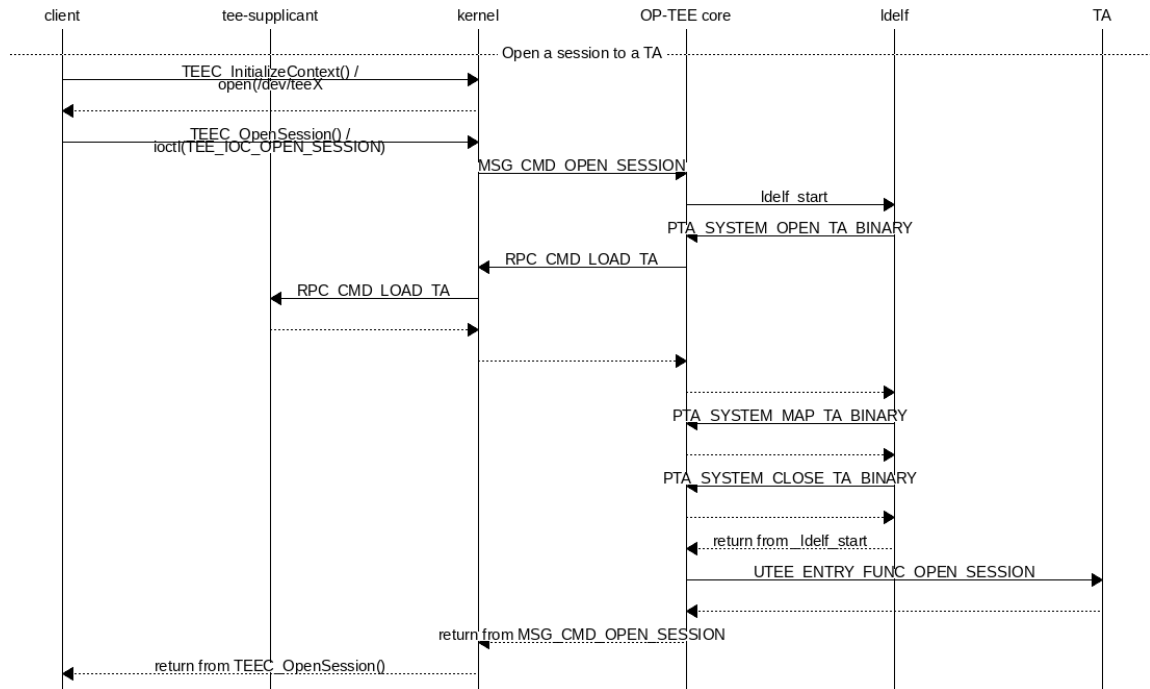


Fig. 21: Open session to a TA

2.12.5 TA Properties

This section give a more in depth description of the TA properties (see *Trusted Applications* also).

GlobalPlatform Properties

Standard TA properties must be defined through property flag in macro `TA_FLAGS` in `user_ta_header_defines.h`

Single Instance

"`gpd.ta.singleInstance`" is a boolean property of the TA. This property defines if one instance of the TA must be created and will receive all open session request, or if a new specific TA instance must be created for each incoming open session request. OP-TEE TA flag `TA_FLAG_SINGLE_INSTANCE` sets to configuration of this property. The boolean property is set to true if `TA_FLAGS` sets bit `TA_FLAG_SINGLE_INSTANCE`, otherwise the boolean property is set to false.

Multi-session

"`gpd.ta.multiSession`" is a boolean property of the TA. This property defines if the TA instance can handle several sessions. If disabled, TA instance support only one session. In such case, if the TA already has a opened session, any open session request will return with a busy error status.

Note: This property is **meaningless** if TA is **NOT** SingleInstance TA.

OP-TEE TA flag `TA_FLAG_MULTI_SESSION` sets to configuration of this property. The boolean property is set to `true` if `TA_FLAGS` sets bit `TA_FLAG_MULTI_SESSION`, otherwise the boolean property is set to `false`.

Keep Alive

"`gpd.ta.instanceKeepAlive`" is a boolean property of the TA. This property defines if the TA instance created must be destroyed or not when all sessions opened towards the TA are closed. If the property is enabled, TA instance, once created (at 1st open session request), is never removed unless the TEE itself is restarted (boot/reboot).

Note: This property is **meaningless** if TA is **NOT** SingleInstance TA.

OP-TEE TA flag `TA_FLAG_INSTANCE_KEEP_ALIVE` sets to configuration of this property. The boolean property is set to `true` if `TA_FLAGS` sets bit `TA_FLAG_INSTANCE_KEEP_ALIVE`, otherwise the boolean property is set to `false`.

Heap Size

"`gpd.ta.dataSize`" is a 32bit integer property of the TA. This property defines the size in bytes of the TA allocation pool, in which `TEE_Malloc()` and friends allocate memory. The value of the property must be defined by the macro `TA_DATA_SIZE` in `user_ta_header_defines.h` (see [TA Properties](#)).

Stack Size

"`gpd.ta.stackSize`" is a 32bit integer property of the TA. This property defines the size in bytes of the stack used for TA execution. The value of the property must be defined by the macro `TA_STACK_SIZE` in `user_ta_header_defines.h` (see [TA Properties](#)).

Property Extensions

Secure Data Path Flag

`TA_FLAG_SECURE_DATA_PATH` is a bit flag supported by `TA_FLAGS`. This property flag claims the secure data support from the OP-TEE OS for the TA. Refer to the OP-TEE OS for secure data path support. TAs that do not set `TA_FLAG_SECURE_DATA_PATH` in the value of `TA_FLAGS` will **not** be able to handle memory reference invocation parameters that relate to secure data path buffers.

Cache maintenance Flag

`TA_FLAG_CACHE_MAINTENANCE` is a bit flag supported by `TA_FLAGS`. This property flag, when enabled, allows Trusted Application to use the cache maintenance API extension of the Internal Core API described in [Cache Maintenance Support](#). TAs that do not set `TA_FLAG_CACHE_MAINTENANCE` in the value of their `TA_FLAGS` will not be able to call the cache maintenance API.

Deprecated Property Flags

Older versions of OP-TEE used to define extended property flags that are deprecated and meaningless to current OP-TEE. These are `TA_FLAG_USER_MODE`, `TA_FLAG_EXEC_DDR` and `TA_FLAG_REMAP_SUPPORT`.

2.13 Virtualization

OP-TEE have experimental virtualization support. This is when one OP-TEE instance can run TAs from multiple virtual machines. OP-TEE isolates all VM-related states, so one VM can't affect another in any way.

With virtualization support enabled, OP-TEE will rely on a hypervisor, because only the hypervisor knows which VM is calling OP-TEE. Also, naturally the hypervisor should inform OP-TEE about creation and destruction of VMs. Besides, in almost all cases, hypervisor enables two-stage MMU translation, so VMs does not see real physical address of memory, instead they work with intermediate physical addresses (IPAs). On other hand OP-TEE can't translate IPA to PA, so this is a hypervisor's responsibility to do this kind of translation. So, hypervisor should include a component that knows about OP-TEE protocol internals and can do this translation. We call this component "TEE mediator" and right now only XEN hypervisor have OP-TEE mediator.

2.13.1 Configuration

Virtualization support is enabled with `CFG_VIRTUALIZATION` configuration option. When this option is enabled, OP-TEE will **not** work without compatible a hypervisor. This is because the hypervisor should send `OPTEE_SMC_VM_CREATED` SMC with VM ID before any standard SMC can be received from client.

`CFG_VIRT_GUEST_COUNT` controls the maximum number of supported VMs. As OP-TEE have limited size of available memory, increasing this count will decrease amount of memory available to one VM. Because we want VMs to be independent, OP-TEE splits available memory in equal portions to every VM, so one VM can't consume all memory and cause DoS to other VMs.

2.13.2 Requirements for hypervisor

As said earlier, hypervisor should be aware of OP-TEE and SMCs from virtual guests to OP-TEE. This is a list of things, that compatible hypervisor should perform:

1. When new OP-TEE-capable VM is created, hypervisor should inform OP-TEE about it with SMC `OPTEE_SMC_VM_CREATED`. `a1` parameter should contain VM id. ID 0 is defined as `HYP_CLNT_ID` and is reserved for hypervisor itself.
2. When OP-TEE-capable VM is being destroyed, hypervisor should stop all VCPUs (this will ensure that OP-TEE have no active threads for that VMs) and send SMC `OPTEE_SMC_VM_DESTROYED` with the same parameters as for `OPTEE_SMC_VM_CREATED`.
3. Any SMC to OP-TEE should have VM ID in `a7` parameter. This is either `HYP_CLNT_ID` if call originates from hypervisor or VM ID that was passed in `OPTEE_SMC_VM_CREATED` call.
4. Hypervisor should perform IPA<->PA address translation for all SMCs. This includes both arguments in `a1-a6` registers and in in-memory command buffers.
5. Hypervisor should pin memory pages that VM shares with OP-TEE. This means, that hypervisor should ensure that pinned page will reside at the original PA as long, as it is shared with OP-TEE. Also it should still belong to the VM that shared it. For example, the hypervisor should not swap out this page, transfer ownership to another VM, unmap it from VM address space and so on.

6. Naturally, the hypervisor should correctly handle the OP-TEE protocol, so for any VM it should look like it is working with OP-TEE directly.

2.13.3 Limitations

Virtualization support is in experimental state and it have some limitations, user should be aware of.

Platforms support

Only Armv8 architecture is supported. There is no hard restriction, but currently Armv7-specific code (like MMU or thread manipulation) just know nothing about virtualization. Only one platform has been tested right now and that is QEMU-V8 (aka qemu that emulates Arm Versatile Express with Armv8 architecture). Support for Rcar Gen3 should be added soon.

Static VMs guest count and memory allocation

Currently, a user should configure maximum number of guests. OP-TEE will split memory into equal chunks, so every VM will have the same amount of memory. For example, if you have 6MB for your TAs, you can set `CFG_VIRT_GUEST_COUNT` to 3 and every VM would be able to use 2MB maximum, even if there is no other VMs running. This is okay for embedded setups when you know exact number and roles of VMs, but can be inconvenient for server applications. Also, it is impossible to configure amount of memory available for a given VM. Every VM instance will have exactly the same amount of memory.

Sharing hardware resources and PTAs

Right now only HW that can be used by multiple VMs simultaneously is serial console, used for logging. Devices like HW crypto accelerators, secure storage devices (e.g. external flash storage, accessed directly from OP-TEE) and others are not supported right now. Drivers should be made virtualization-aware before they can be used with virtualization extensions.

Every VM will have own PTA states, which is a good thing in most cases. But if one wants PTA to have some global state that is shared between VMs, he need to write PTA accordingly.

No compatibility with “normal” mode

OP-TEE built with `CFG_VIRTUALIZATION=y` will not work without a hypervisor, because before executing any standard SMC, `OPTEE_SMC_VM_CREATED` must be called. This can be inconvenient if one wants to switch between virtualized and non-virtualized environment frequently. On other hand, it is not a big deal in a production environment. Simple workaround can be made for this: if OP-TEE receives standard SMC prior to `OPTEE_SMC_VM_CREATED`, it implicitly creates VM context and uses it for all subsequent calls.

Implementation details

OP-TEE as a whole can be split into two entities. Let us call them “nexus” and TEE. Nexus is a core part of OP-TEE that takes care of low level things: SMC handling, memory management, threads creation and so on. TEE is a part that does the actual job: handles requests, loads TAs, executes them, and so on. So, it is natural to have one nexus instance and multiple instances of TEE, one TEE instance per registered VM. This can be done either explicitly or implicitly.

Explicit way is to move TEE state in some sort of structure and make all code to access fields of this structure. Something like `struct task_struct` and `current` in linux kernel. Then it is easy to allocate such structure for every VM instance. But this approach basically requires to rewrite all OP-TEE code.

Implicit way is to have banked memory sections for TEE/VM instances. So memory layout can look something like that:

```
+-----+
|          Nexus: .nex_bss, .nex_data, ...          |
+-----+
|          TEE states                               |
|          |                                         | |
| VM1 TEE state | VM 2 TEE state | VM 3 TEE state |
| .bss, .data   | .bss, .data   | .bss, .data,   |
+-----+
```

This approach requires no changes in TEE code and requires some changes into nexus code. So, idea that Nexus state resides in separate sections (`.nex_data`, `.nex_bss`, `.nex_nozi`, `.nex_heap` and others) and is always mapped.

TEE state resides in standard sections (like `.data`, `.bss`, `.heap` and so on). There is a separate set of this sections for every VM registered and Nexus maps them only when it receives call from corresponding VM.

As Nexus and TEE have separate heaps, `bget` allocator was extended to work with multiple “contexts”. `malloc()`, `free()` with friends work with one context. `nex_malloc()` (and other `nex_` functions) were added. They use different context, so now Nexus can use separate heap, which is always mapped into OP-TEE address space. When virtualization support is disabled, all those `nex_` functions are defined to point to standard `malloc()` counterparts.

To change memory mappings in run-time, in MMU code we have added a new entity, named “partition”, which is defined by `struct mmu_partition`. It holds information about all page-tables, so the whole MMU mapping can be switched by one write to TTBR register.

There is the default partition, it holds MMU state when there is no VM context active, so no TEE state is mapped. When OP-TEE receives `OPTEE_SMC_VM_CREATED` call, it copies default partition into new one and then maps sections with TEE data. This is done by `prepare_memory_map()` function in `virtualization.c`.

When OP-TEE receives STD call it checks that the supplied VM ID is valid and then activates corresponding MMU partition, so TEE code can access its own data. This is basically how virtualization support is working.

2.14 SPMC

This document describes the SPMC (S-EL1) implementation for OP-TEE. More information on the SPMC can be found in the FF-A specification can be found in the [FF-A spec](#).

2.14.1 SPMC Responsibilities

The SPMC is a critical component in the FF-A flow. Some of its major responsibilities are:

- **Initialisation and run-time management of the SPs:**
The SPMC component is responsible for initialisation of the Secure Partitions (loading the image, setting up the stack, heap, ...).
- **Routing messages between endpoints:**
The SPMC is responsible for passing FF-A messages from normal world to SPs and back. It also responsible for passing FF-A messages between SPs.
- **Memory management:**
The SPMC is responsible for the memory management of the SPs. Memory can be shared between SPs and between a SP to the normal world.

This document describes OP-TEE as a S-EL1 SPMC.

2.14.2 Secure Partitions

Secure Partitions (SPs) are the endpoints used in the FF-A protocol. When OP-TEE is used as a SPMC SPs run primarily inside S-EL0.

OP-TEE will use FF-A for its transport layer when the OP-TEE `CFG_CORE_FFA=y` configuration flag is enabled. The SPMC will expose the OP-TEE core, privileged mode, as a secure endpoint itself. This is used to handle all GlobalPlatform programming mode operations. All GlobalPlatform messages are encapsulated inside FF-A messages. The OP-TEE endpoint will unpack the messages and afterwards handle them as standard OP-TEE calls. This is needed as TF-A (S-EL3) does only allow FF-A messages to be passed to the secure world when the SPMD is enabled.

SPs run from the initial boot of the system until power down and don't have any built-in session management compared to GPD TEE TAs. The only means of communicating with the outside world is through messages defined in the FF-A specification. The context of a SP is saved between executions.

The [Trusted Service](#) repository includes the libsp library which exports all needed functions to build a S-EL0 SP. It also includes many examples of how to create and implement a SP.

2.14.3 Secure Partition formats

OP-TEE specific ELF format

OP-TEE uses an ELF format for its *Trusted Applications*. It has an OP-TEE specific section which contains a header structure for describing the Trusted Application. A very similar format can be used for Secure Partitions. The same ELF format allows OP-TEE to use the built-in ELF loader (`ldelf`) with all its features like handling relocations or ASLR. In this case a different section is used for the header structure to distinguish between Trusted Applications and Secure Partitions.

SPMC agnostic flat binary format

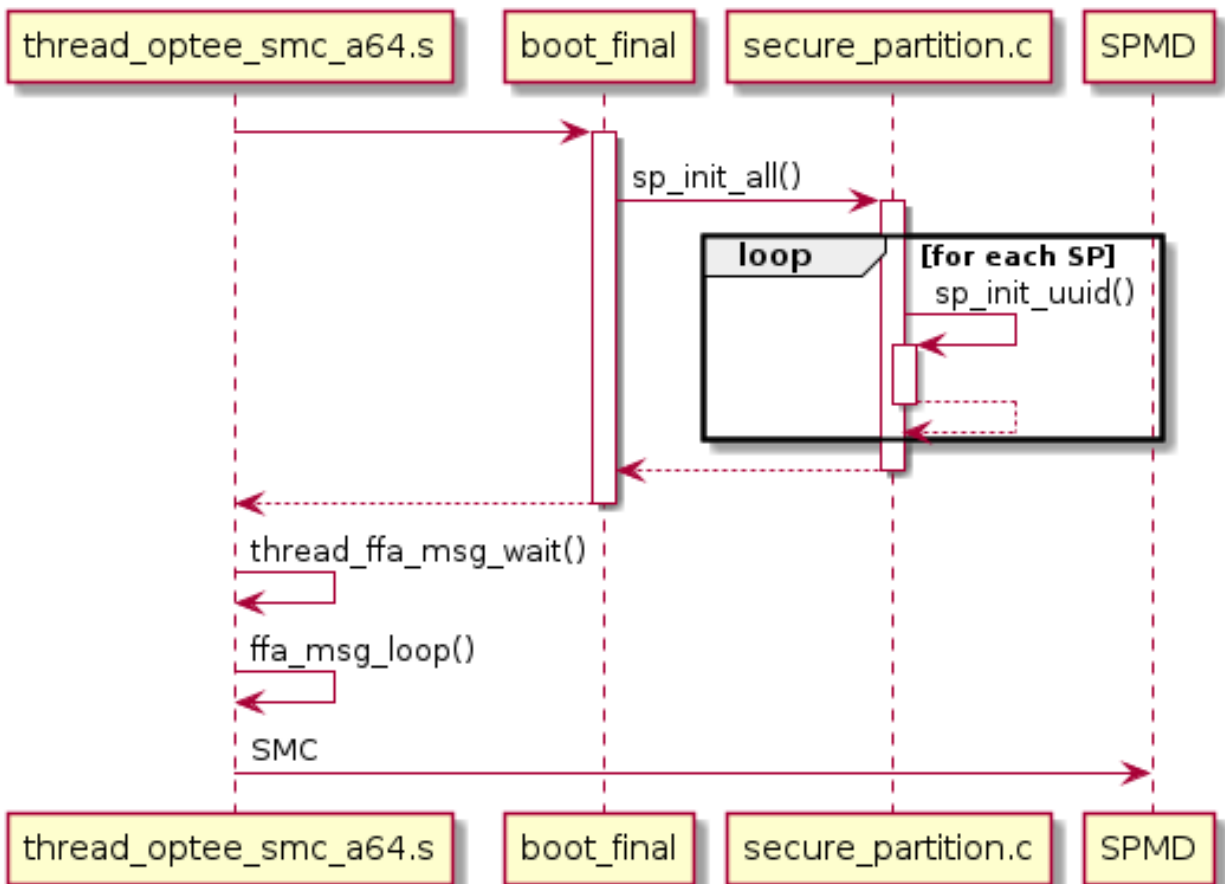
This simple binary format aims for maximum portability between SPMC implementations by removing the dependency on an ELF loader and implementation specific metadata in the SP image. The SPMC can simply copy the binary into the memory and start running it. The relocations, the stack setup and any further initialization steps should be handled by the startup code of the secure partition. The access rights for different sections of the binary can be configured either by adding load relative memory regions to the SP manifest or by using the `FFA_MEM_PERM_SET` interface in the startup code.

2.14.4 SPMC Program Flow

SP images are either embedded into the OP-TEE image or loaded from the FIP by BL2. This makes it possible to start SPs during boot, before the rich OS is available in the normal world.

Starting SPs

SPs are loaded and started as the last step in OP-TEE's initialisation process. This is done by adding `sp_init_all()` to the `boot_final` initcall level.



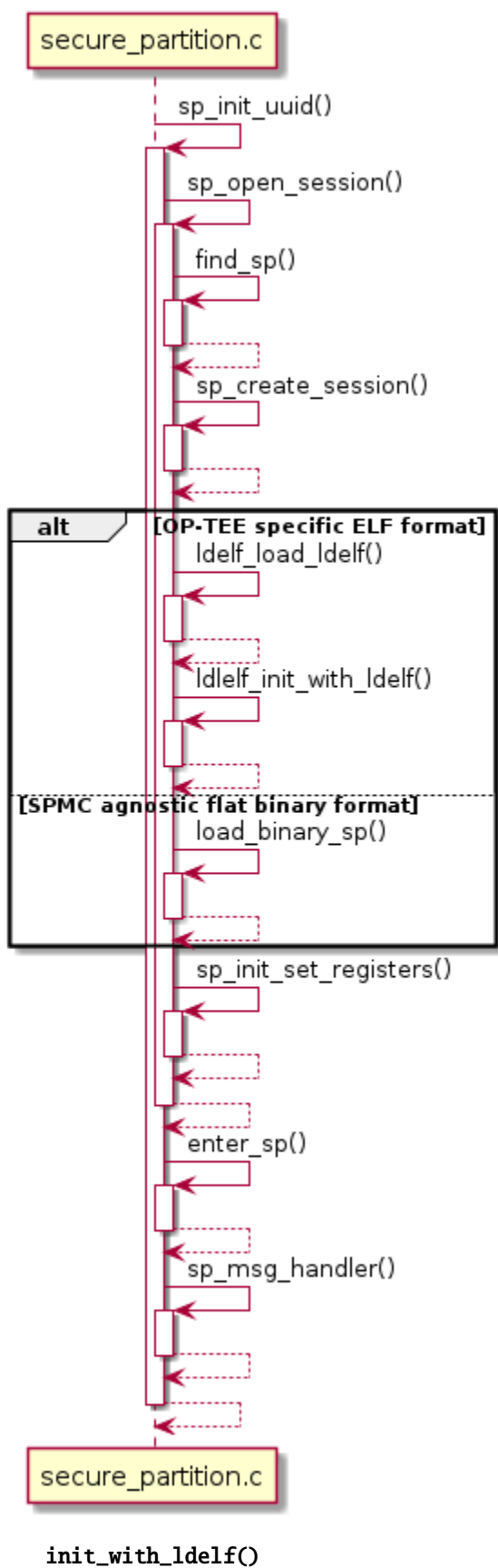
`sp_init_all()`

Initialise all SPs which have been added by the `SP_PATHS` compiler option and run them

`thread_ffa_msg_wait()`

All SPs are loaded and started. A `FFA_MSG_WAIT` message is sent to the Normal World.

Each ELF format SP is loaded into the system using `ldelf` and started. This is based around the same process as loading the early TAs. For each binary format SP a simpler method is used to copy the binary into a suitable memory area. All SPs are run after they are loaded and run until a `FFA_MSG_WAIT` is sent by the SP.



Load the OP-TEE specific ELF format SP

load_binary_sp()

Load the SPMC agnostic flat binary format SP

sp_init_info()

Initialise the struct `ffa_init_info`. The struct `ffa_init_info` is passed to the SP during its first run.

sp_init_set_registers()

Initialise the registers of the SP

sp_msg_handler()

Handle the SPs FF-A message

Once all SPs are loaded and started we return to the SPMD and the Normal World is booted.

SP message handling

The SPMC is split into 2 main message handlers:

thread_spmc_msg_recv() `thread_spmc.c`

Used to handle message coming from the Normal World.

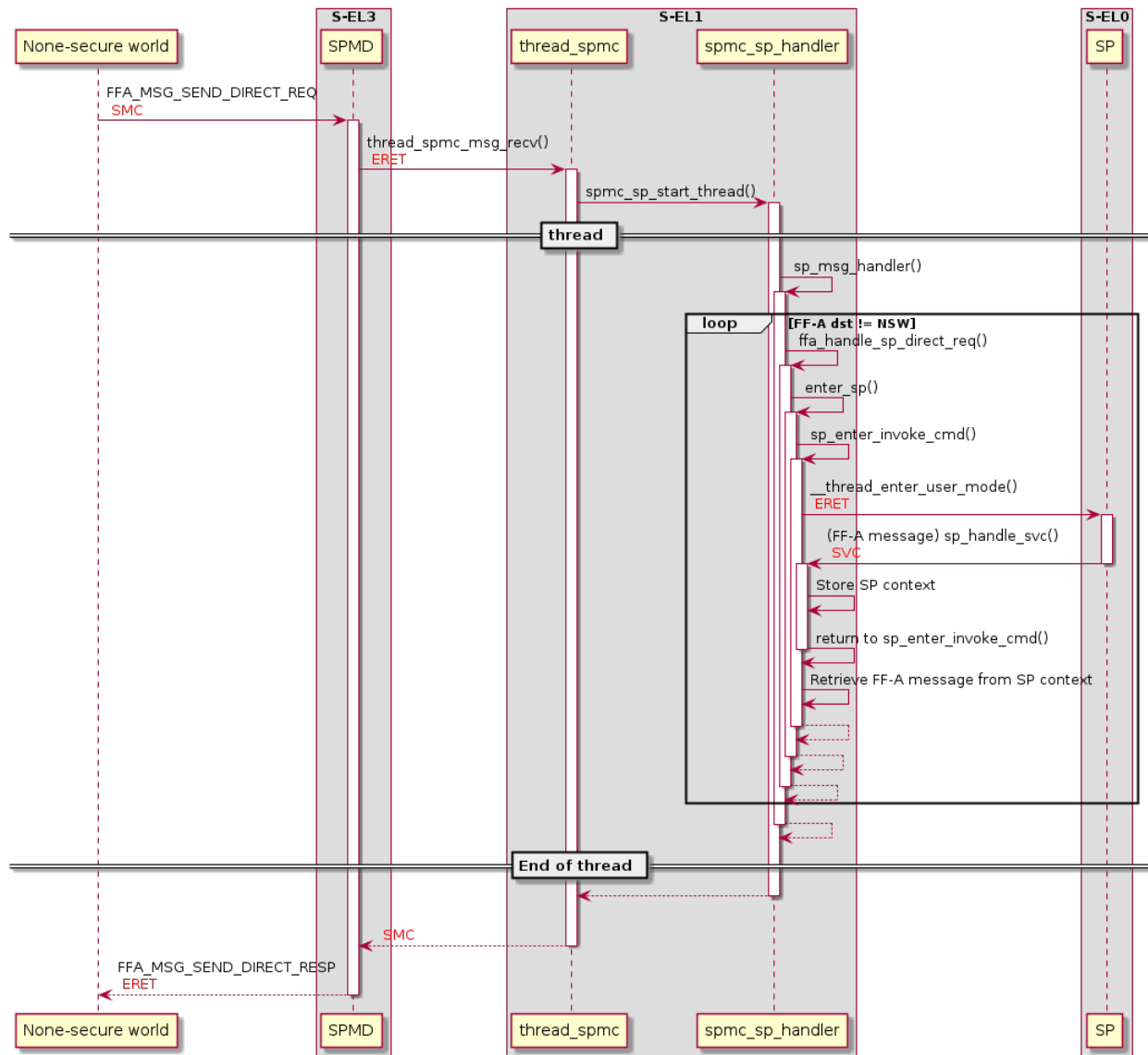
sp_msg_handler() `spmc_sp_handler.c`

Used to handle message where the source or the destination is a SP.

When a `FFA_MSG_SEND_DIRECT_REQ` message is received by the SPMC from the Normal World, a new thread is started. The FF-A message is passed to the thread and it will call the `sp_msg_handler()` function.

Whenever the SPMC (`sp_msg_handler()`) receives a message not intended for one of the SPs, it will exit the thread and return to the Normal World passing the FF-A message.

Currently only a `FFA_MSG_SEND_DIRECT_REQ` can be passed from the Normal World to a SP.



Every message received by the SPMC from the Normal World is handled in the `thread_spmc_msg_rcv()` function.

When entering a SP we need to be running in a OP-TEE thread. This is needed to be able to push the TS session (We push the TS session to get access to the SP memory). Currently the only possibility to enter a SP from the Normal world is via a `FFA_MSG_SEND_DIRECT_REQ`. Whenever we receive a `FFA_MSG_SEND_DIRECT_REQ` message which doesn't have OP-TEE as the endpoint-id, we start a thread and forward the FF-A message to the `sp_msg_handler()`.

The `sp_msg_handler()` is responsible for all messages coming or going to/from a SP. It runs in a while loop and will handle every message until it comes across a messages which is not intended for the secure world. After a message is handled by the SPMC or when it needs to be forwarded to a SP, `sp_enter()` is called. `sp_enter()` will copy the FF-A arguments and resume the SP.

When the SPMC needs to have access to the SPs memory, it will call `ts_push_current_session()` to gain access and `ts_pop_current_session()` to release the access.

Running and exiting SPs

The SPMC resumes/starts the SP by calling the `sp_enter()`. This will set up the SP context and jump into S-EL0. Whenever the SP performs a system call it will end up in `sp_handle_svc()`. `sp_handle_svc()` stores the current context of the SP and makes sure that we don't return to S-EL0 but instead returns to S-EL1 back to `sp_enter()`. `sp_enter()` will pass the FF-A registers (x0-x7) to `spmc_sp_msg_handler()`. This will process the FF-A message.

RxTx buffer management

RxTx buffers are used by the SPMC to exchange information between an endpoint and the SPMC. The `rxtx_buf` struct is used by the SPMC for abstracting buffer management. Every SP has a `struct rxtx_buf` which will be passed to every function that needs access to the rxtx buffer. A separate `struct rxtx_buf` is defined for the Normal World, which gives access to the Normal World buffers.

2.14.5 FF-A compliance

Legend

- Fully supported
- Partially implemented
- Not supported
- Does not apply for the FF-A instance or version

Partition boot protocol

Only FF-A v1.0 partition boot protocol is supported by the SPMC.

Supported partition manifest fields

Field	Mandatory	FF-A v1.0	FF-A v1.1
FF-A version	Yes		
UUID	Yes		
Partition ID	No		
Auxiliary IDs	No		
Name (description)	No		
Number of execution contexts	Yes		
Run-time EL	Yes		
Execution state	Yes		
Load address	No		
Entry point offset	No		
Translation granule	No		
Boot order	No		
RX/TX information	No		
Messaging method	Yes		
Primary scheduler implemented	No		
Run-time model	No		
Tuples	No		

continues on next page

Table 5 – continued from previous page

Field	Mandatory	FF-A v1.0	FF-A v1.1
Memory regions			
Base address	No		
Load address relative offset	No		
Page count	Yes		
Attributes	Yes		
Name	No		
Stream & SMMU IDs	No		
Stream ID access permissions	No		
Device regions			
Physical base address	Yes		
Page count	Yes		
Attributes	Yes		
Interrupts	No		
SMMU IDs	No		
Stream IDs	No		
Exclusive access and ownership	No		
Name	No		

Limitations

- The values of mandatory but not supported fields are ignored by the SP loader. This means all values are accepted but the SPMC might behave differently than expected.
- Memory region attributes doesn't support shareability and cacheability flags.

Supported FF-A interfaces

The table below describes the implementation level of each FF-A interface on different FF-A instances. The two instances are between OP-TEE SPMC and the SPMC and between OP-TEE SPMC and its S-EL0 secure partitions. The FF-A specification uses 'Secure Physical' and 'Secure Virtual' terms for these instances.

Interface	OP-TEE <-> SPMD FF-A v1.0	FF-A v1.1	OP-TEE <-> S-EL0 FF-A v1.0
FFA_ERROR			
FFA_SUCCESS			
FFA_INTERRUPT			
FFA_VERSION			
FFA_FEATURES			
FFA_RX_ACQUIRE			
FFA_RX_RELEASE			
FFA_RXTX_MAP			
FFA_RXTX_UNMAP			
FFA_PARTITION_INFO_GET			
FFA_ID_GET			
FFA_SPM_ID_GET			
FFA_MSG_WAIT			
FFA_YIELD			
FFA_RUN			

Table 6 – continued from previous page

Interface	OP-TEE <-> SPMD FF-A v1.0	FF-A v1.1	OP-TEE <-> S-EL0 FF-A v1.0
FFA_NORMAL_WORLD_RESUME			
FFA_MSG_SEND			
FFA_MSG_SEND2			
FFA_MSG_SEND_DIRECT_REQ			
FFA_MSG_SEND_DIRECT_RESP			
FFA_MSG_POLL			
FFA_MEM_DONATE			
FFA_MEM_LEND			
FFA_MEM_SHARE			
FFA_MEM_RETRIEVE_REQ			
FFA_MEM_RETRIEVE_RESP			
FFA_MEM_RELINQUISH			
FFA_MEM_RECLAIM			
FFA_MEM_PERM_GET			
FFA_MEM_PERM_SET			
FFA_MEM_FRAG_RX			
FFA_MEM_FRAG_TX			
FFA_MEM_OP_PAUSE			
FFA_MEM_OP_RESUME			

Limitations

- FF-A v1.1 error code NO_DATA is not supported.
- FFA_SUCCESS is not supported as a response to an FFA_MSG_SEND_DIRECT_REQ message.
- Non-secure interrupts are not forwarded to the normal world via FFA_INTERRUPT.
- Interrupts cannot be forwarded to S-EL0 secure partitions.
- Only FFA_RXTX_MAP feature query is supported by the FFA_FEATURES interface. FFA_MEM_DONATE, FFA_MEM_LEND, FFA_MEM_SHARE and FFA_MEM_RETRIEVE_REQ feature query is not implemented.
- FF-A v1.1 Flags field in FFA_MSG_SEND_DIRECT_REQ and FFA_MSG_SEND_DIRECT_RESP calls is not supported.
- Transferring memory transaction descriptors in a buffer distinct from the TX buffer is not supported by the secure virtual instance.
- Transferring fragmented memory transaction descriptors is not supported by the secure virtual instance.
- The only supported ‘Memory region attributes descriptor’ value is normal memory, write-back cacheability and inner shareable. All other values are denied on the secure physical instance. The secure virtual instance’s implementation ignores the value of this descriptor but uses the same attributes for the region.
- The NS flag support is not implemented for ‘Memory region attributes descriptor’.
- Only read-write non-executable value can be used in the ‘Memory access permissions descriptor’ at the secure physical instance.
- The Flags field of FFA_MEM_RELINQUISH is ignored.
- The secure physical instance doesn’t implement the receiving of FFA_MEM_RELINQUISH.
- Time slicing of memory management operations is not supported.

2.14.6 Configuration

SPMC config options

To configure OP-TEE as a S-EL1 SPMC with Secure Partition support, the following flags should be set for `optee_os`:

- `CFG_CORE_SEL1_SPMC=y`
- `CFG_SECURE_PARTITION=y`
- `CFG_DT=y`
- `CFG_MAP_EXT_DT_SECURE=y`

Furthermore TF-A should be configured as the SPMD, expecting a S-EL1 SPMC:

- `SPD=spmd`
- `SPMD_SPM_AT_SEL2=0`
- `ARM_SPMC_MANIFEST_DTS=<path to SPMC manifest dts>`

SP loading mechanism

OP-TEE SPMC supports two methods for finding and loading the SP executable images. Currently only ELF executables are supported. In the build repo the loading method can be selected with the `SP_PACKAGING_METHOD` option.

Embedded SP

In this case the early TA mechanism of `optee_os` is reused: the SP ELF files are embedded into the main OP-TEE binary. Each ELF should start with a specific section (`.sp_head`) containing a struct which describes the SP (UUID, stack size, etc.). The images can be added to `optee_os` using the `SP_PATHS` config option, the build repo will set this up automatically when `SP_PACKAGING_METHOD=embedded` is selected. The images passed in `SP_PATHS` are processed by `ts_bin_to_c.py` in `optee_os` and linked into the main binary. At runtime the `for_each_secure_partition()` macro can iterate through these images, so a particular SP can be found by UUID and then loaded.

The SP manifest file [1] used by the SPMC to setup SPs is also handled by `ts_bin_to_c.py`, it will be concatenated to the end of the SP ELF.

FIP SP

In this case the SP ELF files and the corresponding SP manifest DTs are encapsulated into SP packages and packed into the FIP. The goal of providing this alternative flow is to make updating SPs easier (independent of the main OP-TEE binary) and to get aligned with Hafnium (S-EL2 SPMC). For more information about the FIP, please refer to the TF-A documentation [2]. The SP packaging process and the package format is provided by TF-A, detailed description is available at [3]. In the build repo this method can be selected by `SP_PACKAGING_METHOD=fip`, it covers all the necessary setup automatically. In case of using another builds system, the following steps should be implemented:

- TF-A config `SP_LAYOUT_FILE`: provide a JSON file which describes the SPs (path to SP executable and corresponding DT, example [4]). The TF-A builds system will create the SP packages (using `sptool`) based on this and pack them into the FIP.
- TF-A config `ARM_BL2_SP_LIST_DTS`: provide a DT snippet which describes the SPs' UUIDs and load addresses (example: [5]). This will be injected into the SP list in `TB_FW_CONFIG` DT of TF-A, and BL2 will load the SP packages based on this. Note that BL2 doesn't automatically load all images from the FIP: it's necessary to explicitly define them in `TB_FW_CONFIG` (using this injected snippet or manually editing the DT).

- TF-A config `ARM_SPMC_MANIFEST_DTS`: provide the SPMC manifest (example: [6]). This DT is passed to the SPMC as a boot argument (in the TF-A naming convention this is the `TOS_FW_CONFIG`). It should contain the list of SP packages and their load addresses in the `compatible = "arm,sp_pkg"` node.

At boot `optee_os` will parse the SP package load addresses from the SPMC manifest and find the SP packages already loaded by BL2. Iterating through the SP packages, based on the SP package header in each package it will map the SP executable image and the corresponding manifest DT and collect these to the `fip_sp_list` list. Later when initialising the SPs, the `for_each_fip_sp` macro is used to iterate this list and load the executables, just like for the embedded SP case.

[1] <https://trustedfirmware-a.readthedocs.io/en/v2.6/components/ffa-manifest-binding.html>

[2] <https://trustedfirmware-a.readthedocs.io/en/v2.6/design/firmware-design.html#firmware-image-package-fip>

[3] <https://trustedfirmware-a.readthedocs.io/en/v2.6/components/secure-partition-manager.html#secure-partition-packages>

[4] <https://trustedfirmware-a.readthedocs.io/en/v2.6/components/secure-partition-manager.html#describing-secure-partitions>

[5] https://github.com/OP-TEE/build/blob/master/fvp/bl2_sp_list.dtsi

[6] https://github.com/OP-TEE/build/blob/master/fvp/spmc_manifest.dts

2.15 Arm Security Extensions

2.15.1 Branch Target Identification

Branch Target Identification (BTI) is an ARMv8.5 extension that provides Control Flow Integrity (CFI) around indirect branches and their targets, thus helping to limit the JOP (Jump Oriented Programming) attacks.

With this extension, ARM8.5-A introduces Branch Target Instructions (BTIs). BTIs are also called landing pads. The processor can be configured so that indirect branches (BR and BLR) only allows target landing pad instructions. If the target of an indirect branch is not a landing pad, a Branch Target Exception is generated.

How to enable BTI for OP-TEE core

To make use of BTI in TEE core on CPU's that support it, enable the option `CFG_CORE_BTI`.

OP-TEE core makes use of some built-ins in the GCC/clang toolchains. So, in order to use the option `CFG_CORE_BTI`, make sure that GCC toolchain has been built with `--enable-standard-branch-protection` is used else OP-TEE will fail to build. By default libraries such as `libgcc.a` are built with flags `(-mbranch-protection=none)`, hence are incompatible with branch protection enabled. The Arm GNU compiler team is looking for ways of providing users easy access to BTI-enabled libraries. In the short-term, they plan to create documentation to make it easier for users to build BTI-enabled libraries themselves. Longer-term, they will begin discussions on how to ensure BTI-enabled libraries are available automatically to users. Please contact GCC team for more information on same. In the meantime, building a BTI-enabled GCC toolchain is possible as described in *Q: How can I build GCC with BTI enabled?*.

The same problem is also there with clang toolchain. So, when using clang to build OP-TEE with `CFG_CORE_BTI=y`, builtins (found in llvm's "compiler-rt" project) must be built with BTI protection enabled. We have some instructions on how to build the compiler-rt with BTI enabled. These are available in *Q: How can I build LLVM compiler-rt with BTI enabled ?*.

How to enable BTI for TA's

To make use of BTI support for TA's and user mode libraries, enable the option `CFG_TA_BTI`. This will ensure that all libraries provided by OP-TEE to the TA's as well as the TA's are built with BTI option.

When the TA's are loaded by `ldelf`, they are checked at run time for the BTI NOTE property in ELF before enabling the protection for the TA.

When building TA's, you need to ensure that any external library used has been built with branch-protection. This can be done by checking the library using `readelf` command with option `-n`. The BTI enabled libraries will have BTI NOTE property in `.note.gnu.property` section. If that is not the case, compilation will stop with a warning. This is done intentionally to warn the user.

Note: The BTI support is currently not compatible with options `CFG_VIRTUALIZATION` and `CFG_WITH_PAGER`.

2.16 Platform documentation

2.16.1 NXP

Security Disclaimer

- NXP i.MX processors have various security-relevant modules that may be configured by the customer to effectively secure the device.
- **These security modules vary by the i.MX product family and may include:**
 - The **Central Security Unit (CSU)** that manages the system security policy for peripheral access on the SoC.
 - The **Resource Domain Controllers (RDC/XRDC/TRDC)** that provide support for the isolation of peripherals and memory.
 - **Arm® TrustZone®** technology-based memory protection for embedded memories such as the on-chip RAM (OCRAM).
 - The **TrustZone® Address Space Controller (TZASC)** that protects and secures data in a trusted execution environment.
 - The **AIPSTZ** bridge that provides programmable access protections for both controllers and peripherals.
- The default security configuration in OP-TEE OS for these security modules is left in an open (non-secure) state because a universal secure configuration that meets all customer requirements is not possible.
- NXP delivers various open-source software components (NXP OP-TEE OS) for customer enablement, however, these are not provided as secure production-ready implementations.
- Using OP-TEE OS upstream releases instead of NXP OPTEE-OS releases may have an impact on the features supported and the security level of the i.MX platforms.
- Customers should optimize the security configuration in OP-TEE OS to lock and secure end products according to their specific security requirements.
- NXP has documented how to securely configure these security modules in the respective [i.MX SoC Reference and Security manuals](#) and also provides a Security Checklist for the i.MX family to help customers secure end products.

- For Further assistance please contact your NXP field representative or submit an [NXP Support ticket](#).

BUILD AND RUN

In this part of the documentation you will find information telling you how to build OP-TEE as a whole developer setup or as individual components. Likewise it will also tell you how to run OP-TEE on various devices.

Since you pressed this, it's likely that you want to know how to build a full OP-TEE developer setup. So a first place to start looking is probably at the “*build*” page to get started.

You may also want to look at “*Q: What is the quickest and easiest way to try OP-TEE?*”.

3.1 Prerequisites

We believe that you can use any Linux distribution to build OP-TEE, but as maintainers of OP-TEE we are mainly using Ubuntu-based distributions and to be able to build and run OP-TEE there are a few packages that needs to be available. Hereafter we provide Docker files which may be used as a reference.

Ubuntu 22.04

Ubuntu 20.04

Older

```
FROM ubuntu:22.04
ARG DEBIAN_FRONTEND=noninteractive
RUN apt update && apt upgrade -y
RUN apt install -y \
    adb \
    acpica-tools \
    autoconf \
    automake \
    bc \
    bison \
    build-essential \
    ccache \
    cpio \
    cscope \
    curl \
    device-tree-compiler \
    e2tools \
    expect \
    fastboot \
    flex \
    ftp-upload \
```

(continues on next page)

(continued from previous page)

```

gdisk \
git \
libattr1-dev \
libcap-ng-dev \
libfdt-dev \
libftdi-dev \
libglib2.0-dev \
libgmp3-dev \
libhidapi-dev \
libmpc-dev \
libncurses5-dev \
libpixman-1-dev \
libslirp-dev \
libssl-dev \
libtool \
libusb-1.0-0-dev \
make \
mtools \
netcat \
ninja-build \
python3-cryptography \
python3-pip \
python3-pyelftools \
python3-serial \
python-is-python3 \
rsync \
swig \
unzip \
uuid-dev \
wget \
xdg-utils \
xterm \
xz-utils \
zlib1g-dev
RUN curl https://storage.googleapis.com/git-repo-downloads/repo > /bin/repo && chmod a+rx
↪ /bin/repo
RUN mkdir /optee
WORKDIR /optee
RUN repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml && repo sync -j10
WORKDIR /optee/build
RUN make -j2 toolchains
RUN make -j$(nproc) check

```

```

FROM ubuntu:20.04
ARG DEBIAN_FRONTEND=noninteractive
RUN apt update && apt upgrade -y
RUN apt install -y \
    android-tools-adb \
    android-tools-fastboot \
    autoconf \
    automake \
    bc \

```

(continues on next page)

(continued from previous page)

```

bison \
build-essential \
ccache \
cpio \
cscope \
curl \
device-tree-compiler \
expect \
flex \
ftp-upload \
gdisk \
git \
iasl \
libattr1-dev \
libcap-ng-dev \
libfdt-dev \
libftdi-dev \
libglib2.0-dev \
libgmp3-dev \
libhidapi-dev \
libmpc-dev \
libncurses5-dev \
libpixman-1-dev \
libslirp-dev \
libssl-dev \
libtool \
make \
mtools \
netcat \
ninja-build \
python-is-python3 \
python3-crypto \
python3-cryptography \
python3-pip \
python3-pyelftools \
python3-serial \
rsync \
unzip \
uuid-dev \
wget \
xdg-utils \
xterm \
xz-utils \
zlib1g-dev
RUN curl https://storage.googleapis.com/git-repo-downloads/repo > /bin/repo && chmod a+rx
↪ /bin/repo
RUN mkdir /optee
WORKDIR /optee
RUN repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml && repo sync -j10
WORKDIR /optee/build
RUN make -j2 toolchains
RUN make -j$(nproc) check

```

Note: No longer supported by the OP-TEE community!

Due to all changes over the years with different names of Python packages and different requirement in time for Python2 and/or Python3 packages, it's not really possible to build more recent versions of OP-TEE with something that is older than Ubuntu 18.04. If you for some reason need to rebuild OP-TEE using a very old distro, then the best strategy for doing is to check an earlier version of this documentation and start with the build instructions from there.

3.2 Device specific information

3.2.1 AMD-Xilinx Versal ACAP VCK190

Instructions below show how to run OP-TEE on the [VCK190](#) development board. Details of the Versal ACAP can be found in the Versal Technical Reference Manual ([Versal_TRM](#)).

Supported boards

This makefile supports the VCK190 but also supports the [VMK180](#) development board as well.

Setting up the toolchain

This build chain relies on Petalinux 2022.1, therefore the first step will be to download and [install](#) it from the AMD-Xilinx website ([Downloads](#)).

Then, you will also need to download the board support package (BSP) from the AMD-Xilinx website ([Downloads](#)). It contains prebuilt firmwares and hardware definition files required to assemble a bootable image.

Note: You will need a free AMD-Xilinx account to proceed with the two previous steps.

Configuring and building for VCK190

Lets summarize the steps taken so far; these are common to all boards.

```
$ mkdir ~/optee-project
$ cd ~/optee-project
$ repo init -u https://github.com/OP-TEE/manifest.git -m versal.xml
$ repo sync -j4 --no-clone-bundle
$ cd build
$ make -j8 toolchains
$ make -j8
```

At this point we have a working directory `~/optee-project` with all the repositories required with the exception of the Versal ACAP board support package. A pre-requisite to unpacking the BSP file is installing Petalinux ([install](#)) as previously mentioned.

Having done that, now is the time to unpack the BSP:

```
$ cd ~/optee-project
$ cp ~/Downloads/xilinx-vck190-v2022.1-04191534.bsp .
$ source /path/to/petalinux.2022.1/settings.sh
$ petalinux-create --type project -s xilinx-vck190-v2022.1-04191534.bsp
$ ls
    xilinx-vck190-2022.1
```

In order for the Versal OP-TEE port to work correctly, the PLM needs to be updated to add the XilNvm and XilPuf libraries. This can be accomplished by the following steps within the PetaLinux workspace created above:

```
$ mkdir project-spec/meta-user/recipes-bsp/embeddedsw
$ cp ~/optee-project/build/versal/plm-firmware_%.bbappend project-spec/meta-user/recipes-
  ↳ bsp/embeddedsw
$ petalinux-build -c plm
```

The newly created PLM will be located in the folder `images/linux/plm.elf`.

Note: Replace the VCK190 BSP with the VMK180 BSP if you want to build this project for the VMK180 development board.

Before building the release, you will need to edit the Boot Image File (BIF) `build/versal/bootImage-versal-vck190.bif` to point to the required BSP files. The paths for the following files in the BIF will need to be updated *before* proceeding:

- `vpl_gen_fixed.pdi`
- `plm.elf`
- `psmfw.elf`

Note: The default PLM **only** contains the `xilsecure` library. If you would like to take advantage of all of hardware cryptographic features implemented for Versal, you **must** enable the `xilpuf` and `xilnvm` libraries by following the steps above for customizing the PLM ([PLM_Customization](#)).

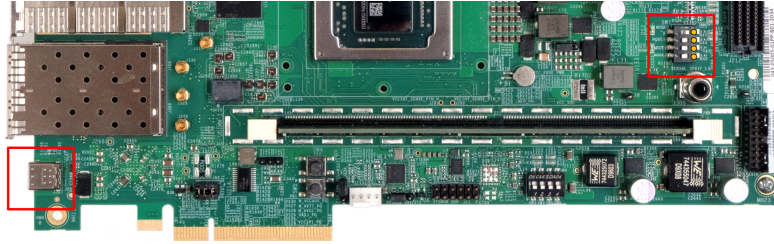
The `xilpuf` library enables support of the physically unclonable function (PUF) and the `xilnvm` library enables support of reading and writing to eFUSEs. Once these libraries are enabled, be sure to point to the updated PLM firmware in the previously mentioned BIF file.

After you have done that you can build the images as follows:

```
$ cd ~/optee-project
$ cd build
$ make -f versal.mk image
$ ls versal | grep -E 'BIN|ub'
    BOOT.BIN
    versal-vck190.ub
```

JTAG boot to U-Boot shell

To run the bootable image `BOOT.BIN` via JTAG, configure the boot switches as seen below and then power up the board.



Then run the `boot_jtag.sh` script.

This script will first ask for the path of the Petalinux installation; once entered, it will download and execute the image on the Versal ACAP platform.

```
$ cd ~/optee-project/build/versal/  
$ ./boot_jtag.sh
```

SD card creation and boot

Prepare a SD card with a single **bootable** partition large enough to hold both of the built files.

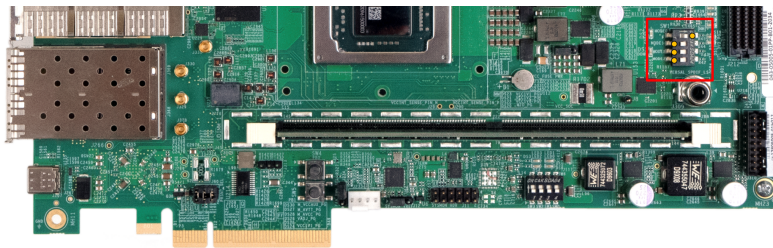
Using `gparted` or any other partition manager tool create a single partition on the card (remember to flag it as bootable)

- 1GB FAT32 bootable partition (i.e: `/dev/sdc1`).

Once SD card is partitioned, mount it on your file system and copy the images:

```
$ cp ~/optee-project/build/versal/BOOT.BIN <mount_point>/  
$ cp ~/optee-project/build/versal/versal-vck190.ub <mount_point>/  
$ sync  
$ umount <mount_point>
```

Now you can use the newly created SD card to boot your board. Make sure the boot switches are configured for SD boot.



Unless you have modified the default U-boot boot command, you will need to stop the sequence at the U-boot shell and issue these three additional commands to boot to Linux:

```
uboot shell$ mmc dev 0  
uboot shell$ fatload mmc 0:1 0x20000000 versal-vck190.ub  
uboot shell$ bootm 0x20000000
```

3.2.2 DeveloperBox

The instructions here will tell how to build OP-TEE for [DeveloperBox](#).

Build instructions

1. Follow the “*Get and build the solution*” in *build* from step 1 to step 3.
2. Initialize EDK2 submodule

```
1 $ cd <optee-project>/edk2
2 $ git submodule update --init
```

3. Follow “*Get and build the solution*” step 4 & 5
4. Stage a new OP-TEE update capsule. This updates TF-A, OP-TEE and UEFI.

```
1 $ fwupdate --apply {50b94ce5-8b63-4849-8af4-ea479356f0e3} \
2 > <optee-project>/edk2-platforms/Build/DeveloperBox/RELEASE_GCC5/FV/\
3 > SYNQUACERFIRMWAREUPDATECAPSULEFMPPKCS7.Cap
```

Hint: Change RELEASE_GCC5 to DEBUG_GCC5 for debug build.

5. Reboot to update.
6. Follow the rest of “*Get and build the solution*” from step 7

3.2.3 FVP

The instructions here will tell how to build and run OP-TEE using Foundation Models.

Build instructions

Start out by following the “*Get and build the solution*” as described in *build*. However, stop before doing “*Step 5 - Build the solution*”.

Next you should obtain the [Armv8-A Foundation Platform \(For Linux Hosts Only\)](#). To download FVPs you’ll need to log in to Arm Self Service. That binary should be untar’ed to the root of the repo forest, i.e., like this: `<fpv-project>/Foundation_Platformpkg`. In the end after cloning all source code, getting the toolchains and “installing” Foundation_Platformpkg you should have a folder structure that looks like this:

```
$ ls -al
drwxrwxr-x 15 jbech jbech 4096 Feb  5 09:10 .
drwxr-xr-x 22 jbech jbech 4096 Jan 15 12:45 ..
drwxrwxr-x 18 jbech jbech 4096 Feb  5 09:10 arm-trusted-firmware
drwxrwxr-x  9 jbech jbech 4096 Feb  5 09:10 build
drwxrwxr-x 15 jbech jbech 4096 Feb  5 09:10 buildroot
drwxrwxr-x 51 jbech jbech 4096 Feb  5 09:10 edk2
drwxrwxr-x  5 jbech jbech 4096 Feb  5 09:10 edk2-platforms
drwxrwxr-x  6 jbech jbech 4096 Mar 15 2018 Foundation_Platformpkg
drwxrwxr-x 15 jbech jbech 4096 Feb  5 09:10 grub
drwxrwxr-x 26 jbech jbech 4096 Feb  5 09:10 linux
```

(continues on next page)

(continued from previous page)

```
drwxrwxr-x  6 jbech jbech 4096 Feb  5 09:10 optee_client
drwxrwxr-x 10 jbech jbech 4096 Feb  5 09:10 optee_examples
drwxrwxr-x 11 jbech jbech 4096 Feb  5 09:10 optee_os
drwxrwxr-x  8 jbech jbech 4096 Feb  5 09:10 optee_test
drwxrwxr-x  7 jbech jbech 4096 Feb  5 09:10 .repo
lrwxrwxrwx  1 jbech jbech  23 Feb  5 09:09 toolchains
```

When this pre-condition met you can simply continue with

```
$ make run
```

and then FVP should build the rootfs and then start the simulation and when you have a terminal you can log in and run xtest (as described at [Step 9 - Run xtest](#)).

3.2.4 HiKey 620

The instructions here will tell how to run OP-TEE on [HiKey 620](#).

Multiple sources for HiKey and OP-TEE instructions?

First you must understand that the HiKey project as such is led by the 96Boards project. So, if you **aren't** interested in running OP-TEE on the device, then you should stop reading here and instead have a look at the [official HiKey documentation](#).

For OP-TEE using HiKey you will still find information in more than one place. There are a couple of reasons for that.

- **96Boards:** The official 96Boards project used to host some OP-TEE instructions and they include OP-TEE in their official releases.
- **Google:** has an [AOSP HiKey branch](#), where OP-TEE is supported to some extent.
- **Linaro-SWG:** The OP-TEE team has done some work related to AOSP (see the [AOSP](#) page) and there HiKey has been one of the devices in use.

If you have questions regarding the configurations above, please reach out to the people on the right forum (96Boards, Google and Linaro-SWG).

This particular guide is maintained by the OP-TEE [core team](#) and this is what we use when we are doing are stable releases for our OP-TEE developer builds. I.e, for OP-TEE this should be considered as a well maintained guide with a fully working setup.

Supported HiKey boards

There are four different versions of the HiKey board.

Name	Manufacturer	Memory	Flash	Comment
HiKey	CircuitCo	1GB	4GB	Green solder mask
HiKey	LeMaker	1GB	8GB	Black solder mask
HiKey	LeMaker	2GB	8GB	Black solder mask

All of them works, but where differences apply we have default configurations that works for the LeMaker 8GB eMMC versions.

UART adapter board

Everything is configured to use the 96Boards [UART Adapter Board](#). The UART is by default configured to UART3. If you don't have any UART adapter board and instead would like to use UART0, then you need to change that before building. See `CFG_NW_CONSOLE_UART` and `CFG_NW_CONSOLE_UART` in [hikey.mk](#).

Build instructions

Just follow the “*Get and build the solution*” as described in [build](#). The `make flash` step will tell you how you should set the jumpers on the board.

Recovery

If you manage to corrupt the device, so that fastboot doesn't load automatically on boot, then you will need to run the recovery procedure. Basically what you will need to do is use another make target and change some jumpers. All that is described when you run the target:

```
$ make recovery
```

3.2.5 HiKey 960

The instructions here will tell how to run OP-TEE on [HiKey 960](#).

Supported HiKey960 boards

There are three different versions of the HiKey960 board.

Name	Manufacturer	Mem-ory	Flash	Comment
HiKey960	Archer-mind/LeMaker	4GB	32GB	v2 uses DIP Switches (SW2201), rev B has 4GB RAM
HiKey960	Archer-mind/LeMaker	3GB	32GB	v2 uses DIP Switches (SW2201), rev A has 3GB RAM
HiKey960	Archer-mind/LeMaker	3GB	32GB	v1 uses Jumpers (J2001)

UART adapter board

Everything is configured to use the 96Boards [UART Serial](#) adapter. The UART is by default configured to UART6. If you have a v1 board and need to use UART5, then you need to change that before building. See `CFG_CONSOLE_UART` in [hikey960.mk](#).

Build instructions

Just follow the instructions at “*Get and build the solution*”. If `make flash` doesn’t work, try `make recovery`.

For the 4GB RAM board version (rev B), an update to the `CFG_DRAM_SIZE_GB` setting in `conf.mk` is needed. Either update the value from 3 to 4 in `conf.mk` before building, or set the value on the command line when building, i.e `make PLATFORM=hikey-hikey960 CFG_DRAM_SIZE_GB=4`

Recovery

If you manage to corrupt the device, such that fastboot doesn’t load automatically on boot, then you will need to run the recovery procedure. Basically what you will need to do is use another make target and change some jumpers. All that is described when you run the target:

External guide

<https://github.com/ARM-software/arm-trusted-firmware/blob/master/docs/plat/hikey960.rst>

```
$ make recovery
```

3.2.6 Juno

The instructions here will tell how to run OP-TEE on the Juno board. The instructions has been tested and verified on the Juno r0 revision (see [Juno revisions](#) for more details).

Regular build

First step is to start out by following the instructions in the *Get and build the solution* as described in *build*.

Deploy files on the device

Enter the firmware console on the Juno board and press **enter** to stop the auto boot.

```
ARM V2M_Juno Firmware v1.3.9
Build Date: Nov 11 2015

Time : 12:50:45
Date : 29:03:2016

Press Enter to stop auto boot...
```

Enable FTP at the firmware prompt.

```
Cmd> ftp_on
Enabling ftp server...
MAC address: xxxxxxxxxxxx

IP address: 192.168.1.158

Local host name = V2M-JUNO-A2
```

Flash the binary by running

Note: Use the **IP address** from output from previous command.

```
$ make JUNO_IP=192.168.1.158 flash
```

Once all binaries have been transferred, reboot the board:

```
Cmd> reboot
```

Update the flash layout

The flash layout for Juno may need to be updated for the flashing above to work. If flashing fails or if TF-A refuses to boot due to wrong version of the SCP binary, then the flash(-layout) needs to be updated. To update the flash please follow the instructions at Arm's [old release notes](#) page selecting one of the zips under "Development boards / Juno / Prebuilt configurations" and flash it as described at [Run the Arm Platforms deliverables on Juno](#).

GlobalPlatform testsuite support

Note: Depending on the Juno pre-built configuration, the built `ramdisk.img` size with GlobalPlatform testsuite may exceed its pre-defined Juno flash memory reserved location (`image.txt` file). In that case, you will need to extend the Juno flash block size reserved location for the `ramdisk.img` in the `image.txt` file accordingly and follow the instructions under "5.7.1 Update flash and its layout".

Example

Example with `juno-latest-busybox-uboot.zip`. The current `ramdisk.img` size with GlobalPlatform testsuite is 8.6 MBytes and that is too big to fit in the default configuration, therefore we need to make adjustments to the flash layout. You will do that by making changes to `/JUNO/SITE1/HBI0262B/images.txt`. I.e., from:

```
1 NOR4UPDATE: AUTO           ;Image Update:NONE/AUTO/FORCE
2 NOR4ADDRESS: 0x01800000    ;Image Flash Address
3 NOR4FILE: \SOFTWARE\ramdisk.img ;Image File Name
4 NOR4NAME: ramdisk.img
5 NOR4LOAD: 00000000         ;Image Load Address
6 NOR4ENTRY: 00000000        ;Image Entry Point
```

to extending the *Image Flash Address* to 16MB

```
1 NOR4UPDATE: AUTO           ;Image Update:NONE/AUTO/FORCE
2 NOR4ADDRESS: 0x01000000    ;Image Flash Address
3 NOR4FILE: \SOFTWARE\ramdisk.img ;Image File Name
4 NOR4NAME: ramdisk.img
5 NOR4LOAD: 00000000         ;Image Load Address
6 NOR4ENTRY: 00000000        ;Image Entry Point
```

GCC > 5.x support

Note: In case you are using the **latest version** of the OP-TEE Arm Juno build (i.e., `juno.xml` manifest), then the `ramdisk.img` built with a GCC version newer than 5.x will be bigger than built with older GCC versions. This means that you will need to update the sections in `image.txt` that tells where various images will start (see the `image.txt` file).

To solve this problem you will need to extend the Juno flash block size reserved location for the `ramdisk.img` and decrease the size for other images in the `image.txt` file accordingly in the same manner as described in the previous section above.

For example with `juno-latest-busybox-uboot.zip`. The current `ramdisk.img` size with GCC 5.x compiler is 29.15MB and therefore we will need to extend that size for that to 32MB. You do that by changing the highlighted ones (i.e., *Image Flash Address*) in file `/JUNO/SITE1/HBI0262B/images.txt`.

```

1  NOR2UPDATE: AUTO                ;Image Update:NONE/AUTO/FORCE
2  NOR2ADDRESS: 0x00100000         ;Image Flash Address
3  NOR2FILE: \SOFTWARE\Image       ;Image File Name
4  NOR2NAME: norkern               ;Rename kernel to norkern
5  NOR2LOAD: 00000000              ;Image Load Address
6  NOR2ENTRY: 00000000             ;Image Entry Point
7
8  NOR3UPDATE: AUTO                ;Image Update:NONE/AUTO/FORCE
9  NOR3ADDRESS: 0x02C00000         ;Image Flash Address
10 NOR3FILE: \SOFTWARE\juno.dtb    ;Image File Name
11 NOR3NAME: board.dtb             ;Specify target filename to preserve file extension
12 NOR3LOAD: 00000000              ;Image Load Address
13 NOR3ENTRY: 00000000             ;Image Entry Point
14
15 NOR4UPDATE: AUTO                ;Image Update:NONE/AUTO/FORCE
16 NOR4ADDRESS: 0x00D00000         ;Image Flash Address
17 NOR4FILE: \SOFTWARE\ramdisk.img ;Image File Name
18 NOR4NAME: ramdisk.img
19 NOR4LOAD: 00000000              ;Image Load Address
20 NOR4ENTRY: 00000000             ;Image Entry Point
21
22 NOR5UPDATE: AUTO                ;Image Update:NONE/AUTO/FORCE
23 NOR5ADDRESS: 0x02D00000         ;Image Flash Address
24 NOR5FILE: \SOFTWARE\hdlcdclk.dat ;Image File Name
25 NOR5LOAD: 00000000              ;Image Load Address
26 NOR5ENTRY: 00000000             ;Image Entry Point

```

3.2.7 NUVOTON

The instructions here will tell how to build and run OP-TEE OS for Nuvoton platform standalone (not as a part of `openbmc` image).

Build instructions

Pre-requirements:

1. Install prerequisites according to the [Prerequisites](#) page.
2. Download the latest IGPS from the Nuvoton Israel GitHub repository

```
$ git clone https://github.com/Nuvoton-Israel/igps-npcm8xx
```

#. Download and extract the latest Linux based Arm GNU Toolchain for aarch64 bare-metal target, for example, 12.2

```
$ cd /opt
$ wget https://developer.arm.com/-/media/Files/downloads/gnu/12.2.rel1/binrel/
  ↪ arm-gnu-toolchain-12.2.rel1-x86_64-aarch64-none-elf.tar.xz?
  ↪ rev=28d5199f6db34e5980aae1062e5a6703&
  ↪ hash=D87D4B558F0A2247B255BA15C32A94A9F354E6A8
$ tar xvf arm-gnu-toolchain-12.2.rel1-x86_64-aarch64-none-elf.tar.xz
```

1. Add the Arm GNU Toolchain binary to the \$PATH

```
$ cd ~
$ vi .bashrc
export PATH=$PATH:/opt/arm-gnu-toolchain-12.2.rel1-x86_64-aarch64-none-elf/
  ↪ bin
```

2. Clone the latest OP-TEE OS code from the GitHub repository

```
$ git clone https://github.com/OP-TEE/optee_os.git
$ cd optee_os
```

Build process:

1. build OP-TEE OS for Nuvoton platform

```
$ make CROSS_COMPILE64=aarch64-none-elf- PLATFORM=nuvoton -j $(nproc)
```

Note: you can use additional debug flag for compilation to get debug prints to console

```
$ make CROSS_COMPILE64=aarch64-none-elf- PLATFORM=nuvoton CFG_NPCM_DEBUG=y -
  ↪ j $(nproc)
```

2. Update binary input files in IGPS

```
$ cd ../igps-npcm8xx/py_scripts
$ python ./UpdateInputsBinaries_Arbel_A1_EB.py
```

3. Copy the compiled out/arm-plat-nuvoton/core/tee.bin file into IGPS_3.8.6/py_scripts/ImageGeneration/inputs
4. Generate new image file

```
$ python ./GenerateAll.py
```

5. Program the new image to flash:

```
$ python ./ProgramAll_Secure.py
```

6. After programming, enable terminal connection to the ArbelEVB, and if you compiled with the `CFG_NPCM_DEBUG=y` flag, you will see OP-TEE version before U-Boot console trace messages, for example:

```
I/TC: >===== I/TC: OP-TEE OS Version 3.21.0-1127-gaf809d0ab-dev (gcc version 12.2 (Arm GNU Toolchain 12.2.Rel1)) #1 Thu May 18 05:24:18 UTC 2023 aarch64 I/TC: >=====
```

On this page you will find device specific information for QEMU v7 (Armv7-A) and QEMU v8 (Armv8-A).

3.2.8 QEMU v7

The instructions here will tell how to run OP-TEE using QEMU for Armv7-A.

Build instructions

As long as you pick the v7 manifest, i.e., `default.xml` the “*Get and build the solution*” tells all you need to know to build and boot up QEMU v7.

A usual short shell sequence to fetch, build and run OP-TEE using QEMU for Armv7-A is like the one below:

```
$ mkdir optee
$ cd optee
$ repo init -u https://github.com/OP-TEE/manifest.git
$ repo sync
$ cd build
$ make toolchains
$ make run
```

Hint: If you do not want to check out the latest version of OP-TEE, but rather a specific tagged version, you can use `repo init -u https://github.com/OP-TEE/manifest.git -b <branchname>`. e.g., `repo init -u https://github.com/OP-TEE/manifest.git -b 3.16.0`. You can see valid branch names by inspecting the OP-TEE/manifest git repository on <https://github.com/OP-TEE/manifest/branches>.

To speed up your build, you can make use of the parallel make feature. For example, use `make -j32 run` to have 32 build processes running concurrently. Note that this will make it much more difficult to spot errors if something fails; therefore fall back to sequential builds to view build errors and produce logs for bug reports.

Consoles

After running `make run` you will end up in the QEMU console and it will also spawn two UART consoles. One console containing the UART for secure world and one console containing the UART for normal world. You will see that it stops waiting for input on the QEMU console. To continue, do:

```
(qemu) c
```

Host-Guest folder sharing

You can use the VirtFS QEMU feature to avoid changing rootfs CPIO archive every time you need to add additional files or modify existing files. To do this, you share a folder between the guest and host operating systems. To enable and use this feature you have to provide additional arguments when running make, example:

```
$ make QEMU_VIRTFS_ENABLE=y QEMU_USERNET_ENABLE=y
```

Hint: You can also add QEMU_VIRTFS_HOST_DIR=<share> in case you don't want to use the default sharing location (which is the root of <qemu-v7-project>).

When QEMU with OP-TEE is up and running, you can mount the host folder in QEMU (normal world UART).

```
# mount -t 9p -o trans=virtio host <mount_point>
```

<mount_point> here is folder in the QEMU where you want to mount the host PC's shared folder. So if you want to mount it at /mnt/host you typically do this from QEMU NW/UART.

```
# mkdir -p /mnt/host
# mount -t 9p -o trans=virtio host /mnt/host
```

Networking

After booting QEMU, eth0 will automatically receive an IP address from QEMU via DHCP using the SLiRP user networking feature. QEMU will act as a gateway to the host network SLiRP.

Please note that ICMP won't work in the guest unless additional configuration is made, so the ping utility won't work.

GDB - Normal world

If you need to debug a client application, using GDB in a remote debugging configuration may be useful. Remote debugging means gdb runs on your PC, where it can access the source code, while the program being debugged runs on the remote system (in this case, in the QEMU environment in normal world). Here is how to do that. On your PC, build with GDBSERVER=y:

```
$ cd <qemu-v7-project>/build
# You **only** need to rm -rf the first time you build with the new flag.
# If you omit doing so, it's likely that you will see "stamp" errors in the
# build log.
$ rm -rf <qemu-v7-project>/out-br
$ make -j8 run GDBSERVER=y
```

Boot up as usual

```
(qemu) c
```

Inside QEMU (Normal World UART), run your application with gdbserver (for example xtest 4002):

```
# gdbserver :12345 xtest 4002
Process xtest created; pid = 654
Listening on port 12345
```

Back on your PC, open another terminal, start GDB and connect to the target:

```
$ <qemu-v7-project>/out-br/host/bin/arm-buildroot-linux-gnueabi-hf-gdb
(gdb) set sysroot <qemu-v7-project>/out-br/host/arm-buildroot-linux-gnueabi-hf/sysroot
(gdb) target remote :12345
```

Now GDB is connected to the remote application. You may use GDB normally.

```
(gdb) b main
(gdb) c
```

GDB - Secure world

TEE core debugging

To debug TEE core running QEMU with GDB, you need to disable TEE ASLR with `CFG_CORE_ASLR=n` flag. Furthermore, note that it's easier to debug if you have optimization disabled. Other than that you will have four consoles that you are working with.

- Qemu console
- NW UART console
- SW UART console
- GDB console

All of them but the GDB console are consoles you normally will see/use when running OP-TEE/xtest using QEMU. The first thing is to start QEMU, i.e.,

```
$ cd <qemu-v7-project>/build
# make run-only also works if you don't want to rebuild things
$ make run CFG_CORE_ASLR=n
```

Next launch another console for GDB and do this

```
$ cd <qemu-v7-project>/toolchains/aarch32/bin
$ ./arm-linux-gnueabi-hf-gdb -q
```

In the GDB console connect to the QEMU GDB server, like this (the output is included to show what you normally will see).

```
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x00000000 in ?? ()
```

Still in the GDB console, load the symbols for TEE core

```
(gdb) symbol-file <qemu-v7-project>/optee_os/out/arm/core/tee.elf
Reading symbols from <qemu-v7-project>/optee_os/out/arm/core/tee.elf...done.
```

Now you can set a breakpoint for any symbol in OP-TEE, for example

```
(gdb) b tee_entry_std
Breakpoint 1 at 0xe103012: file core/arch/arm/tee/entry_std.c, line 526.
```

Last step is to initiate the boot, do that also from the GDB console

```
(gdb) c
Continuing.
```

At this point will see UART output in the Normal world console as well as the Secure world UART console. If you now for example [Run xtest](#), then you will rather soon hit the breakpoint we previously set and you will see something like this in the GDB console:

```
Continuing.
[Switching to Thread 2]

Thread 2 hit Breakpoint 1, tee_entry_std (smc_args=0xe183f18
<stack_thread+8216>) at core/arch/arm/tee/entry_std.c:526
526          struct optee_msg_arg *arg = NULL;          /* fix gcc warning */
(gdb)
```

From here you can start to poke around with GDB, single step, read memory, read registers, print variables and all sorts of things that you normally do with a debugger.

Hint: Some people find it easier to also see the source code while debugging. You can enable the “TUI mode” to see the source code in GDB. To enable that, run GDB with

```
$ ./arm-linux-gnueabi-gdb -q -tui
```

3.2.9 QEMU v8

The instructions here will tell how to run OP-TEE using QEMU for Armv8-A.

Build instructions

As long as you pick the v8 manifest, i.e., `qemu_v8.xml` the “[Get and build the solution](#)” tells all you need to know to build and boot up QEMU v8.

A usual short shell sequence to fetch, build and run OP-TEE using QEMU for Armv8-A is like the one below:

```
$ mkdir optee
$ cd optee
$ repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
$ repo sync
$ cd build
$ make toolchains
$ make run
```

All other things (networking, GDB etc) in the v7 section above is also applicable on QEMU v8 as long as you replace `<qemu-v7-project>` with `<qemu-v8-project>` to get the correct paths relative to your QEMU v8 setup.

3.2.10 ROCK Pi 4

The instructions here will tell how to run OP-TEE on [ROCK Pi 4 / ROCK 4](#) boards.

Supported ROCK Pi 4 boards

There are several versions of the ROCK Pi 4 board, each with 1GB, 2GB or 4GB RAM options. OP-TEE has been tested and is known to work with a Rock Pi 4 Model B OP1 4GB. Other variants will likely work too.

UART

The console can be accessed using a USB adapter connected to the board as described in the [serial console](#) documentation.

Build instructions

Just follow the “*Get and build the solution*” as described in [build](#). To flash the board you will need a USB type A to type A cable. The `make flash` step will tell you how to connect the cable and use the buttons.

3.2.11 Raspberry Pi 3

[Sequitur Labs](#) did the initial OP-TEE port which at the time also came with modifications in U-Boot, Trusted Firmware A and Linux kernel. Since that initial port more and more patches have found mainline trees and today the OP-TEE setup for Raspberry Pi 3 uses only upstream tree’s with the exception of Linux kernel.

Disclaimer

Warning: This port of Trusted Firmware A and OP-TEE to Raspberry Pi 3 **IS NOT SECURE!** Although the Raspberry Pi3 processor provides ARM TrustZone exception states, the mechanisms and hardware required to implement secure boot, memory, peripherals or other secure functions are not available. Use of OP-TEE or TrustZone capabilities within this package **does not result** in a secure implementation. This package is provided solely for **educational purposes** and **prototyping**.

What is expected to work?

First, note that all OP-TEE developer builds (ref, [build](#)) have rather simple overall goals:

- Successfully build OP-TEE for certain devices.
- Run `xtest` and `optee_example` binaries successfully with no regressions using UART(s).

I.e., it is important to understand that our “*OP-TEE developer builds*” shall not be compared with full Linux distributions which supports “everything”. As a couple of examples, we don’t enable any particular drivers in Linux kernel, we don’t include all sorts of daemons, we do not include an X-environment etc. At the same time this doesn’t mean that you cannot use OP-TEE in real environments. It is usually perfectly fine to run on all sorts of devices, environments etc. It’s just that for the OP-TEE developer builds we have intentionally stripped down the environment to make it rather fast to get all the source code, build it all and run `xtest`.

We are highlighting this here, since over the years we have had many questions at GitHub about things that people usually find working on their Raspberry Pi devices when they are using Raspbian (which this is not). The table below

describes what is *officially* supported in the Raspberry Pi 3 OP-TEE developer builds and right after that follows sections for each of giving a bit more context to it.

Name	Supported?
Buildroot	Yes
HDMI	No
NFS	Yes
Random packages	Maybe
Raspbian	No
Secure boot	Maybe
TFTP	Yes
UART	Yes
Wi-Fi	No

Buildroot

We are using Buildroot as the tool to create a stripped down filesystem for Linux where we also put OP-TEE binaries like Trusted Applications, client libraries and TEE supplicant. If a user wants to add/enable additional packages, then that is also possible by adding new lines in `common.mk` in *build* (search for `BR2_PACKAGE_` in the git to see how it's done).

HDMI

X isn't enabled and we have not built nor enabled any drivers for graphics.

NFS

Works to boot up a Linux root filesystem, more on that further down.

Random packages

See the *Buildroot* section above. You can enable packages supported by Buildroot, but as mentioned initially in this section, lack of drivers and other daemons etc might make it impossible to run.

Raspbian

We are not using it. However, people (from [Sequitur Labs](#)) have successfully been able to add OP-TEE to Raspbian builds. But since we're not using it and haven't tried, we simply don't support it.

Secure boot

First pay attention to the initial warning on this page. I.e., no matter what you are doing with Raspberry Pi and TrustZone / OP-TEE you **cannot** make it secure. But that doesn't mean that you cannot "enable" secure features as such for prototyping and to learn how to build and use those. That kind of knowledge can later on be transferred and used on other devices which have all the necessary secure capabilities needed to make a secure system. We haven't tested to enable secure boot on Raspberry Pi 3. But we believe that a good starting point would be Trusted Firmware A's documentation about the "[Authentication Framework](#)" and [RPi3 in TF-A](#).

TFTP

When you reach U-Boot (see [Boot sequence](#)), then you can start using TFTP to load boot firmware etc. Note that if you overwrite `armstub8.bin` for example and that happens to be faulty, then you will need to re-mount the BOOT partition on the SD-card and put a new working version of it. Also note that changing early boot binaries (TF-A, OP-TEE core etc) will require you to reboot the device see the changes.

UART

Fully supported, for more details look at the UART section further down.

Wi-Fi

Even though Raspberry Pi 3 has a Wi-Fi chip, we do not support it in our stripped down builds.

What versions of Raspberry Pi will work?

Below is a table of supported hardware in our OP-TEE developer builds. We have only used the Raspberry Pi 3 Model B, i.e., the first RPi 3 device that was released. But we know that people have successfully been able to use it with both RPi 2's as well as the newer RPi 3 B+. But as long as we in the [core team](#) doesn't have those at hands we cannot guarantee anything, therefore we simply say "No" below.

Hardware	Supported?
Raspberry Pi 1 Model A	No
Raspberry Pi 1 Model B	No
Raspberry Pi 1+ Model A	No
Raspberry Pi 1+ Model B	No
Raspberry Pi 2 Model B	No
Raspberry Pi 2 Model B v1.2	No
Raspberry Pi 3+ Model A	No
Raspberry Pi 3 Model B	Yes
Raspberry Pi 3+ Model B	Yes
Raspberry Pi 4	No
Zero - all versions	No
Compute module - all versions	No

Boot sequence

- The **GPU** starts executing the first stage bootloader, which is stored in ROM on the SoC. The first stage bootloader reads the SD-card, and loads the second stage bootloader (`bootcode.bin`) into the L2 cache, and runs it.
- `bootcode.bin` enables SDRAM, and reads the third stage bootloader `loader.bin` from the SD-card into RAM, and runs it.
- `loader.bin` reads the GPU firmware (`start.elf`).
- `start.elf` reads `config.txt`, pre-loads `armstub8.bin` (which contains: BL1/TF-A + BL2/TF-A + BL31/TF-A + BL32/OP-TEE + BL33/U-boot) to `0x0` and jumps to the first instruction.
- A traditional boot sequence of TF-A -> OP-TEE -> U-boot is performed, i.e., BL1 loads BL2, then BL2 loads and run BL31(SM), BL32(OP-TEE), BL33(U-boot) (one after another)
- U-Boot runs `fatload/booti` sequence to load from eMMC to RAM both `zImage` and then DTB and boot.

Build instructions

1. Start by following the [Get and build the solution](#) as described in [build](#), but stop at the “[Step 6 - Flash the device](#)” step (i.e., **don’t** run the `make flash` command!).
2. Next step is to partition and format the memory card and to put the files onto the same. That is something we don’t want to automate, since if anything goes wrong, in worst case it might wipe one of your regular hard disks. Instead what we have done, is that we have created another makefile target that will tell you exactly what to do. Run that command and follow the instructions there.

```
$ make img-help
```

Note: The mention of `/dev/sdx1` and `/dev/sdx2` when running the command above are just examples. You need to figure out and replace that with the correct name(s) for your computer and SD-card (typically run `dmesg` and look for the device name matching your SD-card).

3. Put the SD-card back into the Raspberry Pi 3.
4. Plug in the UART cable and attach to the UART

```
$ picocom -b 115200 /dev/ttyUSB0
```

Note: Install `picocom` if not already installed `$ sudo apt-get install picocom`.

5. Power up the Raspberry Pi 3 and the system shall start booting which you will see on the UART (not [HDMI](#)).
6. When you have a shell, then it’s simply just to follow the “[Step 9 - Run xtest](#)” instructions.

NFS boot

Booting via NFS is quite useful for several reasons, but the obvious reason when working with Raspberry Pi is that you don't have to move the SD-card back and forth between the host machine and the Raspberry Pi 3 itself when working with **Normal World** files, like Linux kernel and user space programs. Here we will describe how to setup NFS server, so the rootfs can be mounted via NFS.

Warning: This guide doesn't focus on any desktop security, so eventually you would need to harden your setup.

In the description below we will use the following terminology, IP addresses and paths. The reader of this guide is supposed to update this to match his own environment.

192.168.1.100 <--- This is your desktop computer (NFS server)
192.168.1.200 <--- This is the Raspberry Pi
/srv/nfs/rpi <--- Location for the NFS share

Configure NFS

Start by installing the NFS server

```
$ sudo apt-get install nfs-kernel-server
```

Then edit the exports file,

```
$ sudo vim /etc/exports
```

In this file you shall tell where your files/folder are and the IP's allowed to access the files. The way it's written below will make it available to every machine on the same subnet (again, be careful about security here). Let's add this line to the file (it's the only line necessary in the file, but if you have several different filesystems available, then you should of course add them too, one line for each share).

```
/srv/nfs/rpi 192.168.1.0/24(rw,sync,no_root_squash,no_subtree_check)
```

Next create the folder where you are going to put the root filesystem

```
$ sudo mkdir /srv/nfs/rpi
```

After this, restart the NFS kernel server

```
$ service nfs-kernel-server restart
```

Hint: To see that your shares are correctly setup and that the NFS server is running, you can run: `$ showmount --all localhost` and you should get a list of `IP:<path>'s` based on what you have added in your exports file. If you get nothing there, then your NFS server hasn't been setup correctly.

Prepare files to be shared

We are now going to put the root filesystem on the location we prepared in the previous section.

Note: The path to the `rootfs.cpio.gz` refers to `<rp13-project>`, replace this so it matches your setup.

```
$ cd /srv/nfs/rpi
$ sudo gunzip -cd <rp13-project>/out-br/images/rootfs.cpio.gz | sudo cpio -idmv
$ sudo rm -rf /srv/nfs/rpi/boot/*
```

uboot.env configuration

The file `uboot.env` contains boot configurations that tells what binaries to load and at what addresses. When using NFS you need to tell U-Boot where the NFS server is located (IP and path). Since the exact IP and path varies for each user, we must update `uboot.env` accordingly.

There are two ways to update `uboot.env`, one is to update `uboot.env.txt` (in *build*) and the other is to update directly from the U-Boot console. Pick the one that you suits your needs. We will cover each of them separately here.

Change uboot.env.txt

In an editor open: `<rp13-project>/build/rp13/firmware/uboot.env.txt` and change:

- `nfsserverip` to match the IP address of your NFS server.
- `gatewayip` to the IP address of your router.
- `nfspath` to the exported filesystem in your NFS share.

As an example a section of `uboot.env.txt` could look like this:

```
# NFS/TFTP boot configuraton
gatewayip=192.168.1.1
netmask=255.255.255.0
nfsserverip=192.168.1.100
nfspath=/srv/nfs/rpi
```

Next, you need to re-generate `uboot.env`:

```
$ cd <rp13-project>/build
$ make u-boot-env-clean
$ make u-boot-env
```

Finally, you need to copy the updated `<rp13-project>/out/uboot.env` to the **BOOT** partition of your SD-card (mount it as described in *Build instructions* and then just overwrite (cp) the file on the **BOOT** partition of your SD-card).

Update u-boot.env from U-Boot console

Boot up the device until you see U-Boot running and counting down, then hit any key and will see the U-Boot> prompt. You can then update the nfsserverip, gatewayip and nfspath by writing

```
U-Boot> setenv nfsserverip '192.168.1.100'  
U-Boot> setenv gatewayip '192.168.1.1'  
U-Boot> setenv nfspath '/srv/nfs/rpi'
```

If you want those environment variables to persist between boots, then type.

```
U-Boot> saveenv
```

Boot up with NFS

With all preparations above done correctly, you should now be able to boot up the device and kernel, secure side OP-TEE and the entire root filesystem should be loaded from the network shares (NFS). Power up the Raspberry, halt in U-Boot and then type.

```
U-Boot> run nfsboot
```

If everything works, you can simply copy paste files like `xtest`, Trusted Applications and other things that usually resides on the host PC's filesystem, i.e., directly from your build folders to the `/srv/nfs/rpi/...` folders. By doing so you don't have to reboot the device when doing development and testing. Just rebuild and copy is sufficient.

Note: You **cannot** make symlinks in the NFS share to the built files, i.e., you must copy them!

JTAG

To enable JTAG you need to add a line saying `enable_jtag_gpio=1` in `config.txt`. There are two ways you can do this, both requires that you to mount the **BOOT** partition on the SD-card at your computer (see the `make img-help` step under [Build instructions](#)). **After** you have mounted the BOOT partition continue with whichever way is most suitable for you.

Change config.txt directly

With your editor, open `/media/boot/config.txt` and add a line `enable_jtag_gpio=1`, save the file, unmount the BOOT partition and you're good to go after rebooting the device.

Rebuild and untar

1. With your editor, open `<rpi3-project>/build/rpi3/firmware/config.txt` and add a line `enable_jtag_gpio=1`, save the file.
2. `$ cd <rpi3-project>/build && make`
3. `$ cd /media`

4. `$ sudo gunzip -cd <rpi3-project>/out-br/images/rootfs.cpio.gz | sudo cpio -idmv "boot/*"`

Note: You didn't forget to mount the BOOT partition before trying this step?

5. Unmount the BOOT partition and you're good to go after rebooting the device.

JTAG/RPi3 cable

We have created our own cables that consists of a standard 20-pin JTAG connector and a 22-pin connector for the Raspberry Pi 3 itself. Then using a ribbon cable we have connected the cables according to the table below (JTAG pin <-> Raspberry Pi 3 Header pin).

JTAG pin	Signal	GPIO	Mode	RPi3 Header pin
1	3v3	N/A	N/A	1
3	nTRST	GPIO22	ALT4	15
5	TDI	GPIO26	ALT4	37
7	TMS	GPIO27	ALT4	13
9	TCK	GPIO25	ALT4	22
11	RTCK	GPIO23	ALT4	16
13	TDO	GPIO24	ALT4	18
18	GND	N/A	N/A	14
20	GND	N/A	N/A	20

Warning: Be careful and cross check the wiring as incorrect wiring might **damage** your device! Also be careful to connect the cable correctly at both ends (don't flip it and don't put it at the wrong pins in the Raspberry Pi 3 side).

UART/RPi3 cable

In addition to the JTAG connections we have also wired up the RX/TX to be able to use the UART. Note, for this you don't need to do JTAG wirings, i.e., it's perfectly fine to just wire up the UART only. There are many ready made cables for this on the net ([eBay](#)) and cost almost nothing. Get one of those if you **don't** intend to use JTAG.

UART pin	Signal	GPIO	Mode	RPi3 Header pin
Black (GND)	GND	N/A	N/A	6
White (RXD)	TXD	GPIO14	ALT0	8
Green (TXD)	RXD	GPIO15	ALT0	10

Warning: Be careful and cross check the wiring as incorrect wiring might **damage** your device!

OpenOCD

Build OpenOCD

Before building OpenOCD, ensure that you have the `libusb-dev` installed.

```
$ sudo apt-get install libusb-1.0-0-dev
```

We are using the [official OpenOCD](#) release, simply clone that to your computer and then building is like a lot of other software, i.e.,

```
$ git clone http://repo.or.cz/openocd.git
$ cd openocd
$ ./bootstrap
$ ./configure
$ make
```

Note: In recent versions of OpenOCD, the legacy `ft232` support has been deprecated. All these devices now use `libftdi` instead. From OpenOCD release notes: “*GPL-incompatible FTDI D2XX library support dropped (Presto, OpenJTAG and USB-Blaster I are using libftdi only now)*”.

We leave it up to the reader of this guide to decide if he wants to install it properly (`make install`) or if he will just run it from the tree directly. The rest of this guide will just run it from the tree.

OpenOCD RPi3 configuration file

Unfortunately, the necessary [RPi3 OpenOCD config](#) isn't upstreamed yet into the [official OpenOCD](#) repository, so you should use the one stored here `<rpi3-project>/build/rpi3/debugger/pi3.cfg`.

Running OpenOCD

Depending on the JTAG debugger you are using you'll need to find and use the interface file for that particular debugger. We've been using [J-Link debuggers](#) and [Bus Blaster](#) successfully. To start an OpenOCD session using a J-Link device you type:

```
$ cd <openocd>
$ ./src/openocd -f ./tcl/interface/jlink.cfg -f <rpi3-project>/build/rpi3/debugger/pi3.
↪ cfg
```

For Bus Blaster type:

```
$ ./src/openocd -f ./tcl/interface/ftdi/dp_busblaster.cfg \ -f <rpi3_repo_dir>/build/
↪ rpi3/debugger/pi3.cfg
```

To be able to write commands directly to OpenOCD, you simply open up another shell and type:

```
$ nc localhost 4444
```

From there you can set breakpoints, examine memory etc ("`> help`" will give you a list of available commands). Having that said, if you connect to OpenOCD using GDB, then there is not much incentive connecting to OpenOCD directly, since you will be able to do the same in GDB by the `monitor` command.

Use GDB

OpenOCD will by default listen to GDB connections on port 3333. So after starting OpenOCD, make a connection to GDB.

```
# Ensure that you have "gdb" in your $PATH
$ aarch64-linux-gnu-gdb -q
(gdb) target remote localhost:3333
```

To load symbols you just use the `symbol-file <path/to/my.elf>` as usual. For convenience you can create an alias in the `~/.gdbinit` file. For TEE core debugging this works:

```
define jtag_rpi3
    target remote localhost:3333
    symbol-file <rpi3-project>/optee_os/out/arm/core/tee.elf
end
```

So, when running GDB, you simply type: `(gdb) jtag_rpi3` and it will both connect and load the symbols for TEE core. For Linux kernel and other binaries you would do the same.

Debug session example

After making an initial Raspberry Pi 3 build for OP-TEE where you've enabled JTAG, installed and built OpenOCD, connected the JTAG cable, then you're ready for debugging OP-TEE using JTAG on Raspberry 3. Boot up the Raspberry Pi 3 until you are in Linux and ready to run `xtest`. Start a new shell (on the host machine) where you run OpenOCD:

```
$ cd <openocd>
$ ./src/openocd -f ./tcl/interface/jlink.cfg -f <rpi3-project>/build/rpi3/debugger/pi3.
↪ cfg
```

Start another shell, where you run GDB

```
$ <rpi3-project>/toolchains/aarch64/bin/aarch64-linux-gnu-gdb -q
(gdb) target remote localhost:3333
(gdb) symbol-file <rpi3-project>/optee_os/out/arm/core/tee.elf
```

Next, try to set a breakpoint for the function `hmac_init`, here use **hardware** breakpoints (i.e., `hb`)!

```
(gdb) hb hmac_init
Hardware assisted breakpoint 2 at 0x1012a178: file core/lib/libtomcrypt/src/mac/hmac/
↪ hmac_init.c, line 65.
(gdb) c
Continuing.
```

In the UART console (RPI3/Linux), run `xtest`.

```
# xtest
```

And shortly thereafter you will see GDB stops on your breakpoint and from there you can debug using normal GDB commands.

3.2.12 STM32MP1

The instructions here will tell how to run OP-TEE on one of the supported STM32MP1 boards.

Supported boards

Board Name	Manufacturer	Boot media	Hardware Description
STM32MP135F-DK	STMicroelectronics	SDcard	Wiki STM32MP135x-DK
STM32MP157A-DK1	STMicroelectronics	SDcard	Wiki STM32MP157x-DKx
STM32MP157D-DK1			
STM32MP157C-DK2	STMicroelectronics	SDcard	Wiki STM32MP157x-DKx
STM32MP157F-DK2			
STM32MP157C-EV1	STMicroelectronics	SDCard (1)	Wiki STM32MP157x-EV1
STM32MP157F-EV1			

(1): STM32MP157x-EV1 boards also integrate an eMMC device, a NOR flash and a Nand flash the system can boot on. OP-TEE distribution however only supports booting from the SDcard slot.

Build instructions

Follow the instructions at “[Get and build the solution](#)”.

Configuration switch PLATFORM can be used to specify the target device as listed in table below:

Board Name	Build configuration directive
STM32MP135F-DK	PLATFORM=stm32mp1-135F_DK
STM32MP157A-DK1 STM32MP157D-DK1	PLATFORM=stm32mp1-157A_DK1
STM32MP157C-DK2 STM32MP157F-DK2	PLATFORM=stm32mp1-157C_DK2
STM32MP157C-EV1 STM32MP157F-EV1	PLATFORM=stm32mp1-157C_EV1

When the build completes, generated image file sdcard.img can be found in the generated binary images directory `../out/bin/` from build root path. The image is a GPT multipartition image you can raw copy to the target SDcard using a tool like `dd`.

A usual short fetch/build/load shell sequence is like the one below:

```
$ repo init -u https://github.com/OP-TEE/manifest.git -m stm32mp1.xml
$ repo sync
$ cd build
$ make toolchains
$ make PLATFORM=stm32mp1-157C_DK2 all
$ dd if=../out/bin/sdcard.img of=/dev/sdX conv=fdatasync status=progress
$ sgdisk -e /dev/sdX
```

Command `sgdisk -e` fixes the GPT backup data which location depends on storage device effective size.

3.2.13 Texas Instruments SoCs

The instructions here will tell how to run OP-TEE on Texas Instruments devices. Secure TI devices require a boot image that is authenticated by ROM code to function. Without this, even JTAG remains locked. In order to create a valid boot image for a secure device from TI, the initial public software image must be signed and combined with various headers, certificates, and other binary images.

Information on the details on the complete boot image format can be obtained from Texas Instruments. The tools used to generate boot images for secure devices are part of a secure development package (SECDEV) that can be downloaded from:

<http://www.ti.com/mysecuresoftware> (login required)

The secure development package is access controlled due to NDA and export control restrictions. Access must be requested and granted by TI before the package is viewable and downloadable. Contact TI, either online or by way of a local TI representative, to request access.

Regular build

Start out by following the *Get and build the solution* as described in *build*. Stop before the section on flashing the device, this is currently not supported automatically.

Booting the device

SD Card boot

Create two partitions on an SD card, boot of type FAT16 and rootfs of type EXT4. To prevent accidental data loss we do not attempt this automatically (the RPI3 *Build instructions* use a similar SD card layout, you can refer to that page for details).

Extract the generated rootfs to the rootfs partition

```
$ cd <SD card rootfs partition>
$ gunzip -cd <repo directory>/gen_rootfs/filesystem.cpio.gz | sudo cpio -idm
```

Add the bootloader to the boot partition

```
$ cd <SD card boot partition>
$ cp <repo directory>/u-boot/u-boot-spl_HS_MLO MLO
$ cp <repo directory>/u-boot/u-boot_HS.img u-boot.img
```

3.2.14 Zynq MPSoC

Instructions below show how to run OP-TEE on Zynq MPSoC based boards.

Supported boards

Board Name	Manufacturer	Hardware Description
ZCU102	Xilinx/AMD	ZCU102 Website
ZCU104	Xilinx/AMD	ZCU104 Website
ZCU106	Xilinx/AMD	ZCU106 Website
Ultra96	Avnet	Ultra96 Website

Boot Firmware

Xilinx Zynq MPSoC device requires two firmware images, one to configure the device (First Stage Bootloader) and one for runtime platform management (PMU Firmware). The scope of OP-TEE build Makefile does not cover building these two firmware images therefore pre built binaries are required to generate a valid boot image. The pre built images can be found in the following [Xilinx wiki](#) page.

Note: For Ultra96 board, the firmware binaries can be found in the Avnet website.

Build instructions

Follow the instructions at “[Get and build the solution](#)” page.

Configuration switch PLATFORM can be used to specify the target device as listed in table below:

Board Name	Build configuration directive
ZCU102	PLATFORM=zynqmp-zcu102
ZCU104	PLATFORM=zynqmp-zcu104
ZCU106	PLATFORM=zynqmp-zcu106
Ultra96	PLATFORM=zynqmp-ultra96

An example of fetch and build commands is:

```
$ repo init -u https://github.com/OP-TEE/manifest.git -m zynqmp.xml
$ repo sync
$ cd build
$ make toolchains
$ make PLATFORM=zynqmp-zcu102 all
```

After completion of the building process, two new files will be generated within the zynqmp/ folder, BOOT.bin and <platform-name>.ub. The first one is the boot image composed of the FSBL, PMU Firmware, ARM Trusted Firmware, OP-TEE and U-Boot. The second one is a FIT image containing the Linux kernel, the device-tree blob and the initramfs root file system.

Note: If the firmware image is not provided to the build script the boot image will not be generated.

Petalinux build instructions

OP-TEE build can be additionally integrated within Xilinx Petalinux tool for Embedded Linux development. As Petalinux is built on top of Yocto, the integration is performed through adding some existing recipes and few customizations. Use the previous build [Makefile](#) based on Petalinux 2020.2 release as reference.

Bootting the device

SD Card boot

Place both generated images in a single partition within the SD card. Boot the board in SD boot mode and stop the U-Boot autoboot process once the prompt is displayed in the serial port.

Use the bellow commands to load the FIT image to RAM and boot.

```
ZynqMP> fatload mmc 0 0x30000000 zynqmp-zcu102.ub
27803872 bytes read in 1827 ms (14.5 MiB/s)
ZynqMP> bootm 0x30000000
```

3.3 AOSP

This page contains information that tells how to get OP-TEE up and running on HiKey devices (see [HiKey 620](#), [HiKey 960](#)) together with AOSP.

Warning: The build used to be based on the latest OP-TEE release and used to be updated every quarter together with the regular OP-TEE releases. However, the AOSP build hasn't been updated since July 2021 and is **no longer maintained**.

Note: We **only** use and support this static/stable configuration. If you try using it with the latest available AOSP, there is a risk that both OP-TEE and other parts are not working as expected.

3.3.1 Prerequisites

- You should already be able to build AOSP for Hikey according to the [official instructions](#). Note that the official build is **NOT** part of the OP-TEE build. It is a separate and non-related build used only to verify and make sure that your system has everything needed to build AOSP without any issues.
- Distro should have necessary packages installed, and the repo tool should be installed. Note that AOSP is built with Java. Also make sure that the `mtools` package is installed, which is needed to make the hikey boot image.
- In addition, you will need the pre-requisites necessary to build optee-os.

After following the AOSP setup instructions, the following additional packages from main [Prerequisites](#) page are needed. Please install them.

3.3.2 Build instructions

```
$ git clone https://github.com/linaro-swg/optee_android_manifest [-b <release_tag>]
# release tags come in the form of X.Y.Z, e.g. 3.8.0
$ cd optee_android_manifest
```

HiKey620 - LeMaker 8GB

```
$ ./sync-p.sh
$ ./build-p.sh
```

HiKey620 - CircuitCo 4GB

```
$ ./sync-p.sh
$ ./build-p.sh -4g
```

HiKey960

```
$ ./sync-p-hikey960.sh
$ ./build-p-hikey960.sh
```

These steps **MUST** finish with **no errors**. For `sync*.sh` scripts, that means there must be no errors prior to the Sync done! console output. For `build*.sh` scripts, that means there must be a `#### build completed successfully (MM:SS (mm:ss)) ####` console output! If there are errors, then there is no point in trying to flash the device.

Warning:

- `--force-sync` is used which means you might **lose your work** so save often, save frequent, and save accordingly, especially before running `sync-p.sh` again!
- **Attention!** Do **NOT** use `git clean` with `-x` or `-X` or `-e` option in `optee_android_manifest/`, else risk **losing all files** in the directory!!!

Hint: You can add the `-squashfs` option to `build.sh` option to make `system.img` size smaller, but this will make `/system` read-only, so you won't be able to push files to it.

Currently, only version P is supported. Other existing files are for internal development purposes **ONLY** and **NOT SUPPORTED!**

3.3.3 Flashing the image

The instructions for flashing the image can be found in detail under `device/linaro/hikey/installer/hikey{960}/README` in the tree.

1. Set jumpers/switches 1-2 and 3-4, and unset 5-6.
2. Reset the board. After that, invoke:

HiKey620

```
$ cp -a out/target/product/hikey/*.img device/linaro/hikey/installer/hikey/
$ sudo ./device/linaro/hikey/installer/hikey/flash-all.sh /dev/ttyUSBn
```

HiKey960

```
$ cp -a out/target/product/hikey960/*.img device/linaro/hikey/installer/hikey960/
$ sudo ./device/linaro/hikey/installer/hikey960/flash-all.sh /dev/ttyUSBn
```

where the `/dev/ttyUSBn` device is the one that appears after rebooting with the 3-4 jumper set. Note that the device only remains in this recovery mode for about 90 seconds. If you take too long to run the flash commands, it will need to be reset again. After flashing, unset the 3-4 jumper again to boot normally.

3.3.4 Partial flashing

The last handful of lines in the `flash-all.sh` script flash various images. After modifying and rebuilding Android, it is only necessary to flash *boot*, *system*, *cache*, *vendor* and *userdata*. If you aren't modifying the kernel, *boot* is not necessary, either.

3.3.5 Experimental prebuilts

Available at <http://snapshots.linaro.org/android> under `android-hikey*` directories. Note that these images do not always work and are **NOT SUPPORTED** as well!

3.3.6 Running xtest

Do NOT try to run `tee-suppllicant` as it has already been started automatically as a service! Once booted to the command prompt, `xtest` can be run immediately from the console or an `adb` shell. For more details about running OP-TEE, please see *Run xtest* at *optee_test*.

3.3.7 Running VTS Gtest unit for Gatekeeper and Keymaster (Optional)

On the device after going into the command prompt, run:

```
$ su
$ ./data/nativetest64/VtsHalGatekeeperV1_0TargetTest/VtsHalGatekeeperV1_0TargetTest
$ ./data/nativetest64/VtsHalKeymasterV3_0TargetTest/VtsHalKeymasterV3_0TargetTest
```

Note: These tests need to be run as root.

3.3.8 Enable adb over USB

Boot the device. On serial console:

```
$ su setprop sys.usb.configfs 1
$ stop adbd
$ start adbd
```

3.3.9 Known issues

- If you don't have a monitor or hdmi emulator (dummy plug) connected to the board, you'll see constant errors scrolling on the console. As a workaround, move `android.hardware.graphics.composer@2.1-service.rc` out of `/vendor/etc/init`. Move it back in when working with a monitor again.
- Adb over USB currently doesn't work on HiKey960. As a workaround, use adb over tcpip. See https://bugs.96boards.org/show_bug.cgi?id=502 for details on how to connect. There are still some limitations however. E.g. running `adb shell` or a second adb instance will break the current adb tcpip connection. This might be due to unstable WiFi (there are periodic error messages like `wlcore: WARNING corrupted packet in RX: status: 0x1 len: 76`) or just incompleteness of the generic HiKey960 builds under P.

3.4 Linux kernel TEE framework

3.5 OP-TEE gits

These are the gits considered as the main OP-TEE gits which together makes up the entire TEE solution.

3.5.1 build

Why this particular git? As it turns out it's totally possible to put together everything on your own. You can build all the individual components, os, client, xtest, Linux kernel, TF-A, TianoCore, QEMU, [Buildroot](#) etc and put all the binaries at correct locations and write your own command lines, Makefiles, shell-scripts etc that will work nicely on the devices you are interested in. If you know how to do that, fine, please go ahead. But for newcomers it's way too much behind the scenes to be able to setup a working environment. Also, if you want to run something in an automated way, then you need something else wrapping it up for you.

With this particular git **built.git** our goal is to make it easy for newcomers to get started with OP-TEE using the devices we've listed in this document.

git location

<https://github.com/OP-TEE/build>

Why repo?

We discussed alternatives, initially we started out with having a simple shell-script, that worked to start with, but after getting more gits in use and support for more devices it started to be difficult to maintain. In the end we ended up choosing between [repo](#) from the Google AOSP project and [git submodules](#). No matter which you choose, there will always be some person arguing that one is better than the other. For us we decided to use repo. Not directly for the features itself from repo, but for the ability to simply work with different manifests containing both stable and non-stable release. Using some tips and tricks you can also speed up setup time significantly. For day to day work with commits, branches etc we tend to use git commands directly.

Root filesystem

The rootfs in the builds that we cover here are as small as possible and is based on a stripped down [Buildroot](#) configuration adding just enough in the rootfs such that one can:

- Boot OP-TEE.
- Run xtest with no regressions.
- Easily add additional developer tools like, strace, valgrind etc.

Note: As a consequence of enabling “just enough”, it is likely that **non-UART** based environments won’t work out of the box. I.e., if you try to boot up an environment using HDMI and connect keyboards and other devices it is likely that things will not work. To make them work, you probably need to rebuild Linux kernel with correct drivers/frameworks enabled and in addition to that enable binaries/daemons in Buildroot that might be necessary (user space tools and drivers).

How do I build using AOSP / OpenEmbedded?

For guides how to build AOSP, please refer to our [AOSP](#) page. For OpenEmbedded we have no guide ready.

Platforms supported by build.git

Below is a table showing the platforms supported by build.git. OP-TEE as such supports many more platforms. To find out how to run OP-TEE on those, please reach out to the maintainer of that platform directly if you have build related questions etc. Please see the [MAINTAINERS](#) file for contact information.

Platform	Composite flag	Publicly available?
AMD/Xilinx Versal ACAP VCK190	PLATFORM=versal	Yes
ARM Juno Board	PLATFORM=vexpress-juno	Yes
ARM Foundation FVP	PLATFORM=vexpress-fvp	Yes
DeveloperBox	PLATFORM=synquacer	Yes
HiKey Kirin 620	PLATFORM=hikey	Yes
HiKey 960	PLATFORM=hikey-hikey960	Yes
MediaTek MT8173 EVB Board (deprecated)	PLATFORM=mediatek-mt8173	No
Poplar	PLATFORM=poplar	Yes
QEMU	PLATFORM=vexpress-qemu_virt	Yes
QEMUv8	PLATFORM=vexpress-qemu_armv8a	Yes
Raspberry Pi 3	PLATFORM=rpi3	Yes
ROCK Pi 4	PLATFORM=rockchip-rk3399	Yes
STM32MP157A-DK1	PLATFORM=stm32mp1-157A_DK1	Yes
STM32MP157C-DK2	PLATFORM=stm32mp1-157C_DK2	Yes
STM32MP157C-EV1	PLATFORM=stm32mp1-157C_EV1	Yes
Texas Instruments DRA7xx	PLATFORM=ti-dra7xx	Yes
Texas Instruments AM57xx	PLATFORM=ti-am57xx	Yes
Texas Instruments AM43xx	PLATFORM=ti-am43xx	Yes

Manifests

Current version

Here is a list of manifests for the devices currently supported in `build.git`. With these you will get a setup containing the all necessary software components to run OP-TEE on the chosen device. Beware that this will run latest available on OP-TEE gits meaning that if you re-sync then you will most likely get new commits. If you need a stable/tagged version with non-moving gits, then please refer to the next section instead.

Target	Manifest xml	Device documentation
AM43xx	am43xx.xml	<i>Texas Instruments SoCs</i>
AM57xx	am57xx.xml	<i>Texas Instruments SoCs</i>
DeveloperBox	synquacer.xml	<i>DeveloperBox</i>
ARM Juno board	juno.xml	<i>Juno</i>
DRA7xx	dra7xx.xml	<i>Texas Instruments SoCs</i>
FVP	fvp.xml	<i>FVP</i>
HiKey 960	hikey960.xml	<i>HiKey 960</i>
HiKey	hikey.xml	<i>HiKey 620</i>
Poplar Debian	poplar.xml	
QEMU	default.xml	<i>QEMU v7</i>
QEMUv8	qemu_v8.xml	<i>QEMU v8</i>
Raspberry Pi 3	rpi3.xml	<i>Raspberry Pi 3</i>
ROCK Pi 4	rockpi4.xml	<i>ROCK Pi 4</i>
STM32MP1	stm32mp1.xml	<i>STM32MP1</i>
VCK190	versal.xml	<i>AMD-Xilinx Versal ACAP VCK190</i>

Stable releases

Starting from OP-TEE v3.1 you can check out stable releases by using the same manifests as for current version above, but with the difference that **you also need to specify a branch** where the name corresponds to the release version. I.e., when we are doing releases we are creating a branch with a name corresponding to the release version. So, let's for example say that you want to checkout a stable OP-TEE v3.12 for Raspberry Pi 3, then you do like this instead of what is mentioned further down in section “*Step 3 - Get the source code*” (note the `-b 3.12.0`):

Hint: If there is no strong need for an older version, then we always recommend to use the most recent release. I.e., in the example here we do say `3.12.0`, but there may very well be more recent version when you are reading this. To find out, please have a look at the “*Release dates*” page.

```
...
$ repo init -u https://github.com/OP-TEE/manifest.git -m rpi3.xml -b 3.12.0
...
```

Stable releases prior to OP-TEE v3.1 (v1.0.0 to v3.0.0)

Before OP-TEE v3.1 we used to have separate xml-manifest files for the stable builds. If you for some reason need an older stable release, please refer to “*Build stable releases v1.0.0 to v3.0.0*”.

Stable releases prior to OP-TEE v3.9 (3.1.0 to 3.8.0)

Due to a change in the Google repo tool, you might get an error when cloning OP-TEE repositories before version 3.9.0. In this case please refer to “*Build stable releases 3.1.0 to 3.8.0*”.

Get and build the solution

Below we will describe the general way of how to get the source, build the solution and how to run xtest on the device. For device specific instructions, please see the links in the table in the “*Current version*” section.

Step 1 - Prerequisites

Install prerequisites according to the *Prerequisites* page.

Step 2 - Install Android repo

Note that here you don’t install a huge SDK, it’s simply a Python script that you download and put in your \$PATH, that’s it. Exactly how to “install” repo, can be found at the Google [repo](#) pages, so follow those instructions before continuing.

Step 3 - Get the source code

Choose the manifest corresponding to the platform you intend to use (see the table in section “*Current version*”). For example, if you intend to use Raspberry Pi3, then at line 3 below, \${TARGET}.xml shall be rpi3.xml. The <optee-project> is whatever location where you want to store the entire OP-TEE developer setup.

```

1 $ mkdir -p <optee-project>
2 $ cd <optee-project>
3 $ repo init -u https://github.com/OP-TEE/manifest.git -m ${TARGET}.xml [-b ${BRANCH}]
4 $ repo sync -j4 --no-clone-bundle

```

Hint: By referencing an existing and locally saved repo forest you can save lots of time. We are talking about doing repo sync in 30 seconds instead of 15-30 minutes (see the *Tips and Tricks* section for more details).

Step 4 - Get the toolchains

In OP-TEE we're using different toolchains for different targets (depends on ARMv7-A ARMv8-A 64/32bit solutions). In any case start by downloading the toolchains by:

```
$ cd <optee-project>/build
$ make -j2 toolchains
```

Step 5 - Build the solution

We've configured our repo manifests, so that repo will always automatically symlink the Makefile to the correct device specific makefile, that means that you simply start the build by running (still in <optee-project>/build)

```
$ make -j `nproc`
```

This step will also take some time, but you can speed up subsequent builds by enabling *ccache* (again see *Tips and Tricks*).

Hint: If you're having build issues, then you can pipe the entire build log to a file, which makes it easier to search for the issue using a regular editor. In that case also avoid the `-j` flag so it's easier to see in what order things are happening. To create a `build.log` file do: `$ make 2>&1 | tee build.log`

Step 6 - Flash the device

On **non-emulated** solutions (this means that you shouldn't do this step when you are running QEMU-v7/v8 and FVP), you will need to flash the software in some way. We've tried to "hide" that under the following make target:

```
$ make flash
```

But, since some devices are trickier to flash than others, please see the *Device specific information*. See this just as a general instruction.

Step 7 - Boot up the device

This is device specific (see *Device specific information*).

Step 8 - Load tee-supplciant

On **most** solutions tee-supplciant is already running (check by running `$ ps aux | grep tee-supplciant`) on others not. If it's **not** running, then start it by running:

```
$ tee-supplciant -d
```

Note: If you've built using our manifest you should not need to modprobe any OP-TEE/TEE kernel driver since it's built into the kernel in all our setups.

Step 9 - Run xtest

The entire xtest test suite has been deployed when you we're making the builds in previous steps, i.e, in general there is no need to copy any binaries manually. Everything has been put into the *Root filesystem* automatically. So, to run xtest, you simply type:

```
$ xtest
```

If there are no regressions / issues found, xtest should end with something like this:

```
...
+-----
23476 subtests of which 0 failed
67 test cases of which 0 failed
0 test case was skipped
TEE test application done!
```

Hint: For other ways to run xtest, please refer to the “*Run xtest*” page at *optee_test*.

Tips and Tricks

Reference existing project to speed up repo sync

Doing a `repo init`, `repo sync` from scratch can take a fair amount of time. The main reason for that is simply because of the size of some of the gits we are using, like for the Linux kernel and EDK2. With `repo` you can reference an existing forest and by doing so you can speed up `repo sync` to taking 30 seconds instead of 15-30 minutes. The way to do this are as follows.

1. Start by setup a clean forest that you will not touch, in this example, let us call that `optee-ref` and put that under for `$HOME/devel/optee-ref`. This step will take somewhere between 15- to 45 minutes, depending on your connection speed to internet.
2. Then setup a cronjob (`crontab -e`) that does a `repo sync` in this folder particular folder once a night (that is more than enough).
3. Now you should setup your actual tree which you are going to use as your working tree. The way to do this is almost the same as stated in the instructions above (see the “*Step 3 - Get the source code*” section) , the only difference is that you **also** reference the other local forest when running `repo init`, like this

```
$ repo init -u https://github.com/OP-TEE/manifest.git --partial-clone --reference
↪ $HOME/devel/optee-ref
```

4. The rest is the same above, but now it will only take less than a minute to clone a forest.

Normally ‘1’ and ‘2’ above is something you will only do once. Also if you ignore step ‘2’, then you will **still** get the latest from official git trees, since `repo` will also check for updates that aren’t at the local reference.

Use ccache

`ccache` is a tool that caches build object-files etc locally on the disc and can speed up build time significantly in subsequent builds. On Debian-based systems (Ubuntu, Mint etc) you simply install it by running:

```
$ sudo apt-get install ccache
```

The makefiles in `build.git` are configured to automatically find and use `ccache` if `ccache` is installed on your system, so other than having it installed you don't have to think about anything.

3.5.2 Build stable releases v1.0.0 to v3.0.0

Before OP-TEE v3.1.0 we used to have separate `xml-manifest` files for the stable builds. If you for some reason need such an older stable release, then you can use the `xyz_stable.xml` file corresponding to your device. The way to `init repo` is almost the same as described above, the major difference is the name of manifest being referenced (`-m xyz_stable.xml`) and that we are referring to a tag instead of a branch (`-b refs/tags/MAJOR.MINOR.PATCH`). So as an example, if you need to setup the 2.1.0 stable release for HiKey, then you would do like this instead of what is mentioned further down in section “[Step 3 - Get the source code](#)”.

```
...
repo init -u https://github.com/OP-TEE/manifest.git -m hikey_stable.xml -b refs/tags/2.1.
↪ 0
...
```

Here is a list of targets and the names of the stable manifests files which were supported by older releases:

Target	Stable manifest xml
AM43xx	am43xx_stable.xml
AM57xx	am57xx_stable.xml
ARM Juno board	juno_stable.xml
DRA7xx	dra7xx_stable.xml
FVP	fvp_stable.xml
HiKey 960	hikey960_stable.xml
HiKey Debian	hikey_debian_stable.xml
HiKey	hikey_stable.xml
MTK8173	mt8173-evb_stable.xml
QEMU	default_stable.xml
QEMUv8	qemu_v8_stable.xml
Raspberry Pi 3	rpi3_stable.xml

3.5.3 Build stable releases 3.1.0 to 3.8.0

If you have a recent enough version of the Google `repo` tool (`>= 2.0.0`) and follow the normal build procedure at “[Get and build the solution](#)”, you will likely get an error during `repo init` with the following OP-TEE versions and platforms:

- 3.1.0 to 3.5.0: `qemu_v8.xml`, `rpi3.xml`
- 3.6.0 to 3.8.0: `default.xml` (QEMU), `qemu_v8.xml`, `rpi3.xml`

The typical error message is:

```
$ repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml -b 3.3.0
[...]
fatal: manifest 'qemu_v8.xml' not available
fatal: <linkfile> invalid "src": ../toolchains/aarch64/bin/aarch64-linux-gnu-gdb: bad_
↪ component: ..
```

The workaround is to checkout repo version 1.13.9 manually:

```
$ repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml -b 3.3.0
# Above error occurs, ignore it
$ (cd .repo/repo; git checkout v1.13.9)
$ repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml -b 3.3.0
# Should not error out. Then proceed with 'repo sync' and build.
```

3.5.4 manifest

This page contains a couple of guidelines and rules that we want to try to follow when it comes to managing the manifests.

git location

<https://github.com/OP-TEE/manifest>

Remotes

Since most of our projects can be found on GitHub, we are using that as the main remote. If you need to include other remotes for some reason, then that is OK, but please double check of there is any **maintained** (and preferably official) mirror for the project at GitHub before adding a new remote.

Sections

To have some kind of structure of the files, we have split them up in three sections, one for pure OP-TEE gits, one for OP-TEE supporting gits found at [linaro-swg](#) and then a third, misc section where everything else can be found. I.e., a template looks like this (this also includes the default remote for clarity):

```
<?xml version="1.0" encoding="UTF-8"?>
<manifest>
  <remote name="github" fetch="https://github.com" />

  <default remote="github" revision="master" />

  <!-- OP-TEE gits -->
  <!-- linaro-swg gits -->
  <!-- Misc gits -->
</manifest>
```

Project XML elements

All `<project ... >` lines should be on the format as shown below with the attributes in this order. The reason for this is to have it uniformly done across all manifests and that it will make it easier when comparing various versions of manifests with diff tools. All three attributes are **mandatory**. The only exception is `revision` which does not have to be stated if it is `master` that we are tracking.

```
<project path="name_and_path_on_disk" name="upstream_name.git" revision="git_revsn" />
```

Alphabetic order

Within each of the three sections, all `<project ... >` lines **shall** be sorted in alphabetic order (this is again for making it easier to diff manifests). The only exception here is `build.git` which uses the `linkfile` element. Having that at the end makes it look cleaner.

Additional XML attributes

If you are using another remote than the default, then that should come **after** the `revision` attribute (this is true for all attributes other than the `path`, `name` and `revision`).

Alignment of XML attributes

The three mandatory XML attributes `path`, `name` and `revision` should be column aligned. Alignment of additional XML attributes are optional.

When to use `clone-depth="1"`?

With `clone-depth="1"` you are telling `repo` and `git` that you only want a certain commit and not the entire git log history. You can only use this under two conditions and that is when `revision` is either a branch or a tag. Pure SHA-1's does not work and will even raise `repo` and `git` sync errors in some cases. So, the rules are, if you use either `revision="refs/tags/my_tag"` or `revision="refs/heads/my_branch"`, then you shall add `clone-depth="1"` right after the `revision` attribute.

Spaces or tabs?

Only use spaces!

Example

Here is an example showing the basis for an OP-TEE manifest. The names are fictive etc, but it describes everything said above.

```
<?xml version="1.0" encoding="UTF-8"?>
<manifest>
  <remote name="github" fetch="https://github.com" />
  <remote name="other" fetch="https://someotherlocation.com" />

  <default remote="github" revision="master" />
```

(continues on next page)

(continued from previous page)

```

<!-- OP-TEE gits -->
<project path="optee_abc" name="OP-TEE/optee_abc.git" />
<project path="optee_def" name="OP-TEE/optee_def.git" />

<!-- linaro-swg gits -->
<project path="lswg_abc" name="linaro-swg/lswg-abc.git" revision=
↪ "aaaabbbbcccc93e64c2fdd6ae8b0be14a8c45719" />
<project path="lswg_def" name="linaro-swg/lswg-def.git" revision=
↪ "ddddeeeeffff83e64c2fdd6ae8b0be14a8c45719" />

<!-- Misc gits -->
<project path="my_other" name="my_other.git" revision="refs/tags/
↪ 2017.11" clone-depth="1" remote="other" />
</manifest>

```

3.5.5 optee_client

optee_client git contains the source code for the TEE client library in Linux. This component provides the TEE Client API as defined by the GlobalPlatform TEE standard. It is distributed under the BSD 2-clause open source license.

In this git there are two main targets/binaries to build. There is `libtee.so`, which is the library that contains that API for communication with the Trusted OS. Then there is `tee-supplc` which is a daemon serving the Trusted OS in secure world with miscellaneous features, such as file system access.

git location

https://github.com/OP-TEE/optee_client

License

The software is provided under the [BSD 2-Clause](#) license.

Build instructions

You can build the code in this git only or build it as part of the entire system, i.e. as a part of a full OP-TEE developer setup. For the latter, please refer to instructions at the [build](#) page. For standalone builds we currently support building with both CMake as well as with regular GNU Makefiles.

Configure the toolchain

First step is to download and configure a toolchain, see the [Toolchains](#) page for instructions.

Clone optee_client

```
$ git clone https://github.com/OP-TEE/optee_client
$ cd optee_client
```

Build using CMake

```
$ mkdir build
$ cd build
$ cmake -DCMAKE_C_COMPILER=arm-linux-gnueabi-gcc ..
$ make
```

Note: This example uses the 32-bit toolchain (arm-linux-gnueabi-gcc), the same works using the 64-bit toolchain (aarch64-linux-gnu-).

After this step the compiled binaries can be found in sub-folders of build. If you have a need or preference to install the binaries at some specific location, then on the cmake line above add -DCMAKE_INSTALL_PREFIX=<my-install-path> as an additional argument. With that you can then run make install and the binaries etc will be copied to the location that you gave as an argument. In this example /tmp/optee_client.

```
$ cmake -DCMAKE_C_COMPILER=arm-linux-gnueabi-gcc -DCMAKE_INSTALL_PREFIX=/tmp/optee_
↪client ..
$ make
$ make install
```

Build using GNU Make

The Makefile is configured to use arm-linux-gnueabi-gcc by default.

```
$ make
```

Note: For a 64-bit builds (or any other toolchain) you will need to use CROSS_COMPILE.

```
$ make CROSS_COMPILE=aarch64-linux-gnu-
```

After this step the compiled binaries can be found in the sub-folder out.

Compiler flags

To be able to see all commands when building you could build using following flags:

GNU Make

```
$ make V=1
```

CMake

```
$ make VERBOSE=1
```

Coding standards

See *Coding standards*.

3.5.6 optee_docs

This is the Git where all official OP-TEE documentation resides and this is what you are reading right now. Here we will give instructions on how to write and build the documentation as well as give some guidelines on what to do and not to do. Note that the documentation is written for [Sphinx](#). So, even though GitHub for example renders *.rst files somewhat OK, that is still not the preferred way to read and view the documentation. Instead head over to <https://optee.readthedocs.io> where the final output is stored and nicely rendered using Sphinx.

git location

https://github.com/OP-TEE/optee_docs

Install Sphinx

Before doing anything else, first install Sphinx and the dependencies.

```
$ sudo apt install graphviz python3-sphinx python3-sphinx-rtd-theme
```

Build optee_docs

```
$ git clone https://github.com/OP-TEE/optee_docs
$ cd optee_docs
$ make html
```

After this step all documentation should have been built and you can open <optee_docs>/_build/html/index.html in your browser to see the result and browse the documentation.

Hint: By using a Linux tool called `entr`. You can automatically rebuild the pages you are working with. First get the package `$ sudo apt install entr`, then:

```
$ cd <optee_docs>
$ find . -name "*.rst" | entr -c make html
```

With this, `entr` will automatically rebuild the documentation everytime you make change and save a file. Which means you only have to save the file in your editor and refresh the browser page to see the changes locally.

General guidelines

Linking

Internal links

Internally within a Sphinx project you can link various pages by referring to a keyword specified right above a section, chapter or subsection. This means that you don't have to make hardlinks to certain files. Instead Sphinx will just figure out where it is for you. Example I have to files, file *compiler.rst* and *toolchain.rst*. They could look like this:

compiler.rst example

```
1 #####
2 Compiler
3 #####
4 Bla bla bla
5
6 .. _compiler_flags:
7
8 Compiler Flags
9 *****
```

toolchain.rst example

```
1 #####
2 Toolchain
3 #####
4 Bla bla bla to see find out more about various flags, please refer
5 :ref:`compiler_flags`.
```

What we can see in the example, is that on line 5 in *toolchain.rst* we refer to the keyword in *compiler.rst* by using `:ref:`compiler_flags``. This would render a direct link to that section in *compiler.rst*.

General recommendation for OP-TEE internal linking

- Things about general things doesn't have to be prefixed with the "document name".
- Things that are specific should be prefixed with the "document name".

Example: the "Contact" section is generic so it's there is no need for prefix. But for example HiKey 620 build instructions are specific to HiKey 620, so there we shall prefix keyword for internal linking.

rst files

The rst files should have descriptive names, but even more important is where you decide to put the files. Even though it's not a problem to move files around, we have to remember that we tend to quite often give links to documentation from at GitHub, emails etc. If we move files, there is a high likelihood that they will become dead links in the future (404's). So think twice before adding a new file or moving an existing file.

Sections, chapters

We have adopted the Sphinx recommended way of using sections, chapters, subsections etc, those are:

- # with overline, for parts
- * with overline, for chapters
- =, for sections
- -, for subsections
- ^, for subsubsections
- “, for paragraphs

3.5.7 optee_examples

This document describes the sample applications that are included in the OP-TEE, that aim to showcase specific functionality and use cases.

For sake of simplicity, all OP-TEE example test application are prefixed with `optee_example_`. All of them works as standalone host and Trusted Application and can be found in separate directories.

git location

https://github.com/linaro-swg/optee_examples

License

The software is provided under the [BSD 2-Clause](#) license.

Build instructions

You can build the code in this git only or build it as part of the entire system, i.e. as a part of a full OP-TEE developer setup. For the latter, please refer to instructions at the [build](#) page. For standalone builds we currently support building with both CMake as well as with regular GNU Makefiles. However, since the both the host and the Trusted Applications have dependencies to files in `optee_client` (libteec.so and headers) as well as `optee_os` (TA-devkit), one **must first** build those and then refer to various files. Below we will show to to build the **hello_world** example for Armv7-A using regular GNU Make.

Configure the toolchain

First step is to download and configure a toolchain, see the [Toolchains](#) page for instructions.

Build the dependencies

Then you must build *optee_os* as well as *optee_client* first. Build instructions for them can be found on their respective pages.

Clone optee_examples

```
$ git clone https://github.com/linaro-swg/optee_examples.git
```

Build using GNU Make

Host application

```
$ cd optee_examples/hello_world/host
$ make \
  CROSS_COMPILE=arm-linux-gnueabi- \
  TEEC_EXPORT=<optee_client>/out/export/usr \
  --no-builtin-variables
```

With this you end up with a binary `optee_example_hello_world` in the host folder where you did the build.

Trusted Application

```
$ cd optee_examples/hello_world/ta
$ make \
  CROSS_COMPILE=arm-linux-gnueabi- \
  PLATFORM=vexpress-qemu_virt \
  TA_DEV_KIT_DIR=<optee_os>/out/arm/export-ta_arm32
```

With this you end up with files named `uuid.{ta,elf,dmp,map}` etc in the ta folder where you did the build.

Note: For a 64-bit builds (or any other toolchain) you will need to change `CROSS_COMPILE` (and also use a `PLATFORM` corresponding to an Armv8-A configuration).

Coding standards

See *Coding standards*.

Example applications

acipher

Application name	UUID
optee_example_acipher	a734eed9-d6a1-4244-aa50-7c99719e7b7b

Generates an RSA key pair of specified size and encrypts a supplied string with it using the GlobalPlatform TEE Internal Core API.

aes

Application name	UUID
optee_example_aes	5dbac793-f574-4871-8ad3-04331ec17f24

Runs an AES encryption and decryption from a TA using the GlobalPlatform TEE Internal Core API. Non secure test application provides the key, initial vector and ciphered data.

hello_world

Application name	UUID
optee_example_hello_world	8aaaf200-2450-11e4-abe2-0002a5d5c51b

This is a very simple Trusted Application to answer a hello command and incrementing an integer value.

hotp

Application name	UUID
optee_example_hotp	484d4143-2d53-4841-3120-4a6f636b6542

HMAC based One Time Password in OP-TEE

HMAC based One Time Passwords or shortly just ‘HOTP’ has been around for many years and was initially defined in [RFC4226](#) back in 2005. Since then it has been a popular choice for doing [two factor authentication](#). With the implementation here we are showing how one could leverage OP-TEE for generating such HMAC based One Time Passwords in a secure manner.

Client (OP-TEE) / Server solution

The most common way of using HOTP is in a client/server setup, where the client needs to authenticate itself to be able to get access to some resources on the server. In those cases the server will ask for an One Time Password, the client will generate that and send it over to the server and if the server is OK with the password it will grant access to the client.

Technically how it is working is that the server and the client needs to agree on shared key ('K') and also start from the same counter ('C'). How that is done in practice is another topic, but [RFC4226](#) has some discussion about it. You should at least have a secure channel between the client and the server when sharing the key, but even better would be if you could establish a secure channel all the way down to the TEE (currently we have TCP/UDP support in OP-TEE, but not TLS).

When both the server and the client knows about and use the same key and counter they can start doing client authentication using HOTP. In short what happens is that both the client and the server computes the same HOTP and the server compares the result of both computations (which should be the same to grant access). How that could work can be seen in the sequence diagram below.

In the current implementation we have OP-TEE acting as a client and the server is a remote service running somewhere else. There is no server implemented, but that should be pretty easy to add in a real scenario. The important thing here is to be able to register the shared key in the TEE and to get HOTP values from the TEE on request.

Since the current implementation works as a client we do not need to think about implementing the look-ahead synchronization window ('s') nor do we have to think about adding throttling (which prevents/slows down brute force attacks).

Sequence diagram - Client / Server

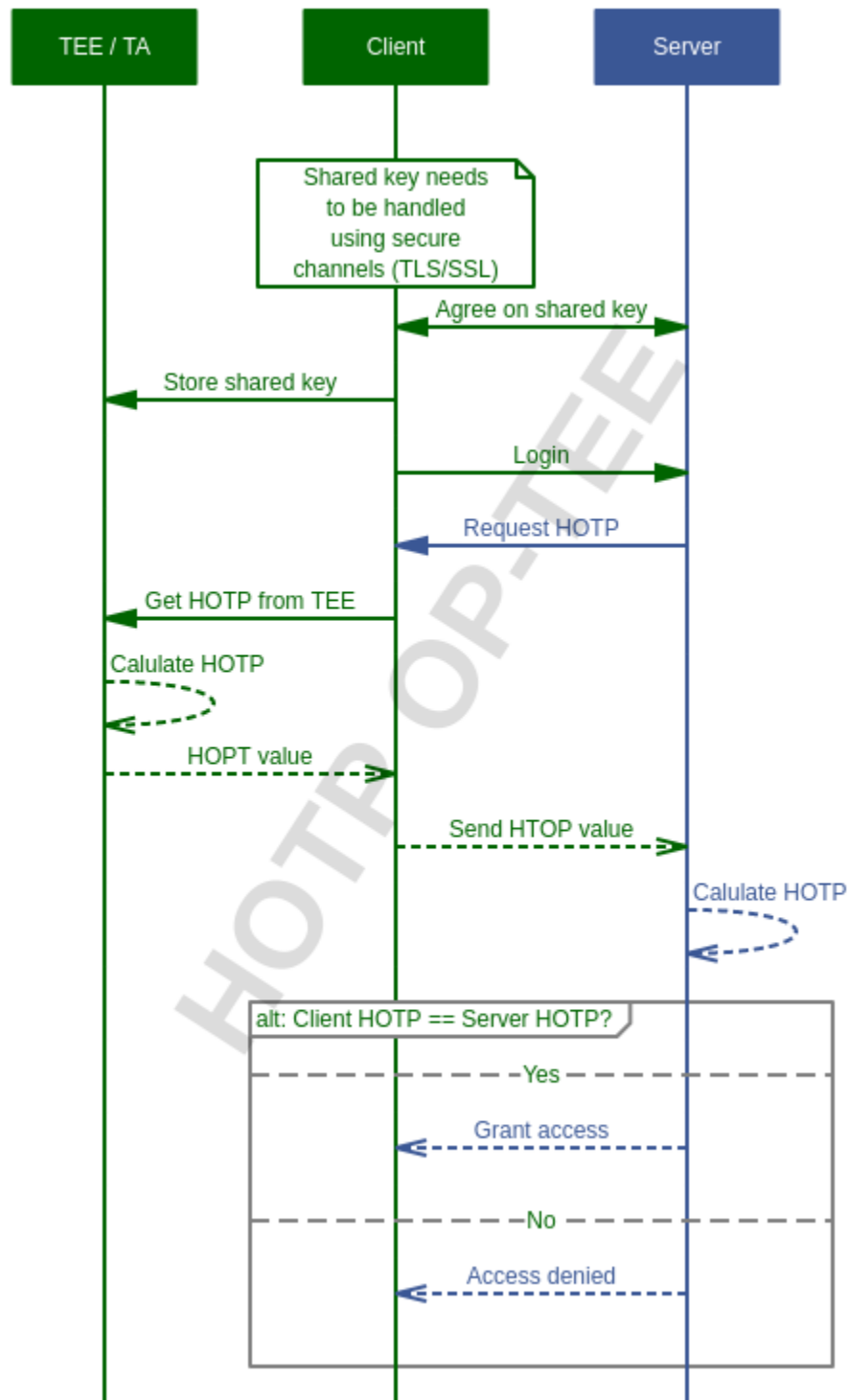
Client / Server (OP-TEE)?

Even though the current implementation works as a HOTP client, there is nothing saying that the implementation cannot be updated to also work as the validating server. One could for example have a simple device (a [security token] only generating one time passwords) and use the TEE as a validating service to open up other secure services.

random

Application name	UUID
optee_example_random	b6c53aba-9669-4668-a7f2-205629d00f86

Generates a random UUID using capabilities of TEE API (TEE_GenerateRandom()).



secure_storage

Application name	UUID
optee_example_secure_storage	f4e750bb-1437-4fbf-8785-8d3580c34994

A Trusted Application to read/write raw data into the OP-TEE secure storage using the GlobalPlatform TEE Internal Core API.

Further reading

Some additional information about how to write and compile Trusted Applications can be found at the [Trusted Applications](#) page.

3.5.8 optee_os

git location

https://github.com/OP-TEE/optee_os

License

The TEE core of optee_os is provided under the [BSD 2-Clause](#) license. But there are also other software such as libraries included in optee_os. This “other” software will have different licenses that are compatible with BSD 2-Clause (i.e., non-contaminating licenses unlike GPL-v2 for example).

Build instructions

You can build the code in this git only or build it as part of the entire system, i.e. as a part of a full OP-TEE developer setup. For the latter, please refer to instructions at the [build](#) page. For standalone builds optee_os uses only regular GNU Makefiles (i.e. **no** CMake support here unlike the other OP-TEE gits).

Configure the toolchain

First step is to download and configure a toolchain, see the [Toolchains](#) page for instructions.

Clone optee_os

```
$ git clone https://github.com/OP-TEE/optee_os
$ cd optee_os
```

Build using GNU Make

Since optee_os supports many devices and configurations it's impossible to give a examples to all variants. But below is how you for example would build for QEMU running Armv7-A (AArch32), with debugging enabled and the benchmark framework disabled and will put all built files in a folder name *out/arm* in the root of the git.

```

1 $ make \
2   CFG_TEE_BENCHMARK=n \
3   CFG_TEE_CORE_LOG_LEVEL=3 \
4   CROSS_COMPILE=arm-linux-gnueabihf- \
5   CROSS_COMPILE_core=arm-linux-gnueabihf- \
6   CROSS_COMPILE_ta_arm32=arm-linux-gnueabihf- \
7   CROSS_COMPILE_ta_arm64=aarch64-linux-gnu- \
8   DEBUG=1 \
9   O=out/arm \
10  PLATFORM=vexpress-qemu_virt

```

The same for an QEMU Armv8-A (AArch64) would look like this:

```

1 $ make \
2   CFG_ARM64_core=y \
3   CFG_TEE_BENCHMARK=n \
4   CFG_TEE_CORE_LOG_LEVEL=3 \
5   CROSS_COMPILE=aarch64-linux-gnu- \
6   CROSS_COMPILE_core=aarch64-linux-gnu- \
7   CROSS_COMPILE_ta_arm32=arm-linux-gnueabihf- \
8   CROSS_COMPILE_ta_arm64=aarch64-linux-gnu- \
9   DEBUG=1 \
10  O=out/arm \
11  PLATFORM=vexpress-qemu_armv8a

```

Hint: To be able to see all commands when building you could build with:

```
$ make V=1
```

Build using LLVM/clang

optee_os can be compiled using llvm/clang. Start by downloading the toolchain (see *LLVM / Clang*). After that you can compile by running.

Note: On line one you need to adjust the path so it matches the version of clang you are using.

```

1 $ export PATH=<optee-project>/toolchains/clang-v9.0.1/bin:$PATH
2 $ make COMPILER=clang

```

Coding standards

See *Coding standards*.

Build system

The build system in optee_os consists of a main **Makefile** in the root of the project together with **sub.mk** files in all source directories. In addition, some supporting files are used to recursively process all **sub.mk** files and generate the build rules.

Name	Description
core/core.mk	Included from Makefile to build the TEE Core
ta/ta.mk	Included from Makefile to create the TA devkit
mk/compile.mk	Create rules to make objects from source files
mk/lib.mk	Create rules to make a libraries (.a)
mk/subdir.mk	Process sub.mk files recursively
mk/config.mk	Global configuration variable
core/arch/\$(ARCH)/\$(ARCH).mk	Arch-specific compiler flags
core/arch/\$(ARCH)/plat-\$(PLATFORM)/conf.mk	Platform-specific compiler flags and configuration variables
core/arch/\$(ARCH)/plat-\$(PLATFORM)/link.mk	Make recipes to link the TEE Core
ta/arch/arm/link.mk	Make recipes to link Trusted Applications
ta/mk/ta_dev_kit.mk	Main Makefile to be included when building Trusted Applications
mk/checkconf.mk	Utility functions to manipulate configuration variables and generate a C header file
sub.mk	List source files and define compiler flags

make is always invoked from the top-level directory; there is no recursive invocation of make itself.

Choosing the build target

The target architecture, platform and build directory may be selected by setting environment or make variables (VAR=value make or make VAR=value).

ARCH - CPU architecture

\$(ARCH) is the CPU architecture to be built. Currently, the only supported value is **arm** for 32-bit or 64-bit Armv7-A or Armv8-A. Please note that contrary to the Linux kernel, \$(ARCH) should **not** be set to **arm64** for 64-bit builds. The ARCH variable does not need to be set explicitly before building either, because the proper instruction set is selected from the \$(PLATFORM) value. For platforms that support both 32-bit and 64-bit builds, CFG_ARM64_core=y should be set to select 64-bit and not set (or set to n) to select 32-bit.

Architecture-specific source code belongs to sub-directories that follow the arch/\$(ARCH) pattern, such as: core/arch/arm, lib/libutee/arch/arm and so on.

CROSS_COMPILE

`$(CROSS_COMPILE)` is the prefix used to invoke the (32-bit) cross-compiler toolchain. The default value is `arm-linux-gnueabi`. This is the variable you want to change in case you want to use `ccache` to speed up recom-pilations:

```
$ make CROSS_COMPILE="ccache arm-linux-gnueabi"
```

If the build includes a mix of 32-bit and 64-bit code, for instance if you set `CFG_ARM64_core=y` to build a 64-bit secure kernel, then two different toolchains are used, that are controlled by `$(CROSS_COMPILE32)` and `$(CROSS_COMPILE64)`. The default value of `$(CROSS_COMPILE32)` is the value of `CROSS_COMPILE`, which defaults to `arm-linux-gnueabi` as mentioned above. The default value of `$(CROSS_COMPILE64)` is `aarch64-linux-gnu`. Examples:

```
# For this example, select HiKey which supports both 32- and 64-bit builds
$ export PLATFORM=hikey

# 1. Build everything 32-bit
$ make

# 2. Same as (1.) but override the toolchain
$ make CROSS_COMPILE="ccache arm-linux-gnueabi"

# 3. Same as (2.)
$ make CROSS_COMPILE32="ccache arm-linux-gnueabi"

# 4. Select 64-bit secure 'core' (and therefore both 32- and 64-bit
# Trusted Application libraries)
$ make CFG_ARM64_core=y

# 5. Same as (4.) but override the toolchains
$ make CFG_ARM64_core=y \
    CROSS_COMPILE32="ccache arm-linux-gnueabi" \
    CROSS_COMPILE64="ccache aarch64-linux-gnu"
```

PLATFORM / PLATFORM_FLAVOR

A *platform* is a family of closely related hardware configurations. A *platform flavor* is a variant of such configurations. When used together they define the target hardware on which OP-TEE will be run.

For instance `PLATFORM=stm PLATFORM_FLAVOR=b2260` will build for the ST Microelectronics 96boards/cannes2 board, while `PLATFORM=vexpress PLATFORM_FLAVOR=qemu_virt` will generate code for a para-virtualized Arm Versatile Express board running on QEMU.

For convenience, the flavor may be appended to the platform name with a dash, so `make PLATFORM=stm-b2260` is a shortcut for `make PLATFORM=stm PLATFORM_FLAVOR=b2260`. Note that in both cases the value of `$(PLATFORM)` is `stm` in the makefiles.

Platform-specific source code belongs to `core/arch/$(ARCH)/plat-$(PLATFORM)`, for instance: `core/arch/arm/plat-vexpress` or `core/arch/arm/plat-stm`.

O - output directory

All output files go into a platform-specific build directory, which is by default `out/${ARCH}-plat-${PLATFORM}`.

The output directory has basically the same structure as the source tree. For instance, assuming `ARCH=arm` `PLATFORM=stm`, `core/kernel/panic.c` will compile into `out/arm-plat-stm/core/kernel/panic.o`.

However, some libraries are compiled several times: once or twice for user mode, and once for kernel mode. This is because they may be used by the TEE Core as well as by the Trusted Applications. As a result, the `lib` source directory gives two or three build directories: `ta_arm{32,64}-lib` and `core-lib`.

The output directory also has an `export-ta_arm{32,64}` directory, which contains:

- All the files needed to build Trusted Applications.
 - In `lib/`: `libutee.a` (the GlobalPlatform Internal API), `libutils.a` (which implements a part of the standard C library), and other libraries which provide additional APIs.
 - In `include/`: header files for the above libraries
 - In `mk/`: `ta_dev_kit.mk`, which is a Make include file with suitable rules to build a TA, and its dependencies
 - `scripts/sign.py`: a Python script used by `ta_dev_kit.mk` to sign TAs.
 - In `src/`: `user_ta_header.c`: source file to add a suitable header to the Trusted Application (as expected by the loader code in the TEE Core).
- Some files needed to build host applications (using the Client API), under `export-ta_arm{32,64}/host_include`.

Finally, the build directory contains the auto-generated configuration file for the TEE Core: `$(0)/include/generated/conf.h` (see below).

Configuration and flags

The following variables are defined in `core/arch/${ARCH}/${ARCH}.mk`:

- `$(core-platform-aflags)`, `$(core-platform-cflags)` and `$(core-platform-cppflags)` are added to the assembler / C compiler / preprocessor flags for all source files compiled for TEE Core including the kernel versions of libraries such as `libutils.a`.
- `$(ta_arm{32,64}-platform-aflags)`, `$(ta_arm{32,64}-platform-cflags)` and `$(ta_arm{32,64}-platform-cppflags)` are added to the assembler / C compiler / preprocessor flags when building the user-mode libraries (`libutee.a`, `libutils.a`) or Trusted Applications.

The following variables are defined in `core/arch/${ARCH}/plat-${PLATFORM}/conf.mk`:

- If `$(arm{32,64}-platform-cflags)`, `$(arm{32,64}-platform-aflags)` and `$(arm{32,64}-platform-cppflags)` are defined their content will be added to `$(\*-platform-\*flags)` when they are initialized in `core/arch/${ARCH}/${ARCH}.mk` as described above.
- `$(core-platform-subdirs)` is the list of the subdirectories that are added to the TEE Core.

Linker scripts

The file `core/arch/$(ARCH)/plat-$(PLATFORM)/link.mk` contains the rules to link the TEE Core and perform any related tasks, such as running `objdump` to produce a dump file. `link.mk` adds files to the `all:` target.

Source files

Each directory that contains source files has a file called `sub.mk`. This makefile defines the source files that should be included in the build, as well as any subdirectories that should be processed, too. For example:

```
# core/arch/arm/sm/sub.mk
srcs-y += sm_asm.S
srcs-y += sm.c
```

```
# core/sub.mk
subdirs-y += kernel
subdirs-y += mm
subdirs-y += tee
subdirs-y += drivers
```

The `-y` suffix is meant to facilitate conditional compilation. See section [Configuration variables](#) below.

`srcs-y` and `subdirs-y` are often not used together in the same `sub.mk`, because source files are usually alone in leaf directories. But this is not a hard rule.

In addition to source files, `sub.mk` may define compiler flags, include directories and/or configuration variables as explained below.

Compiler flags

Default compiler flags are defined in `mk/compile.mk`. Note that platform-specific flags must not appear in this file which is common to all platforms.

To add flags for a given source file, you may use the following variables in `sub.mk`:

- `cflags-<filename>-y` for C files (*.c)
- `aflags-<filename>-y` for assembler files (*.S)
- `cppflags-<filename>-y` for both C and assembler

For instance:

```
# core/lib/libtomcrypt/src/pk/dh/sub.mk
srcs-y += dh.c
cflags-dh.c-y := -Wno-unused-variable
```

Compiler flags may also be removed, as follows:

```
# lib/libutils/isoc/newlib/sub.mk
srcs-y += memmove.c
cflags-remove-memmove.c-y += -Wcast-align
```

Some variables apply to libraries only (that is, when using `mk/lib.mk`) and affect all the source files that belong to the library: `cppflags-lib-y` and `cflags-lib-y`.

Include directories

Include directories may be added to `global-incdirs-y`, in which case they will be accessible from all the source files and will be copied to `export-ta_arm{32,64}/include` and `export-ta_arm{32,64}/host_include`.

When `sub.mk` is used to build a library, `incdirs-lib-y` may receive additional directories that will be used for that library only.

Configuration variables

Some features may be enabled, disabled or otherwise controlled at compile time through makefile variables. Default values are normally provided in makefiles with the `?=` operator so that their value may be easily overridden by environment variables. For instance:

```
PLATFORM ?= stm
PLATFORM_FLAVOR ?= default
```

Some global configuration variables are defined in `mk/config.mk`, but others may be defined in `sub.mk` when then pertain to a specific library for instance.

Variables with the `CFG_` prefix are treated in a special way: their value is automatically reflected in the generated header file `$(out-dir)/include/generated/conf.h`, after all the included makefiles have been processed. `conf.h` is automatically included by the preprocessor when a source file is built.

Depending on their value, variables may be considered either boolean or non-boolean, which affects how they are translated into `conf.h`.

Boolean configuration variables

When a configuration variable controls the presence or absence of a feature, `y` means **enabled**, while anything else means **disabled**. In particular, a variable that is undefined or is defined but has an empty value is also disabled. For example, the following commands are equivalent and would disable feature `CFG_CRYPT0_GCM`:

```
$ make CFG_CRYPT0_GCM=n
```

```
$ make CFG_CRYPT0_GCM=
```

```
$ CFG_CRYPT0_GCM=n make
```

```
$ export CFG_CRYPT0_GCM=n
$ make
```

When a configuration variable is introduced, the proper way to enable it by default is to write `CFG_XXX ?= y`. If it should be default disabled instead, use `CFG_XXX ?= n` rather than leaving the variable undefined, and add a description in a comment.

In general, common settings belong in `mk/config.mk` while platform-specific ones should go in `core/arch/$(arch)/plat-$(platform)/conf.mk` (there are exceptions, for instance crypto settings have their own `crypto.mk` files). Both places may be used in case a platform needs to set a default value that is different from the one that is set in the global configuration file. For example:

```
# CFG_F00 is enabled by default except for PLATFORM=xyz
# Override with "make CFG_F00=y" is allowed

# In core/arch/$(arch)/plat-xyz/conf.mk
CFG_F00 ?= n

# In mk/config.mk
CFG_F00 ?= y
```

```
# CFG_F00 is enabled by default except for PLATFORM=xyz that requires
# CFG_F00 disabled
# Override with "make CFG_F00=y" is NOT allowed

# In core/arch/$(arch)/plat-xyz/conf.mk
$(call force,CFG_F00,n)

# In mk/config.mk
CFG_F00 ?= y
```

y and n can be swapped to achieve the opposite scenario.

Configuration variables can easily be used in `sub.mk` to trigger conditional compilation:

```
# core/lib/libtomcrypt/src/encauth/sub.mk
subdirs-$(CFG_CRYPTOC_CCM) += ccm
subdirs-$(CFG_CRYPTOC_GCM) += gcm
```

It is not recommended to set `CFG_` values in `sub.mk`.

When a configuration variable is **enabled** (y), `<generated/conf.h>` contains a macro with the same name as the variable and the value 1. If it is **disabled**, however, no macro definition is output. This allows the C code to use constructs like:

```
/* core/lib/libtomcrypt/src/tee_ltc_provider.c */

/* ... */

#ifdef CFG_CRYPTOC_GCM
struct tee_gcm_state {
    gcm_state ctx;          /* the gcm state as defined by LTC */
    size_t tag_len;         /* tag length */
};
#endif
```

Non-boolean configuration variables

Configuration variables that are not recognized as booleans are simply output unchanged into `<generated/conf.h>`. For instance:

```
$ make CFG_TEE_CORE_LOG_LEVEL=4
```

```
/* out/arm-plat-vexpress/include/generated/conf.h */  
  
#define CFG_TEE_CORE_LOG_LEVEL 4 /* '4' */
```

The ‘force’ macro

Some platforms may require specific values for some of their configuration variables. For instance, the number of CPU cores in a system is fixed so the value of `CFG_TEE_CORE_NB_CORE` should generally not be changed. Or some feature may not be supported by the hardware, in which case the corresponding configuration variable should always be disabled.

In such cases, the `force` macro should be used. It sets the variable to the specified value while reporting about any conflicting value that may have been set previously, either via a previous assignment in the makefiles or via the command line or environment variables. For example:

```
$(call force,CFG_TEE_CORE_NB_CORE,8)  
$(call force,CFG_ARM64_core,n)  
$(call force,CFG_ARM_GICV3,y)
```

```
$ make -j10 PLATFORM=hikey CFG_TEE_CORE_NB_CORE=4  
core/arch/arm/plat-hikey/conf.mk:5: *** CFG_TEE_CORE_NB_CORE is set to '4' (from command_  
→line) but its value must be '8'. Stop.
```

There are only two ways...

Given what has been explained above, there are only two valid ways to set a `CFG_` variable in a `.mk` file: either with `?=` if the value is a default that may be changed at build time, or with `$(call force,...)` if there is only one acceptable value. Using `=` or `:=` in particular is **not** correct because they allow overrides on the command line (`make CFG_F00=foo`) but not from the environment (`CFG_F00=foo make`).

Configuration dependencies

Some combinations of configuration variables may not be valid. This should be dealt with by custom checks in makefiles. `mk/checkconf.h` provides functions to help detect and deal with such situations.

Import branches

This section is more specifically intended for maintainers.

The optee_os repository contains branches with the `import/` prefix, which we call *import branches* below. This section describes their purpose and how they are used.

Import branches are meant to help import external libraries into the optee_os repository and maintain them:

- *Import* means copy source files from a given upstream version of the library and commit them locally (typically under `optee_os/lib` or `optee_os/core/lib`), along with OP-TEE specific changes (build and configuration files for instance)
- *Maintain* means carry local bug fixes or improvements that did not make their way upstream, and periodically upgrade the library by importing changes from upstream.

Import branches have the version of the imported library in their names. For example: `import/mbedtls-2.6.1`. They are forked from `master`. They record the history of all changes made for OP-TEE to a library for a given library version. For example, the import branch for Mbed TLS 2.6.1 illustrates how the Mbed TLS library was initially imported and later modified:

```
$ BRANCH=github/import/mbedtls-2.6.1
$ BASE=`git merge-base master $BRANCH`
$ git log --oneline --no-merges $BASE..$BRANCH
8ff963a6 (github/import/mbedtls-2.6.1) mbedtls: fix memory leak in mpi_miller_rabin()
213cce52 libmbedtls: mpi_miller_rabin: increase count limit
f934e291 mbedtls: add mbedtls_mpi_init_static()
782fddd1 libmbedtls: add mbedtls_mpi_init_mempool()
33873834 libmbedtls: make mbedtls_mpi_mont*() available
e0186224 libmbedtls: refine mbedtls license header
215609ae mbedtls: configure mbedtls to reach for config
6916dcd9 mbedtls: remove default include/mbedtls/config.h
b60fc42a Import mbedtls-2.6.1
```

Commit `b60fc42a` imports the library under `lib/libmbedtls/mbedtls` with no modification to the code (not the whole library is imported however, since some files are not needed they are deleted as mentioned in the commit description). Then a couple of adjustments are made in commits `6916dcd9` and `215609ae` in order to be able to build with `optee_os`. At this point the initial import is done and subsequent commits are local improvements or bug fixes made later on.

The initial import (commits `b60fc42a`, `6916dcd9` and `215609ae`) is merged into `master` as a “squashed” commit to preserve bisectability – in other words, so that no commit in `master` breaks the build:

```
$ git show --quiet 817466cb
commit 817466cb476de705a8e3dabe1ef165fe27a18c2f
Author: Jens Wiklander <jens.wiklander@linaro.org>
Date: Tue May 22 13:49:31 2018 +0200

    Squashed commit importing mbedtls-2.6.1 source

    Squash merging branch import/mbedtls-2.6.1

    215609ae4d8c ("mbedtls: configure mbedtls to reach for config")
    6916dcd9b9cd ("mbedtls: remove default include/mbedtls/config.h")
    b60fc42a5cd5 ("Import mbedtls-2.6.1")
```

(continues on next page)

(continued from previous page)

Acked-by: Joakim Bech <joakim.bech@linaro.org>
 Signed-off-by: Jens Wiklander <jens.wiklander@linaro.org>

Later changes are reviewed and merged into master normally, and are also recorded on top of the import branch via pull requests against the import branch. Consider for example the two following commits, one is in the master branch and the other is applied on top of the import branch.

```
18c5148d357e ("mbedtls: add mbedtls_mpi_init_static()") # In master
f934e2913b7b ("mbedtls: add mbedtls_mpi_init_static()") # In import/mbedtls-2.6.1
```

The master branch is occasionally merged into the import branches. Otherwise, some patches cherry-picked from master would not apply. Note that it is a “normal” merge (not a rebase), so the commits that are on master can easily be filtered out (`git log --oneline --first-parent import/...`).

When it is time to upgrade a library, a new import branch is created from master, for example: `import/mbedtls-2.16.0`. A pull request is created against this branch with the following commits:

- The first commit deletes the “old” version of the library and imports the new upstream version.
- The subsequent commits are cherry-picked from the previous import branch and adjusted as needed. These commits are effectively “rebased” onto the new library.
- Build files are updated if needed.

Here is the history of the `import/mbedtls-2.16.0` branch, for comparison with the initial import:

```
$ BRANCH=github/import/mbedtls-2.16.0
$ BASE=`git merge-base master $BRANCH`
$ git log --oneline --no-merges $BASE..$BRANCH
68df6eb0 libmbedtls: mbedtls_mpi_exp_mod(): reduce stack usage
f58facc6 libutee: increase MPI mempool size
be040a3e libmbedtls: preserve mempool usage on reinit
ae499f6a libmbedtls: mbedtls_mpi_exp_mod() initialize W
b95a6c5d libmbedtls: fix no CRT issue
ac34734a libmbedtls: add interfaces in mbedtls for context memory operation
9ee2a92d libmbedtls: compile new files added with 2.16.0
9b0818d4 mbedtls: fix memory leak in mpi_miller_rabin()
2d6644ee libmbedtls: mpi_miller_rabin: increase count limit
d831db4c libmbedtls: add mbedtls_mpi_init_mempool()
df0f4886 libmbedtls: make mbedtls_mpi_mont*() available
7b079206 libmbedtls: refine mbedtls license header
2616e2d9 mbedtls: configure mbedtls to reach for config
d686ab1c mbedtls: remove default include/mbedtls/config.h
50a57cfa Import mbedtls-2.16.0
8bfc3de4 libutee: lessen dependency on mbedtls internals
```

Note that the first commit `8bfc3de4` (“libutee: lessen dependency on mbedtls internals”) can be ignored, it was applied to master in anticipation of the 2.16.0 upgrade but the `import/mbedtls-2.16.0` was forked before.

The upgrade from 2.6.1 to 2.16.0 is made of all the commits up to and including commit `9ee2a92d` (“libmbedtls: compile new files added with 2.16.0”):

- Commit `50a57cfa` (“Import mbedtls-2.16.0”) deletes the “old” files and library imports the new ones.
- Commits `d686ab1c..9b0818d4` are cherry-picked from the previous import branch.
- Commit `9ee2a92d` (“libmbedtls: compile new files added with 2.16.0”) adapts the build files to the new version.

The master branch contains a squashed equivalent of the above:

```
$ git show --quiet 3d3b0591
commit 3d3b05918ec9052ba13de82fbcaba204766eb636
Author: Jens Wiklander <jens.wiklander@linaro.org>
Date:   Wed Mar 20 15:30:29 2019 +0100

    Squashed commit upgrading to mbedtls-2.16.0

    Squash merging branch import/mbedtls-2.16.0

    9ee2a92de51f ("libmbedtls: compile new files added with 2.16.0")
    9b0818d48d29 ("mbedtls: fix memory leak in mpi_miller_rabin()")
    2d6644ee0bbe ("libmbedtls: mpi_miller_rabin: increase count limit")
    d831db4c238a ("libmbedtls: add mbedtls_mpi_init_mempool()")
    df0f4886b663 ("libmbedtls: make mbedtls_mpi_mont*() available")
    7b0792062b65 ("libmbedtls: refine mbedtls license header")
    2616e2d9709f ("mbedtls: configure mbedtls to reach for config")
    d686ab1c51b7 ("mbedtls: remove default include/mbedtls/config.h")
    50a57cfac892 ("Import mbedtls-2.16.0")

    Acked-by: Jerome Forissier <jerome.forissier@linaro.org>
    Signed-off-by: Jens Wiklander <jens.wiklander@linaro.org>
```

Subsequent commits in the `import/mbedtls-2.16.0` branch are modifications that happened later in master as a result of OP-TEE development.

3.5.9 optee_test

The `optee_test.git` contains the source code for the TEE sanity test suite in Linux using the ARM(R) TrustZone(R) technology. It is typically referred to as *xtest*. By default there are several thousands of tests when running the code that is in the git only. However, it is also possible to incorporate tests coming from GlobalPlatform (see [Extended test \(GlobalPlatform tests\)](#)). We typically refer to these to as:

- **Standard tests:** These are the test that are included in `optee_test`. They are free and open source.
- **Extended tests:** Those are the tests that are written directly by GlobalPlatform. They are **not** open source and they are **not** freely available (it's free to members of GlobalPlatform and can otherwise be purchased directly from GlobalPlatform).

git location

https://github.com/OP-TEE/optee_test

License

The client applications (`optee_test/host/*`) are provided under the [GPL-2.0](#) license and the user Trusted Applications (`optee_test/ta/*`) are provided under the [BSD 2-Clause](#).

Build instructions

At the moment you can **only** build the code in this git as part of the entire system, i.e. as a part of a full OP-TEE developer setup. So, please refer to the instructions at the [build](#) page to learn how to build a full OP-TEE developer setup. Building purely standalone is **not** possible (*) because:

- the host code (`xtest`) have dependencies to the [optee_client](#) (it links against `libteec`, `openssl` and uses various headers)
- the Trusted Applications have dependencies to the TA-devkit built by [optee_os](#).

Note: (*) It is of course possible to build this without a full OP-TEE developer setup, but it will require a lot of tweaking with paths, flags etc. I.e., one would need to do exactly the same as the full OP-TEE developer setup does under the hood.

Extended test (GlobalPlatform tests)

One can purchase the [GlobalPlatform Compliance Test suite](#) which is GlobalPlatforms own test suite for testing TEE implementations adhering to the GlobalPlatforms interfaces.

Hint: Members of GlobalPlatform can download this for free at the GlobalPlatform members pages. This something that the OP-TEE project **cannot** help with. If you need help with that, please reach out to the liason at GlobalPlatform.

`xtest` can be extended/patched to include the GlobalPlatform Compliance Test suite. This can be done by downloading the GlobalPlatform Compliance Test suite (a *.7z file) and add an additional compiler flag (`GP_PACKAGE`) to the make invocation line, example:

```
$ make GP_PACKAGE=/tmp/TEE_Initial_Configuration-Test_Suite_v2_0_0_2-2017_06_09.7z
```

Note: Starting from OP-TEE v3.11.0, OP-TEE was updated to support the `TEE_Initial_Configuration-Test_Suite_v2_0_0_2-2017_06_09.7z` version from the GlobalPlatform Compliance Test suite. That is the only supported version after OP-TEE v3.11.0. If you need to run an earlier version of the GlobalPlatform Compliance Test suite then you need to follow the instructions in the documentation for OP-TEE v3.9.0 and earlier.

Run xtest

It's important to understand that you run `xtest` on the device itself, i.e., this is nothing that you run on the host machine.

xtest - default

The most simple case is to run the default configuration:

```
$ xtest
```

xtest - all

This runs all tests within the standard `xtest`. Using the `-l` parameter you can tweak the amount of tests you will run. 15 is the most and 0 is the least.

```
$ xtest -l 15
```

xtest - single

To run a single test case, just specify its numbers, for example:

```
$ xtest 1001
```

xtest - family

To run a family (1xxx, 2xxx and so on), just specify its number prefixed with an underscore. This for example will run the 1xxx family.

```
$ xtest _1
```

xtest - benchmark

To run the benchmark tests, run `xtest` like this:

```
$ xtest -t benchmark
```

Here it is also possible to state a number for a certain benchmark test, for example:

```
$ xtest -t benchmark 2001
```

xtest - regression

To run the regression tests, run xtest like this:

```
$ xtest -t regression
```

Here it is also possible to state a number for a certain regression test, for example:

```
$ xtest -t regression 2004
```

xtest - aes-perf

This is benchmark test for AES and you run it like this:

```
$ xtest --aes-perf
```

Note: There is an individual help for `--aes-perf`, i.e.

```
$ xtest --aes-perf -h
```

xtest - sha-perf

This is benchmark test for SHA-xxx and you run it like this:

```
$ xtest --sha-perf
```

Note: There is an individual help for `--sha-perf`, i.e.

```
$ xtest --sha-perf -h
```

There you can select other SHA algorithms etc.

Coding standards

See *Coding standards*.

The *optee_os* repository is required to run the checks. It's location may be passed using the *OPTEE_OS_PATH* environment variable:

```
export OPTEE_OS_PATH=/path/to/optee_os
```

In case *OPTEE_OS_PATH* is unset or empty, the dispatcher script will default to *../optee_os*.

3.6 Toolchains

OP-TEE uses both 32bit as well as 64bit toolchains and it is even possible to mix them in some configurations. In theory you should be able to compile OP-TEE with the Arm toolchains that are coming with your Linux distribution. But instead of using those directly, we instead download the toolchains directly from Arm.

3.6.1 Download/install

We propose two ways to download the toolchains, both will put the toolchains under the same path(s).

Direct download

Go the [Arm GNU Toolchain Downloads](#) page and download the “AArch32 target with soft float (*arm-linux-gnueabi*)” for 32bit builds and the “AArch64 GNU/Linux target (*aarch64-linux-gnu*)” for 64bit builds. When the downloads have finished, you will untar them to a location that you later on will export to your \$PATH. Here is an example

```
$ mkdir -p $HOME/toolchains
$ cd $HOME/toolchains

# Download 32bit toolchain
$ wget https://developer.arm.com/-/media/Files/downloads/gnu-a/8.2-2019.01/gcc-arm-8.2-
  ↳ 2019.01-x86_64-arm-linux-gnueabi.tar.xz
$ mkdir aarch32
$ tar xf gcc-arm-8.2-2019.01-x86_64-arm-linux-gnueabi.tar.xz -C aarch32 --strip-
  ↳ components=1

# Download 64bit toolchain
$ wget https://developer.arm.com/-/media/Files/downloads/gnu-a/8.2-2019.01/gcc-arm-8.2-
  ↳ 2019.01-x86_64-aarch64-linux-gnu.tar.xz
$ mkdir aarch64
$ tar xf gcc-arm-8.2-2019.01-x86_64-aarch64-linux-gnu.tar.xz -C aarch64 --strip-
  ↳ components=1
```

Using build.git

As an alternative, you can let *build.git* download them for you, but this of course involves getting a git that you might not otherwise use.

```
$ cd $HOME
$ git clone https://github.com/OP-TEE/build.git
$ cd build
$ make -f toolchain.mk -j2
```

3.6.2 Export PATH

If you have downloaded the toolchains as described above, you should have them at `$HOME/toolchains/{aarch32/aarch64}`, so now we just need to export the paths and then you are ready to starting compiling OP-TEE components.

```
$ export PATH=$PATH:$HOME/toolchains/aarch32/bin:$HOME/toolchains/aarch64/bin
```

3.6.3 LLVM / Clang

It's possible to also compile *optee_os*.git using llvm/clang. To do that, you can download and extract Clang from the GitHub release page. You'll need an x86_64 cross-compiler capable of generating aarch64 and armv7a code **and** the compiler-rt libraries for these architectures (libclang_rt.*.a).

Clang is configured to be able to cross-compile to all the supported architectures by default (see `<clang path>/bin/llc -version`) which is great, but compiler-rt is included only for the host architecture. Therefore you need to combine several packages into one. Please refer to this [get_clang.sh](#) script for details on creating a llvm/clang toolchain ready to be used.

Using build.git

As an alternative, you can let *build*.git download them for you, but this of course involves getting a git that you might not otherwise use.

```
$ cd $HOME
$ git clone https://github.com/OP-TEE/build.git
$ cd build
$ make -f toolchain.mk clang-toolchains
```

The above instructions will download and install Clang in `$HOME/clang-9.0.1`.

You can also get the toolchain using your package manager or alternatively build it yourself, but these alternative methods risk being incomplete. For example, the Ubuntu clang package does not install the needed `ld.lld` package. The package also does not contain the cross-compiled compiler-rt libraries. Building by yourself is hard for the same reason, i.e. no cross-compiled compiler-rt libraries are generated.

3.7 Trusted Applications

This document tells how to implement a Trusted Application for OP-TEE, using OP-TEE's so called *TA-devkit* to both build and sign the Trusted Application binary. In this document, a *Trusted Application* running in the OP-TEE os is referred to as a *TA*. Note that in the default setup a private test key is distributed along with the *optee_os* source is used for signing Trusted Applications. See *TASign* for more details, including offline signing of TAs.

3.7.1 TA Mandatory files

The Makefile for a Trusted Application must be written to rely on OP-TEE TA-devkit resources in order to successfully build the target application. TA-devkit is built when one builds *optee_os*.

To build a TA, one must provide:

- **Makefile**, a make file that should set some configuration variables and include the TA-devkit make file.
- **sub.mk**, a make file that lists the sources to build (local source files, subdirectories to parse, source file specific build directives).
- **user_ta_header_defines.h**, a specific ANSI-C header file to define most of the TA properties.
- An implementation of at least the TA entry points, as extern functions: `TA_CreateEntryPoint()`, `TA_DestroyEntryPoint()`, `TA_OpenSessionEntryPoint()`, `TA_CloseSessionEntryPoint()`, `TA_InvokeCommandEntryPoint()`

TA file layout example

As an example, *hello_world* looks like this:

```
hello_world/
├── ...
└── ta
    ├── Makefile           BINARY=<uuid>
    ├── Android.mk         Android way to invoke the Makefile
    ├── sub.mk             srcs-y += hello_world_ta.c
    ├── include
    │   └── hello_world_ta.h Header exported to non-secure: TA commands API
    ├── hello_world_ta.c    Implementation of TA entry points
    └── user_ta_header_defines.h TA_UUID, TA_FLAGS, TA_DATA/STACK_SIZE, ...
```

3.7.2 TA Makefile Basics

Required variables

The main TA-devkit make file is located in *optee_os* at `ta/mk/ta_dev_kit.mk`. The make file supports make targets such as `all` and `clean` to build a TA or a library and clean the built objects.

The make file expects a couple of configuration variables:

TA_DEV_KIT_DIR

Base directory of the TA-devkit. Used by the TA-devkit itself to locate its tools.

BINARY and LIBNAME

These are exclusive, meaning that you cannot use both at the same time. If building a TA, `BINARY` shall provide the TA filename used to load the TA. The built and signed TA binary file will be named `${BINARY}.ta`. In native OP-TEE, it is the TA UUID, used by tee-supplciant to identify TAs. If one is building a static library (that will be later linked by a TA), then `LIBNAME` shall provide the name of the library. The generated library binary file will be named `lib${LIBNAME}.a`

CROSS_COMPILE and CROSS_COMPILE32

Cross compiler for the TA or the library source files. `CROSS_COMPILE32` is optional. It allows to target AArch32 builds on AArch64 capable systems. On AArch32 systems, `CROSS_COMPILE32` defaults to `CROSS_COMPILE`.

Optional variables

Some optional configuration variables can be supported, for example:

O

Base directory for build objects filetree. If not set, TA-devkit defaults to **./out** from the TA source tree base directory.

Example Makefile

A typical Makefile for a TA looks something like this

```
# Append specific configuration to the C source build (here log=info)
# The UUID for the Trusted Application
BINARY=8aaaf200-2450-11e4-abe2-0002a5d5c51b

# Source the TA-devkit make file
include $(TA_DEVKIT_DIR)/mk/ta_dev_kit.mk
```

sub.mk directives

The make file expects that current directory contains a file **sub.mk** that is the entry point for listing the source files to build and other specific build directives. Here are a couple of examples of directives one can implement in a **sub.mk** make file:

```
# Adds /hello_world_ta.c from current directory to the list of the source
# file to build and link.
srcs-y += hello_world_ta.c

# Includes path **./include/** from the current directory to the include
# path.
global-incdirs-y += include/

# Adds directive -Wno-strict-prototypes only to the file hello_world_ta.c
cflags-hello_world_ta.c-y += -Wno-strict-prototypes

# Removes directive -Wno-strict-prototypes from the build directives for
# hello_world_ta.c only.
cflags-remove-hello_world_ta.c-y += -Wno-strict-prototypes

# Adds the static library foo to the list of the linker directive -lfoo.
libnames += foo

# Adds the directory path to the libraries pathes list. Archive file
# libfoo.a is expected in this directory.
libdirs += path/to/libfoo/install/directory

# Adds the static library binary to the TA build dependencies.
libdeps += path/to/greatlib/libgreatlib.a
```

3.7.3 Android Build Environment

OP-TEE's TA-devkit supports building in an Android build environment. One can write an `Android.mk` file for the TA (stored side by side with the Makefile). Android's build system will parse the `Android.mk` file for the TA which in turn will parse a TA-devkit Android make file to locate TA build resources. Then the Android build will execute a make command to build the TA through its generic Makefile file.

A typical `Android.mk` file for a TA looks like this (`Android.mk` for *hello_world* is used as an example here).

```
# Define base path for the TA sources filetree
LOCAL_PATH := $(call my-dir)

# Define the module name as the signed TA binary filename.
local_module := 8aaaf200-2450-11e4-abe2-0002a5d5c51b.ta

# Include the devkit Android make script
include $(OPTEE_OS_DIR)/mk/aosp_optee.mk
```

3.7.4 TA Mandatory Entry Points

A TA must implement a couple of mandatory entry points, these are:

```
TEE_Result TA_CreateEntryPoint(void)
{
    /* Allocate some resources, init something, ... */
    ...

    /* Return with a status */
    return TEE_SUCCESS;
}

void TA_DestroyEntryPoint(void)
{
    /* Release resources if required before TA destruction */
    ...
}

TEE_Result TA_OpenSessionEntryPoint(uint32_t ptype,
                                    TEE_Param param[4],
                                    void **session_id_ptr)
{
    /* Check client identity, and alloc/init some session resources if any */
    ...

    /* Return with a status */
    return TEE_SUCCESS;
}

void TA_CloseSessionEntryPoint(void *sess_ptr)
{
    /* check client and handle session resource release, if any */
    ...
}
```

(continues on next page)

(continued from previous page)

```

TEE_Result TA_InvokeCommandEntryPoint(void *session_id,
                                     uint32_t command_id,
                                     uint32_t parameters_type,
                                     TEE_Param parameters[4])
{
    /* Decode the command and process execution of the target service */
    ...

    /* Return with a status */
    return TEE_SUCCESS;
}

```

3.7.5 TA Properties

Trusted Application properties shall be defined in a header file named `user_ta_header_defines.h`, which should contain:

- `TA_UUID` defines the TA uuid value
- `TA_FLAGS` define some of the TA properties
- `TA_STACK_SIZE` defines the RAM size to be reserved for TA stack
- `TA_DATA_SIZE` defines the RAM size to be reserved for TA heap (`TEE_Malloc()` pool)

Refer to *TA Properties* to understand how to configure these macros.

Hint: UUIDs can be generated using python

```
python -c 'import uuid; print(uuid.uuid4())'
```

or in most Linux systems using either

```
cat /proc/sys/kernel/random/uuid # Linux only
uuidgen # available from the util-linux package in most distributions
```

Example of a property header file

```

#ifndef USER_TA_HEADER_DEFINES_H
#define USER_TA_HEADER_DEFINES_H

#define TA_UUID
    { 0x8aaaf200, 0x2450, 0x11e4, \
      { 0xab, 0xe2, 0x00, 0x02, 0xa5, 0xd5, 0xc5, 0x1b} }

#define TA_FLAGS (TA_FLAG_EXEC_DDR | \
                  TA_FLAG_SINGLE_INSTANCE | \
                  TA_FLAG_MULTI_SESSION)
#define TA_STACK_SIZE (2 * 1024)

```

(continues on next page)

(continued from previous page)

```
#define TA_DATA_SIZE                (32 * 1024)

#define TA_CURRENT_TA_EXT_PROPERTIES \
    { "gp.ta.description", USER_TA_PROP_TYPE_STRING, "Foo TA for some purpose." }, \
    { "gp.ta.version", USER_TA_PROP_TYPE_U32, &(const uint32_t){ 0x0100 } }

#endif /* USER_TA_HEADER_DEFINES_H */
```

Note: It is recommended to use the TA_CURRENT_TA_EXT_PROPERTIES as above to define extra properties of the TA.

Note: Generating a fresh UUID with suitable formatting for the header file can be done using:

```
python -c "import uuid; u=uuid.uuid4(); print(u); \
n = [' ', 0x'] * 11; \
n[::2] = ['{:12x}'.format(u.node)[i:i + 2] for i in range(0, 12, 2)]; \
print('\n' + '#define TA_UUID\n\t{ ' + \
'0x{:08x}'.format(u.time_low) + ', ' + \
'0x{:04x}'.format(u.time_mid) + ', ' + \
'0x{:04x}'.format(u.time_hi_version) + ', \x5c\n\t\t{ ' + \
'0x{:02x}'.format(u.clock_seq_hi_variant) + ', ' + \
'0x{:02x}'.format(u.clock_seq_low) + ', ' + \
'0x' + ''.join(n) + '}' }'"
```

3.7.6 Checking TA parameters

GlobalPlatforms TEE Client APIs TEEC_InvokeCommand() and TEE_OpenSession() allow clients to invoke a TA with some invocation parameters: values or references to memory buffers. It is mandatory that TA's verify the parameters types before using the parameters themselves. For this a TA can rely on the macro TEE_PARAM_TYPE_GET(param_type, param_index) to get the type of a parameter and check its value according to the expected parameter.

For example, if a TA expects that command ID 0 comes with params[0] being a input value, params[1] being a output value, and params[2] being a in/out memory reference (buffer), then the TA should implemented the following sequence:

```
TEE_Result handle_command_0(void *session, uint32_t cmd_id,
                           uint32_t param_types, TEE_Param params[4])
{
    if ((TEE_PARAM_TYPE_GET(param_types, 0) != TEE_PARAM_TYPE_VALUE_IN) ||
        (TEE_PARAM_TYPE_GET(param_types, 1) != TEE_PARAM_TYPE_VALUE_OUT) ||
        (TEE_PARAM_TYPE_GET(param_types, 2) != TEE_PARAM_TYPE_MEMREF_INOUT) ||
        (TEE_PARAM_TYPE_GET(param_types, 3) != TEE_PARAM_TYPE_NONE)) {
        return TEE_ERROR_BAD_PARAMETERS
    }

    /* process command */
    ...
}
```

(continues on next page)

(continued from previous page)

```

TEE_Result TA_InvokeCommandEntryPoint(void *session, uint32_t command_id,
                                     uint32_t param_types, TEE_Param params[4])
{
    switch (command_id) {
        case 0:
            return handle_command_0(session, param_types, params);

        default:
            return TEE_ERROR_NOT_SUPPORTED;
    }
}

```

3.7.7 Identifying TA's client

The GP TEE specification is designed to ensure that TEE sessions are reliable once created. A TA instance can identify its client login method when a session is opened. A TA can use the client login credentials to establish or reject the session. A TA can get its client identity from property "gpd.client.identity" with the TEE Internal Core API function `TEE_GetPropertyAsIdentity()`:

```

TEE_Result TA_OpenSessionEntryPoint(uint32_t __unused param_types,
                                    TEE_Param __unused params[4],
                                    void **tee_session)
{
    TEE_Identity identity = { };
    TEE_Result res = TEE_SUCCESS;

    res = TEE_GetPropertyAsIdentity(TEE_PROPSSET_CURRENT_CLIENT,
                                   "gpd.client.identity", &identity);

    if (res)
        return res;

    switch (identity.login) {
        case TEE_LOGIN_PUBLIC:
            return login_public(&identity.uuid, tee_session);
        case TEE_LOGIN_USER:
            return login_user(&identity.uuid, tee_session);
        case TEE_LOGIN_GROUP:
            return login_group(&identity.uuid, tee_session);
        case TEE_LOGIN_REE_KERNEL:
            return login_kernel(&identity.uuid, tee_session);
        case TEE_LOGIN_TRUSTED_APP:
            return login_ta(&identity.uuid, tee_session);
        default:
            return TEE_ERROR_ACCESS_DENIED;
    }
}

```

The value of the UUID found in `identity.uuid` depends on the login method:

- When the client is a TA, `identity.login` is `TEE_LOGIN_TRUSTED_APP` and `identity.uuid` is the client TA UUID;

- When the non-secure client uses TEE_LOGIN_PUBLIC or TEE_LOGIN_REE_KERNEL method, the UUID is not used. By convention, Linux kernel and U-Boot both set nil UUID (all zeroes).
- When the non-secure client uses TEE_LOGIN_USER or TEE_LOGIN_GROUP method, the UUID is generated from the UUIDv5 namespace derivation of a user ID tag ("uid=%x") or a group ID tag ("gid=%x") in tee_client_uuid_ns namespace (below). The derivation is performed by the Linux kernel that verifies that the client's UID/GID is genuine, refer to tee_session_calc_client_uuid().

```
static const uuid_t tee_client_uuid_ns = UUID_INIT(0x58ac9ca0, 0x2086, 0x4683,
                                                    0xa1, 0xb8, 0xec, 0x4b,
                                                    0xc0, 0x8e, 0x01, 0xb6);
```

3.7.8 Signing of TAs

All *REE Filesystem Trusted Applications* need to be signed. The signature is verified by *optee_os* upon loading of the TA. Within the *optee_os* source is a directory `keys`. The public part of `keys/default_ta.pem` will be compiled into the *optee_os* binary and the signature of each TA will be verified against this key upon loading. Currently `keys/default_ta.pem` must contain an RSA key.

Warning: *optee_os* comes with a default **private** key in its source to facilitate easy development, testing, debugging and QA. Never deploy an *optee_os* binary with this key in production. Instead replace this key as soon as possible with a public key and keep the private part of the key offline, preferably on an HSM.

Note: Currently only a single key for signing TAs is supported by *optee_os*.

TAs are signed using the `sign_encrypt.py` script referenced from `ta/mk/ta_dev_kit.mk` in *optee_os*. Its default behaviour is to sign a compiled TA binary and attach the signature to form a complete TA for deployment. For **offline** signing, a three-step process is required: In a first step a digest of the compiled binary has to be generated, in the second step this digest is signed offline using the private key and finally in the third step the binary and its signature are stitched together into the full TA.

Offline Signing of TAs

There are two types of TAs that can be signed offline. The in-tree TAs, which come with the OP-TEE OS (for example the `pkcs11` TA) and are generated during the compilation of the TA DEV KIT. The second type are any external TAs coming from the user. In both cases however, the signing process is the same.

Offline signing is done with the following sequence of steps:

0. (Preparation) Generate a 2048 or 4096 bit RSA key for signing in a secure environment and extract the public key. For example

```
openssl genrsa -out rsa2048.pem 2048
openssl rsa -in rsa2048.pem -pubout -out rsa2048_pub.pem
```

1. Build the OP-TEE OS with the variable `TA_PUBLIC_KEY` set to the public key generated above

```
TA_PUBLIC_KEY=/path/to/public_key.pem make all
```

The build script will do two things:

- It will embed the `TA_PUBLIC_KEY` key into the OP-TEE core image, which will be used to

authenticate the TAs.

- It will generate *.stripped.elf* files of the in-tree TAs and sign them with the dummy key pointed to by `TA_SIGN_KEY`, thus creating *.ta* files. Note that the generated *.ta* files are not to be used as they are not compatible with the public key embedded into the OP-TEE core image.

2. Build any external TA. Same as with the in-tree TAs, the building procedure can use the dummy key pointed to by `TA_SIGN_KEY`, however they are not to be used due to the incompatibility reasons mentioned in the paragraph above.

There are now two ways to generate the final *.ta* files. Either re-sign the *.ta* files with a customized `sign_encrypt.py` script (left to the user to implement) or stitch the *.stripped.elf* files and their signatures together (explained in steps 3-5).

Export the previously generated custom keypair and the UUID of the TA. In this example the UUID of OP-TEE's `pkcs11` in-tree TA is used.

```
export TA_SIGN_KEY=rsa2048.pem
export TA_PUBLIC_KEY=rsa2048_pub.pem
export UUID=fd02c9da-306c-48c7-a49c-bbd827ae86ee
```

3. Manually generate a digest of the generated *.stripped.elf* files using

```
sign_encrypt.py digest --key $TA_PUBLIC_KEY --uuid $UUID \
--elf $UUID.stripped.elf --dig $UUID.dig
```

Note: It may be necessary to make use of the `--ta-version` argument here in some cases, e.g when building Widevine's `oemcrypto`. Check the make output of `optee-os` or the particular TAs and see if the version differs.

4. Sign this digest offline, for example with OpenSSL

```
base64 --decode $UUID.dig | \
openssl pkeyutl -sign -inkey $TA_SIGN_KEY \
-pkeyopt digest:sha256 -pkeyopt rsa_padding_mode:pss \
-pkeyopt rsa_pss_saltlen:digest -pkeyopt rsa_mgf1_md:sha256 | \
base64 > $UUID.sig
```

or using a Nitrokey HSM (assuming a working OpenSSL configuration for the PKCS11 engine is present)

```
base64 -d $UUID.dig | \
openssl pkeyutl -engine pkcs11 -keyform engine \
-sign -inkey "pkcs11:token=<my access token>;type=cert;object=<key label> or id=<key_
↳ id>" \
-pkeyopt digest:sha256 -pkeyopt rsa_padding_mode:pss \
-pkeyopt rsa_pss_saltlen:digest \
-pkeyopt rsa_mgf1_md:sha256 | \
base64 > $UUID.sig
```

When using an HSM, the public key must be extracted and set as `TA_PUBLIC_KEY`. `TA_SIGN_KEY` doesn't need to be set in this case, since it is stored in the HSM module.

5. Manually stitch the TA and signature together

```
sign_encrypt.py stitch --key $TA_PUBLIC_KEY --uuid $UUID \
--elf $UUID.stripped.elf --sig $UUID.sig --out $UUID.ta
```

Note: If the `--ta-version` flag was used in step 3., it needs to be used here as well.

By default, the UUID is taken as the base file name for all files. When signing directly inside the optee-os repository the `$UUID.sig`, `UUID.dig` and `$UUID.ta` arguments can be omitted. They were merely provided in this example for completeness. Consult `sign_encrypt.py --help` for a full list of options and parameters.

3.8 StandAloneMM

StandAloneMM is a PE/COFF binary produced by EDK2. For Arm platforms we can compile and use it, in combination with OP-TEE to store EFI variables in an RPMB partition of our eMMC.

3.8.1 EDK2 Build instructions

```
$ git clone https://github.com/tianocore/edk2.git
$ git clone https://github.com/tianocore/edk2-platforms.git
$ cd edk2
$ git submodule init && git submodule update --init --recursive
$ cd ..
$ export WORKSPACE=$(pwd)
$ export PACKAGES_PATH=$WORKSPACE/edk2:$WORKSPACE/edk2-platforms
$ export ACTIVE_PLATFORM="Platform/StandaloneMm/PlatformStandaloneMmPkg/
→ PlatformStandaloneMmRpmB.dsc"
$ export GCC5_AARCH64_PREFIX=aarch64-linux-gnu-
$ source edk2/edksetup.sh
$ make -C edk2/BaseTools
$ build -p $ACTIVE_PLATFORM -b RELEASE -a AARCH64 -t GCC5 -n `nproc`
```

3.8.2 OP-TEE Build instructions

```
$ git clone https://github.com/OP-TEE/optee_os.git
$ cd optee_os
$ ln -s ../Build/MmStandaloneRpmB/RELEASE_GCC5/FV/BL32_AP_MM.fd
$ export ARCH=arm
$ CROSS_COMPILE32=arm-linux-gnueabi- make -j32 CFG_ARM64_core=y \
  PLATFORM=<myboard> CFG_STMM_PATH=BL32_AP_MM.fd CFG_RPMB_FS=y \
  CFG_RPMB_FS_DEV_ID=0 CFG_CORE_HEAP_SIZE=524288 CFG_RPMB_WRITE_KEY=y \
  CFG_CORE_HEAP_SIZE=524288 CFG_CORE_DYN_SHM=y CFG_RPMB_TESTKEY=y \
  CFG_REE_FS=n CFG_CORE_ARM64_PA_BITS=48 CFG_TEE_CORE_LOG_LEVEL=1 \
  CFG_TEE_TA_LOG_LEVEL=1 CFG_SCTLR_ALIGNMENT_CHECK=n
```

Warning: Check `caveats` regarding `CFG_RPMB_WRITE_KEY` before enabling it

3.8.3 U-Boot Build instructions

Although the StandAloneMM binary comes from EDK2, using and storing the variables is currently available in U-Boot only.

```
$ git clone https://github.com/u-boot/u-boot.git
$ cd u-boot
$ export CROSS_COMPILE=aarch64-linux-gnu-
$ export ARCH=<arch>
$ make <myboard>_defconfig
$ make menuconfig
```

Enable CONFIG_OPTEE, CONFIG_CMD_OPTEE_RPMB and CONFIG_EFI_MM_COMM_TEE

```
$ make -j `nproc`
```

Warning:

- Your OP-TEE platform port must support Dynamic shared memory, since that's the only kind of memory U-Boot supports for now.

3.9 OP-TEE with Rust

This document describes how to build OP-TEE client and trusted applications written in [Rust](#) with [Teaclave TrustZone SDK](#).

3.9.1 Clone OP-TEE repo

Currently, Teaclave TrustZone SDK is compatible with QEMUv8 (aarch64).

Before building examples written with Teaclave TrustZone SDK, you should clone the OP-TEE repo first. For QEMUv8, run:

```
$ mkdir YOUR_OPTEE_DIR && cd YOUR_OPTEE_DIR
$ repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
$ repo sync
```

The source code of Teaclave TrustZone SDK is located in YOUR_OPTEE_DIR/optee_rust/ containing a set of examples written in Rust using the SDK.

For more information about building OP-TEE using QEMUv8, see [run OP-TEE using QEMU](#) .

3.9.2 Compile Rust examples

Rust example applications are located in `optee_rust/examples/`. To build and install them with Buildroot, run:

```
$ (cd build && make toolchains && make)
```

Note that the Rust applications are built by default; add `RUST_ENABLE=n` to disable them.

Then start QEMUv8:

```
$ (cd build && make run-only)
```

Hint: Note that if you are under the environment without GUI, you can use `soc_term` instead.

Access to normal world terminal:

```
$ ./build/soc_term.py 54320
```

Access to secure world terminal:

```
$ ./build/soc_term.py 54321
```

Run QEMU:

```
$ (cd build && make run-only)
```

To differentiate from client applications generated by `optee_examples`, OP-TEE Rust examples are not prefixed with `optee_example_` but suffixed with `-rs`. More description about Rust examples can be found in [Overview of OP-TEE Rust Examples](#).

During the build process, host applications are installed to `/usr/bin/` and TAs are installed to `/lib/optee_armtz/`. After QEMU boots up, you can run host applications in normal world terminal. For example:

```
$ hello_world-rs
original value is 29
inc value is 129
dec value is 29
Success
```

TA log will be printed correspondingly in the secure terminal.

3.9.3 Development Documents

More information about developing OP-TEE applications in Rust can be found in [Teaclave TrustZone SDK Documentation](#).

3.10 Linux userland integration

This document gives pointers on how particular features of OP-TEE may be used from the Linux userland in typical application scenarios.

3.10.1 PKCS#11 driver

A common use-case is the integration of OP-TEE to securely store asymmetric keys inside the secure enclave. For example, when using TLS with client certificates, the corresponding private keys would reside securely within OP-TEE. If this client certificate is then used from within userspace, the corresponding cryptographic primitives are relayed to OP-TEE which establishes the connection using the requested client certificate on behalf of the application. However, the key itself never leaves secure storage (this is where it is created and resides).

The way this is done is via PKCS#11 (aka Cryptoki API). PKCS#11 specifies a number of standard calls to relay cryptographic requests (such as a signing operation) to a third party module. Such a module may be a smart card or, in the case of OP-TEE, it is a software PKCS#11 trusted application that appears to the userland as one. This trusted application is accessed using a shared object (dynamic library) which serves as the “glue” to translate cryptographic requests into OP-TEE calls. This shared object is `libckteec.so` which is part of the [OP-TEE client tools](#).

Once OP-TEE has been compiled with the PKCS#11 TA, the client tools shared object has been built and the OP-TEE supplicant has been started, we can use `pkcs11-tool` of the [OpenSC project](#) to initiate first communication with the emulated smart card. In the following, we assume that `libckteec.so` has been installed in `/usr/lib/libckteec.so`. For simplicity reasons, we define an alias to call `pkcs11-tool` using the appropriate PKCS#11 module.

```
# alias p11="pkcs11-tool --module /usr/lib/libckteec.so"
# p11 --show-info
Cryptoki version 2.40
Manufacturer      Linaro
Library           OP-TEE PKCS11 Cryptoki library (ver 0.1)
Using slot 0 with a present token (0x0)
```

Hint: When testing OP-TEE under QEMU, OpenSC should be built by default as well and the `pkcs11-tool` should be available without modifications to the configuration. It can be explicitly requested by using the

```
make BR2_PACKAGE_OPENSC=y
```

build parameter when compiling OP-TEE.

This tells us the library code is already working. We can now display the different “slots”. You can think of them as different “card readers” for virtual smart cards. In a typical use case, only one slot is used for a single smart card.

```
# p11 --list-slots
Available slots:
Slot 0 (0x0): OP-TEE PKCS11 TA - TEE UUID 94e9ab89-4c43-56ea-8b35-45dc07226830
    token state:  uninitialized
Slot 1 (0x1): OP-TEE PKCS11 TA - TEE UUID 94e9ab89-4c43-56ea-8b35-45dc07226830
    token state:  uninitialized
Slot 2 (0x2): OP-TEE PKCS11 TA - TEE UUID 94e9ab89-4c43-56ea-8b35-45dc07226830
    token state:  uninitialized
```

Observe that the connection to the TA is also successfully working and it is showing three inserted (but “empty”, uninitialized) smart cards/tokens. Before we are able to create keys on these tokens, we need to initialize them with a

SO-PIN and PIN. The SO-PIN is the “super pin”, while the PIN is the “user pin”. The concept is likely familiar to you from the SIM card of your phone, where the PUK acts as the “super pin”.

First, we initialize the SO-PIN of slot 0 and name our token “mytoken”:

```
# p11 --init-token --label mytoken --so-pin 1234567890
Using slot 0 with a present token (0x0)
Token successfully initialized
```

We have successfully initialized the SO-PIN to “1234567890”. Now we “log in” into the token using that SO-PIN and, using the SO-PIN authorization, initialize the PIN of the token to “12345”:

```
# p11 --label mytoken --login --so-pin 1234567890 --init-pin --pin 12345
Using slot 0 with a present token (0x0)
User PIN successfully initialized
```

We can now verify that the token has been successfully initialized:

```
# p11 --list-slots
Available slots:
Slot 0 (0x0): OP-TEE PKCS11 TA - TEE UUID 94e9ab89-4c43-56ea-8b35-45dc07226830
  token label      : mytoken
  token manufacturer : Linaro
  token model      : OP-TEE TA
  token flags      : login required, rng, token initialized, PIN initialized
  hardware version  : 0.0
  firmware version  : 0.1
  serial num       : 000000000000000000
  pin min/max      : 4/128
Slot 1 (0x1): OP-TEE PKCS11 TA - TEE UUID 94e9ab89-4c43-56ea-8b35-45dc07226830
  token state:      uninitialized
Slot 2 (0x2): OP-TEE PKCS11 TA - TEE UUID 94e9ab89-4c43-56ea-8b35-45dc07226830
  token state:      uninitialized
```

Now we have a fully initialized token but it still contains no keys. To list what cryptographic primitives the particular OP-TEE version offers, you can query the supported mechanisms:

```
# p11 --list-mechanisms
Using slot 0 with a present token (0x0)
Supported mechanisms:
  SHA224-RSA-PKCS-PSS, keySize={256,4096}, sign, verify
  SHA224-RSA-PKCS, keySize={256,4096}, sign, verify
  SHA512-RSA-PKCS-PSS, keySize={256,4096}, sign, verify
  SHA384-RSA-PKCS-PSS, keySize={256,4096}, sign, verify
  SHA256-RSA-PKCS-PSS, keySize={256,4096}, sign, verify
  SHA512-RSA-PKCS, keySize={256,4096}, sign, verify
  SHA384-RSA-PKCS, keySize={256,4096}, sign, verify
  SHA256-RSA-PKCS, keySize={256,4096}, sign, verify
  SHA1-RSA-PKCS-PSS, keySize={256,4096}, sign, verify
  RSA-PKCS-OAEP, keySize={256,4096}, encrypt, decrypt
  SHA1-RSA-PKCS, keySize={256,4096}, sign, verify
  MD5-RSA-PKCS, keySize={256,4096}, sign, verify
  RSA-PKCS-PSS, sign, verify
  RSA-PKCS, keySize={256,4096}, encrypt, decrypt, sign, verify
  RSA-PKCS-KEY-PAIR-GEN, keySize={256,4096}, generate_key_pair
```

(continues on next page)

(continued from previous page)

```

ECDSA-SHA512, keySize={160,521}, sign, verify
ECDSA-SHA384, keySize={160,521}, sign, verify
ECDSA-SHA256, keySize={160,521}, sign, verify
ECDSA-SHA224, keySize={160,521}, sign, verify
ECDSA-SHA1, keySize={160,521}, sign, verify
ECDSA, keySize={160,521}, sign, verify
ECDSA-KEY-PAIR-GEN, keySize={160,521}, generate_key_pair
mechtype-0x272, keySize={32,128}, sign, verify
mechtype-0x262, keySize={32,128}, sign, verify
mechtype-0x252, keySize={24,128}, sign, verify
mechtype-0x257, keySize={14,64}, sign, verify
SHA-1-HMAC-GENERAL, keySize={10,64}, sign, verify
MD5-HMAC-GENERAL, keySize={8,64}, sign, verify
SHA512-HMAC, keySize={32,128}, sign, verify
SHA384-HMAC, keySize={32,128}, sign, verify
SHA256-HMAC, keySize={24,128}, sign, verify
SHA224-HMAC, keySize={14,64}, sign, verify
SHA-1-HMAC, keySize={10,64}, sign, verify
MD5-HMAC, keySize={8,64}, sign, verify
SHA512, digest
SHA384, digest
SHA256, digest
SHA224, digest
SHA-1, digest
MD5, digest
GENERIC-SECRET-KEY-GEN, keySize={1,4096}, generate
AES-KEY-GEN, keySize={16,32}, generate
AES-CBC-ENCRYPT-DATA, derive
AES-ECB-ENCRYPT-DATA, derive
mechtype-0x108B, keySize={16,32}, sign, verify
AES-CMAC, keySize={16,32}, sign, verify
mechtype-0x1089, keySize={16,32}, encrypt, decrypt
AES-CTR, keySize={16,32}, encrypt, decrypt
AES-CBC-PAD, keySize={16,32}, encrypt, decrypt
AES-CBC, keySize={16,32}, encrypt, decrypt, wrap, unwrap
AES-ECB, keySize={16,32}, encrypt, decrypt, wrap, unwrap

```

In our case, we would want to create an elliptic curve keypair on P-256 (aka secp256r1 or prime256v1). As you can see, this is supported (“ECDSA-KEY-PAIR-GEN” supports between 160 and 521 bit curves).

```

# p11 -l --pin 12345 --keypairgen --key-type EC:prime256v1 --label mykey
Using slot 0 with a present token (0x0)
Key pair generated:
Private Key Object; EC
  label:      mykey
  Usage:      sign, derive
  Access:     sensitive, always sensitive, never extractable, local
Public Key Object; EC  EC_POINT 256 bits
  EC_POINT:   044104e3f89bd32ac8101ba675815fbaf34c4f34bb7bb2d233589983bad934cfa09795d56811747778d22b94e245028d3af6a
  EC_PARAMS:  06082a8648ce3d030107
  label:      mykey

```

(continues on next page)

(continued from previous page)

```
Usage:      verify, derive
Access:     local
```

You can see the public key, which is a point on the elliptic curve. The byte 04 at byte offset 2 indicates that this point is represented in uncompressed affine representation, i.e., X and Y coordinates follow that byte directly. This format is not ideal to interface common libraries, however. Especially when using PKI with X.509 certificates, we typically want a PEM-formatted CSR to be able to create a certificate from.

For this, we create a small configuration file for OpenSSL and call it `optee_hsm.conf`. It references a library of `libp11` which acts as a driver that enables OpenSSL to interface with a PKCS#11 library.

```
openssl_conf = openssl_conf

[openssl_conf]
engines = engine_section

[engine_section]
pkcs11 = pkcs11_section

[pkcs11_section]
engine_id = pkcs11
dynamic_path = /usr/lib/engines-1.1/libpkcs11.so
MODULE_PATH = /usr/lib/libckteec.so
PIN = 12345

[req]
distinguished_name = req_distinguished_name

[req_distinguished_name]
```

Hint: When testing OP-TEE under QEMU, `libp11` is not compiled by default. For easy access to this library, you can build OP-TEE using the command

```
make BR2_PACKAGE_OPENSC=y BR2_PACKAGE_LIBOPENSSL=y BR2_PACKAGE_LIBOPENSSL_BIN=y BR2_
↳ PACKAGE_LIBP11=y
```

This will ensure that OpenSC (for the command line utility `pkcs11-tool`), OpenSSL, and `libp11` are all built and installed in the QEMU environment. Note that in that environment, `libpkcs11.so` will reside at `/usr/lib/engines-1.1/libpkcs11.so`.

Then, we can ask OpenSSL to create a CSR from the key we have previously created:

```
# OPENSSL_CONF=optee_hsm.conf openssl req -new -engine pkcs11 -keyform engine -key label_
↳ mykey -subj "/CN=My CSR" -out mykey_csr.pem
engine "pkcs11" set.
```

We can then inspect said CSR:

```
$ openssl req -in mykey_csr.pem -text
Certificate Request:
  Data:
    Version: 1 (0x0)
```

(continues on next page)

(continued from previous page)

```

Subject: CN = My CSR
Subject Public Key Info:
  Public Key Algorithm: id-ecPublicKey
  Public-Key: (256 bit)
  pub:
    04:e3:f8:9b:d3:2a:c8:10:1b:a6:75:81:5f:ba:f3:
    4c:4f:34:bb:7b:b2:d2:33:58:99:83:ba:d9:34:cf:
    a0:97:95:d5:68:11:74:77:78:d2:2b:94:e2:45:02:
    8d:3a:f6:af:f9:e6:ab:bb:db:3a:75:fe:14:33:18:
    2c:60:58:68:c7
  ASN1 OID: prime256v1
  NIST CURVE: P-256
Attributes:
  a0:00
Signature Algorithm: ecdsa-with-SHA256
  30:45:02:20:61:7e:05:30:cf:4d:d0:93:22:78:9e:45:cf:af:
  3c:83:bb:04:c4:f0:81:f6:9a:5c:97:cd:ac:1e:94:cd:17:1b:
  02:21:00:e7:7f:88:1d:4f:56:b8:e2:87:be:76:de:28:b3:92:
  68:a7:16:3a:56:af:79:2f:98:bd:fd:6d:b3:82:e1:15:6c

```

Note that the public key matches exactly that which we have previously created (04 e3 f8...). This CSR could then be signed by a CA. For simplicity purposes, we can also use a self-signed certificate and sign with our own OP-TEE contained key:

```

# OPENSSL_CONF=optee_hsm.conf openssl req -new -engine pkcs11 -keyform engine -key label_
mykey -subj "/CN=My CSR" -x509 -out mykey_selfsigned_cert.pem
engine "pkcs11" set.

```

Again we can review this self-signed certificate:

```

$ openssl x509 -in mykey_selfsigned_cert.pem -text
Certificate:
  Data:
    Version: 1 (0x0)
    Serial Number:
      3f:8f:c8:c0:de:a8:75:ca:9d:62:79:31:c2:6c:48:f4:fd:50:22:1d
    Signature Algorithm: ecdsa-with-SHA256
    Issuer: CN = My CSR
    Validity
      Not Before: Mar 22 20:19:15 2023 GMT
      Not After : Apr 21 20:19:15 2023 GMT
    Subject: CN = My CSR
    Subject Public Key Info:
      Public Key Algorithm: id-ecPublicKey
      Public-Key: (256 bit)
      pub:
        04:e3:f8:9b:d3:2a:c8:10:1b:a6:75:81:5f:ba:f3:
        4c:4f:34:bb:7b:b2:d2:33:58:99:83:ba:d9:34:cf:
        a0:97:95:d5:68:11:74:77:78:d2:2b:94:e2:45:02:
        8d:3a:f6:af:f9:e6:ab:bb:db:3a:75:fe:14:33:18:
        2c:60:58:68:c7
      ASN1 OID: prime256v1

```

(continues on next page)

(continued from previous page)

```

NIST CURVE: P-256
Signature Algorithm: ecdsa-with-SHA256
30:45:02:20:4a:9d:63:f2:e0:12:4b:46:eb:eb:62:34:9e:86:
3d:d4:c8:cf:5f:c0:44:fe:8b:71:a0:b8:fa:41:d9:0b:60:3a:
02:21:00:fb:c2:b3:0a:7b:54:e9:bb:66:7b:8e:f7:11:52:81:
69:81:a6:cc:d0:bf:a2:7c:f7:2a:67:db:ab:f1:f3:2c:9f
-----BEGIN CERTIFICATE-----
MIIBHDCBwwIUP4/IwN6odcqdYnkxwmXI9P1QIh0wCgYIKoZIzj0EAwIwETEPMA0G
A1UEAwGTXkgQ1NSMB4XDTIzMDMyMjIwMTkxNVoXDTIzMDQyMTIwMTkxNVowETEP
MA0GA1UEAwGTXkgQ1NSMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE4/ib0yrI
EBumdYFfuvNMTzS7e7LSM1iZg7rZNM+gl5XVaBF0d3jSK5TiRQKN0vav+earu9s6
df4UMxgsYFhoxzAKBggqhkJOPQQDAgNIADBFAiBKnpPy4BJLRuvrYjSehj3UyM9f
wET+i3GguPpB2QtgOgIhAPvCswp7V0m7Znu09xFSgWmBpszQv6J89ypn26vx8yyf
-----END CERTIFICATE-----

```

To test our self-signed certificate as a client certificate, we first need to initialize a TLS server. This can either be done on a remote machine or locally. For the server we will again use a self-signed certificate (but simply store the corresponding private key in a file).

```

$ openssl ecparam -genkey -name prime256v1 -out server_key.pem
$ openssl req -new -x509 -key server_key.pem -subj '/CN=Server' -out server_cert.pem
$ openssl s_server -accept 9876 -cert server_cert.pem -key server_key.pem -www -Verify 1
verify depth is 1, must return a certificate
Using default temp DH parameters
ACCEPT

```

This starts a HTTPS server which listens at port 9876 and requires a TLS client certificate. We can validate that the connection to the server is refused if no client certificate is provided. Assume that 192.168.178.34 is the IPv4 address of the server:

```

$ curl -k https://192.168.178.34:9876
curl: (56) OpenSSL SSL_read: error:0A00045C:SSL routines::tlsv13 alert certificate_
required, errno 0

```

Now on our OP-TEE device we can use OpenSSL to establish a connection using our OP-TEE stored client certificate:

```

# OPENSSL_CONF=optee_hsm.conf openssl s_client -engine pkcs11 -connect 192.168.178.
34:9876 -cert mykey_selfsigned_cert.pem -keyform engine -key label_mykey
engine "pkcs11" set.
CONNECTED(000000004)
Can't use SSL_get_servername
depth=0 CN = Server
verify error:num=18:self signed certificate
verify return:1
depth=0 CN = Server
verify return:1
---
Certificate chain
0 s:CN = Server
i:CN = Server
---
Server certificate
-----BEGIN CERTIFICATE-----

```

(continues on next page)

(continued from previous page)

```

MIIBeDCCAR2gAwIBAgIUdNuzOcNS9AgeJhvVmp73wF5DwxQwCgYIKoZIzj0EAwIw
ETEPMA0GA1UEAwGU2VydMvYMB4XDTIzMDMyMjIwMjMwMloXDTIzMDQyMTIwMjMw
MlowETEPMA0GA1UEAwGU2VydMvYMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE
qnCvLjLa1XWBtY10QjaHa60re5vnZ2WY55XSsFCe2RoF7wGBDDrdXKkQz9Vy0t4
d5OC6VMcFhia967nGa5zPqNTMFEwHQYDVR00BBYEFBKTMLG057a/a2exmeF7dHVH
85D0MB8GA1UdIwQYMBaAFBKTMLG057a/a2exmeF7dHVH85D0MA8GA1UdEwEB/wQF
MAMBAf8wCgYIKoZIzj0EAwIDSQAwwRgIhAIEKwlgHskhA8zvpXsl9y6WSCXo9fRzt
DSl6myUsgac/AiEAhipKSjVQAvJAqXIecmMylqjY79XVzrbxKWYjsL1XdLw=
-----END CERTIFICATE-----
subject=CN = Server

issuer=CN = Server

---
No client certificate CA names sent
Requested Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA-
PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-
PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224
Shared Requested Signature Algorithms:
ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:Ed448:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-
PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-
PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512
Peer signing digest: SHA256
Peer signature type: ECDSA
Server Temp Key: X25519, 253 bits

---
SSL handshake has read 817 bytes and written 797 bytes
Verification error: self signed certificate

---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 256 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 18 (self signed certificate)
[...]

```

When connected, you can type “GET /” and press return to get a HTML response back from the HTTPS server, which will echo your client certificate inside a HTML page.

DEBUGGING TECHNIQUES

4.1 Abort dumps / call stack

When OP-TEE encounters a serious error condition, it prints diagnostic information to the secure console. The message contains a call stack if `CFG_UNWIND=y` (enabled by default).

The following errors will trigger a dump:

- Data or prefetch abort exception in the TEE core (kernel mode) or in a TA (user mode),
- When a user-mode Trusted Application panics, either by calling `TEE_Panic()` directly or due to some error detected by the TEE Core Internal API,
- When the TEE core detects a fatal error and decides to hang the system because there is no way to proceed safely (core panic).

The messages look slightly different depending on:

- Whether the error is an exception or a panic,
- The exception/privilege level when the exception occurred (PL0/EL0 if a user mode Trusted Application was running, PL1/EL1 if it was the TEE core),
- Whether the TEE and TA are 32 or 64 bits,
- The exact type of exception (data or prefetch abort, translation fault, read or write permission fault, alignment errors etc).

Here is an example of a panic in a 32-bit Trusted Application, running on a 32-bit TEE core (QEMU):

```
E/TC:0 TA panicked with code 0x0
E/TC:0 Status of TA 484d4143-2d53-4841-3120-4a6f636b6542 (0xe07ba50) (active)
E/TC:0 arch: arm load address: 0x101000 ctx-idr: 1
E/TC:0 stack: 0x100000 4096
E/TC:0 region 0: va 0x100000 pa 0xe31d000 size 0x1000 flags rw-
E/TC:0 region 1: va 0x101000 pa 0xe300000 size 0xf000 flags r-x
E/TC:0 region 2: va 0x110000 pa 0xe30f000 size 0x3000 flags r--
E/TC:0 region 3: va 0x113000 pa 0xe312000 size 0xb000 flags rw-
E/TC:0 region 4: va 0 pa 0 size 0 flags ---
E/TC:0 region 5: va 0 pa 0 size 0 flags ---
E/TC:0 region 6: va 0 pa 0 size 0 flags ---
E/TC:0 region 7: va 0 pa 0 size 0 flags ---
E/TC:0 Call stack:
E/TC:0 0x001044a8
E/TC:0 0x0010ba59
```

(continues on next page)

(continued from previous page)

```

E/TC:0 0x00101093
E/TC:0 0x001013ed
E/TC:0 0x00101545
E/TC:0 0x0010441b
E/TC:0 0x00104477
D/TC:0 user_ta_enter:452 tee_user_ta_enter: TA panicked with code 0x0
D/TC:0 tee_ta_invoke_command:649 Error: ffff3024 of 3
D/TC:0 tee_ta_close_session:402 tee_ta_close_session(0xe07be98)
D/TC:0 tee_ta_close_session:421 Destroy session
D/TC:0 tee_ta_close_session:447 Destroy TA ctx

```

The above dump was triggered by the TA when entering an irrecoverable error ending up in a TEE_Panic(0) call.

OP-TEE provides a helper script called `symbolize.py` to facilitate the analysis of such issues. It is located in the OP-TEE OS source tree in `scripts/symbolize.py` and is also copied to the TA development kit. Whenever you are confronted with an error message reporting a serious error and containing a "Call stack:" line, you may use the `symbolize` script. It is meant to be run on the host system (build environment), not on the target.

`symbolize.py` reads its input from `stdin` and writes extended debug information to `stdout`. The `-d` (directories) option tells the script where to look for TA ELF file(s) (`<uuid>.stripped.elf`) or for `tee.elf` (the TEE core). Please refer to `symbolize.py --help` for details.

Typical output:

```

$ cat dump.txt | ./optee_os/scripts/symbolize.py -d ./optee_examples/*/ta
# (or run the script, copy and paste the dump, then press Ctrl+D)
E/TC:0 TA panicked with code 0x0
E/TC:0 Status of TA 484d4143-2d53-4841-3120-4a6f636b6542 (0xe07ba50) (active)
E/TC:0 arch: arm load address: 0x101000 ctx-idr: 1
E/TC:0 stack: 0x100000 4096
E/TC:0 region 0: va 0x100000 pa 0xe31d000 size 0x1000 flags rw-
E/TC:0 region 1: va 0x101000 pa 0xe300000 size 0xf000 flags r-x .ta_head .text .rodata
E/TC:0 region 2: va 0x110000 pa 0xe30f000 size 0x3000 flags r-- .rodata .ARM.extab .ARM.
↳ extab.text.uttee_panic .ARM.extab.text.__aeabi_ldivmod .ARM.extab.text.__aeabi_uldivmod
↳ .ARM.exidx .got .dynsym .rel.got .dynamic .dynstr .hash .rel.dyn
E/TC:0 region 3: va 0x113000 pa 0xe312000 size 0xb000 flags rw- .data .bss
E/TC:0 region 4: va 0 pa 0 size 0 flags ---
E/TC:0 region 5: va 0 pa 0 size 0 flags ---
E/TC:0 region 6: va 0 pa 0 size 0 flags ---
E/TC:0 region 7: va 0 pa 0 size 0 flags ---
E/TC:0 Call stack:
E/TC:0 0x001044a8 uttee_panic at optee_os/lib/libutee/arch/arm/utee_syscalls_a32.S:74
E/TC:0 0x0010ba59 TEE_Panic at optee_os/lib/libutee/tee_api_panic.c:35
E/TC:0 0x00101093 hmac_shal at optee_examples/hotp/ta/hotp_ta.c:63
E/TC:0 0x001013ed get_hotp at optee_examples/hotp/ta/hotp_ta.c:171
E/TC:0 0x00101545 TA_InvokeCommandEntryPoint at optee_examples/hotp/ta/hotp_ta.c:225
E/TC:0 0x0010441b entry_invoke_command at optee_os/lib/libutee/arch/arm/user_ta_entry.
↳ c:207
E/TC:0 0x00104477 __utee_entry at optee_os/lib/libutee/arch/arm/user_ta_entry.c:235
D/TC:0 user_ta_enter:452 tee_user_ta_enter: TA panicked with code 0x0 ???
D/TC:0 tee_ta_invoke_command:649 Error: ffff3024 of 3
D/TC:0 tee_ta_close_session:402 tee_ta_close_session(0xe07be98)
D/TC:0 tee_ta_close_session:421 Destroy session
D/TC:0 tee_ta_close_session:447 Destroy TA ctx

```

The Python script uses several tools from the GNU Binutils package to perform the following tasks:

1. Translate the call stack addresses into function names, file names and line numbers.
2. Convert the abort address to a symbol plus some offset and/or an ELF section name plus some offset.
3. Print the names of the ELF sections contained in each memory region of a TA.

Note that to successfully run `symbolize.py` you must also make your toolchain visible on the PATH (i.e., `export PATH=<my-toolchain-path>/bin:$PATH`).

4.2 Ftrace (function tracing)

This section describes how to generate a function call graph for user Trusted Applications using `ftrace`. The name comes from the Linux framework which has a similar purpose, but the OP-TEE `ftrace` is very much specific.

A call graph logs all the calls to instrumented functions and contains timing information. It is therefore a valuable tool to troubleshoot performance problems or to optimize the code in general.

The configuration option `CFG_FTRACE_SUPPORT=y` enables OP-TEE to collect function graph information from Trusted Applications running in user mode and compiled with `-pg`. Once collected, the function graph data is sent to `tee-supplciant` via RPC, so they can be saved to disk, later processed and displayed using helper scripts (`ftrace_format.py` and `symbolize.py` which can be found in `optee_os/scripts`).

Another configuration option `CFG_SYSCALL_FTRACE=y` in addition to `CFG_FTRACE_SUPPORT=y` enables OP-TEE to collect function graph information for syscalls as well while running in kernel mode on behalf of Trusted Applications. Note that a small number of kernel functions cannot be traced; they have the `__noprof` attribute in the source code.

A third configuration option `CFG_ULIBS_MCOUNT=y` enables tracing of user space libraries contained in `optee_os` and used by TAs (such as `libutee` and `libutils`).

4.2.1 Usage

- Build OP-TEE OS with `CFG_FTRACE_SUPPORT=y` and optionally `CFG_ULIBS_MCOUNT=y` and `CFG_SYSCALL_FTRACE=y`.
- Build user TAs with `-pg`, for instance enable `CFG_TA_MCOUNT=y` to instrument the whole TA. Also, in case user wants to set `-pg` for a particular file, following should go in corresponding `sub.mk`: `cflags-<file-name>-y+=-pg`. Note that instrumented TAs have a larger `.bss` section. The memory overhead depends on `CFG_FTRACE_BUF_SIZE` macro which can be configured specific to user TAs using config: `CFG_FTRACE_BUF_SIZE=4096` (default value: 2048, refer to the TA linker script for details: `ta/arch/${ARCH}/ta.ld.S`).
- Run the application normally. When the current session exits or there is any abort during TA execution, `tee-supplciant` will write function graph data to `/tmp/ftrace-<ta_uuid>.out`. If the file already exists, a number is appended, such as: `ftrace-<ta_uuid>.1.out`.
- Run helper scripts called `ftrace_format.py` to translate the function graph binary data into human readable text and `symbolize.py` to convert function addresses into function names: `optee_os/scripts/ftrace_format.py ftrace-<ta_uuid>.out | optee_os/scripts/symbolize.py -d <ta_uuid>.elf -d tee.elf`
- Refer to [commit 5c2c0fb31efb](#) for a full usage example on QEMU.

4.2.2 Typical output

```

TEE load address @ 0x5ab04000
Function graph for TA: cb3e5ba0-adf1-11e0-998b-0002a5d5c51b @ 80085000
    | 1 | __ta_entry() {
    | 2 |   __utee_entry() {
43.840 us | 3 |     ta_header_get_session()
 7.216 us | 3 |     tahead_get_trace_level()
14.480 us | 3 |     trace_set_level()
    | 3 |     malloc_add_pool() {
    | 4 |       raw_malloc_add_pool() {
46.032 us | 5 |         bpool()
    | 5 |         raw_realloc() {
166.256 us | 6 |           bget()
 23.056 us | 6 |           raw_malloc_return_hook()
267.952 us | 5 |         }
398.720 us | 4 |       }
426.992 us | 3 |     }
    | 3 |     TEE_GetPropertyAsU32() {
 23.600 us | 4 |       is_propset_pseudo_handle()
    | 4 |       __utee_check_instring_annotation() {
 26.416 us | 5 |         strlen()
    | 5 |         check_access() {
    | 6 |           TEE_CheckMemoryAccessRights() {
    | 7 |             _utee_check_access_rights() {
    | 8 |               syscall_check_access_rights() {
    | 9 |                 ts_get_current_session() {
 4.304 us |10 |                 ts_get_current_session_may_fail()
10.976 us | 9 |               }
    | 9 |             to_user_ta_ctx() {
 2.496 us |10 |               is_user_ta_ctx()
 8.096 us | 9 |             }
    | 9 |           vm_check_access_rights() {
    |10 |             vm_buf_is_inside_um_private() {
    |11 |               core_is_buffer_inside() {
...

```

The duration (function's time of execution) is displayed on the closing bracket line of a function or on the same line in case the function is the leaf one. In other words, duration is displayed whenever an instrumented function returns. It comprises the time spent executing the function and any of its callees. The Counter-timer Physical Count register (CNTPCT) and the Counter-timer Frequency register (CNTFRQ) are used to compute durations. Time spent servicing foreign interrupts is subtracted.

The second column is the stack depth for the current function. It helps visually match function entries and exit.

4.3 Gprof

This describes to do profiling of user Trusted Applications with `gprof`.

The configuration option `CFG_TA_GPROF_SUPPORT=y` enables OP-TEE to collect profiling information from Trusted Applications running in user mode and compiled with `-pg`. Once collected, the profiling data are formatted in the `gmon.out` format and sent to `tee-suppllicant` via RPC, so they can be saved to disk and later processed and displayed by the standard `gprof` tool.

4.3.1 Usage

- Build OP-TEE OS with `CFG_TA_GPROF_SUPPORT=y`. You may also set `CFG_ULIBS_MCOUNT=y` to instrument the user TA libraries contained in `optee_os` (such as `libutee` and `libutils`).
- Build user TAs with `-pg`, for instance enable: `CFG_TA_MCOUNT=y` to instrument whole user TA. Note that instrumented TAs have a larger `.bss` section. The memory overhead is 1.36 times the `.text` size for 32-bit TAs, and 1.77 times for 64-bit ones (refer to the TA linker script for details: `ta/arch/arm/ta.ld.S`).
- Run the application normally. When the last session exits, `tee-suppllicant` will write profiling data to `/tmp/gmon-<ta_uuid>.out`. If the file already exists, a number is appended, such as: `gmon-<ta_uuid>.1.out`.
- Run `gprof` on the TA ELF file and profiling output: `gprof <ta_uuid>.elf gmon-<ta_uuid>.out`

4.3.2 Implementation

Part of the profiling is implemented in `libutee`. Another part is done in the TEE core by a pseudo-TA (`core/arch/arm/sta/gprof.c`). Two types of data are collected:

1. Call graph information

- When TA source files are compiled with the `-pg` switch, the compiler generates extra code into each function prologue to call the instrumentation entry point (`__gnu_mcount_nc` or `_mcount` depending on the architecture). Each time an instrumented function is called, `libutee` records a pair of program counters (one is the caller and the other one is the callee) as well as the number of times this specific arc of the call graph has been invoked.

2. PC distribution over time

- When an instrumented TA starts, `libutee` calls the pseudo-TA to start PC sampling for the current session. Sampling data are written into the user-space buffer directly by the TEE core.
- Whenever the TA execution is interrupted, the TEE core records the current program counter value and builds a histogram of program locations (i.e., relative amount of time spent for each value of the PC). This is later used by the `gprof` tool to derive the time spent in each function. The sampling rate, which is assumed to be roughly constant, is computed by keeping track of the time spent executing user TA code and dividing the number of interrupts by the total time.
- The profiling buffer into which call graph and sampling data are recorded is allocated in the TA's `.bss` section. Some space is reserved by the linker script, only when the TA is instrumented.

FREQUENTLY ASKED QUESTIONS

Table of Contents

- *Frequently Asked Questions*
 - *Abbreviations*
 - *Architecture*
 - * *Q: Which platforms/architectures are supported?*
 - * *Q: Are 32-bit as well as 64-bit support?*
 - * *Q: Does OP-TEE support mixed-mode, i.e., both AArch32 and AArch64 Trusted Applications on top of an AArch64 core?*
 - * *Q: What's the maximum size for heap and stack? Can it be changed?*
 - * *Q: What is the size of OP-TEE itself?*
 - * *Q: Can NEON optimizations be done in OP-TEE?*
 - * *Q: Can I use C++ libraries in OP-TEE?*
 - * *Q: Would using malloc() in OP-TEE give physically contiguous memory?*
 - * *Q: Can I limit what CPUs / cores OP-TEE runs on?*
 - * *Q: How is OP-TEE being scheduled?*
 - *Board support*
 - * *Q: How do I port OP-TEE to another platform?*
 - *Building*
 - * *Q: I got build errors running latest, why?*
 - * *Q: I got build errors running stable tag x.y.z, why?*
 - * *Q: I get gcc XYZ or g++ XYZ compiler error messages?*
 - * *Q: I found this build.git, what is that?*
 - * *Q: When running make from build.git it fails to download the toolchains?*
 - * *Q: How can I build LLVM compiler-rt with BTI enabled ?*
 - * *Q: How can I build GCC with BTI enabled?*
 - * *Q: What is the quickest and easiest way to try OP-TEE?*

- *Certification and security reviews*
 - * *Q: Will TrustedFirmware.org be involved in GlobalPlatform certification/qualification?*
 - * *Q: Has any test lab been testing OP-TEE?*
 - * *Q: Where are listed security vulnerabilities addressed in OP-TEE*
 - * *Q: Have there been any code audit / code review done?*
- *Contribution*
 - * *Q: How do I contribute?*
 - * *Q: Where can I get help?*
 - * *Q: I'm new to OP-TEE but I would like to help out, what can I do?*
- *Interfaces*
 - * *Q: Which API's have been implemented in OP-TEE?*
 - * *Q: Which Linux kernel version supports <some OP-TEE feature>?*
- *Hardware and peripherals*
 - * *Q: Can I use my own hardware IP for crypto acceleration?*
- *License*
 - * *Q: Under what license is OP-TEE released?*
 - * *Q: GlobalPlatform click-through license*
 - * *Q: I've modified OP-TEE by using code with non BSD 2-Clause license, will you accept it?*
- *Promotion*
 - * *Q: I want to get my company logo on op-tee.org, how?*
- *Security vulnerabilities*
 - * *Q: I have found a security flaw in OP-TEE, how can I disclose it with you?*
- *Source code*
 - * *Q: Where is the source code?*
 - * *Q: Where do I download the test suite called xtest?*
 - * *Q: Where is the Linux kernel TEE driver?*
- *Testing*
 - * *Q: How are you testing OP-TEE?*
- *Trusted Applications*
 - * *Q: How do I write a Trusted Application (TA)?*
 - * *Q: How do I link a library into a Trusted Application?*
 - * *Q: Where should I put my compiled Trusted Application on the device?*
 - * *Q: What is a Pseudo TA and how do I write one?*
 - * *Q: Are Pseudo **user space** TAs supported?*
 - * *Q: Can a static TA Open/Invoke dynamic TA?*

- * *Q: How can I extend the GlobalPlatform Internal Core API?*
 - * *Q: How are Trusted Applications verified?*
 - * *Q: Is multi-core TA supported?*
 - * *Q: Is multi-threading supported in a TA?*
 - * *Q: How can I use or call OP-TEE from native Android (apk) applications?*
 - * *Q: I've heard that there is a Widevine and PlayReady TA, how do I get access?*
-

5.1 Abbreviations

OP-TEE

Open Portable TEE

TA

Trusted Application

TEE

Trusted Execution Environment

TZASC

TrustZone Address Space Controller

TZPC

TrustZone Protection Controller

5.2 Architecture

5.2.1 Q: Which platforms/architectures are supported?

- The *Platforms supported* page lists all platforms and architectures currently supported in the official tree.

5.2.2 Q: Are 32-bit as well as 64-bit support?

- Both 32- and 64-bit are fully supported for all OP-TEE components.

5.2.3 Q: Does OP-TEE support mixed-mode, i.e., both AArch32 and AArch64 Trusted Applications on top of an AArch64 core?

- Yes!

5.2.4 Q: What's the maximum size for heap and stack? Can it be changed?

- Yes, it can be changed. In the current setup (for vexpress for example), there are 32MB DDR dedicated for OP-TEE. 1MB for TEE RAM and 1MB for PUB RAM, this leaves 30MB for Trusted Applications. In the Trusted Applications, you set `TA_STACK_SIZE` and `TA_DATA_SIZE`. Typically, we set stack to 2KB and data to 32K. But you are free to adjust those according to the amount of memory you have available. If you need them to be bigger than 1MB then you also must adjust TA's MMU L1 table accordingly, since default section mapping is 1MB.

5.2.5 Q: What is the size of OP-TEE itself?

- As of 2016.01, `optee_os` is about 244KB (release build). It is preferred to run *optee_os* entirely in SRAM, but if there is not enough room, DRAM can be used and protected with TZASC. We are also looking into the possibility of creating a 'minimal' OP-TEE, i.e. a limited OP-TEE usable even in a very memory constrained environment, by eliminating as many memory-hungry parts as possible. There is however no ETA for this at the moment.
- You can check the memory usage by using the `make mem_usage` target in *optee_os*, for example:

```
$ make ... mem_usage
# Which will output a file with the figures here:
# out/arm/core/tee.mem_usage
```

You will of course get different sizes depending on what compile time flags you have enabled when running *make mem_usage*.

5.2.6 Q: Can NEON optimizations be done in OP-TEE?

- Yes (for additional information, please also see [Issue#953](#))

5.2.7 Q: Can I use C++ libraries in OP-TEE?

- C++ libraries are currently not supported. Technically, it is possible but will require a fair amount of work to implement, especially more so if exceptions are required. There are currently no plans to do this.
- See [Issue#2628](#) for related information.

5.2.8 Q: Would using *malloc()* in OP-TEE give physically contiguous memory?

- `malloc()` in OP-TEE currently gives physically contiguous memory. It is not guaranteed as it is not mentioned anywhere in the documentation, but in practice the heap only has physically contiguous memory in the pool(s). The heap in OP-TEE is normally quite small, ~24KiB, and could be a bit fragmented.

5.2.9 Q: Can I limit what CPUs / cores OP-TEE runs on?

- Currently it's up to the kernel to decide which core it runs on, i.e. it will be the same core as the one initiating the SMC in Linux. Please also see [Issue#1194](#).

5.2.10 Q: How is OP-TEE being scheduled?

- OP-TEE does not have its own scheduler, instead it is being scheduled by Linux kernel. For more information, please see [Issue#1036](#) and [Issue#1183](#).
-

5.3 Board support

5.3.1 Q: How do I port OP-TEE to another platform?

- Start by reading the [Porting guidelines](#).
 - See the [Presentations](#) page. There might be some interesting information in the “LCU14-302 How To Port OP-TEE To Another Platform” deck and video. Beware that the presentation is more than five years old, so even though it is a good source, there might be parts that are not relevant any longer.
 - As a good example for
 - **Armv8-A** patch enabling OP-TEE support on a new device, please see the [ZynqMP port](#) that enabled support for running OP-TEE on *Xilinx UltraScale+ Zynq MPSoC*. Besides that there are similar patches for [Juno port](#), [Raspberry Pi3 port](#), [HiKey port](#).
 - **ARMv7-A**, please have a look at the [Freescale ls1021a port](#), another example would be the [TI DRA7xx port](#).
-

5.4 Building

5.4.1 Q: I got build errors running latest, why?

- What did you try to build? Only [optee_os](#)? A full OP-TEE developer setup using QEMU, HiKey, RPi3, Juno using repo? AOSP? OpenEmbedded? What we build on daily basis are the OP-TEE developer setups (see [Platforms supported by build.git](#)), but other builds like AOSP and OpenEmbedded are builds that we try from time to time, but we have no CI/regression testing configured for those builds.
- By running latest instead of stable also comes with a risk of getting build errors due to version and/or interdependency skew which can result in build error. Now, such issues most often affects running xtest and not the building. If you however clean all gits and do a `repo sync -d`. Then we're almost 100% sure you will get back to a working state again, since as mentioned in next bullet, we build (and run xtest) on all QEMU on all patches sent to OP-TEE.
- Every pull request in OP-TEE are tested on hardware (see [Q: How are you testing OP-TEE?](#)).

5.4.2 Q: I got build errors running stable tag x.y.z, why?

- Stable releases are quite well tested both in terms of building for all supported platforms and running xtest on all platforms, so if you cannot get that to build and run, then there is a great chance you have something wrong on your side. All platforms that has been tested on a stable release can be found in [CHANGELOG.md](#) file. Having that said, we do make mistakes on stable builds also from time to time.

5.4.3 Q: I get gcc XYZ or g++ XYZ compiler error messages?

- Most likely you're trying to build OP-TEE using the regular x86 compiler and not the using the Arm toolchain. Please install the [Prerequisites](#) and make sure you have gotten and installed the Arm toolchains as described at the [Toolchains](#) page. (for additional information, please see [Issue#846](#)).

5.4.4 Q: I found this build.git, what is that?

- *build* is a git that is used in conjunction with the *manifest* to create full OP-TEE developer builds. It contains helper makefiles that makes it easy to get OP-TEE up and running on the setups that are using repo.

5.4.5 Q: When running make from build.git it fails to download the toolchains?

- We try to stay somewhat up to date with running recent GCC versions. But just like everywhere else on the net things moves around. In some cases like [Issue#1195](#), the URL was changed without us noticing it. If you find and fix such an issue, please send the fix as pull request and we will be happy to merge it.

5.4.6 Q: How can I build LLVM compiler-rt with BTI enabled ?

- Download the llvm-12 sources either from the releases page or you can checkout the "release/12.x" from llvm's github. (12 to match your chosen clang version).
- Make a build directory and cd into that.
- Run this cmake command to configure a standalone build of compiler-rt.

```
cmake -G Ninja <llvm sources>/compiler-rt/ -DCMAKE_BUILD_TYPE=Release \  
-DLLVM_CONFIG_PATH=<path to>/llvm-config" \  
-DCMAKE_CXX_FLAGS="-mbranch-protection=bti" \  
-DCMAKE_C_FLAGS="-mbranch-protection=bti" \  
-DCMAKE_ASM_FLAGS="-mbranch-protection=bti" \  
-DCOMPILER_RT_BUILD_SANITIZERS=OFF \  
-DCOMPILER_RT_BUILD_XRAY=OFF \  
-DCOMPILER_RT_BUILD_LIBFUZZER=OFF \  
-DCOMPILER_RT_BUILD_PROFILE=OFF \  
-DCOMPILER_RT_BUILD_MEMPROF=OFF
```

Replace the path to llvm-config with the path to the clang install you want to use to compile. What this does is enable BTI protection for c/cxx/assembly files (all the types in compiler-rt) and disable some parts of the build that you wouldn't need. If you need more components you can find cmake options for them in `compiler-rt/CMakeLists.txt`.

Once you've built that you will find the libraries in `<build folder>/lib/linux`. You can verify that each object in the builtins has the BTI marker by doing the following:

```

/build-llvm-aarch64/lib/linux$ mkdir tmp && cd tmp
/build-llvm-aarch64/lib/linux/tmp$ cp ../libclang_rt.builtins-aarch64.a .
/build-llvm-aarch64/lib/linux/tmp$ ar x libclang_rt.builtins-aarch64.a
/build-llvm-aarch64/lib/linux/tmp$ rm libclang_rt.builtins-aarch64.a
/build-llvm-aarch64/lib/linux/tmp$ for i in *.o; do echo "$i:" &&
readelf -a $i | grep -i bti ; done

```

This should find a BTI line for every file.

```

$ for i in *.o; do echo "$i:" && readelf -a $i | grep -i bti ; done | wc -l
502
$ ls | wc -l
251

```

$251 * 2 = 502$ so all objects in the archive are bti compatible.

How you take this set of libraries and integrate it into your overall build system is up to you. The major thing to note is that the name of the library does not change when you enable BTI protection

5.4.7 Q: How can I build GCC with BTI enabled?

- A GCC toolchain with BTI enabled can easily be built using Crosstool-NG:

```

$ git clone https://github.com/crosstool-ng/crosstool-ng
$ cd crosstool-ng
$ ./bootstrap && ./configure --enable-local && make
$ ./ct-ng aarch64-unknown-linux-gnu
$ cat >>.config <<_EOF_
CT_CC_GCC_EXTRA_CONFIG_ARRAY="--enable-standard-branch-protection"
CT_CC_GCC_CORE_EXTRA_CONFIG_ARRAY="--enable-standard-branch-protection"
_EOF_
$ ./ct-ng build.$(nproc)

```

The above commands will install the new toolchain in `~/x-tools/aarch64-unknown-linux-gnu`. You can then use this toolchain to build and run OP-TEE for [QEMU v8](#) with full BTI support by adding a few arguments to the `make run` command:

```

$ make CFG_CORE_BTI=y CFG_TA_BTI=y CFG_USER_TA_TARGETS=ta_arm64 \
AARCH64_CROSS_COMPILE=~/.x-tools/aarch64-unknown-linux-gnu/bin/aarch64-linux-gnu- \
run

```

5.4.8 Q: What is the quickest and easiest way to try OP-TEE?

- That would be running it on QEMU on a local PC. To do that you would need to:
 - Install the OP-TEE [Prerequisites](#).
 - Build for QEMU according to the instructions at [QEMU v7](#).
 - And [Run xtest](#).
- By summarizing the above, you would need to:

```
$ sudo apt-get install [pre-reqs]
$ mkdir optee-qemu && cd optee-qemu
$ repo init -u https://github.com/OP-TEE/manifest.git
$ repo sync
$ cd build
$ make toolchains -j2
$ make run
QEMU console:      (qemu) c
Normal world shell: # xtest
```

5.5 Certification and security reviews

5.5.1 Q: Will TrustedFirmware.org be involved in GlobalPlatform certification/qualification?

- No, not as of now. Most often certification is performed using a certain software version and on a unique device. I.e., it is the combination software + hardware that gets certified. This is typically something that the SoC or OEM needs to do on their own.
- But it is worth mentioning that since OP-TEE is coming from a proprietary TEE solution that was GlobalPlatform certified on some products in the past and we regularly have people from some member companies running the extended test suite from GlobalPlatform we know that the gap to become GlobalPlatform certified/qualified isn't that big.

5.5.2 Q: Has any test lab been testing OP-TEE?

- [Applus Laboratories](#) have done some side-channel attack testing and fault injection testing on OP-TEE using the *HiKey 620* device. Their findings and fixes can be found at the [Security Advisories](#) page at optee.org.
- [Riscure](#) did a mini-audit of OP-TEE which generated a couple of patches (see *PR#2745*). The [OP-TEE OS Security Advisories](#) page on Github will be updated with more information regarding that in the future.

5.5.3 Q: Where are listed security vulnerabilities addressed in OP-TEE

- Please see [OP-TEE OS Security Advisories](#) page.

5.5.4 Q: Have there been any code audit / code review done?

- Full audit? No! But in the past Linaro have been collaborating with Riscure trying to identify and fix potential security issues. There has also been some companies that have done audits internally and they have then shared the result with us and where relevant, we have created patches resolving the issues reported to us (see *Q: Has any test lab been testing OP-TEE?*).
- Code review, yes! Every single patch going into OP-TEE has been reviewed in a pull request on GitHub. We more or less have a requirement that every patch going into OP-TEE shall at least have one “Reviewed-by” tag in the patch.
- Third party / test lab code review, no! Again some companies have reviewed internally and shared the result with us, but other than that no (see related *Q: Has any test lab been testing OP-TEE?*)

5.6 Contribution

5.6.1 Q: How do I contribute?

- Please see the [Contribute](#) page.

5.6.2 Q: Where can I get help?

- Please see the [Contact](#) page.

5.6.3 Q: I'm new to OP-TEE but I would like to help out, what can I do?

- We always need help with code reviews, feel free to review any of the open [OP-TEE OS Pull Requests](#). Please also note that there could be open pull request in the other [OP-TEE gits](#) that needs reviews too.
 - We always need help answering all the questions asked at [OP-TEE OS Issues](#).
 - If you want to try to solve a bug, please have a look at the [OP-TEE OS Bugs](#) or the [OP-TEE OS Enhancements](#).
 - Documentation tends to become obsolete if not maintained on regular basis. We try to do our best, but we're not perfect. Please have a look at [optee_docs](#) and try to update where you find gaps.
 - Enable *repo* for the device in *manifest* and *build* (and also *Platforms supported*) currently not using *repo*.
 - If you would like to implement a bigger feature, please reach out to us (see [Contact](#)) and we can discuss what is most relevant to look into for the moment. If you already have an idea, feel free to send the proposal to us.
-

5.7 Interfaces

5.7.1 Q: Which API's have been implemented in OP-TEE?

- **GlobalPlatform** (see [GlobalPlatform API](#) for more details).
 - GlobalPlatform's TEE Client API v1.0 (Errata and Precisions 2.0) specification
 - GlobalPlatform's TEE Internal Core API v1.3.1 specification.
 - GlobalPlatform's Secure Elements v1.0 (**now deprecated**, see `git log`).
 - GlobalPlatform's Socket API v1.0 (TCP and UDP, but not TLS).
- AOSP [Keymaster](#) (v3) and AOSP [Gatekeeper](#) (see [AOSP](#) for more details).
- [Android Verified Boot 2.0](#) (AVB 2.0)

5.7.2 Q: Which Linux kernel version supports <some OP-TEE feature>?

- The OP-TEE Linux driver is maintained in the official Linux tree at kernel.org under *drivers/tee*. This is normally where you find the latest code. That being said, some platforms need minor customizations, such as device tree updates, in order to be used in the OP-TEE developer builds (*manifest* files). That is why the [linaro-swg](#) kernel branch *optee* is used in the manifest files. It is rebased onto upstream on a regular basis.
- Older kernels may lack support for newer OP-TEE features. In order to assess in which kernel version some commit has been introduced, you may use the following shell command:

```
$ cd linux
$ git log --no-merges --oneline drivers/tee | \
  while read hash sub; do \
    name=$(git name-rev --tags --name-only $hash | sed 's/\([^~]*\)~.*/[\1]/'); \
    printf "%-20s %s %s\n" "$name" "$hash" "$sub"; \
  done
```

The output looks like this:

```
[v5.12-rc4]          6417f03132a6 module: remove never implemented MODULE_SUPPORTED_
↳DEVICE
[v5.12-rc1-dontuse]  67bc80975279 optee: simplify i2c access
[v5.12-rc1-dontuse]  958567600517 tee: optee: remove need_resched() before cond_
↳resched()
[v5.12-rc1-dontuse]  617d8e8b347e optee: sync OP-TEE headers
[v5.12-rc1-dontuse]  bed13b5fc4f3 tee: optee: fix 'physical' typos
[v5.12-rc1-dontuse]  fda90b29e271 drivers: optee: use flexible-array member instead_
↳of zero-length array
[v5.11-rc6]          dcb3b06d9c34 tee: optee: replace might_sleep with cond_resched
[v5.10-rc6]          853735e40424 optee: add writeback to valid memory type
[v5.11-rc1]          a24d22b225ce crypto: sha - split sha.h into sha1.h and sha2.h
[v5.10-rc5]          be353be27874 tee: amdtee: synchronize access to shm list
...
```

5.8 Hardware and peripherals

5.8.1 Q: Can I use my own hardware IP for crypto acceleration?

- Yes, OP-TEE has a Crypto Abstraction Layer (see *Cryptographic implementation* that was designed mainly to make it easy to add support for hardware crypto acceleration. There you will find information about the abstraction layer itself and what you need to do to be able to support new software/hardware “drivers” in OP-TEE.
-

5.9 License

5.9.1 Q: Under what license is OP-TEE released?

- The software is mostly provided under the [BSD 2-Clause](#) license.
- The TEE kernel driver is released under GPLv2 for obvious reasons.
- xtest (*optee_test*) uses BSD 2-Clause for code running in secure world (Trusted Applications etc) and GPLv2 for code running in normal world (client code).

5.9.2 Q: GlobalPlatform click-through license

- Since OP-TEE is a GlobalPlatform based TEE which implements the APIs as specified by GlobalPlatform one has to accept, the click-through license which is presented when trying to download the *GlobalPlatform API* specifications before start using OP-TEE.

5.9.3 Q: I've modified OP-TEE by using code with non BSD 2-Clause license, will you accept it?

- That is something we deal with case by case. But as a general answer, if it does not contaminate the BSD 2-Clause license we will accept it. Reach out to us (see [Contact](#)) and we will take it from there.
-

5.10 Promotion

5.10.1 Q: I want to get my company logo on op-tee.org, how?

- If your company has done significant contributions to OP-TEE, then please [Contact](#) us and we will do our best to include your company. Pay attention to that we will review this on regular basis and inactive supporting companies might be removed in the future again.
-

5.11 Security vulnerabilities

5.11.1 Q: I have found a security flaw in OP-TEE, how can I disclose it with you?

- Please see the [Contact](#) page.
-

5.12 Source code

5.12.1 Q: Where is the source code?

- It is located on GitHub under the project [OP-TEE](#) and [linaro-swg](#).

5.12.2 Q: Where do I download the test suite called xtest?

- All the source code for that can be found in the git called [optee_test](#).
- The *Extended test (GlobalPlatform tests)* can be purchased separately.

5.12.3 Q: Where is the Linux kernel TEE driver?

- You can find both the generic TEE framework including the OP-TEE driver included in the official Linux kernel project since v4.12. Having that said, we “buffer up” pending patches on a our [Linux kernel TEE framework](#) branch. I.e., that is where we keep new features being developed for OP-TEE. In the long run we aim to completely stop using our own branch and just send all patches to the official Linux kernel tree directly. But as of now we cannot do that.
-

5.13 Testing

5.13.1 Q: How are you testing OP-TEE?

- There is a test suite called xtest that tests the complete TEE-solution to ensure that the communication between all architectural layers is working as it should. The test suite also tests the majority of the GlobalPlatform TEE Internal Core API. It has close to 50,000 and ever increasing test cases, and is also extendable to include the official GlobalPlatform test suite (see *Extended test (GlobalPlatform tests)*).
 - Every pull request in OP-TEE are built for a multitude of different platforms automatically using [Azure DevOps pipelines](#) and [IBART](#). Please have a look there to see whether it failed building on the platform you’re using before submitting any issue about build errors.
 - For more information see [optee_test](#).
-

5.14 Trusted Applications

5.14.1 Q: How do I write a Trusted Application (TA)?

- Have a look at the [Trusted Applications](#) page as well as the [optee_examples](#) page. Those provides guidelines and examples on how to implement basic Trusted Applications.
- If you want to see more advanced uses cases of Trusted Applications, then we encourage that you have a look at the Trusted Applications [optee_test](#).

5.14.2 Q: How do I link a library into a Trusted Application?

- See the example in *sub.mk directives*.
- Also see [Issue#280](#), [Issue#601](#), [Issue#901](#), [Issue#1003](#).

5.14.3 Q: Where should I put my compiled Trusted Application on the device?

- `/lib/optee_armtz`, that is the default location where tee-supplciant will look for Trusted Applications.

5.14.4 Q: What is a Pseudo TA and how do I write one?

- A Pseudo TA is an OP-TEE firmware service offered through the generic API used to invoke Trusted Applications. Pseudo TA interface and services all runs in TEE kernel / core context. I.e., it will have access to the same functions, memory and hardware etc as the TEE core itself. If we're talking ARMv8-A it is running in S-EL1.

5.14.5 Q: Are Pseudo user space TAs supported?

- No!

5.14.6 Q: Can a static TA Open/Invoke dynamic TA?

- Yes, for a longer discussion see [Issue#967](#), [Issue#1085](#), [Issue#1132](#).

5.14.7 Q: How can I extend the GlobalPlatform Internal Core API?

- You may develop your own “Pseudo TA”, which is part of the core (see [Q: What is a Pseudo TA and how do I write one?](#) for more information about the Pseudo TA).

5.14.8 Q: How are Trusted Applications verified?

- Please see the section *Trusted Application private/public keypair* in the *Porting guidelines*.
- Alternatively one can also build a Trusted Application and embed its raw binary content into the OP-TEE firmware binary. At runtime, if invoked, the Trusted Application will be loaded from the OP-TEE firmware image instead of being fetched from the normal world and authenticated in the secure world (see [Early TA](#) for more information).

5.14.9 Q: Is multi-core TA supported?

- Yes, you can have two or more TAs running simultaneously. Please see also [Issue#1194](#).

5.14.10 Q: Is multi-threading supported in a TA?

- No, there is no such concept as `pthreads` or similar. I.e, you cannot spawn thread from a TA. If you need to run tasks in parallel, then you should probably look into running two TAs or more simultaneously and then let them communicate with each other using the TA2TA interface.

5.14.11 Q: How can I use or call OP-TEE from native Android (apk) applications?

- Use the [Java Native Interface \(JNI\)](#).
- First get familiar with [sample_hellojni.html](#) and make sure you can run the sample. After that, replace the C-side Implementation with for example [hello_world](#) or one of the other examples in [optee_examples](#).

Note: Note that [hello_world](#) and other binaries in `optee_examples` are built as executables, and have to be modified to be built as a `.so` shared library instead so that it can be loaded by the Java-side Implementation.

- Note that `*.apk` apps by default have no access to the TEE driver. See [Issue#903](#) for details. The workaround is to disable SELinux before launching any `*.apk` app that calls into OP-TEE. The solution is to create/write SELinux domains/rules to allow any required access, but since this is not a TEE-related issue, it is left as an exercise for the users.
- For a reference implementation contributed by one of our community users, see [optee_android_hello_world_example](#).

5.14.12 Q: I've heard that there is a Widevine and PlayReady TA, how do I get access?

- TrustedFirmware have no such implementation, but Linaro do have reference implementations for that that they share with their members who have signed the WMLA and NDA/MLA with Google and Microsoft. So the advice is to reach out to Linaro if you have questions about that.